
A R C H I T E C T U R E S P E C

2D 탑다운 웹 MMORPG 기술 아키텍처 설계서

모듈 1~10 · 항목 001~100 — 클라이언트 엔진부터 라이브 서비스 운영까지

문서 버전 v1.0

작성일 2026-09-05

기술 스택 uWebSockets.js · Kysely · Phaser 3 · Tiled · Redis

목 차

제 1 장 개요 — 문서 구성 및 아키텍처 총람.....	1
제 2 장 모듈 1 — 웹 & 크로스플랫폼 기술 아키텍처 (001~010).....	2
001. WebGL/Canvas 기반 2D 렌더링 엔진 기본 루프 구조.....	2
002. uWebSockets.js 서버 연결용 바이너리(ArrayBuffer) 소켓 핸들러.....	15
003. PC/모바일 가로형 반응형 화면 스케일링 모듈.....	29
004. 점진적 자원 로딩(Lazy Loading) 및 에셋 번들러.....	40
005. 네트워크 끊김 시 소켓 자동 재접속 및 세션 복구 로직.....	51
006. IndexedDB 기반 정적 에셋(스프라이트, 음원) 캐싱 시스템.....	64
007. 마우스/터치 입력 디바이스 자동 감지 및 조작부 전환.....	77
008. 브라우저 탭 비활성화(Background Thread) 시 메모리 관리 및 동기화 일시정지	91
009. 미사용 스프라이트 가비지 컬렉션(GC) 및 메모리 해제 최적화.....	100
010. WebAssembly(WASM) 연동을 위한 인터페이스 구조.....	109
제 3 장 모듈 2 — 네트워크 및 동기화 시스템 (011~020).....	120
011. 클라이언트 위치 예측(Client Prediction) 엔진.....	121
012. 데드 레코닝(Dead Reckoning) & 위치 보간(Interpolation) 연산 모듈.....	131
013. uWebSockets.js 내장 Pub/Sub 기반 AOI 타일 격자 브로드캐스팅.....	139
014. Delta Compression 기반 상태 변경점 압축 바이너리 패킷 구조.....	149
015. 레이턴시 보상(Lag Compensation) 판정 연산.....	161
016. Server-Authoritative 위치 및 이동 검증 로직.....	170
017. 인스턴스 채널 분산 처리 및 룸(Room) 관리 스키마.....	178
018. Kysely ORM 기반 재화/아이템 안전 이동 분산 DB 트랜잭션.....	190
019. 클라이언트 RTT(Ping) 측정 및 상태 모니터링.....	203

020. 좌표 오차 초과 시 위치 롤백(Rollback) 및 보정 연산.....	213
제 4 장 모듈 3 – 캐릭터 및 성장 시스템 (021~030).....	221
021. 주/보조 스탯 계산 및 최종 수치 연산 시스템.....	222
022. 레벨별 요구 경험치 곡선 및 스탯 상승 공식.....	235
023. 스킬 트리 & 특성 포인트 투자/초기화 데이터 구조.....	244
024. 장비 부위별 착용 슬롯 인벤토리 메커니즘.....	257
025. 장비 강화/인챈트 확률 및 스탯 부여 연산.....	267
026. 클래스 전직 시스템 및 스킬 개방 테이블.....	280
027. 외형(코스튬/스킨) 레이어 분리 및 착용 처리.....	292
028. 칭호/업적 달성 조건 및 패시브 스탯 부여 연산.....	300
029. 장비 무작위 옵션 리롤(Random Option Reroll) 로직.....	313
030. 도감/수집품 달성률 계산 및 전역 보상 연산.....	324
제 5 장 모듈 4 – 전투 및 액션 메커니즘 (031~040).....	336
031. 2D Hitbox / Hurtbox 충돌 판정 (원, 부채꼴, 직사각형).....	336
032. 스킬 쿨타임 및 자원(MP, 기력) 소모/회복 관리자.....	350
033. 모바일용 오토 배틀(Auto-Battle) 타겟팅 & 이동 AI.....	362
034. 다중 타겟팅(Tab 키, 거리순, 클릭) 선택 관리자.....	374
035. CC(기절, 둔화, 속박, 밀쳐내기) 상태 이상 타이머 및 연산.....	386
036. 투사체(직선, 유도, 곡선) 이동 궤적 연산 모듈.....	395
037. 몬스터 어그로(Aggro) 수치 계산 및 타겟 전환 AI.....	406
038. 범위 내 파티 버프 및 시너지 연산.....	416
039. 속성(불, 물, 바람, 땅 등) 상성 피해 비율 공식.....	427
040. 전투 피드백 연출 (화면 흔들림, 데미지 텍스트 팝업, 피격 이펙트).....	439
제 6 장 모듈 5 – 월드 & 환경 시스템 (041~050).....	452
041. Tiled 타일맵 레이어 (바닥, 벽, 상단 가림막) 파싱 및 렌더링.....	452
042. Y-Sorting (Y 좌표 기준 캐릭터/오브젝트 깊이 정렬) 엔진.....	464

043. 2D A* (A-Star) 길찾기 알고리즘 및 이동 가능 영역 Grid 연산.....	473
044. 실시간 동적 2D 조명 & 낮밤 시간 변화 셰이더/파티클.....	490
045. 날씨(비, 눈, 안개) 환경 파티클 시스템.....	501
046. 포탈 진입 및 인스턴스 맵 이공간 텔레포트 매핑.....	512
047. 필드 채집/채광 리젠 오브젝트 상호작용.....	521
048. 미니맵 동기화 및 안개(Fog of War) 탐색 가림 처리.....	532
049. 필드 돌발 이벤트 영역 감지 및 트리거.....	542
050. 핫스팟 휴식/앉기 등 오브젝트 상호작용 연출.....	553
제 7 장 모듈 6 – 소설 및 커뮤니티 시스템 (051~060).....	564
051. uWebSockets.js 토픽 기반 다중 채널(전체, 필드, 파티, 길드, 귓속말) 채팅 시스템.....	564
052. 길드 창고, 출석, 기술 연구, 권한 관리 데이터 구조 및 DDL.....	574
053. 파티 결성, 실시간 파티원 위치 공유 및 아이템 분배(순번/균등/루팅) 로직	589
054. 친구 목록, 실시간 접속 상태 모니터링 및 차단 시스템.....	599
055. 1:1 개인 거래 시스템 (아이템/재화 상호 수락 및 Kysely DB 트랜잭션).....	610
056. 통합 경매장/위탁 판매소 (검색, 시세, 등록, 즉시 구매).....	623
057. 감정표현(Emote) 및 픽셀 애니메이션 동기화.....	637
058. 영지전 / 공성전 대규모 PvP 인스턴스 권한 시스템.....	648
059. Redis ZSET 기반 실시간 레벨/PvP/길드 랭킹 산출.....	659
060. 멘토링 및 추천인 시스템 연계 보상 로직.....	668
제 8 장 모듈 7 – 주요 콘텐츠 시스템 (061~070).....	681
061. 퀘스트 트래킹(메인, 서브, 일일) 및 대화/조건 달성 체커.....	682
062. 일일/주간 미션 및 활동도 게이지 보상 시스템.....	695
063. 인스턴스 던전 솔로/파티 매칭 및 전용 룸 세션 관리.....	707
064. 필드 보스 스폰 타이머, 알림 및 기여도(딜량)별 아이템 분배 로직.....	718

065. PvP 전장 및 PK 카오스(성향치) 시스템 매커니즘.....	730
066. 제작/연금술 재료 합성 레시피 연산 모듈.....	739
067. 펫/탈것 이동속도 보정 및 자동 루팅 AI.....	751
068. 하우스/개인 영지 가구 배치 및 인스턴스 공간 생성.....	761
069. 캐주얼 미니게임(낚시 타이밍 미니게임 등) 상호작용.....	770
070. 시즌 패스 / 배틀 패스 달성도 및 보상 지급 로직.....	779
제 9 장 모듈 8 – UI/UX 및 크로스플랫폼 가용성 (071~080).....	791
071. 모바일 가상 조이스틱 (Touch Virtual Joystick) 반응형 구현.....	791
072. PC 용 마우스 클릭 이동 및 WASD/방향키 조작 통합 컨트롤러.....	803
073. 해상도 반응형 HUD 스케일링 및 모바일 터치 영역 최적화.....	815
074. 터치 제스처 (핀치 투 줌, 스와이프) 이벤트 핸들러.....	825
075. 드래그 앤 드롭 & 터치 인벤토리 아이템 그리드 인터페이스.....	836
076. 퀵슬롯 커스텀 배치 및 다중 페이지 스와이프 UI.....	851
077. 착용 장비 vs 선택 장비 스탯 비교 툴팁 레이어.....	864
078. 퀘스트 자동 길찾기 경로 내비게이션 라인 표시.....	873
079. 모바일 절전 모드 (프레임 제한 및 화면 잠전 처리).....	886
080. 브라우저 웹 푸시 알림 (Web Push Notification) 연동.....	896
제 10 장 모듈 9 – 경제 및 BM(수익화) 시스템 (081~090).....	907
081. 다중 재화(골드, 캐시, 이벤트 코인, 길드 포인트) 관리 시스템.....	907
082. 웹 PG 결제 API 및 모바일 인앱 결제(IAP) 영수증 서버 검증.....	921
083. 확률형 상품(가챠) 뽑기 및 천장(Pity) 시스템 연산.....	932
084. VIP / 월정액 매일 자동 보상 지급 스케줄러.....	945
085. 인플레이션 방지를 위한 재화 소모(sink) 매커니즘.....	955
086. 귀속 / 비귀속 아이템 판정 및 거래 제한 상태 관리.....	963
087. 어뷰징/현금거래 방지용 거래소 시세 상하한가 제어 로직.....	972
088. 이벤트 코인 교환소 기간 제한 매커니즘.....	983

089. 무료/프리미엄 배틀패스 보상 트랙 분기 처리.....	994
090. 스테미너 / 피로도 자동 회복 및 차감 연산.....	1003
제 11 장 모듈 10 – 보안, 어드민 및 라이브 서비스 (091~100).....	1014
091. 클라이언트 메모리 변조 및 스피드해킹 탐지 검증.....	1014
092. 서버 측 이동 거리 및 타일 충돌 재연산 (벽뚫기 방지).....	1025
093. 매크로/봇 패턴 탐지 및 자동 캡차(CAPTCHA) 트리거.....	1034
094. 어드민 웹 대시보드 (유저 조회, 아이템 지급, 공지, 제재 API).....	1043
095. 인게임 재화 유동성 및 유저 데이터 로그/텔레메트리 수집.....	1056
096. 웹 브라우저 무중단 핫픽스(Hot-Fix) 리소스 업데이트.....	1070
097. 다국어(i18n) localization 텍스트 파싱 및 폰트 레이아웃.....	1082
098. 소셜 로그인(구글, 카카오 등) OAuth2 & 게스트 계정 연동/이전.....	1094
099. 계정/IP/디바이스 단위 제재(Ban) 및 채팅 금지(Mute) 미들웨어.....	1110
100. DB 자동 스냅샷 백업 및 분산 서버 패일오버(Failover).....	1119

※ 본 목차는 필드 코드로 생성되었습니다. 문서 편집 후 목차를 우클릭하고 '필드 업데이트'를 선택하면
페이지 번호가 갱신됩니다.

제 1 장 개요 — 문서 구성 및 아키텍처 총람

본 설계서는 2D 탑다운 웹 MMORPG 를 브라우저에서 상용 서비스 수준으로 구동하기 위한 100 개 항목의 모듈 설계서와 핵심 구현 코드를 담고 있다. 문서는 10 개 모듈(항목 001~100)로 구성되며, 각 항목은 (1) 역할 설명, (2) 즉시 실행 가능한 TypeScript/SQL 구현 코드, (3) 성능 최적화 팁의 3 단 구조로 작성되었다. 코드는 지정 기술 스택(uWebSockets.js, Kysely, Phaser 3/PixiJS, Tiled, Redis, Fastify)을 기준으로 작성되었고, 클라이언트와 서버가 공유하는 모듈은 shared/ 네임스페이스로 구분했다.

전체 아키텍처는 5 계층으로 요약된다. 표현 계층(모듈 1·5·8)은 렌더링·월드·UI 를 담당하고, 통신 계층(모듈 2·6)은 위치 동기화와 소셜 채널을 운반한다. 데이터 계층(모듈 3·9)은 스탯·성장·경제의 원자성을 보장하며, 콘텐츠 계층(모듈 4·7)은 전투와 게임플레이 시스템을 구성한다. 마지막으로 운영 계층(모듈 10)은 보안·어드민·라이브 서비스 관제를 맡는다. 계층 간 의존 방향은 표현→통신→데이터→운영의 단방향을 원칙으로 하여, 순환 의존에 의한 결합도 상승을 차단한다.

각 모듈은 독립적으로 읽을 수 있도록 설계했지만, 몇 가지 공통 패턴이 문서 전체에서 반복된다. 서버 권위(Server-Authoritative) 판정, 먹등 지급(ledger ref), 이벤트 버스 구독 체커, 더티 플래그 재계산, TypedArray 기반 바이너리 코덱이 그것이다. 실무 적용 시에는 먼저 모듈 1~2(엔진·동기화)와 모듈 3(스탯 파이프라인)을 기반으로 구축하고, 이후 모듈 4~9 를 콘텐츠 우선순위에 따라 단계적으로 채택한 뒤, 모듈 10 을 라이브 오픈 전 완성하는 순서를 권장한다.

표 1-1. 시스템 5 계층 구성

계층	포함 모듈	핵심 역할
표현 계층	모듈 1, 5, 8	렌더링 루프·월드 렌더· 조작부와 HUD
통신 계층	모듈 2, 6	위치 동기화·AOI·채팅·실시간 소셜
데이터 계층	모듈 3, 9	스탯·성장·재화·경제의 원자적 연산
콘텐츠 계층	모듈 4, 7	전투·퀘스트·던전·경제 콘텐츠
운영 계층	모듈 10	보안·어드민·텔레메트리· 가용성

표 1-2. 모듈 구성 및 기술 스택

모듈	항목	주제	핵심 기술 스택
1	001~010	웹 & 크로스플랫폼 기술 아키텍처	TypeScript · Phaser 3/PixiJS · uWebSockets.js(Client Ws) · IndexedDB
2	011~020	네트워크 및 동기화 시스템	Node.js · uWebSockets.js · ArrayBuffer(TypedArray) · Kysely(PostgreSQL)
3	021~030	캐릭터 및 성장 시스템	PostgreSQL DDL · Kysely · TypeScript Seed · TypeScript
4	031~040	전투 및 액션 메커니즘	TypeScript · Phaser 3/PixiJS 물리 · uWebSockets.js
5	041~050	월드 & 환경 시스템	Tiled Map Editor(JSON) · Phaser 3/PixiJS · TypeScript
6	051~060	소셜 및 커뮤니티 시스템	uWebSockets.js(Pub/ Sub Topics) · Kysely · Redis · TypeScript
7	061~070	주요 콘텐츠 시스템	TypeScript · Kysely · uWebSockets.js
8	071~080	UI/UX 및 크로스플랫폼 가용성	HTML5/DOM · Phaser 3 UI · TypeScript · Web Push API
9	081~090	경제 및 BM(수익화) 시스템	Node.js · Kysely(PostgreSQL) · 결제 PG/IAP API · TypeScript
10	091~100	보안, 어드민 및 라이브 서비스	uWebSockets.js · Kysely · Fastify(어드민 API) · OAuth2 · TypeScript

제 2 장 모듈 1 — 웹 & 크로스플랫폼 기술 아키텍처 (001~010)

기술 스택 : TypeScript · Phaser 3/PixiJS · uWebSockets.js(Client Ws) · IndexedDB
본 장은 클라이언트 엔진의 뼈대를 구성하는 10개 항목이다. 렌더링 루프와 바이너리 소켓 핸들러가 성능의 1차 결정 요소이며, 반응형 스케일링·자연 로딩·IndexedDB 캐싱·재접속 복구는 웹 플랫폼 특유의 제약을 해소하는 장치다. 백그라운드 탭 관리·스프라이트 GC·WASM 인터페이스는 장기 세션 안정성과 확장성을 위한 기반으로, 이후 모든 모듈이 이 장의 구조물 위에 얹힌다.

001. WebGL/Canvas 기반 2D 렌더링 엔진 기본 루프 구조

(1) 역할 설명

2D 탑다운 MMORPG의 모든 화면 출력은 requestAnimationFrame(rAF) 기반 고정 스텝 시뮬레이션 + 가변 렌더 구조에서 출발한다. Phaser 3는 내부적으로 이 구조를 갖추고 있지만 대규모 씬 전환·커스텀 매니저 운용을 위해서는 프레임 루프의 3단계(입력 수집 → 고정 스텝 업데이트 → 렌더)를 명시적으로 분리한 게임 루프 관리자가 필요하다. 고정 스텝(FIXED_STEP)은 물리·전투 판정의 결정론성을 보장하고, 렌더는 누적 시간을 보간해 프레임율과 무관하게 부드러운 움직임을 만든다. 렌더러는 WebGL을 우선 사용하고 WebGL 컨텍스트 상실 시 Canvas2D로 자동 폴백한다.

이 모듈은 Phaser 게임 인스턴스의 부팅 순서(부트 → 프리로드 → 월드 → UI)를 통제하며, 프레임 타임아웃 감시(스파이크 감지)와 FPS/드로우콜 계측 혹은 함께 제공한다. 특히 모바일 저사양 기기에서는 렌더 해상도 스케일을 동적으로 낮추어 30fps 하한을 지키는 적응형 품질 제어의 기반이 된다.

(2) 구현 코드 — src/engine/RenderLoop.ts

```
import Phaser from "phaser";

export const FIXED_STEP = 1000 / 60; // 60Hz 고정 시뮬레이션 스텝(ms)

export interface LoopStats {
  fps: number; drawCalls: number; entities: number; spikeMs: number;
}

export class RenderLoop {
  private acc = 0;
  private last = 0;
  private alpha = 0; // 보간 계수 (0~1)
  private stats: LoopStats = { fps: 0, drawCalls: 0, entities: 0, spikeMs: 0 };
  private frameCount = 0;
  private fpsTimer = 0;
  private renderScale = 1;
```

```

constructor(
  private game: Phaser.Game,
  private onFixedUpdate: (dtMs: number) => void,
  private onRender?: (alpha: number) => void,
) {
  // WebGL 컨텍스트 상실 감지 → Canvas 폴백 유도
  const canvas = game.canvas;
  canvas.addEventListener("webglcontextlost", (e) => {
    e.preventDefault();
    console.warn("[RenderLoop] WebGL context lost — pause & hint reload");
    this.game.loop.sleep();
  });
  canvas.addEventListener("webglcontextrestored", () => {
    this.game.loop.wake();
  });
}

/** Phaser 실행 밖에서 구동하는 독립 루프(커스텀 매니저용) */
start() {
  this.last = performance.now();
  const tick = (now: number) => {
    requestAnimationFrame(tick);
    let frame = now - this.last;
    this.last = now;
    if (frame > 250) frame = 250; // 탭 복귀 등 스파이크 클램프
    this.stats.spikeMs = Math.max(this.stats.spikeMs, frame);

    this.acc += frame;
    let steps = 0;
    while (this.acc >= FIXED_STEP && steps < 5) { // 스파이럴 오브 데스 방지
      this.onFixedUpdate(FIXED_STEP);
      this.acc -= FIXED_STEP;
      steps++;
    }
    this.alpha = this.acc / FIXED_STEP;
    this.onRender?.(this.alpha);

    this.measure(frame, now);
  };
  requestAnimationFrame(tick);
}

private measure(frameMs: number, now: number) {
  this.frameCount++; this.fpsTimer += frameMs;
  if (this.fpsTimer >= 1000) {
    this.stats.fps = Math.round(this.frameCount * 1000 / this.fpsTimer);
    this.frameCount = 0; this.fpsTimer = 0;
    this.adaptQuality(); // 1 초마다 품질 자동 조정
  }
}

```

```

/** 적응형 품질: fps 하방 미달 시 렌더 스케일 축소 */
private adaptQuality() {
  if (this.stats.fps < 28 && this.renderScale > 0.6) {
    this.renderScale = Math.max(0.6, this.renderScale - 0.1);
    this.game.scale.setZoom(this.renderScale);
  } else if (this.stats.fps > 55 && this.renderScale < 1) {
    this.renderScale = Math.min(1, this.renderScale + 0.05);
    this.game.scale.setZoom(this.renderScale);
  }
}

getStats(): LoopStats { return this.stats; }
}

// — Phaser 부팅 설정 예시

export function bootGame(parent: HTMLElement): Phaser.Game {
  return new Phaser.Game({
    type: Phaser.AUTO,           // WebGL 우선, 미지원 시 Canvas
    parent,
    width: 1280, height: 720,
    backgroundColor: "#0b1c2c",
    render: { antialias: false, powerPreference: "high-performance" },
    scale: { mode: Phaser.Scale.FIT, autoCenter: Phaser.Scale.CENTER_BOTH },
    fps: { target: 60, forceSetTimeout: false },
    scene: [],                   // 씬은 AssetFlow 이후 등록
  });
}

```

(3) 성능 최적화 팁

- 고정 스텝과 렌더 보간(alpha)을 분리하면 144Hz 모니터와 30fps 모바일에서 동일한 전투 판정 결과를 얻는다.
- 스파이럴 오브 데스 방지를 위해 한 프레임의 최대 업데이트 스텝(여기서는 5 회)을 반드시 제한한다.
- 드로우콜은 텍스처 아틀라스 단위로 줄어드므로, 스프라이트는 반드시 정적 아틀라스 또는 동적 텍스처 아틀라스로 묶는다.
- powerPreference: high-performance 는 하이브리드 GPU 노트북에서 전용 GPU 를 우선 사용하게 한다.

002. uWebSockets.js 서버 연결용 바이너리(ArrayBuffer) 소켓 핸들러

(1) 역할 설명

MMORPG 트래픽의 대부분은 위치·액션·상태 동기화 패킷이며, JSON 텍스트 대신 ArrayBuffer 바이너리 프로토콜을 사용하면 패킷 크기를 1/5~1/10 수준으로 줄일 수 있다. 이

핸들러는 클라이언트와 서버(uWebSockets.js)가 공유하는 패킷 ID 체계를 기준으로 DataView로 직렬화/역직렬화를 수행하고, 수신 버퍼를 패킷 단위로 분할하는 스트림 파서를 포함한다. 모든 패킷은 [u16 길이][u8 타입][페이로드] 헤더 구조를 따라 파서가 단순하고 서버 측 C++ 코어(uWS)의 메모리 복사를 최소화한다.

핸들러는 연결 생명주기(오픈/클로즈/에러) 이벤트를 EventBus로 중계하고, 송신 큐에 백프레시(전송 지연 누적)를 감시해 네트워크 포화 시 패킷 우선순위 드롭(장식 이펙트 패킷 폐기)을 적용한다. 이는 100명 이상 동시 접속 구역에서의 프레임 드랍을 방지하는 핵심 장치다.

(2) 구현 코드 — src/net/packet.ts — 공유 패킷 정의 및 코덱

```
// 클라이언트/서버 공유 패킷 ID (uWebSockets.js 서버와 동일 상수 사용)
export const enum PKT {
  C2S_MOVE      = 1, // 이동 의도
  C2S_SKILL     = 2, // 스킬 사용
  C2S_PING      = 3,
  S2C_SNAPSHOT  = 64, // AOI 스냅샷(델타)
  S2C_ENTITY_SPAWN = 65,
  S2C_ENTITY_DESPAWN = 66,
  S2C_PONG      = 67,
  S2C_KICK      = 68,
}

export interface MoveIntent { seq: number; x: number; y: number; dx: number; dy: number; }

export function encodeMove(m: MoveIntent): ArrayBuffer {
  const buf = new ArrayBuffer(1 + 4 + 4 + 4 + 4 + 4); // type + seq + x + y + (dx,dy packed)
  const v = new DataView(buf);
  let o = 0;
  v.setUint8(o++, PKT.C2S_MOVE);
  v.setUint32(o, m.seq); o += 4;
  v.setInt16(o, Math.round(m.x * 10)); o += 2; // x100 배율 고정소수
  v.setInt16(o, Math.round(m.y * 10)); o += 2;
  v.setInt8(o, Math.round(m.dx * 127)); o += 1; // -1~1 정규화 → i8
  v.setInt8(o, Math.round(m.dy * 127)); o += 1;
  return buf.slice(0, o); // 실제 길이만큼 절단
}

export function decodeMove(buf: ArrayBuffer): MoveIntent {
  const v = new DataView(buf);
  let o = 1; // skip type
  const seq = v.getUint32(o); o += 4;
  const x = v.getInt16(o) / 10; o += 2;
  const y = v.getInt16(o) / 10; o += 2;
  const dx = v.getInt8(o) / 127; o += 1;
  const dy = v.getInt8(o) / 127;
  return { seq, x, y, dx, dy };
}
```

보조 코드 — src/net/BinarySocket.ts — 핸들러 본체

```
import { PKT } from "../packet";

type Handler = (type: number, payload: ArrayBuffer) => void;

export class BinarySocket {
  private ws: WebSocket | null = null;
  private rx = new Uint8Array(0); // 수신 스트림 버퍼
  private outQueue: ArrayBuffer[] = [];
  private outBytes = 0;
  private static MAX_OUT_BYTES = 256 * 1024; // 백프레서 한계(256KB)

  constructor(private url: string, private onPacket: Handler,
    private onState: (s: "open" | "closed" | "error") => void) {}

  connect() {
    this.ws = new WebSocket(this.url);
    this.ws.binaryType = "arraybuffer";
    this.ws.onopen = () => this.onState("open");
    this.ws.onclose = () => this.onState("closed");
    this.ws.onerror = () => this.onState("error");
    this.ws.onmessage = (ev) => this.consume(ev.data as ArrayBuffer);
  }

  /** 스트림 버퍼에서 [u16 len][u8 type][payload] 단위로 절단 */
  private consume(chunk: ArrayBuffer) {
    const merged = new Uint8Array(this.rx.length + chunk.byteLength);
    merged.set(this.rx); merged.set(new Uint8Array(chunk), this.rx.length);
    this.rx = merged;

    const v = new DataView(this.rx.buffer);
    let off = 0;
    while (this.rx.length - off >= 3) {
      const len = v.getUint16(off); // 페이로드 길이
      if (this.rx.length - off < 3 + len) break; // 아직 덜 도착
      const type = v.getUint8(off + 2);
      const payload = this.rx.buffer.slice(off + 3, off + 3 + len);
      this.onPacket(type, payload);
      off += 3 + len;
    }
    if (off > 0) this.rx = this.rx.slice(off);
  }

  /** 우선순위 송신: 큐 포화 시 low 패킷 폐기 */
  send(data: ArrayBuffer, priority: "high" | "low" = "high") {
    if (priority === "low" && this.outBytes > BinarySocket.MAX_OUT_BYTES * 0.7) return;
    this.outQueue.push(data);
    this.outBytes += data.byteLength;
    this.flush();
  }
}
```

```
private flush() {
  if (!this.ws || this.ws.readyState !== WebSocket.OPEN) return;
  while (this.outQueue.length) {
    const item = this.outQueue.shift();
    this.ws.send(item);
    this.outBytes -= item.byteLength;
  }
}

close() { this.ws?.close(); this.outQueue = []; this.outBytes = 0; }
}
```

(3) 성능 최적화 팁

- uWebSockets.js 서버 측에서는 message 이벤트가 ArrayBuffer 가 아닌 Buffer 를 주므로, 서버 코덱은 Buffer/DataView 양쪽에서 동작하도록 만들어 둔다.
- 위치는 절대 float64 대신 i16 고정소수(×10 배율)로 보내면 패킷이 절반 이하로 줄어든다.
- 수신 버퍼 절단 로직은 반드시 부분 패킷(3+len 미만)을 남기고 대기해야 TLS 단편화에서도 안전하다.
- 백프레시 드롭은 채팅/전투 패킷에 적용하면 안 되며, 이펙트·파티클 등 재생성 가능한 트래픽에만 적용한다.

003. PC/모바일 가로형 반응형 화면 스케일링 모듈

(1) 역할 설명

웹 MMORPG 는 4K 모니터부터 폴더블 폰, 세로 회전까지 화면 환경이 극단적으로 다양하다. 이 모듈은 가로형(landscape) 게임 기준 해상도(예: 1280x720)를 베이스로, 화면 종횡비에 따라 Phaser Scale.FIT / EXPAND 모드를 자동 전환하고 HUD(DOM 오버레이)의 크기·터치 영역을 디바이스 등급별로 스케일링한다. 세로로 회전하면 '화면을 가로로 돌려주세요' 가이드 오버레이를 표시해 모바일 UX 파손을 막는다.

또한 devicePixelRatio(DPR)를 상한(2.0)으로 클램프해 고해상도 폰에서의 필 레이트 폭증을 방지하고, resize 이벤트를 디바운싱하여 아이폰 사파리 주소창 수축/확장으로 인한 수십 회의 리사이즈 폭풍을 흡수한다. 게임 캔버스와 DOM HUD 의 좌표계가 어긋나지 않도록 두 영역에 동일한 스케일 팩터를 적용하는 것이 핵심 설계 포인트다.

(2) 구현 코드 — src/engine/ResponsiveScaler.ts

```
import Phaser from "phaser";

export interface ScaleInfo {
  width: number; height: number; // CSS 픽셀 기준 뷰포트
  dpr: number; // 클램프된 devicePixelRatio
  isMobile: boolean; // 터치 + 화면 폭 기준 판정
```

```

    landscape: boolean;
}

const BASE_W = 1280, BASE_H = 720;
const DPR_MAX = 2.0;

export class ResponsiveScaler {
    private info: ScaleInfo;
    private rotateOverlay: HTMLElement;
    private resizeTimer = 0;

    constructor(private game: Phaser.Game, private hudRoot: HTMLElement) {
        this.rotateOverlay = document.getElementById("rotate-guide");
        this.info = this.measure();
        this.apply();
        this.bind();
    }

    private measure(): ScaleInfo {
        const vw = window.innerWidth, vh = window.innerHeight;
        const rawDpr = window.devicePixelRatio || 1;
        const isMobile = matchMedia("(pointer: coarse)").matches && Math.min(vw, vh) < 820;
        return {
            width: vw, height: vh,
            dpr: Math.min(rawDpr, DPR_MAX),
            isMobile,
            landscape: vw >= vh,
        };
    }

    private bind() {
        // 디바운스 200ms: 사파리 주소창 리사이즈 폭풍 흡수
        window.addEventListener("resize", () => {
            clearTimeout(this.resizeTimer);
            this.resizeTimer = window.setTimeout(() => {
                this.info = this.measure();
                this.apply();
            }, 200);
        });
        window.addEventListener("orientationchange", () =>
            setTimeout(() => { this.info = this.measure(); this.apply(); }, 120));
    }

    private apply() {
        const { width, height, dpr, isMobile, landscape } = this.info;

        // 1) 세로 화면 + 모바일 → 회전 가이드
        this.rotateOverlay.style.display = (!landscape && isMobile) ? "flex" : "none";
        if (!landscape && isMobile) { this.game.loop.sleep(); return; }
        this.game.loop.wake();
    }
}

```

```

// 2) 종횡비에 따른 Scale 모드 전환
const ratio = width / height;
const mode = ratio > BASE_W / BASE_H
  ? Phaser.Scale.FIT      // 더 넓으면 게임 전체 표시(레터박스 좌우)
  : Phaser.Scale.EXPAND;  // 더 좁으면 가로 절단 없이 확장
this.game.scale.setGameSize(BASE_W, BASE_H);
this.game.scale.scaleManager.setMode(mode);
this.game.scale.refresh();

// 3) 캔버스 해상도 = CSS 크기 x 클램프 DPR
this.game.canvas.style.width = width + "px";
this.game.canvas.style.height = height + "px";

// 4) HUD 스케일: 모바일에서 터치 타겟 44px 이상 보장
const hudScale = isMobile
  ? Math.max(1, Math.min(width / BASE_W, 1.6))
  : Math.max(0.8, Math.min(width / BASE_W, 1.35));
this.hudRoot.style.setProperty("--hud-scale", hudScale.toFixed(3));
this.hudRoot.style.setProperty("--touch-min", isMobile ? "44px" : "32px");
}

getInfo(): ScaleInfo { return this.info; }
}

// HUD CSS 예: .hud-panel { transform: scale(var(--hud-scale)); transform-origin: top left; }
//           .btn { min-width: var(--touch-min); min-height: var(--touch-min); }

```

(3) 성능 최적화 팁

- DPR 클램프(2.0)만으로 고해상도 폰에서 fill-rate 병목을 크게 줄일 수 있다 — 3x/4x DPR 기기는 픽셀 수가 9~16 배다.
- resize 디바운스는 200ms 가 적정이며, orientationchange 는 물리 회전이 끝난 뒤 치수가 확정되므로 120ms 지연 후 측정한다.
- 게임 캔버스(Phaser)와 DOM HUD 의 스케일 팩터를 동일 변수(--hud-scale)로 공유해야 좌표가 어긋나지 않는다.
- EXPAND 모드는 렌더 영역이 커지므로, 카메라 경계와 미니맵 계산에 game.scale.gameSize 를 기준으로 사용한다.

004. 점진적 자원 로딩(Lazy Loading) 및 에셋 번들러

(1) 역할 설명

MMORPG 는 맵·스프라이트·음원·이펙트 수천 개를 다루지만, 플레이어가 동시에 필요로 하는 자원은 현재 구역 주변뿐이다. 이 모듈은 에셋을 '번들(bundle)' 단위로 정의하고(예: village-base, village-npc, dungeon-fire), 현재 씬에서 도달 가능한 번들을 우선순위 큐로

로딩하며, 거리·스테이지 전환 예측에 따라 프리페치한다. 로딩은 우선순위 + 동시성 제한(4개)으로 진행되어 대역폭 경쟁으로 인한 지연 폭증을 막는다.

번들러(build 타임)는 에셋 파일을 해시명으로 카피하고 manifest.json 을 생성한다. 매니페스트에는 각 번들의 파일 목록·용량·해시가 기록되어, 클라이언트는 이를 참조해 캐시 히트(IndexedDB, 006 번)를 먼저 확인하고 누락분만 네트워크로 받는다. 이 구조는 이후 핫픽스(096 번)와도 연결되어 부분 업데이트의 기반이 된다.

(2) 구현 코드 — src/assets/BundleManifest.ts — 빌드 타임 매니페스트 타임

```
export interface AssetEntry {
  key: string;      // 게임 내 참조 키 (예: "hero_idle")
  url: string;      // 해시 경로 (예: "/a/s/hero_idle.a1b2c3.png")
  bytes: number;
  hash: string;     // SHA-1(내용) — 캐시 무효화 판정
  type: "image" | "atlas" | "audio" | "json" | "bitmapfont";
}
export interface AssetBundle {
  id: string;        // "village-base"
  priority: number;  // 낮을수록 먼저 로딩
  entries: AssetEntry[];
  preloadWith?: string[]; // 의존 번들
}
export interface Manifest { version: string; bundles: Record<string, AssetBundle>; }
```

보조 코드 — src/assets/AssetFlow.ts — 우선순위 지연 로딩 관리자

```
import Phaser from "phaser";
import type { Manifest, AssetBundle } from "../BundleManifest";

export class AssetFlow {
  private loaded = new Set<string>(); // 번들 id
  private queue: AssetBundle[] = [];
  private active = 0;
  private static CONCURRENCY = 4;

  constructor(private game: Phaser.Game, private manifest: Manifest,
    private cache: { has(key: string, hash: string): boolean }) {}

  /** 현재 구역 진입 시 호출 — 필요 번들을 우선순위로 적재 */
  request(bundleIds: string[], onProgress?: (p: number) => void) {
    const need = bundleIds
      .map(id => this.manifest.bundles[id])
      .filter(b => b && !this.loaded.has(b.id));
    this.queue.push(...need);
    this.queue.sort((a, b) => a.priority - b.priority);
    this.pump(onProgress);
  }

  private pump(onProgress?: (p: number) => void) {
```

```

while (this.active < AssetFlow.CONCURRENCY && this.queue.length) {
  const bundle = this.queue.shift(!);
  this.active++;
  this.loadBundle(bundle, onProgress).finally(() => {
    this.active--;
    this.loaded.add(bundle.id);
    this.pump(onProgress);
  });
}

private async loadBundle(b: AssetBundle, onProgress?: (p: number) => void) {
  const loader = this.game.load;
  let done = 0;
  for (const e of b.entries) {
    if (this.cache.has(e.key, e.hash)) { done++; continue; } // IndexedDB 히트
    switch (e.type) {
      case "image": loader.image(e.key, e.url); break;
      case "atlas": loader.atlas(e.key, e.url, e.url.replace(".png", ".json")); break;
      case "audio": loader.audio(e.key, e.url); break;
      case "json": loader.json(e.key, e.url); break;
      case "bitmapfont": loader.bitmapFont(e.key, e.url); break;
    }
    loader.once("filecomplete-" + e.type + "-" + e.key, () => {
      done++;
      onProgress?.(done / b.entries.length);
    });
  }
  if (!loader.isLoading()) loader.start();
  await new Promise<void>(res => loader.once("complete", () => res()));
}

/** 스테이지 전환 예측 프리페치 — idle 시 낮은 우선순위로 미리 받기 */
prefetch(bundleIds: string[]) {
  setTimeout(() => this.request(bundleIds), 1500);
}
}

```

(3) 성능 최적화 팁

- 번들 경계는 '한 구역에서 반드시 함께 필요한 세트'로 자르는 것이 원칙이며, 개별 파일 단위 로딩은 HTTP 오버헤드로 오히려 느려진다.
- 매니페스트 해시로 캐시 무효화를 판정하면 배포 후 같은 URL 을 재사용해도 안전하다 (캐시 폭독 방지).
- 동시성 4~6 개가 일반적인 적정값이며, HTTP/2 환경에서는 이후 6~8 개로 올려도 된다.
- 오디오는 무손실 대신 m4a/ogg 이중 인코딩으로 제공해 총 번들 용량을 60~70% 줄일 수 있다.

005. 네트워크 끊김 시 소켓 자동 재접속 및 세션 복구 로직

(1) 역할 설명

이동 중 지하철 진입, Wi-Fi 전환, 서버 배포 등으로 소켓은 언제든지 끊긴다. 재접속 모듈의 목표는 두 가지다. 첫째, 지수 백오프(exponential backoff) + 지터(jitter)로 재접속 폭풍(스웜)을 막으면서 빠르게 재연결하고, 둘째, 재연결 후 세션 토큰으로 서버에 '이어하기'를 요청해 캐릭터 상태·위치·파티 정보를 복원한다. 오프라인 동안의 로컬 입력은 큐잉했다가 복구 시 서버와 조정(reconcile)한다.

세션 복구의 핵심은 '재접속에 성공해도 세션이 살아 있으면 썸을 다시 로드하지 않는다'는 것이다. 서버는 세션 토큰과 마지막 처리 시퀀스 번호를 유지하며, 클라이언트가 보낸 미처리 입력을 seq 기준으로 거르고, 누락 구간의 스냅샷을 재전송한다. 이를 통해 끊김 복구가 유저 눈에 보이는 '재접속 로딩'이 아니라 1~2 초의 반투명 오버레이로 끝난다.

(2) 구현 코드 — src/net/ReconnectingSocket.ts

```
import { BinarySocket } from "./BinarySocket";

export interface SessionRestore {
  sessionToken: string;
  lastAckSeq: number; // 서버가 마지막으로 승인한 입력 seq
  characterSnapshot: unknown; // HP/MP/버프/위치 등
}

export class ReconnectingSocket {
  private sock: BinarySocket | null = null;
  private attempts = 0;
  private token = localStorage.getItem("sessionToken") || "";
  private pendingInputs: ArrayBuffer[] = []; // 오프라인 입력 큐
  private alive = true;
  onRestore?: (r: SessionRestore) => void;

  constructor(private makeSocket: () => BinarySocket,
    private ui: { show: (msg: string) => void; hide: () => void } ) {}

  start() {
    this.sock = this.makeSocket();
    this.sock.connect();
  }

  onDisconnected() {
    if (!this.alive) return;
    this.attempts++;
    if (this.attempts > 8) { this.ui.show("연결이 불안정합니다. 앱을 다시 시작해 주세요."); return; }

    // 지수 백오프 + 지터: 0.5s, 1s, 2s, 4s ... 최대 15s
    const base = Math.min(15000, 500 * Math.pow(2, this.attempts - 1));
```

```

const wait = base * (0.7 + Math.random() * 0.6); // ±30% 지터
this.ui.show("연결 끊김 — " + Math.round(wait / 1000) + "초 후 재접속합니다 (" + this.attempts
+ "회차)");
setTimeout(() => this.tryReconnect(), wait);
}

private tryReconnect() {
  this.sock = this.makeSocket();
  this.sock.connect();
  // 실제로는 onOpen 콜백에서 tryResume() 호출
}

/** 연결 성공 직후: 세션 토큰으로 이어하기 시도 */
tryResume() {
  this.ui.show("세션 복구 중...");
  const hello = new TextEncoder().encode(JSON.stringify({
    t: "resume", token: this.token, ts: Date.now(),
  }));
  this.sock!.send(hello.buffer, "high");
  // 서버 응답 S2C_RESTORE 수신 시 onRestore 콜백으로 스냅샷 반영
}

/** 복구 성공: 끊긴 동안 쌓인 입력 재전송(서버가 seq 로 중복 거름) */
flushPendingInputs() {
  for (const p of this.pendingInputs) this.sock!.send(p, "high");
  this.pendingInputs = [];
  this.attempts = 0;
  this.ui.hide();
}

/** 서버가 세션을 폐기한 경우 — 전체 재로그인 플로우로 전환 */
fallbackFullLogin() {
  localStorage.removeItem("sessionToken");
  location.reload();
}

destroy() { this.alive = false; this.sock?.close(); }
}

```

보조 코드 — 서버 측 세션 유지 개념 (uWebSockets.js 핸들러 발췌)

```

// server/ws/session.ts (개념 발췌)
import uWS from "uWebSockets.js";

interface LiveSession {
  userId: number; token: string; lastAckSeq: number;
  roomId: string; dirtyAt: number;
}

const sessions = new Map<string, LiveSession>(); // token → session
const GRACE_MS = 20_000; // 끊김 유예 20 초

```

```
uWS.App().ws("/*", {
  open(ws) { /* 인증 후 세션 생성 */ },
  message(ws, msg, isBinary) {
    const s = sessions.get(wsToken(ws));
    if (s) s.dirtyAt = Date.now();
    // ... 패킷 처리: seq <= s.lastAckSeq 인 입력은 중복 폐기
  },
  close(ws) {
    const s = sessions.get(wsToken(ws));
    if (s) setTimeout(() => { // 유예 시간 내 재접속 시 세션 보존
      if (Date.now() - s.dirtyAt > GRACE_MS) sessions.delete(s.token);
    }, GRACE_MS);
  },
}).listen(3001, (t) => console.log("ws ready", t));
```

(3) 성능 최적화 팁

- 재접속 백오프에는 반드시 지터를 섞는다 — 접속자 수천 명이 동시에 끊기는 순간 (서버 롤링 배포) 동시 재시도는 서버를 다시 죽인다.
- 세션 유예(grace) 20 초는 모바일 엘리베이터/지하 구간을 커버하는 실무적 값이며, 게임 성격에 따라 10~60 초로 조정한다.
- 입력 seq 기반 중복 폐기는 멍등성의 기본이며, 재화 소모 같은 비가역 연산은 seq 가 아니라 서버 트랜잭션 ID 로 이중 방어한다.
- 재접속 UI 는 반드시 유저가 조작을 계속할 수 있게 남겨두고, 복구가 되면 조용히 사라지게 한다 — 재시도 블로킹 화면은 이탈률을 올린다.

006. IndexedDB 기반 정적 에셋(스프라이트, 음원) 캐싱 시스템

(1) 역할 설명

브라우저 HTTP 캐시는 용량 상한과 퇴거 정책이 불투명해, 수백 MB 의 게임 에셋을 신뢰할 수 없다. IndexedDB 는 오리진당 수백 MB~수 GB 를 안정적으로 보관할 수 있어, 스프라이트·아틀라스·음원을 로컬에 저장하고 재방문 시 네트워크 없이 즉시 부팅하는 '설치형 웹 게임' 경험을 만든다. 이 모듈은 에셋을 Blob 으로 저장하고 매니페스트 해시(004 번)로 신선도를 판정하며, 용량 한계 도달 시 LRU(최근 사용 기준)로 퇴거시킨다.

주의할 점은 저장 API 가 비동기라는 것이고, Phaser 로더는 URL 기반이라는 점이다. 해법으로 Blob URL.createObjectURL 을 사용해 캐시 히트 시 실제 HTTP 요청을 아예 발생시키지 않는다. 파이어폭스의 프라이빗 모드, iOS 사파리의 7일 미접속 데이터 삭제 정책 같은 예외 상황에서는 캐시가 실패해도 게임이 정상 동작하도록 폴백(네트워크 로딩)을 유지한다.

(2) 구현 코드 — src/assets/AssetCache.ts

```
export interface CacheRecord {
  key: string; // 에셋 키
```

```

hash: string;    // 매니페스트 해시 — 불일치 시 무효
blob: Blob;
usedAt: number;  // LRU 퇴거 기준
bytes: number;
}

const DB_NAME = "sertz-assets";
const STORE = "assets";
const QUOTA_SOFT_LIMIT = 450 * 1024 * 1024; // 450MB 소프트 리밋

export class AssetCache {
  private db: IDBDatabase | null = null;
  private memUrls = new Map<string, string>(); // key → blobURL

  async open(): Promise<boolean> {
    try {
      this.db = await new Promise((res, rej) => {
        const req = indexedDB.open(DB_NAME, 1);
        req.onupgradeneeded = () => {
          req.result.createObjectStore(STORE, { keyPath: "key" });
        };
        req.onsuccess = () => res(req.result);
        req.onerror = () => rej(req.error);
      });
      return true;
    } catch {
      return false; // 프라이빗 모드 등 — 네트워크 폴백
    }
  }

  /** 캐시 히트면 Blob URL 반환, 미스면 null */
  async lookup(key: string, hash: string): Promise<string | null> {
    if (this.memUrls.has(key)) return this.memUrls.get(key)!;
    if (!this.db) return null;

    const rec = await this.get<CacheRecord>(key);
    if (!rec || rec.hash !== hash) return null; // 신선도 실패 → 무효

    rec.usedAt = Date.now();
    this.put(rec); // LRU 갱신(비동기 fire & forget)
    const url = URL.createObjectURL(rec.blob);
    this.memUrls.set(key, url);
    return url;
  }

  /** 다운로드한 에셋을 저장(이미 있으면 스킵) */
  async store(key: string, hash: string, blob: Blob) {
    if (!this.db) return;
    await this.put({ key, hash, blob, usedAt: Date.now(), bytes: blob.size } as CacheRecord);
    await this.evictIfNeeded();
  }
}

```

```

}

/** 소프트 리밋 초과 시 가장 오래된 에셋부터 퇴거 */
private async evictIfNeeded() {
  let total = await this.totalBytes();
  if (total < QUOTA_SOFT_LIMIT) return;
  const tx = this.db!.transaction(STORE, "readwrite");
  const store = tx.objectStore(STORE);
  const all: CacheRecord[] = await new Promise(res => {
    const r = store.getAll(); r.onsuccess = () => res(r.result); });
  all.sort((a, b) => a.usedAt - b.usedAt);
  for (const rec of all) {
    if (total <= QUOTA_SOFT_LIMIT * 0.8) break; // 20% 여유까지 축소
    store.delete(rec.key);
    total -= rec.bytes;
  }
}

private get<T>(key: string): Promise<T | null> {
  return new Promise(res => {
    const r = this.db!.transaction(STORE).objectStore(STORE).get(key);
    r.onsuccess = () => res(r.result ?? null);
    r.onerror = () => res(null);
  });
}

private put(rec: CacheRecord): Promise<void> {
  return new Promise(res => {
    const tx = this.db!.transaction(STORE, "readwrite");
    tx.objectStore(STORE).put(rec);
    tx.oncomplete = () => res(); tx.onerror = () => res();
  });
}

private totalBytes(): Promise<number> {
  return new Promise(res => {
    const r = this.db!.transaction(STORE).objectStore(STORE).getAll();
    r.onsuccess = () => res((r.result as CacheRecord[]).reduce((s, x) => s + x.bytes, 0));
    r.onerror = () => res(0);
  });
}

// AssetFlow(004 번)와의 결합: loader.url 을 캐시 조회 결과로 교체
// const url = (await cache.lookup(e.key, e.hash)) ?? e.url;
// loader.image(e.key, url);

```

(3) 성능 최적화 팁

- Blob URL 은 페이지 생명주기와 같으므로, 재부팅 시 매번 lookup 으로 재생성한다 — memUrls 캐시로 중복 생성을 막는다.
- 스토리지 용량은 navigator.storage.estimate()로 확인해 설정 UI 에 표시하면 유저가 캐시를 관리할 수 있다.

- 음원은 크기가 크므로 별도 스토어로 분리해, 이미지만 먼저 지우는 선택적 청소를 지원한다.
- iOS 사파리는 7일 미사용 시 IndexedDB 를 지울 수 있다 — 매 접속 시 매니페스트 검증은 반드시 거쳐야 깨진 텍스처를 막는다.

007. 마우스/터치 입력 디바이스 자동 감지 및 조작부 전환

(1) 역할 설명

PC 유저에게 터치 UI 를, 모바일 유저에게 마우스 UI 를 보여주는 것은 최악의 UX 다. 이 모듈은 pointer: coarse/fine 미디어쿼리, 최초 입력 이벤트 타입, 최대 터치 포인트 수를 종합해 입력 디바이스를 판정하고, 가상 조이스틱·스킬 버튼(터치 세트) 또는 클릭 이동·단축키 안내(키보드 세트)를 런타임에 전환한다. 판정은 한 번으로 끝내지 않고, 하이브리드 기기(터치 노트북, 태블릿+키보드)에서는 마지막 입력 타입을 계속 추적해 조작부를 동적으로 바꾼다.

입 력 계 층 은 InputController 인터페이스 아래에 TouchController 와 MouseKeyboardController 를 둔 전략 패턴으로 구현한다. 썬 코드는 인터페이스만 의존하므로, 조작부 교체나 신규 디바이스(게이패드) 추가 시 게임 로직은 전혀 수정하지 않는다. Phaser 의 input 이벤트를 컨트롤러가 흡수해, 게임 썬에는 '이동 의도 벡터'와 '액션 이벤트'만 표준화된 형태로 전달된다.

(2) 구현 코드 — src/input/InputRouter.ts

```
export interface MoveIntentVec { x: number; y: number; } // -1~1
export type ActionEvent =
  | { kind: "skill"; slot: number }
  | { kind: "worldTap"; wx: number; wy: number }
  | { kind: "interact" };

export interface InputController {
  readonly name: "touch" | "kb";
  readonly moveVec: MoveIntentVec; // 조이스틱/방향키 합산
  onAction: ((e: ActionEvent) => void) | null; // 썬이 구독
  destroy(): void;
}

export class InputRouter {
  private ctrl: InputController | null = null;
  private lastPointerType: "mouse" | "touch" | "pen" | null = null;

  constructor(private game: Phaser.Game,
    private buildTouch: () => InputController,
    private buildKB: () => InputController) {}

  /** 부팅 시 1 회 초기 판정 */
  autoDetect() {
```



```

const coarse = matchMedia("(pointer: coarse)").matches;
const touchCapable = navigator.maxTouchPoints > 0;
const fine = matchMedia("(any-pointer: fine)").matches;
this.switchTo(coarse && touchCapable && !fine ? "touch" : "kb");
}

/** 하이브리드 기기: 마지막 입력으로 런타임 전환 */
watchHybrid() {
  window.addEventListener("pointerdown", (e) => {
    if (e.pointerType === this.lastPointerType) return;
    this.lastPointerType = e.pointerType as any;
    if (e.pointerType === "touch") this.switchTo("touch");
    else if (e.pointerType === "mouse") this.switchTo("kb");
  }, { passive: true });
}

private switchTo(kind: "touch" | "kb") {
  if (this.ctrl?.name === kind) return;
  this.ctrl?.destroy();
  this.ctrl = kind === "touch" ? this.buildTouch() : this.buildKB();
  this.game.events.emit("input-mode-changed", kind); // HUD 가 조이스틱 표시 전환
}

get current(): InputController { return this.ctrl!; }
}

```

보조 코드 — src/input/KBController.ts — PC 조작부 예시

```

import Phaser from "phaser";
import type { ActionEvent, InputController, MoveIntentVec } from "../InputRouter";

export class KBController implements InputController {
  readonly name = "kb" as const;
  readonly moveVec: MoveIntentVec = { x: 0, y: 0 };
  onAction: ((e: ActionEvent) => void) | null = null;

  private keys: Record<string, Phaser.Input.Keyboard.Key> = {};
  private static KEYMAP = {
    up: "W", down: "S", left: "A", right: "D",
    alt_up: "UP", alt_down: "DOWN", alt_left: "LEFT", alt_right: "RIGHT",
  };

  constructor(private scene: Phaser.Scene) {
    const kb = scene.input.keyboard!;
    for (const [slot, code] of Object.entries(KBController.KEYMAP)) {
      this.keys[slot] = kb.addKey(code);
    }
    // 스킬 슬롯 1~6, 상호작용 E, Tab 타겟팅은 KeyDown 이벤트로
    kb.on("keydown", (ev: KeyboardEvent) => {
      if (ev.code.startsWith("Digit")) {
        const slot = Number(ev.code.slice(5)) - 1;
        if (slot >= 0 && slot < 6) this.onAction?.({ kind: "skill", slot });
      }
    });
  }
}

```

```

    }
    if (ev.code === "KeyE") this.onAction?({ kind: "interact" });
  });
  // 월드 클릭 이동: 카메라 좌표로 변환
  scene.input.on("pointerdown", (p: Phaser.Input.Pointer) => {
    const cam = scene.cameras.main;
    const wx = cam.getWorldPoint(p.x, p.y).x;
    const wy = cam.getWorldPoint(p.x, p.y).y;
    this.onAction?({ kind: "worldTap", wx, wy });
  });
}

/** RenderLoop 고정 업데이트에서 호출 — 방향키 합산 벡터 */
poll(): MoveIntentVec {
  const k = this.keys;
  let x = (k.right.isDown || k.alt_right.isDown ? 1 : 0) -
    (k.left.isDown || k.alt_left.isDown ? 1 : 0);
  let y = (k.down.isDown || k.alt_down.isDown ? 1 : 0) -
    (k.up.isDown || k.alt_up.isDown ? 1 : 0);
  const len = Math.hypot(x, y);
  if (len > 1) { x /= len; y /= len; } // 대각선 정규화
  this.moveVec.x = x; this.moveVec.y = y;
  return this.moveVec;
}

destroy() {
  this.scene.input.keyboard?.removeAllListeners();
  this.scene.input.removeAllListeners("pointerdown");
}
}

```

(3) 성능 최적화 팁

- 초기 판정은 pointer 미디어쿼리가 1 순위다 — userAgent 스니핑은 폴더블·터치 노트북 시대에 신뢰할 수 없다.
- 하이브리드 전환 시 조이스틱 DOM 을 페이드 트랜지션으로 바꾸면 유저가 인지 부담 없이 새 조작부를 받아들인다.
- 대각선 이동 벡터는 반드시 정규화해야 키보드 대각 이동이 41% 빨라지는 버그를 막을 수 있다.
- 게이패드 지원 확장 시 Gamepad API 를 InputController 구현체로만 추가하면 썬 코드는 무수정으로 동작한다.

008. 브라우저 탭 비활성화(Background Thread) 시 메모리 관리 및 동기화 일시정지

(1) 역할 설명

탭이 백그라운드로 가면 rAF 는 정지되지만 setTimeout/setInterval 은 1Hz 로 스로틀되고, 서버로는 여전히 패킷이 흐른다. 이 상태를 방치하면 세 가지 문제가 생긴다. 백그라운드에서도 동기화 패킷을 계속 보내 대역폭을 낭비하고, 복귀 시 수분 분의 시뮬레이션을 한 프레임에 처리하려다 프레임 폭발이 일어나며, iOS 등에서는 오디오/타이머가 불규칙해져 상태가 어긋난다. 이 모듈은 visibilitychange 와 blur/focus 를 감시해 '일시정지 → 상태 스냅샷 → 동기화 최소화' 프로토콜을 수행한다.

일시정지 시에는 서버에 AFK 상태 패킷을 보내 몬스터 어그로 목록에서 제외시키고, 로컬로는 오디오를 정지·텍스처를 유지하되 GPU 메모리 회수 가능한 파티클/이펙트 인스턴스를 정리한다. 복귀 시에는 서버 스냅샷 요청 한 번으로 현재 상태를 재동기화하고, 데드레코닝(012 번) 보간 버퍼를 리셋해 캐릭터가 화면을 가로질러 튀는 현상을 막는다.

(2) 구현 코드 — src/engine/BackgroundGuard.ts

```
import Phaser from "phaser";

type SyncCtl = {
  sendAfk(on: boolean): void;      // 서버에 AFK 전환 통보
  requestResync(): Promise<unknown>; // 복귀 시 전체 스냅샷 요청
};

export class BackgroundGuard {
  private hiddenAt = 0;
  private wasHidden = false;
  private detachFns: Array<() => void> = [];

  constructor(private game: Phaser.Game,
    private audio: Phaser.Sound.BaseSoundManager,
    private sync: SyncCtl,
    private onResumed: (snapshot: unknown, awayMs: number) => void) {}

  attach() {
    const onVis = () => (document.hidden ? this.pause() : this.resume());
    document.addEventListener("visibilitychange", onVis);
    window.addEventListener("blur", () => this.pause()); // 탭 전환만으로는 hidden 미발생 브라우저
    대응
    window.addEventListener("focus", () => this.resume());
    this.detachFns.push(() => {
      document.removeEventListener("visibilitychange", onVis);
    });
  }
}
```

```

private pause() {
  if (this.isHidden) return;
  this.isHidden = true;
  this.hiddenAt = performance.now();

  this.sync.sendAfk(true);          // 어그로·전장 참여 해제
  this.audio.pauseAll();
  this.game.loop.sleep();           // rAF 루프 정지
  // GPU 부담 정리: 파티클 emitter 정지, 동적 텍스처 flush 는 생략(비용 큼)
  this.game.scene.getScenes(true).forEach(s =>
    (s as any).particles?.emitters?.forEach?.((em: any) => em.stop?.()));
}

private async resume() {
  if (!this.isHidden) return;
  this.isHidden = false;
  const awayMs = performance.now() - this.hiddenAt;

  const snapshot = await this.sync.requestResync(); // 서버 현재 상태
  this.onResumed(snapshot, awayMs); // 데드레코닝 버퍼 리셋 + 퀘스트 UI 갱신
  this.audio.resumeAll();
  this.game.loop.wake();

  // rAF 재개 직후 첫 프레임 delta 클램프는 RenderLoop(001)이 처리
  if (awayMs > 5 * 60_000) {
    // 5 분 이상 이탈: 세션 유효성 재확인 (토큰 만료 대응)
    this.sync.sendAfk(false);
  }
}

detach() { this.detachFns.forEach(f => f()); this.detachFns = []; }
}

```

(3) 성능 최적화 팁

- `visibilitychange` 가 `blur` 보다 신뢰도가 높지만, OS 멀티데스크톱 전환 등에서 `blur` 만 발생하는 케이스가 있어 둘 다 잡는다.
- 백그라운드 중에도 서버는 유저를 계속 시뮬레이션해야 하나(피격 등), 어그로 대상에서는 빼는 것이 PVP/PVE 모두에서 공정하다.
- 복귀 스냅샷은 '델타'가 아니라 '전체 AOI 스냅샷'으로 받는 것이 안전하다 — 이탈 구간의 델타를 재생하는 것은 비용 대비 신뢰도가 낮다.
- Page Lifecycle API(`document.wasDiscarded`, `freeze/resume` 이벤트)를 함께 감시하면 브라우저가 탭을 디스카드한 경우까지 감지할 수 있다.

009. 미사용 스프라이트 가비지 컬렉션(GC) 및 메모리 해제 최적화

(1) 역할 설명

Phaser 는 텍스처를 키 기반으로 캐시하며, 게임은 장기 세션에서 수십 개 구역을 오가며 수천 개의 텍스처·사운드·애니메이션을 적재한다. 이를 해제하지 않으면 WebGL 텍스처 메모리가 계속 누적되어 모바일 브라우저가 1~2 시간 내에 크래시한다. 이 모듈은 구역 전환 시점에 '현재 구역에서 참조되지 않는 자원'을 매니페스트 기반으로 계산해 텍스처·오디오·동적 텍스처를 명시적으로 release 하고, 참조 카운트로 공용 자원(UI, 아이콘)을 보호한다.

핵심은 두 계층의 분류다. 퍼시스턴트(UI, 폰트, 공용 이펙트)는 절대 해제하지 않고, 휘발성(구역 스프라이트, BGM)은 구역 이탈 시 전부 해제한다. 또한 중간 계층으로 세션 캐시(이번 세션에서 다시 방문 가능성 높은 구역)를 두어, 재진입 비용과 메모리 사이에서 트레이드오프를 조절한다. 해제 후에는 WebGL 텍스처가 실제로 GPU 에서 내려갔는지 renderer 비용 로그로 검증하는 루틴을 포함한다.

(2) 구현 코드 — src/assets/MemorySweeper.ts

```
import Phaser from "phaser";
import type { Manifest } from "../BundleManifest";

export class MemorySweeper {
  private keepAlive = new Set<string>(); // 퍼시스턴트 키(UI/폰트/공용)
  private sessionPin = new Map<string, number>(); // key → 마지막 접근 시각

  constructor(private game: Phaser.Game, private manifest: Manifest,
    private maxPinned = 3) {} // 세션 핀 유지 구역 수

  markPersistent(keys: string[]) { keys.forEach(k => this.keepAlive.add(k)); }

  /** 구역 진입 시 휘발성 키를 핀으로 갱신 */
  pinZone(zoneId: string) {
    const b = this.manifest.bundles[zoneId];
    b?.entries.forEach(e => this.sessionPin.set(e.key, Date.now()));
    this.trimPins();
  }

  private trimPins() {
    if (this.sessionPin.size <= this.maxPinned * 200) return; // 대략적 상한
    const sorted = [...this.sessionPin.entries()].sort((a, b) => a[1] - b[1]);
    while (sorted.length > this.maxPinned * 200) {
      const [key] = sorted.shift()!;
      this.sessionPin.delete(key);
    }
  }

  /** 구역 이탈 시 호출 — 핀에 없는 휘발성 자원 전부 해제 */
}
```

```

sweepZone(zoneId: string) {
  const tex = this.game.textures, snd = this.game.cache.audio;
  const b = this.manifest.bundles[zoneId];
  if (!b) return;

  let freed = 0;
  for (const e of b.entries) {
    if (this.keepAlive.has(e.key) || this.sessionPin.has(e.key)) continue;
    if (tex.exists(e.key)) { tex.remove(e.key); freed++; } // GPU 텍스처 해제 포함
    if (e.type === "audio" && snd.exists(e.key)) snd.remove(e.key);
    if (e.type === "json" && this.game.cache.json.exists(e.key))
      this.game.cache.json.remove(e.key);
  }
  // 커스텀 애니메이션 정리: 전역 anims 에 남은 구역 전용 프레임 제거
  this.game.anims.getAnimsFrom?.(zoneId)?.forEach?.((a: Phaser.Animations.Animation) =>
    this.game.anims.remove(a.key));
  console.log("[MemorySweeper] freed " + freed + " textures for " + zoneId);
}

/** 진단: 현재 텍스처 메모리 요약(개발용) */
diagnose(): { count: number; bytes: number } {
  let bytes = 0, count = 0;
  this.game.textures.getTextureKeys().forEach(k => {
    const t = this.game.textures.get(k);
    if (!t || k === "__DEFAULT") return;
    count++;
    for (let i = 0; i < t.source.length; i++) {
      const s = t.source[i].image as HTMLImageElement | undefined;
      if (s?.width) bytes += s.width * s.height * 4; // RGBA 가정
    }
  });
  return { count, bytes };
}
}

```

(3) 성능 최적화 팁

- textures.remove(key)는 WebGL 텍스처까지 해제하지만, 씬 오브젝트가 참조 중이면 화이트 프레임이 된다 — 반드시 매니페스트 기반으로 '해당 구역 전용'만 지운다.
- atlas 한 장을 지우면 그 아틀라스의 모든 프레임이 사라진다 — 서로 다른 구역이 한 아틀라스를 공유하면 안 된다.
- Audio 는 decodeAudioData 결과(AudioBuffer)가 가장 크다 — BGM 은 구역 이탈 시 즉시 stop+remove 가 유효하다.
- 장기 세션 테스트는 30 분마다 diagnose() 로그를 남겨 그래프 추이로 누수를 조기 발견하는 것이 표준이다.

010. WebAssembly(WASM) 연동을 위한 인터페이스 구조

(1) 역할 설명

A* 길찾기(1000x1000 그리드), 대규모 충돌 판정, 데미지 시뮬레이션 같은 CPU 집약 연산은 JS로 구현하면 GC 압박과 JIT 한계로 프레임 드랍이 발생한다. WASM으로 이런 연산을 이식하면 3~10 배의 속도 향상을 얻지만, 모듈 로딩·메모리 관리·JS 경계 비용이라는 새로운 관리 지점이 생긴다. 이 모듈은 WASM 모듈을 동일한 인터페이스(JS 구현체와 교체 가능)로 감싸는 어댑터 계층을 제공한다 — WASM이 준비되기 전에는 JS 풀백으로 즉시 게임이 동작하고, 로딩 완료 시 런타임에 구현체를 교체(핫스왑)한다.

메모리 모델은 WASM 선형 메모리를 소유자(WASM 모듈)로 두고, JS는 오프셋 핸들만 받아 다루는 단방향 소유 구조를 채택한다. 이는 구조화된 복사 비용을 없애고, WASM 내부에서 사용 후 폐기하는 대규모 임시 배열(오픈리스트 등)이 JS GC에 부하를 주지 않게 한다. 인터페이스는 init/resize/dispose 라이프사이클을 명시해 다중 실행에서 안전하게 재사용한다.

(2) 구현 코드 — src/wasm/PathFinder.ts — 인터페이스 + JS 풀백 + WASM 어댑터

```
// — 공용 인터페이스: WASM과 JS 구현체가 모두 만족 —
export interface IPathFinder {
  readonly backend: "wasm" | "js";
  setGrid(w: number, h: number, blocked: Uint8Array): void;
  find(sx: number, sy: number, tx: number, ty: number): Int32Array; // [x0,y0,x1,y1,...]
  dispose(): void;
}

// — JS 풀백(간단화: 실제로는 A* 전체 구현) —
export class JSPathFinder implements IPathFinder {
  readonly backend = "js" as const;
  private w = 0; private h = 0; private blocked!: Uint8Array;
  setGrid(w: number, h: number, blocked: Uint8Array) {
    this.w = w; this.h = h; this.blocked = blocked;
  }
  find(sx: number, sy: number, tx: number, ty: number): Int32Array {
    // 데모: 직선 경로(실제 구현은 A* — 043 번 참조)
    return new Int32Array([sx, sy, tx, ty]);
  }
  dispose() {}
}

// — WASM 어댑터: 로딩 후 핫스왑 —
export class WasmPathFinder implements IPathFinder {
  readonly backend = "wasm" as const;
  private exports: any = null; // wasm 인스턴스 exports
  private gridPtr = 0; private outPtr = 0; private outLen = 0;

  static async load(url: string): Promise<WasmPathFinder> {
```

```

const self = new WasmPathFinder();
// Rust/AssemblyScript 로 빌드된 모듈 가정: alloc/setGrid/find/free export
const { instance } = await WebAssembly.instantiateStreaming(fetch(url), {
  env: { abort: () => { throw new Error("wasm abort"); } },
});
self.exports = instance.exports;
self.outPtr = self.exports.alloc(4096);    // 결과 버퍼 재사용
return self;
}

setGrid(w: number, h: number, blocked: Uint8Array) {
  if (this.gridPtr) this.exports.free(this.gridPtr);
  this.gridPtr = this.exports.alloc(blocked.length);
  new Uint8Array(this.exports.memory.buffer,
    this.gridPtr, blocked.length).set(blocked);
  this.exports.setGrid(this.gridPtr, w, h);
}

find(sx: number, sy: number, tx: number, ty: number): Int32Array {
  const n = this.exports.find(sx, sy, tx, ty, this.outPtr); // 길이 반환
  // 복사는 결과 경로만 — 대규모 중간 배열은 WASM 내부에 존재
  return new Int32Array(this.exports.memory.buffer, this.outPtr, n * 2).slice();
}

dispose() {
  if (this.gridPtr) this.exports.free(this.gridPtr);
  if (this.outPtr) this.exports.free(this.outPtr);
  this.exports = null;
}
}

// — 매니저: 폴백으로 시작 → WASM 준비되면 교체 —
export class PathfinderProvider {
  private impl: IPathFinder = new JSPathFinder();
  async upgradeToWasm(url: string) {
    try {
      const wasm = await WasmPathFinder.load(url);
      wasm.setGrid(this.gridW, this.gridH, this.grid!);
      this.impl.dispose();
      this.impl = wasm;          // 이후 호출은 전부 WASM 경유
    } catch (e) { console.warn("WASM upgrade failed, keep JS", e); }
  }
  private gridW = 0; private gridH = 0; private grid: Uint8Array | null = null;
  setGrid(w: number, h: number, blocked: Uint8Array) {
    this.gridW = w; this.gridH = h; this.grid = blocked;
    this.impl.setGrid(w, h, blocked);
  }
  find(sx: number, sy: number, tx: number, ty: number) {
    return this.impl.find(sx, sy, tx, ty);
  }
}

```


(3) 성능 최적화 팁

- JS ↔ WASM 경계 통과는 호출당 수백 ns 비용이다 — 프레임당 수만 회 호출되는 소형 연산보다 큰 덩어리 연산을 이식해야 이득이다.
- WASM 선형 메모리는 GC 대상이 아니므로 alloc/free 규율이 필수다 — 포인터 소유자를 명확히 두지 않으면 이중 free 로 즉시 크래시된다.
- memory.buffer 는 grow 시 교체된다 — Uint8Array 뷰를 캐싱하지 말고 매 접근 시 새로 만든다.
- 풀백 우선 부팅은 코어 웹 바이탈(LCP)을 지키면서 점진적으로 성능을 올리는 표준 전략이다.

제 3 장 모듈 2 — 네트워크 및 동기화 시스템 (011~020)

기술 스택: Node.js · uWebSockets.js · ArrayBuffer(TypedArray) · Kysely(PostgreSQL) 본 장은 멀티플레이의 체감 품질을 결정하는 동기화 계층이다. 클라이언트 예측과 서버 권위 검증의 이중 구조가 핵심 설계이며, AOI 타일 격자 브로드캐스팅과 델타 압축으로 대규모 필드의 트래픽을 통제한다. 랙 컴펜세이션·롤백 보정은 높은 지연 환경에서도 공정한 전투 판정을 보장하고, Kysely 트랜잭션은 재화·아이템 이동의 원자성을 담당한다.

011. 클라이언트 위치 예측(Client Prediction) 엔진

(1) 역할 설명

입력 후 서버 응답(RTT 50~200ms)을 기다렸다가 캐릭터를 움직이면 조작감이 끊긴다. 클라이언트 예측은 입력을 즉시 로컬에 적용하고, 서버 승인이 도착하면 예측 결과와 비교해 오차만큼만 교정한다. 핵심 자료구조는 입력 버퍼((seq, dx, dy, dt) 튜플)와 서버 승인 상태(마지막 승인 seq, 승인 위치)다. 승인이 도착하면 예측 버퍼에서 해당 seq 이전 항목을 제거하고, 남은 입력을 승인 위치부터 다시 재생(replay)한다.

오차가 임계값(예: 0.5 타일) 이하면 부드러운 스무딩으로 수렴시키고, 초과하면 텔레포트 보정을 한다. 예측은 '결정론적 이동 함수'를 클라이언트와 서버가 공유해야 성립하므로, 이동 규칙(속도, 가속, 충돌 반경)은 양쪽에서 임포트하는 shared 모듈로 작성한다. 서버도 같은 함수로 검증(016 번)하므로 이중 구현이 아니라 단일 소스가 된다.

(2) 구현 코드 — shared/movement.ts — 클라이언트/서버 공유 결정론적 이동

```
// 양쪽에서 동일하게 임포트하는 이동 규칙(결정론 필수: 부동소수 연산 순서 고정)
export interface Input {
  seq: number; dx: number; dy: number; dt: number; // 방향(-1~1), 경과 ms
}
export interface MoveState { x: number; y: number; speed: number; }
```

```

export const TILE = 32;
export const WALK_SPEED = 140;      // px/s

export function stepMove(s: MoveState, i: Input, blocked: (x: number, y: number) => boolean):
MoveState {
  const len = Math.hypot(i.dx, i.dy);
  if (len === 0) return s;
  const dtSec = i.dt / 1000;
  const dist = s.speed * dtSec;
  // X/Y 분리 충돌: 벽 끼임(코너 걸림) 방지를 위해 축별로 이동
  const nx = s.x + (i.dx / len) * dist;
  if (!blocked(nx, s.y)) s = { ...s, x: nx };
  const ny = s.y + (i.dy / len) * dist;
  if (!blocked(s.x, ny)) s = { ...s, y: ny };
  return s;
}

```

보조 코드 — src/net/PredictionEngine.ts

```

import { Input, MoveState, stepMove } from "../shared/movement";

export class PredictionEngine {
  private pending: Input[] = [];      // 미승인 입력 버퍼
  private seq = 0;
  /** 서버 마지막 승인 상태 */
  private authState: MoveState;
  /** 화면 표시용 상태(보정 스무딩 적용) */
  display: MoveState;

  constructor(spawn: MoveState, private blocked: (x: number, y: number) => boolean) {
    this.authState = { ...spawn };
    this.display = { ...spawn };
  }

  /** 플레이어 입력 발생(고정 스텝마다) — 즉시 예측 적용 + 서버 전송용 반환 */
  applyLocal(dx: number, dy: number, dt: number): Input {
    const input: Input = { seq: ++this.seq, dx, dy, dt };
    this.display = stepMove(this.display, input, this.blocked);
    this.pending.push(input);
    return input;      // 호출자가 서버로 전송(C2S_MOVE)
  }

  /** 서버 S2C_MOVE_ACK 수신(seq, x, y) — 예측 재생(reconciliation) */
  onServerAck(seq: number, x: number, y: number) {
    // 1) 승인된 입력 이전 항목 폐기
    this.pending = this.pending.filter(i => i.seq > seq);
    this.authState = { ...this.authState, x, y };

    // 2) 승인 위치부터 미승인 입력 재생
    let sim: MoveState = { ...this.authState };
    for (const i of this.pending) sim = stepMove(sim, i, this.blocked);
  }
}

```

```

// 3) 오차 평가: 작으면 스무딩, 크면 스냅
const err = Math.hypot(sim.x - this.display.x, sim.y - this.display.y);
if (err > TILE * 1.5) {
  this.display = { ...sim }; // 텔레포트 보정
} else if (err > 0.5) {
  // 지수 감쇠 스무딩: 복귀 프레임에서 급격히 튀지 않게
  const k = Math.min(1, err > 8 ? 0.35 : 0.15);
  this.display.x += (sim.x - this.display.x) * k;
  this.display.y += (sim.y - this.display.y) * k;
}
}

/** 고정 업데이트: 미승인 입력이 계속 쌓이면 계속 예측 진행 */
fixedUpdate(dt: number, currentInput: { dx: number; dy: number }) {
  if (this.pending.length > 0 || currentInput.dx || currentInput.dy) {
    const ghost: Input = { seq: this.seq, dx: currentInput.dx, dy: currentInput.dy, dt };
    this.display = stepMove(this.display, ghost, this.blocked);
  }
}

get pendingCount() { return this.pending.length; }
}

```

(3) 성능 최적화 팁

- 예측 재생은 '서버 승인 위치 + 미승인 입력 전부'로 다시 시뮬레이션하는 것이 정석이며, 델타만 반영하면 드리프트가 누적된다.
- 오차 스무딩 계수(k)를 오차 크기에 따라 가변으로 두면 자잘한 보정은 티가 나지 않고, 큰 보정은 빠르게 수렴한다.
- pending 입력이 20 개(약 300ms)를 넘으면 네트워크 이상 신호로 간주해 예측을 멈추고 서버 상태를 신뢰하는 안전장치를 둔다.
- 결정론이 깨지는 요인(부동소수 순서, Math.random)은 shared 모듈에서 금지 — 랜덤은 서버 전용 값으로만 주입한다.

012. 데드 레코닝(Dead Reckoning) & 위치 보간(Interpolation) 연산 모듈

(1) 역할 설명

다른 플레이어·몬스터의 위치는 서버가 10~20Hz 로만 알려주지만 화면은 60fps 로 그려야 한다. 데드 레코닝은 마지막 알려진 위치와 속도 벡터로 현재 시각의 위치를 외삽(extrapolation)하고, 보간은 두 스냅샷 사이를 부드럽게 채운다. 실무에서 정석은 '보간 지연 버퍼' 방식이다. 렌더 시각을 현재보다 $RTT/2$ + 안전 마진(보통 100ms)만큼 의도적으로 과거로 두고, 그 시각을 기준으로 두 스냅샷 사이를 선형보간한다.

이 방식은 패킷 지연 지터를 완전히 흡수해 NPC 움직임이 결코 떨리지 않는다. 스냅샷이 버퍼를 초과해 늦게 도착하면(예: 스파이크) 속도 외삽으로 한시적으로 메우고, 새 스냅샷이

연속 도착하면 렌더 시각을 앞으로 당겨 재수렴한다. 유닛별 상태(스폰/사망/스턴)도 함께 보관해 스프라이트 애니메이션 전환이 자연스럽게 이어진다.

(2) 구현 코드 — src/net/DeadReckoning.ts

```
export interface Snapshot {
  id: number; t: number;           // 유닛 id, 서버 타임스탬프(ms)
  x: number; y: number; vx: number; vy: number; // 위치(px), 속도(px/s)
}

const INTERP_DELAY = 100;          // 보간 지연(ms) — RTT/2 + 마진

export class RemoteUnitInterp {
  private buf: Snapshot[] = [];     // 시간 오름차순 유지
  /** 렌더 시각을 과거로 당기는 기준: 로컬 수신 시각 기준 관리 */
  private origin = 0;

  push(s: Snapshot, nowLocal: number) {
    if (this.buf.length && s.t <= this.buf[this.buf.length - 1].t) return; // 지각분 폐기
    this.buf.push(s);
    if (this.buf.length > 30) this.buf.shift(); // 버퍼 상한
    if (!this.origin) this.origin = nowLocal - s.t; // 서버-로컬 시계 정렬
  }

  /** 렌더 프레임마다 호출 — 표시 위치 반환 */
  sample(nowLocal: number): { x: number; y: number } {
    const renderT = nowLocal - this.origin - INTERP_DELAY;
    if (this.buf.length === 0) return { x: 0, y: 0 };

    const last = this.buf[this.buf.length - 1];
    if (renderT >= last.t) {
      // 버퍼 끝 초과(스파이크) → 속도 외삽으로 한시 메우기
      const over = (renderT - last.t) / 1000;
      return { x: last.x + last.vx * over, y: last.y + last.vy * over };
    }

    // renderT 를 포함하는 구간 [a, b] 찾기 → 선형보간
    for (let i = this.buf.length - 2; i >= 0; i--) {
      const a = this.buf[i], b = this.buf[i + 1];
      if (renderT >= a.t) {
        const r = (renderT - a.t) / Math.max(1, b.t - a.t);
        // 스냅샷에 속도가 있으면 이차(헤르미테) 보간에 근사:
        // 선형보간 + 속도 항 절반 보정으로 커브 자연스러움 추가
        const lin = (1 - r) * a.x + r * b.x;
        const linY = (1 - r) * a.y + r * b.y;
        const curvX = (a.vx - b.vx) * 0.5 * (b.t - a.t) / 1000 * r * (1 - r);
        const curvY = (a.vy - b.vy) * 0.5 * (b.t - a.t) / 1000 * r * (1 - r);
        return { x: lin + curvX, y: linY + curvY };
      }
    }
  }

  const first = this.buf[0];
}
```

```

    return { x: first.x, y: first.y };
  }

  /** 재접속/타겟 전환 등 버퍼 전체 리셋 */
  reset() { this.buf = []; this.origin = 0; }
}

```

(3) 성능 최적화 팁

- 보간 지연 100ms 는 대부분의 MMORPG 에서 최적점이다 — 줄이면 지터가 티나고, 늘리면 반응이 둔해 보인다.
- 지각 패킷($t \leq$ 마지막 t)은 반드시 폐기한다 — 정렬 상태가 깨지면 보간이 역주행해 스프라이트가 왔다 갔다 한다.
- 외삽은 한시적 안전망일 뿐이다 — 250ms 이상 지속되면 렌더 시각을 앞당겨(버퍼 소진 유도) 재수렴시키는 게 낫다.
- 몬스터 AI 는 서버가 10Hz 면 충분하지만, 유저 캐릭터는 15~20Hz 스냅샷이 시각적 품질의 하한선이다.

013. uWebSockets.js 내장 Pub/Sub 기반 AOI 타일 격자 브로드캐스팅

(1) 역할 설명

1000 명이 한 맵에 있을 때 모두에게 모든 상태를 브로드캐스트하면 $O(N^2)$ 로 서버가 즉사한다. AOI(Area of Interest)는 각 플레이어에게 '보이는 범위(예: 30x30 타일)'의 엔티티만 전송하는 필터다. uWebSockets.js 는 C++ 코어에 pub/sub 토픽을 내장하고 있어, 구독/발행이 유저 공간 JS 루프 없이 네이티브로 처리된다. 이 모듈은 맵을 NxN 타일 격자로 나누고 각 타일을 토픽(zone:5:12 형식)으로 만들어, 플레이어는 자기 주변 9개(3x3) 타일만 구독한다.

엔티티 상태 변경은 그 엔티티가 속한 타일 토픽에만 publish 하므로, 패킷은 인접 플레이어에게만 전달된다. 타일 경계 이동 시에는 이전 3x3 에서 unsubscribe, 새 3x3 에서 subscribe 하되, uWS 의 publish 옵션으로 자기 자신은 제외한다. 결과적으로 서버 CPU 부하는 유저 수가 아닌 밀도에 비례하게 되어, 밀집 구역(도시 광장 등)은 채널 분산(017 번)으로 해소한다.

(2) 구현 코드 — server/aoi/AoiGrid.ts — uWebSockets.js 토픽 기반 AOI

```

import uWS from "uWebSockets.js";

const TOPIC_PREFIX = "t:"; // 토픽명: "t:<mapId>:<gx>:<gy>"
const CELL = 8; // AOI 격자 한 칸 = 8x8 타일

export function topicOf(mapId: string, x: number, y: number) {
  return TOPIC_PREFIX + mapId + ":" + Math.floor(x / CELL) + ":" + Math.floor(y / CELL);
}

```

```

/** 3x3 이웃 타일 토픽 목록 */
function neighbors(mapId: string, x: number, y: number): string[] {
  const gx = Math.floor(x / CELL), gy = Math.floor(y / CELL);
  const out: string[] = [];
  for (let dx = -1; dx <= 1; dx++)
    for (let dy = -1; dy <= 1; dy++)
      out.push(TOPIC_PREFIX + mapId + ":" + (gx + dx) + ":" + (gy + dy));
  return out;
}

interface Peer {
  id: number; mapId: string; x: number; y: number;
  subs: Set<string>;
}

export class AoiGrid {
  private peers = new Map<uWS.WebSocket, Peer>();

  subscribeAround(ws: uWS.WebSocket, mapId: string, x: number, y: number) {
    const p = this.peers.get(ws)!;
    const topics = neighbors(mapId, x, y);
    for (const t of topics) if (!p.subs.has(t)) { ws.subscribe(t); p.subs.add(t); }
    p.mapId = mapId; p.x = x; p.y = y;
  }

  /** 타일 이동 시 구독 재계산 — 이탤 토픽 해지로 트래픽 최소화 */
  reposition(ws: uWS.WebSocket, x: number, y: number) {
    const p = this.peers.get(ws)!;
    const need = new Set(neighbors(p.mapId, x, y));
    for (const t of p.subs) if (!need.has(t)) { ws.unsubscribe(t); p.subs.delete(t); }
    for (const t of need) if (!p.subs.has(t)) { ws.subscribe(t); p.subs.add(t); }
    p.x = x; p.y = y;
  }

  /** 엔티티 상태 변경 → 해당 타일 3x3 에만 발행(자기 자신 제외) */
  publishState(this: AoiGrid, ws: uWS.WebSocket, payload: ArrayBuffer) {
    const p = this.peers.get(ws)!;
    const t = topicOf(p.mapId, p.x, p.y);
    ws.publish(t, payload, false); // false = 자기 자신 미포함
  }

  /** 맵 전환: 구독 전부 해지 후 새 맵 구독 */
  switchMap(ws: uWS.WebSocket, newMapId: string, x: number, y: number) {
    const p = this.peers.get(ws)!;
    for (const t of p.subs) ws.unsubscribe(t);
    p.subs.clear();
    this.subscribeAround(ws, newMapId, x, y);
  }
}

// — uWS 서버 결합 예시 —

```

```
export function attachAoi(app: uWS.TemplatedApp, grid: AoiGrid) {
  app.ws("/ws", {
    compression: uWS.SHARED_COMPRESSOR, // permessage-deflate 공유 사전
    open(ws) {
      grid["peers"].set(ws, { id: 0, mapId: "world", x: 0, y: 0, subs: new Set() });
      grid.subscribeAround(ws, "world", 100, 100);
    },
    message(ws, msg, isBinary) {
      if (!isBinary) return;
      grid.publishState(ws, msg.slice(0) as ArrayBuffer); // AOI 필터링 전송
    },
    close(ws) { grid["peers"].delete(ws); },
  });
}
```

(3) 성능 최적화 팁

- uWS publish 는 C++ 코어에서 수신자별 복사가 일어나므로 JS 루프로 수신자 목록을 만드는 것보다 수 배 빠르다.
- 격자 크기(CELL)는 시야 반경보다 한 칸 크게 — 8x8 타일(256px)은 표준 시야(약 300px)에 맞는 값이다.
- 밀집 지역은 토픽 하나에 구독자가 수백 명이라 publish 비용이 다시 커진다 — 채널 인스턴스(017 번)와 결합해 인원 상한을 둔다.
- spawn/despawn 이벤트는 인접 토픽들에 모두 보내야 한다 — 재구독 직후 1 회 전체 스냅샷을 주는 방식과 병행한다.

014. Delta Compression 기반 상태 변경점 압축 바이너리 패킷 구조

(1) 역할 설명

AOI 스냅샷을 매 틱 통째로 보내면 밀집 구역에서 1 유저당 수백 KB/s 가 된다. 델타 압축은 이전 스냅샷과의 차이(변경된 필드만)를 비트 마스크로 전송한다. 각 엔티티를 16 비트 필드 마스크 + 변경 필드만 직렬화하는 구조로 만들면, 정지 상태가 많은 필드 맵에서는 90% 이상 절약된다. 이 항목은 엔티티 상태를 고정 스키마(필드 번호)로 정의하고, 마스크 기반 인코더/디코더를 클라이언트·서버 공유 모듈로 구현한다.

설계 원칙은 세 가지다. 첫째, 필드 번호는 절대 변경하지 않는다(프로토콜 버전으로만 확장). 둘째, 압축 대상 필드는 16 비트 고정 소수로 양자화한다. 셋째, uWS 의 SHARED_COMPRESSOR(permessage-deflate)와 조합하면 반복되는 헤더 바이트까지 사전 압축된다. 신뢰성을 위해 델타는 '마지막 승인 스냅샷 기준'으로 생성하며, 유실 감지(시퀀스 건너뛴) 시 전체 스냅샷(full state)으로 복구한다.

(2) 구현 코드 — shared/delta.ts — 필드 마스크 기반 델타 코덱

```
// 엔티티 필드 비트 정의 (하위 16 비트, 추가는 상위 버전에서)
export const F = {
```

```

X: 1 << 0, Y: 1 << 1, VX: 1 << 2, VY: 1 << 3,
HP: 1 << 4, MP: 1 << 5, LVL: 1 << 6, STATE: 1 << 7, // 0=idle 1=move 2=atk 3=dead
TARGET: 1 << 8, BUFF: 1 << 9, SKIN: 1 << 10,
};

export interface EntityState {
  id: number; mask: number;
  x: number; y: number; vx: number; vy: number;
  hp: number; mp: number; lvl: number; state: number;
  target: number; buff: number; skin: number;
}

const Q = 10; // 위치 양자화 배율(x10)
export function diff(prev: EntityState | null, cur: EntityState): ArrayBuffer {
  const mask = prev ? buildMask(prev, cur) : 0xFFFF; // 신규 엔티티는 full
  const size = 4 + 2 + payloadSize(cur, mask); // id(4) + mask(2) + 필드들
  const buf = new ArrayBuffer(size), v = new DataView(buf);
  let o = 0;
  v.setUint32(o, cur.id); o += 4;
  v.setUint16(o, mask); o += 2;
  if (mask & F.X) { v.setInt16(o, Math.round(cur.x * Q)); o += 2; }
  if (mask & F.Y) { v.setInt16(o, Math.round(cur.y * Q)); o += 2; }
  if (mask & F.VX) { v.setInt8(o, Math.round(cur.vx / 4)); o += 1; }
  if (mask & F.VY) { v.setInt8(o, Math.round(cur.vy / 4)); o += 1; }
  if (mask & F.HP) { v.setUint16(o, cur.hp); o += 2; }
  if (mask & F.MP) { v.setUint16(o, cur.mp); o += 2; }
  if (mask & F.LVL) { v.setUint8(o, cur.lvl); o += 1; }
  if (mask & F.STATE) { v.setUint8(o, cur.state); o += 1; }
  if (mask & F.TARGET) { v.setUint32(o, cur.target); o += 4; }
  if (mask & F.BUFF) { v.setUint16(o, cur.buff); o += 2; }
  if (mask & F.SKIN) { v.setUint16(o, cur.skin); o += 2; }
  return buf;
}

function buildMask(p: EntityState, c: EntityState): number {
  let m = 0;
  if (Math.round(p.x * Q) !== Math.round(c.x * Q)) m |= F.X;
  if (Math.round(p.y * Q) !== Math.round(c.y * Q)) m |= F.Y;
  if (p.vx !== c.vx) m |= F.VX;
  if (p.vy !== c.vy) m |= F.VY;
  if (p.hp !== c.hp) m |= F.HP;
  if (p.mp !== c.mp) m |= F.MP;
  if (p.lvl !== c.lvl) m |= F.LVL;
  if (p.state !== c.state) m |= F.STATE;
  if (p.target !== c.target) m |= F.TARGET;
  if (p.buff !== c.buff) m |= F.BUFF;
  if (p.skin !== c.skin) m |= F.SKIN;
  return m;
}

function payloadSize(e: EntityState, m: number): number {
  let n = 0;
  if (m & F.X) n += 2; if (m & F.Y) n += 2;
  if (m & F.VX) n += 1; if (m & F.VY) n += 1;
  if (m & F.HP) n += 2; if (m & F.MP) n += 2;

```



```

    if (m & F.LVL) n += 1; if (m & F.STATE) n += 1;
    if (m & F.TARGET) n += 4; if (m & F.BUFF) n += 2; if (m & F.SKIN) n += 2;
    return n;
  }
}

export function applyDelta(prev: EntityState, delta: DataView): EntityState {
  const e = { ...prev };
  let o = 6; // id(4)+mask(2) 건너뛰
  e.id = delta.getUint32(0);
  const mask = delta.getUint16(4);
  if (mask & F.X) e.x = delta.getInt16(o) / Q; o += 2;
  if (mask & F.Y) e.y = delta.getInt16(o) / Q; o += 2;
  if (mask & F.VX) e.vx = delta.getInt8(o) * 4; o += 1;
  if (mask & F.VY) e.vy = delta.getInt8(o) * 4; o += 1;
  if (mask & F.HP) e.hp = delta.getUint16(o); o += 2;
  if (mask & F.MP) e.mp = delta.getUint16(o); o += 2;
  if (mask & F.LVL) e.lvl = delta.getUint8(o); o += 1;
  if (mask & F.STATE) e.state = delta.getUint8(o); o += 1;
  if (mask & F.TARGET) e.target = delta.getUint32(o); o += 4;
  if (mask & F.BUFF) e.buff = delta.getUint16(o); o += 2;
  if (mask & F.SKIN) e.skin = delta.getUint16(o);
  return e;
}

```

(3) 성능 최적화 팁

- 델타 기준점은 '수신자가 마지막으로 승인한 스냅샷'이어야 한다 — 서버가 유저별 lastAckSeq 를 유지해야 유실 후 정확한 재기준이 가능하다.
- 연속 델타 유실이 3 회 이상이면 즉시 full 스냅샷으로 복구하는 정책이 실무 표준이다.
- 마스크에 버전 비트를 2~3 개 남겨두면 스키마 확장(신규 필드)이 하위 호환을 유지한 채 가능하다.
- permessage-deflate(SHARED_COMPRESSOR)는 델타의 반복 헤더를 추가로 눌러준다 — 이미 양자화된 바이트열에는 대체로 20~30% 추가 절약이다.

015. 레이턴시 보상(Lag Compensation) 판정 연산

(1) 역할 설명

유저가 150ms 라그 상태에서 몬스터를 향해 베기를 눌렀을 때, 화면의 몬스터와 서버의 몬스터 위치는 다르다. 서버가 현재 위치로만 판정하면 유저는 '명백히 맞았는데 빗나갔다'고 느낀다. 락 컴펜세이션은 서버가 과거 시점(RTT 만큼 이전)의 월드 상태를 버퍼에 보관해 두었다가, 유저의 공격을 '유저가 본 시점'의 상태로 판정하는 기법이다. FPS 의 원격 shooter rewind 와 동일한 원리가 근접/타겟팅 MMORPG 에 적용된 형태다.

구현의 핵심은 두 가지다. 첫째, 서버는 모든 동적 엔티티의 과거 상태를 일정 틱(보통 1 초 = 60 틱)만큼 링 버퍼로 보관한다. 둘째, 공격 패킷에는 클라이언트가 조준 당시 기준으로

삼은 서버 시각(clientTick)을 실어 보내고, 서버는 해당 틱 상태로 히트박스를 되감아 판정한다. 판정 결과는 현재 상태에 적용하되(과거로 되돌리지 않음), 보정량이 너무 크면(스폰 직후 등) 현재 상태 기준으로 절충한다.

(2) 구현 코드 — server/combat/LagComp.ts

```
import { distPointSector, distPointRect, distPointCircle } from "./hitbox";

export interface TickState {          // 틱별 저장 상태(엔티티 단위)
  id: number; x: number; y: number; dead: boolean;
}
export interface AttackPacket {
  attackerId: number;
  skillId: number;
  aimTick: number;                  // 클라가 본 서버 틱(클라가 마지막 받은 tick)
  origin: { x: number; y: number }; // 공격자 클라이언트 위치(참고용)
  shape: { kind: "circle" | "sector" | "rect";
    r?: number; angle?: number; dir?: number;
    w?: number; h?: number };
}

const HISTORY_TICKS = 60;          // 1 초 버퍼(60Hz 가정)
const MAX_REWIND_TICKS = 30;       // 되감기 상한 0.5 초 — 어뷰징 방지

export class LagCompensator {
  private history: Map<number, TickState[]> = new Map(); // tick → entities

  pushTick(tick: number, states: TickState[]) {
    this.history.set(tick, states);
    this.history.delete(tick - HISTORY_TICKS); // 링 버퍼
  }

  judge(pkt: AttackPacket, currentTick: number, currentStates: TickState[],
    maxLatencyTicks = MAX_REWIND_TICKS): number[] {
    // 1) 되감기 틱 산정: 클라 요청 틱 사용, 단 상한 클램프 + 미래 금지
    let rewind = currentTick - pkt.aimTick;
    if (rewind < 0) rewind = 0; // 미래 틱 조작 차단
    if (rewind > maxLatencyTicks) rewind = maxLatencyTicks;
    const targetTick = currentTick - rewind;
    const rewind = this.history.get(targetTick) ?? currentStates;

    // 2) 되감은 상태에서 히트박스 판정
    const attacker = rewind.find(s => s.id === pkt.attackerId);
    if (!attacker) return [];
    const ox = pkt.origin.x && rewind > 0 ? attacker.x : attacker.x;
    const oy = pkt.origin.y && rewind > 0 ? attacker.y : attacker.y;

    const hits: number[] = [];
    for (const s of rewind) {
```

```

if (s.id === pkt.attackerId || s.dead) continue;
let hit = false;
switch (pkt.shape.kind) {
  case "circle": hit = distPointCircle(s.x, s.y, ox, oy, pkt.shape.r!) <= 0; break;
  case "sector": hit = distPointSector(s.x, s.y, ox, oy,
    pkt.shape.r!, pkt.shape.angle!, pkt.shape.dir!) <= 0; break;
  case "rect": hit = distPointRect(s.x, s.y, ox, oy,
    pkt.shape.w!, pkt.shape.h!, pkt.shape.dir!) <= 0; break;
}
if (hit) hits.push(s.id);
}

// 3) 되감기 페널티 절충: 되감은 대상이 현재 위치로 크게 이동했다면
//   현재 상태 재검증(스폰 회피/텔레포트 어뷰징 방지)
return hits.filter(id => {
  const now = currentStates.find(s => s.id === id);
  if (!now) return false;
  const past = rewound.find(s => s.id === id!);
  return Math.hypot(now.x - past.x, now.y - past.y) < 200; // 200px 이내만 유효
});
}
}

```

(3) 성능 최적화 팁

- rewind 상한(0.5 초)이 없으면 핵 유저가 aimTick 을 조작해 '5 초 전 몬스터'를 때릴 수 있다 — 반드시 클램프한다.
- 판정은 과거 상태로 하되 데미지 적용은 현재 틱에 한다 — 상태를 과거로 되돌리면 다른 유저 판정이 꼬인다.
- 파티/보스전처럼 유저 밀도가 높은 전투는 되감기 페널티(이동량 상한)를 느슨하게, PVP 는 엄격하게 조정한다.
- 클라이언트는 서버 시각 동기화(RTT/2 추정, 019 번과 연계)로 aimTick 을 정확히 계산해야 보상 정확도가 나온다.

016. Server-Authoritative 위치 및 이동 검증 로직

(1) 역할 설명

클라이언트 예측(011 번)이 아무리 정교해도 서버가 클라 좌표를 그대로 믿으면 월핵은 그대로다. 서버 권위 모델에서 클라이언트의 C2S_MOVE 는 '위치'가 아니라 '입력 의도'이며, 서버는 shared/movement.ts 의 동일한 stepMove 함수로 직접 시뮬레이션해 자신의 결과만 진짜 위치로 유지한다. 클라가 보낸 dx, dy 는 반드시 [-1,1] 정규화 클램프를 거치고, 이동 스텝의 dt 도 서버 틱 간격으로 대체해 클라가 조작한 시간값을 신뢰하지 않는다.

추가 방어로 세 가지를 둔다. 첫째, 입력 스로틀(틱당 최대 2 개 입력)로 패킷 플러딩을 막는다. 둘째, 순간 이동 감지(이전 승인 위치에서 시뮬레이션 상한 속도보다 큰 변위)는 자동

롤백(020 번) 대상이 된다. 셋째, 컨텍스트 속도 보정(가속 버프, 탈것, 스킬 대시)은 서버의 버프 상태에서만 유효 속도를 계산해, 클라가 '속도 버프가 있다'고 허위 보고하는 것을 차단한다.

(2) 구현 코드 — server/move/ServerAuthority.ts — uWebSockets.js 핸들러 결합

```
import { Input, MoveState, WALK_SPEED, stepMove } from "../shared/movement";
import type uWS from "uWebSockets.js";

const MAX_SPEED_MULT = 1.5;          // 네트워크 지터 허용 배율
const INPUTS_PER_TICK = 2;          // 틱당 입력 수용 상한

interface Peer {
  userId: number; state: MoveState;
  speedMult: number;          // 버프 계산 결과(서버 소유)
  buffered: Input[]; lastSeq: number;
  tickInputCount: number;
}

export class MoveAuthority {
  private peers = new Map<uWS.WebSocket, Peer>();

  /** C2S_MOVE 수신 — 클라 좌표를 무시하고 입력만 채택 */
  onMove(ws: uWS.WebSocket, inSeq: number, dx: number, dy: number, _clientDt: number) {
    const p = this.peers.get(ws);
    if (!p || inSeq <= p.lastSeq) return;          // 재전송/조작 시퀀스 폐기
    if (p.tickInputCount >= INPUTS_PER_TICK) return; // 스톱(풀러딩 방지)
    p.tickInputCount++;
    p.lastSeq = inSeq;

    const len = Math.hypot(dx, dy) || 1;
    p.buffered.push({
      seq: inSeq,
      dx: Math.max(-1, Math.min(1, dx / len)),      // 정규화 클램프
      dy: Math.max(-1, Math.min(1, dy / len)),
      dt: 16.67,                                     // 클라 dt 무시, 서버 틱 고정
    });
  }

  /** 서버 고정 틱(60Hz) — 권위 시뮬레이션 */
  tick(now: number, blocked: (x: number, y: number) => boolean) {
    for (const p of this.peers.values()) {
      p.tickInputCount = 0;
      // 유효 속도는 서버 버프에서만 결정
      const prev = { ...p.state };
      p.state.speed = WALK_SPEED * p.speedMult;
      while (p.buffered.length) {
        const inp = p.buffered.shift();
        p.state = stepMove(p.state, inp, blocked);
      }
    }
  }
}
```

```

    }
    // 순간 이동 감지: 한 틱 허용 변위 상한 = 속도 * 틱 * 허용배율
    const maxDisp = WALK_SPEED * p.speedMult * 0.05 * MAX_SPEED_MULT * 2;
    const disp = Math.hypot(p.state.x - prev.x, p.state.y - prev.y);
    if (disp > maxDisp) {
      p.state = prev; // 롤백(020 번 연계)
      this.onViolation?.(p.userId, "teleport", disp);
    }
  }
}
}

onViolation: ((userId: number, kind: "teleport" | "speed", value: number) => void) | null = null;
}

```

(3) 성능 최적화 팁

- 클라이언트가 보내는 모든 수치(좌표, dt, 속도)는 힌트일 뿐이며, 서버가 다시 계산한 값만 유일한 진실이다.
- 입력 스톱들과 시퀀스 중복 폐기만으로도 월핵 자동화 스크립트의 대부분은 무력화된다.
- 유효 속도 배율(speedMult)은 반드시 서버 버프/탈것 상태에서 계산하고 클라 보고를 참조하지 않는다.
- 위반은 첫 회에 즉시 밴이 아니라 롤백 + 로그 기록(095 번 텔레메트리)으로 남겨 패턴 분석에 쓴다.

017. 인스턴스 채널 분산 처리 및 룸(Room) 관리 스키마

(1) 역할 설명

한 맵에 유저가 몰리면 AOI(013 번)로도 밀집 구역 publish 비용이 커지고, 보스·던전은 파티 단위 격리가 필요하다. 룸 관리 스키마는 '월드 맵 = 채널 1..n, 던전/보스 = 전용 룸'의 이중 구조로 이를 해결한다. 필드 맵은 채널(인스턴스) 1~5 를 두고, 유저가 맵 진입 시 가장 인원이 적은 채널에 자동 배정되며, 파티원이 있으면 같은 채널로 강제 동기화한다. 던전·레이드는 매칭 성사 시 전용 룸 인스턴스를 생성하고, 완료 시 회수한다.

룸은 프로세스 내 Map 과 멀티프로세스 Redis 레지스트리 두 계층으로 관리한다. 프로세스 내 Map 은 룸의 라이브 상태(엔티티, 틱 루프)를 담당하고, Redis 는 '어느 프로세스가 어떤 룸을 소유하는지'의 메타만 담당해 스케일아웃 시 라우팅 정보의 단일 진실이 되게 한다. 룸은 최대 수명/유휴 타임아웃/유저 하한을 갖는 생명주기 상태머신(CREATING/ACTIVE/DRAINING/CLOSED)으로 운영된다.

(2) 구현 코드 — server/room/RoomRegistry.ts

```

import Redis from "ioredis";

export type RoomKind = "field" | "dungeon" | "raid" | "housing";

```

```

export type RoomPhase = "CREATING" | "ACTIVE" | "DRAINING" | "CLOSED";

export interface RoomMeta {
  roomId: string;          // "field:world:ch2" / "dungeon:d01:run421"
  kind: RoomKind;
  mapId: string;
  ownerId: string;         // 프로세스 노드 id
  capacity: number;
  pop: number;
  phase: RoomPhase;
  ttlAt: number;           // 유희 회수 예정 시각
}

const ROOM_KEY = "rooms:"; // Redis 해시 — 멀티노드 라우팅의 단일 진실

export class RoomRegistry {
  constructor(private redis: Redis, private nodeId: string) {}

  /** 맵 진입: 가장 여유 있는 채널 선택(파티 정책 우선) */
  async pickFieldChannel(mapId: string, partySize: number, partyRoom?: string):
  Promise<RoomMeta> {
    if (partyRoom) {
      const pr = await this.get(partyRoom);
      if (pr && pr.phase === "ACTIVE" && pr.pop + partySize <= pr.capacity) return pr;
    }
    const rooms = await this.listByKind("field", mapId);
    const open = rooms
      .filter(r => r.phase === "ACTIVE" && r.pop + partySize <= r.capacity)
      .sort((a, b) => a.pop - b.pop);
    if (open.length) return open[0];
    return this.create({ kind: "field", mapId, capacity: 80 }); // 채널 자동 증설
  }

  async create(spec: { kind: RoomKind; mapId: string; capacity: number }):
  Promise<RoomMeta> {
    const roomId = spec.kind + ":" + spec.mapId + ":" + this.nodeId + ":" + Date.now();
    const meta: RoomMeta = {
      roomId, kind: spec.kind, mapId: spec.mapId, ownerId: this.nodeId,
      capacity: spec.capacity, pop: 0, phase: "CREATING",
      ttlAt: Date.now() + 60_000, // 60 초 내 유저 없으면 자동 회수
    };
    await this.redis.hset(ROOM_KEY, roomId, JSON.stringify(meta));
    return meta;
  }

  async join(roomId: string): Promise<RoomMeta | null> {
    const r = await this.get(roomId);
    if (!r || r.phase === "CLOSED" || r.pop >= r.capacity) return null;
    r.pop++; r.ttlAt = 0; // 활성화 — TTL 해제
    await this.redis.hset(ROOM_KEY, roomId, JSON.stringify(r));
    return r;
  }
}

```

```

}

async leave(roomId: string) {
  const r = await this.get(roomId);
  if (!r) return;
  r.pop = Math.max(0, r.pop - 1);
  if (r.pop === 0 && r.kind !== "field") r.phase = "DRAINING"; // 던전은 즉시 회수 대기
  await this.redis.hset(ROOM_KEY, roomId, JSON.stringify(r));
}

private async get(roomId: string): Promise<RoomMeta | null> {
  const raw = await this.redis.hget(ROOM_KEY, roomId);
  return raw ? JSON.parse(raw) : null;
}

private async listByKind(kind: RoomKind, mapId: string): Promise<RoomMeta[]> {
  const all = await this.redis.hvals(ROOM_KEY);
  return all.map(JSON.parse)
    .filter((r: RoomMeta) => r.kind === kind && r.mapId === mapId);
}

/** 주기 청소: DRAINING 상태 30 초 경과 룸 폐기 + 메모리 룸 detach */
async sweep(onClose: (roomId: string) => void) {
  const all = await this.redis.hvals(ROOM_KEY);
  for (const raw of all) {
    const r: RoomMeta = JSON.parse(raw);
    const expired = (r.phase === "DRAINING" && r.ttlAt === 0 && r.pop === 0)
      || (r.phase === "CREATING" && r.ttlAt > 0 && Date.now() > r.ttlAt);
    if (expired) {
      onClose(r.roomId);
      await this.redis.hdel(ROOM_KEY, r.roomId);
    }
  }
}
}

```

(3) 성능 최적화 팁

- 채널 자동 증설 기준은 '최고 밀도 채널이 capacity 의 80%를 넘을 때'가 안정적이며, 감축은 50% 이하가 10 분 지속될 때 수행한다.
- 파티원 강제 동기화(같은 채널)는 유저 경험의 핵심이다 — 파티 진입 시 partyRoom 을 Redis 에 기록해 재접속에도 유지한다.
- roomId 에 노드 id 를 포함하면 라우팅 계층이 '어느 프로세스로 접속을 보낼지'를 O(1)로 알 수 있다.
- DRAINING 상태 룸에는 신규 입장을 막되 기존 유저는 전투를 마치고 나갈 수 있게 해야 강제 추방 불만을 막는다.

018. Kysely ORM 기반 재화/아이템 안전 이동 분산 DB 트랜잭션

(1) 역할 설명

거래, 우편 첨부, 파티 분배, 경매 결제는 모두 '두 유저 간 재화·아이템 이동'이며, 단 한 번이라도 절반만 적용되면 서비스 신뢰가 무너진다. PostgreSQL 트랜잭션 + SELECT FOR UPDATE(행 잠금) + 낙관적 버전 검증으로 원자성을 보장하고, Kysely의 타입 세이프 쿼리 빌더로 이 구조를 컴파일 타임에 강제한다. 이 항목은 (1) 스키마 타입 정의, (2) 재화 이동 트랜잭션, (3) 아이템 소유권 이전 트랜잭션 세 부분으로 구성한다.

분산 환경(멀티 노드)에서는 두 트랜잭션이 같은 행을 건드리지 않도록 잠금 순서(항상 user id 오름차순으로 잠금)를 표준화해 교착 상태(deadlock)를 예방한다. 재화는 절대 음수가 될 수 없다는 CHECK 제약을 DB 레벨에 두고, 애플리케이션 로직 버그가 있어도 DB가 최후의 방어선이 되게 한다. 모든 이동은 ledger 테이블에 흐름 기록을 남겨 감사·복구(100 번 백업과 연계)가 가능하다.

(2) 구현 코드 — db/schema.d.ts — Kysely 타입 정의

```
export interface UserRow {
  id: number; name: string;
  gold: number; cash: number; created_at: Date;
}
export interface ItemRow {
  id: number; owner_id: number;
  item_code: string; qty: number;
  binding: "bound" | "tradeable"; // 귀속/비귀속(086 번과 연계)
  version: number; // 낙관적 잠금용
}
export interface LedgerRow {
  id: number; at: Date; kind: "gold" | "item";
  from_user: number | null; to_user: number | null;
  ref: string; // 거래/우편/경매 참조 id
  payload: string; // { qty, itemCode, ... } JSON
}

export interface Database {
  users: UserRow;
  items: ItemRow;
  ledger: LedgerRow;
}
```

보조 코드 — db/transfer.ts — Kysely 재화/아이템 이동 트랜잭션

```
import { Kysely, sql } from "kysely";
import type { Database } from "../schema";

export class TransferError extends Error {
  constructor(public code: "NOT_FOUND" | "NO_FUNDS" | "BOUND_ITEM" | "LOCK_TIMEOUT",
    msg: string) { super(msg); }
}
```



```

/** 재화 이동: 잠금 순서 표준화(id 오름차순)로 교착 예방 */
export async function transferGold(
  db: Kysely<Database>, fromId: number, toId: number, amount: number, ref: string,
) {
  if (amount <= 0) throw new TransferError("NO_FUNDS", "amount must be positive");
  const [lock1, lock2] = fromId < toId ? [fromId, toId] : [toId, fromId];

  return await db.transaction().execute(async (tx) => {
    // 순서대로 잠금 — SELECT FOR UPDATE
    for (const id of [lock1, lock2]) {
      await tx.selectFrom("users").where("id", "=", id)
        .forUpdate().select("id").execute();
    }
    const from = await tx.selectFrom("users").where("id", "=", fromId)
      .select(["gold"]).executeTakeFirstOrThrow();
    if (from.gold < amount) throw new TransferError("NO_FUNDS", "insufficient gold");

    await tx.updateTable("users").set(eb => ({ gold: eb("gold", "-", amount) }))
      .where("id", "=", fromId).execute();
    await tx.updateTable("users").set(eb => ({ gold: eb("gold", "+", amount) }))
      .where("id", "=", toId).execute();

    await tx.insertInto("ledger").values({
      at: new Date(), kind: "gold", from_user: fromId, to_user: toId,
      ref, payload: JSON.stringify({ amount }),
    }).execute();
  });
}

/** 아이템 소유권 이전: 낙관적 버전 검증 + 귀속 판정 */
export async function transferItem(
  db: Kysely<Database>, itemId: number, fromId: number, toId: number, ref: string,
) {
  return await db.transaction().execute(async (tx) => {
    const item = await tx.selectFrom("items").where("id", "=", itemId)
      .forUpdate().select(["owner_id", "binding", "version"]).executeTakeFirst();
    if (!item) throw new TransferError("NOT_FOUND", "item missing");
    if (item.owner_id !== fromId) throw new TransferError("NOT_FOUND", "not owner");
    if (item.binding === "bound") throw new TransferError("BOUND_ITEM", "bound item");

    // 낙관적 잠금: version 일치 시에만 갱신(동시 소유 이전 차단)
    const updated = await tx.updateTable("items")
      .set({ owner_id: toId, version: item.version + 1 })
      .where(eb => eb.and([
        eb("id", "=", itemId),
        eb("version", "=", item.version),
      ]))
      .executeTakeFirst();
    if (Number(updated.numUpdatedRows) !== 1n)
      throw new TransferError("LOCK_TIMEOUT", "concurrent transfer detected");
  });
}

```

```

await tx.insertInto("ledger").values({
  at: new Date(), kind: "item", from_user: fromId, to_user: toId,
  ref, payload: JSON.stringify({ itemId }),
}).execute();
});
}

// CHECK 제약(DB 레벨 최후 방어):
// ALTER TABLE users ADD CONSTRAINT gold_nonneg CHECK (gold >= 0);
// ALTER TABLE items ADD CONSTRAINT qty_nonneg CHECK (qty >= 0);

```

(3) 성능 최적화 팁

- 잠금 순서 표준화(id 오름차순)는 PostgreSQL 에서 교착 상태를 구조적으로 제거하는 가장 저비용 방법이다.
- CHECK 제약(gold >= 0)은 애플리케이션 버그의 최후 방어선이다 — 반드시 DB 레벨에 둔다.
- ledger 흐름 테이블은 파티셔닝(월 단위)으로 운영하면 감사 조회와 용량 관리가 모두 풀린다.
- Kysely 의 .forUpdate()는 트랜잭션 내에서만 유효하다 — 커넥션 풀 트랜잭션 바깥에서 실수로 쓰지 않도록 레포지토리 계층으로 감싼다.

019. 클라이언트 RTT(Ping) 측정 및 상태 모니터링

(1) 역할 설명

RTT 는 예측 보정(011), 랙 컴펜세이션(015), 보간 지연(012)의 공통 입력값이다. 측정은 바이너리 PING/PONG 왕복(1~2 초 간격)으로 수행하고, 지수 이동 평균(EMA) 과 최댓값/분위수를 함께 유지해 '지금 느린가, 지속적으로 느린가'를 구분한다. 순수 RTT 외에 서버 처리 시간(pong 응답에 서버 수신 타임스탬프 포함)을 분리 측정하면 병목이 네트워크인지 서버인지 구분할 수 있다.

모니터링은 클라/서버 양쪽에서 이루어진다. 클라는 RTT 급상승 시 예측 안전장치를 강화(pending 입력 상한 축소)하고 HUD 에 네트워크 상태 아이콘을 표시한다. 서버는 유저별 RTT 분포를 수집해 지역/ISP 단위 품질 대시보드를 만들고, P95 가 임계를 넘는 구역은 리전 증설(017 번과 연계) 판단 근거로 쓴다. 텔레메트리(095 번)로 주기 배출한다.

(2) 구현 코드 — `src/net/RttMonitor.ts`

```

export interface RttStats {
  last: number; ema: number; p95: number; lossRate: number;
  serverDelay: number;           // 서버 처리 소요(ms)
}

const WINDOW = 20;           // 분위수 계산 윈도우

```

```

export class RttMonitor {
  private inflight = new Map<number, number>(); // seq → 송신 시각
  private samples: number[] = [];
  private emaVal = 0;
  private lost = 0; sent = 0;
  private serverDelayEma = 0;
  private seq = 0;
  private timer = 0;

  constructor(
    private sendPing: (payload: ArrayBuffer) => void,
    private onStats: (s: RttStats) => void,
    private intervalMs = 2000,
  ) {}

  start() {
    this.timer = window.setInterval(() => this.probe(), this.intervalMs);
  }
  stop() { clearInterval(this.timer); }

  private probe() {
    const seq = ++this.seq;
    const v = new DataView(new ArrayBuffer(5));
    v.setUint8(0, 3 /* PKT.C2S_PING */);
    v.setUint32(1, seq);
    this.inflight.set(seq, performance.now());
    this.sent++;
    if (this.inflight.size > 3) { // 3 개 이상 미응답 = 유실 추정
      this.lost += this.inflight.size - 3;
      const oldest = Math.min(...this.inflight.keys());
      this.inflight.delete(oldest);
    }
    this.sendPing(v.buffer);
  }

  /** S2C_PONG 수신: [type u8][seq u32][serverRecvMs f32][serverSendMs f32] */
  onPong(buf: ArrayBuffer) {
    const v = new DataView(buf);
    const seq = v.getUint32(1);
    const sentAt = this.inflight.get(seq);
    if (sentAt == null) return;
    this.inflight.delete(seq);

    const rtt = performance.now() - sentAt;
    const serverDelay = v.getFloat32(9) - v.getFloat32(5);
    this.samples.push(rtt);
    if (this.samples.length > WINDOW) this.samples.shift();
    this.emaVal = this.emaVal === 0 ? rtt : this.emaVal * 0.7 + rtt * 0.3;
    this.serverDelayEma = this.serverDelayEma === 0
      ? serverDelay : this.serverDelayEma * 0.8 + serverDelay * 0.2;
  }
}

```

```
const sorted = [...this.samples].sort((a, b) => a - b);
const p95 = sorted[Math.floor(sorted.length * 0.95)] ?? rtt;
this.onStats({
  last: rtt, ema: Math.round(this.emaVal), p95,
  lossRate: this.lost / Math.max(1, this.sent),
  serverDelay: Math.round(this.serverDelayEma),
});
}
```

// 사용: HUD 표시 임계 — ema < 80 초록 / < 150 노랑 / 이상 또는 lossRate>5% 빨강

(3) 성능 최적화 팁

- ping 주기 2 초는 정확도와 트래픽의 균형점이며, 전투 중에는 1 초로 단축해 렉 컴펜세이션 정확도를 높인다.
- RTT 에서 서버 처리 시간을 빼야 진짜 네트워크 지연이다 — pong 에 서버 수신/송신 타임스탬프를 둘 다 담는다.
- EMA($\alpha=0.3$)는 스파이크에 둔감해 UI 표시에 적합하고, p95 는 예측 안전장치 튜닝에 적합하다 — 용도별로 분리해 사용한다.
- 서버는 수집한 RTT 를 유저 단위가 아닌 지역 단위로 집계해 개인정보 이슈 없이 리전 운영 판단에 쓴다.

020. 좌표 오차 초과 시 위치 롤백(Rollback) 및 보정 연산

(1) 역할 설명

예측(011)과 서버 권위(016)가 병행되면 클라 표시 위치와 서버 승인 위치 사이에 오차가 존재하는 것이 정상이다. 문제는 오차가 임계를 넘는 순간이다. 작은 오차는 스무딩으로 흡수하지만, 텔레포트 수준의 오차(월핵, 재접속, 서버 보정)는 즉시 롤백해서 서버 위치로 되돌려야 조작 신뢰가 유지된다. 이 모듈은 오차 원인별로 다른 보정 정책(스무딩/스냅/롤백+재동기화)을 단일 상태머신으로 수행한다.

롤백의 시각적 품질은 '보정 이펙트'가 좌우한다. 순간이동 보정 시에는 이동 궤적에 잔상/파티클을 뿌려 유저가 '기술 효과'로 느끼게 하고, 롤백 원인이 서버 정정이면 로그에 violation 마킹(016 번)해 어뷰징 판정 근거로 남긴다. 서버도 클라의 재동기화 요청(resync)을 받으면 자신의 권위 상태와 마지막 승인 seq 를 재전송해 양쪽이 같은 기준점을 갖게 한다.

(2) 구현 코드 — [src/net/CorrectionPolicy.ts](#)

```
import type { MoveState } from "../shared/movement";

export type Correction = "smooth" | "snap" | "rollback";

export interface CorrectionCtx {
  display: MoveState; // 클라가 그리는 위치
```

```

    auth: MoveState;          // 서버 승인 위치
    lastViolationMs: number;   // 최근 서버 위반 알림 시각
  }

const TILE = 32;
export function decideCorrection(ctx: CorrectionCtx): Correction {
  const err = Math.hypot(ctx.display.x - ctx.auth.x, ctx.display.y - ctx.auth.y);
  const sinceViolation = Date.now() - ctx.lastViolationMs;

  // 1) 서버 위반(롤백 판정) 후 500ms 내 — 서버 기준 즉시 스냅
  if (sinceViolation < 500) return "snap";
  // 2) 오차 < 0.5 타일 — 스무딩으로 흡수(티 나지 않음)
  if (err < TILE * 0.5) return "smooth";
  // 3) 오차 < 2 타일 — 큰 스무딩(0.5 프레임당 20% 수렴)
  if (err < TILE * 2) return "smooth";
  // 4) 그 이상(텔레포트급) — 롤백 스냅 + 이펙트
  return "rollback";
}

export class PositionCorrector {
  private lastViolationMs = 0;
  private smoothingQueue: MoveState[] = []; // 스무딩용 중간 목표점

  onServerViolationNotice() { this.lastViolationMs = Date.now(); }

  /** 프레임마다: display 를 auth 쪽으로 정책에 따라 수렴 */
  step(display: MoveState, auth: MoveState, dtMs: number): { state: MoveState; fx: "none" | "trail" } {
    const kind = decideCorrection({
      display, auth, lastViolationMs: this.lastViolationMs,
    });
    switch (kind) {
      case "smooth": {
        // 초당 60% 수렴 — 프레임율 독립: k = 1 - 0.4^(dt/1000)
        const k = 1 - Math.pow(0.4, dtMs / 1000);
        return {
          state: {
            x: display.x + (auth.x - display.x) * k,
            y: display.y + (auth.y - display.y) * k,
            speed: display.speed,
          },
          fx: "none",
        };
      }
      case "rollback":
      case "snap": {
        // 궤적 이펙트(trail)로 시각적 완충 — 유저가 기술로 인지하게
        this.smoothingQueue = [];
        return { state: { ...auth }, fx: kind === "rollback" ? "trail" : "none" };
      }
    }
  }
}

```

```

}
}

// 서버 측 resync 응답: 권위 상태 + lastAckSeq 재전송
// { t: "resync", seq, x, y, speed } → 클라 PredictionEngine.onServerAck 재사용

```

(3) 성능 최적화 팁

- 스무딩 계수는 프레임율 독립으로 계산한다(지수 감쇠 공식) — 60fps 와 144fps 에서 수렴 속도가 같아야 한다.
- 롤백 직후 0.5 초는 무조건 snap 정책을 유지해야 보정 루프(오차 → 스무딩 → 다시 오차)에 빠지지 않는다.
- 순간이동 보정에는 반드시 시각 이펙트를 곁한다 — 보정이 잦아도 유저 체감은 '게임 특성'으로 수용된다.
- 오차 원인 구분(네트워크 지연 vs 위반 vs 버그)은 로그에 correction kind + 당시 RTT(019 번)를 함께 남겨야 분석이 가능하다.

제 4 장 모듈 3 — 캐릭터 및 성장 시스템 (021~030)

기술 스택: PostgreSQL DDL · Kysely · TypeScript Seed · TypeScript 본 장은 캐릭터의 모든 수치는 하나의 스탯 파이프라인(합산 순서 고정)에서 산출된다. 경험치 곡선·스킬 트리·장비 슬롯·강화 인첸트·전직·코스튬·칭호·랜덤 옵션·도감까지, 성장의 모든 요소가 데이터 테이블과 순수 함수로 정규화된다. 이 장의 원칙은 '기획 변경은 데이터 삽입으로 끝난다'는 것이며, 서버와 클라가 같은 연산 코드를 공유해 결정론을 유지한다.

021. 주/보조 스탯 계산 및 최종 수치 연산 시스템

(1) 역할 설명

캐릭터의 최종 스탯은 기본치(레벨/클래스), 장비, 버프, 패시브(칭호/도감), 특성 투자 등 다수 소스의 합성이다. 이 시스템은 모든 소스를 동일한 StatMod 구조(flat/add + percent/mul)로 정규화하고, 순서를 고정한 합산 파이프라인(기본치 → 장비 flat → 패시브 flat → 전체 % → 상한 클램프)으로 최종값을 산출한다. 계산은 순수 함수로 만들어 서버(판정)와 클라(UI 표시)가 동일 결과를 보장한다.

성능을 위해 스탯 변경 이벤트(장비 착용, 레벨업, 버프 적용)가 발생할 때만 재계산하는 더티 플래그 방식을 쓰고, 결과를 캐릭터 캐시에 저장한다. 합산 순서가 중요한 이유는 곱연산의 교환 법칙이 소수점 환경에서 성립하지 않기 때문이며, 서버/클라 동일 순서 강제는 결정론의 전제다.

(2) 구현 코드 — PostgreSQL DDL

```
CREATE TABLE characters (
```

```

id          BIGSERIAL PRIMARY KEY,
account_id  BIGINT    NOT NULL,
name        VARCHAR(20) NOT NULL UNIQUE,
class_code  VARCHAR(20) NOT NULL,
level       INT       NOT NULL DEFAULT 1,
exp         BIGINT    NOT NULL DEFAULT 0,
base_str    INT NOT NULL DEFAULT 5, -- 기본 주 스탯(성장으로 증가)
base_dex    INT NOT NULL DEFAULT 5,
base_int    INT NOT NULL DEFAULT 5,
base_vit    INT NOT NULL DEFAULT 5,
stat_points INT NOT NULL DEFAULT 0, -- 자유 분배 포인트
allocated   JSONB NOT NULL DEFAULT '{}', -- {"str":3,"vit":2}
created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 장비(착용 상태 포함) — 024 번과 연결
CREATE TABLE character_items (
  id          BIGSERIAL PRIMARY KEY,
  character_id BIGINT NOT NULL REFERENCES characters(id) ON DELETE CASCADE,
  item_code    VARCHAR(40) NOT NULL,
  slot         VARCHAR(16),          -- NULL = 인벤토리, 착용 시 부위
  enhance      SMALLINT NOT NULL DEFAULT 0,
  mods         JSONB NOT NULL DEFAULT '[]', -- [{"stat":"atk", flat:12}, ...]
  version      INT NOT NULL DEFAULT 0
);

CREATE INDEX idx_citems_char_slot ON character_items(character_id, slot);

-- 스탯 최종 캐시(재계산 결과 저장 — 조회 최적화)
CREATE TABLE character_stats_cache (
  character_id BIGINT PRIMARY KEY REFERENCES characters(id) ON DELETE CASCADE,
  final         JSONB NOT NULL,          -- {"atk":120.5, "hp":3200, ...}
  dirty        BOOLEAN NOT NULL DEFAULT TRUE,
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

보조 코드 — src/game/StatMath.ts — 순수 스탯 연산

```

export type StatKey =
  | "str" | "dex" | "int" | "vit"    // 1 차
  | "atk" | "matk" | "def" | "hp" | "mp" | "crit" | "aspd"; // 2 차

export interface StatMod {
  stat: StatKey;
  flat?: number;    // 가산
  mul?: number;     // 승산(0.1 = +10%)
}

export interface StatSource {
  kind: "base" | "equip" | "passive" | "buff";
  mods: StatMod[];
}

```

```

/** 합산 순서 고정: base flat → equip flat → passive flat → 전체 mul → 클램프 */
export function computeFinal(base: Record<string, number>, sources: StatSource[]):
Record<string, number> {
  const flat: Record<string, number> = { ...base };
  let mulAll: Record<string, number> = {};

  for (const src of sources) {
    for (const m of src.mods) {
      if (m.flat) flat[m.stat] = (flat[m.stat] ?? 0) + m.flat;
      if (m.mul) mulAll[m.stat] = (mulAll[m.stat] ?? 0) + m.mul;
    }
  }
  const out: Record<string, number> = {};
  for (const k of Object.keys(flat)) {
    const mul = 1 + (mulAll[k] ?? 0);
    out[k] = Math.max(0, Math.round(flat[k] * mul * 100) / 100);
  }
  return out;
}

/** 2 차 스탯 도출(1 차 → 2 차 변환 공식도 여기서 관리) */
export function deriveSecondaries(primary: Record<string, number>): Record<string,
number> {
  return {
    atk: Math.round(primary.str * 2.2 + primary.dex * 0.6),
    matk: Math.round(primary.int * 2.4),
    def: Math.round(primary.vit * 1.8),
    hp: Math.round(80 + primary.vit * 22 + primary.str * 4),
    mp: Math.round(40 + primary.int * 15),
    crit: Math.min(50, Math.round(primary.dex * 0.25)), // % 상한 50
    aspd: Math.min(190, 100 + Math.round(primary.dex * 0.4)), // % 상한 190
    ...primary,
  };
}

/** 더티 플래그 재계산 혹은 — 장비/버프/레벨 변화 시 호출 */
export function recompute(
  char: { level: number; base: Record<string, number>; allocated: Record<string, number> },
  sources: StatSource[],
): Record<string, number> {
  const base = { ...char.base };
  for (const [k, v] of Object.entries(char.allocated)) base[k] = (base[k] ?? 0) + (v as number);
  const lvlBonus = { str: char.level * 0.4, dex: char.level * 0.4,
    int: char.level * 0.4, vit: char.level * 0.5 };
  for (const [k, v] of Object.entries(lvlBonus)) base[k] = (base[k] ?? 0) + v;
  return deriveSecondaries(computeFinal(base, sources));
}

```

(3) 성능 최적화 팁

- 합산 순서를 절대 바꾸지 마라 — 'flat 먼저, mul 나중'이 산수상 이득이 아니라 '결정론'의 문제다.

- 2차 스탯 상한(crit 50% 등)은 deriveSecondaries 한 곳에서만 관리해야 기획 변경 시 한 줄 수정으로 끝난다.
- 스탯 캐시 테이블의 dirty 플래그는 로그아웃 시 저장, 접속 시 1회 재계산 정책과 결합하면 DB 쓰기를 크게 줄인다.
- 클라미는 서버 산출값을 표시만 하고 자체 계산하지 않는다 — 양쪽 계산은 반드시 공유 코드(이 파일) 하나로.

022. 레벨별 요구 경험치 곡선 및 스탯 상승 공식

(1) 역할 설명

경험치 곡선은 게임 페이스의 뼈대다. 이 시스템은 닫힌 형태의 공식(예: $\text{next} = \text{floor}(60 * L^{2.4} + 120 * L)$)으로 레벨별 요구치를 계산하되, 기획 튜닝을 위해 구간별 보정 계수 테이블(exp_curve 세그먼트)을 DB에 둔다. 스탯 상승은 레벨당 자동 증가분 + 자유 분배 포인트 두 트랙으로 나누고, 성장 곡선 지수(level curve)는 클래스별로 다르게 둔다.

레벨업은 서버에서만 승인한다. 클라가 exp 증가를 보고하면 서버는 사냥 판정 히스토리로 검증(016 번 철학의 확장)하고, 레벨업이 확정되면 스탯 재계산(021 번), 신규 스킬 개방(026 번), 이펙트 알림을 하나의 트랜잭션으로 처리한다. exp 손실 없는 안전장치로, 레벨업 직전 오버플로 exp는 누적해 다음 레벨 요구치에 이월한다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE exp_curve (
  min_level INT NOT NULL,
  max_level INT NOT NULL,
  formula VARCHAR(40) NOT NULL, -- 'poly24' | 'poly30' | 'linear'
  k NUMERIC(10,4) NOT NULL, -- 계수
  stat_point INT NOT NULL DEFAULT 3, -- 레벨당 자유 포인트
  PRIMARY KEY (min_level)
);
INSERT INTO exp_curve (min_level, max_level, formula, k, stat_point) VALUES
(1, 30, 'poly24', 60, 5), -- 초반: 60*L^2.4 — 빠른 성장 체감
(31, 70, 'poly27', 45, 3), -- 중반: 45*L^2.7
(71, 99, 'poly30', 30, 2); -- 후반: 30*L^3.0 — 장기 목표

CREATE TABLE class_growth (
  class_code VARCHAR(20) PRIMARY KEY,
  curve JSONB NOT NULL -- 클래스별 레벨 성장 지수
  -- {"str":0.35,"dex":0.35,"int":0.35,"vit":0.5}
);
```

보조 코드 — src/game/Growth.ts — 경험치 곡선·스탯 상승 연산

```
export interface CurveRow { min_level: number; max_level: number; formula: string; k: number;
stat_point: number; }
```

```

/** 세그먼트 공식 평가 — 서버 부팅 시 전체 테이블을 메모리에 올려 사용 */
export function expForLevel(level: number, curve: CurveRow[]): number {
  const row = curve.find(c => level >= c.min_level && level <= c.max_level)
    ?? curve[curve.length - 1];
  switch (row.formula) {
    case "poly24": return Math.floor(row.k * Math.pow(level, 2.4) + 120 * level);
    case "poly27": return Math.floor(row.k * Math.pow(level, 2.7) + 200 * level);
    case "poly30": return Math.floor(row.k * Math.pow(level, 3.0) + 300 * level);
    default:      return Math.floor(row.k * level * 100);
  }
}

/** 누적 exp → (레벨, 남은 exp) 변환 — 세이브는 누적 exp 만 저장하는 형태 권장 */
export function levelFromTotalExp(total: number, curve: CurveRow[], maxLevel = 99) {
  let level = 1, rest = total;
  while (level < maxLevel) {
    const need = expForLevel(level, curve);
    if (rest < need) break;
    rest -= need; level++;
  }
  return { level, rest, statPoint: level * (curve.find(c => c.min_level <= level)!.stat_point) };
}

/** 레벨업 확정 트랜잭션(서버) */
import { Kysely } from "kysely";
type DB = import("./schema").Database;
export async function applyExp(db: Kysely<DB>, charId: number, gain: number, curve: CurveRow[]) {
  return db.transaction().execute(async tx => {
    const ch = await tx.selectFrom("characters").where("id", "=", charId)
      .select(["level", "exp", "stat_points"]).forUpdate().executeTakeFirstOrThrow();
    const total = Number(ch.exp) + gain;
    const next = levelFromTotalExp(total, curve);
    await tx.updateTable("characters").set({
      level: next.level,
      exp: total,
      stat_points: ch.stat_points + (next.level - ch.level) * 5,
    }).where("id", "=", charId).execute();
    return { leveledUp: next.level > ch.level, level: next.level };
  });
}

```

(3) 성능 최적화 팁

- 요구치 공식은 지수 2.4~3.0 구간이 MMORPG 표준 밴드다 — 2.4 미만은 콘텐츠 소모가 빠르고 3.0 초과는 후반 이탈률이 급증한다.
- 세이브는 '레벨+구간 내 exp'보다 '누적 exp' 단일 값이 안전하다 — 곡선 리밸런싱에도 유저 자산이 보존된다.
- 레벨업 보너스(포인트 5 개 등)는 곡선 세그먼트별로 다르게 두면 구간 페이스 조절이 쉬워진다.

- exp 게인은 서버가 몬스터 처치 판정과 함께만 승인하고 클라 보고 수치를 그대로 더하지 않는다.

023. 스킬 트리 & 특성 포인트 투자/초기화 데이터 구조

(1) 역할 설명

스킬 트리는 '노드(스킬) + 간선(선행 스킬 요구)'의 DAG 이며, 각 노드는 최대 투자 레벨과 레벨별 효과 공식을 가진다. 데이터 구조는 세 계층으로 나뉜다. skill_def(스킬 정의), skill_node(트리 배치·선행 요구), character_skill(투자 상태). 투자는 포인트 소비 트랜잭션으로, 선행 노드 요구 레벨 검증을 서버가 항상 수행한다 — 클라 UI는 미리보기일 뿐이다.

초기화(리스펙)는 유료/무료 정책에 따라 소모 재화를 결정하고, 투자 기록 전체를 한번에 리셋하되 '환급 포인트'를 정확히 합산해 돌려준다. 리스펙은 남용되면 경제 지표가 흔들리므로 주 1회 무료 + 유료 아이템 정책을 서버 규칙으로 강제한다. 트리 구조는 프론트에서 렌더링 가능하도록 x/y 좌표와 프리뷰 아이콘을 정의에 포함한다.

(2) 구현 코드 — PostgreSQL DDL

```
CREATE TABLE skill_def (
  code      VARCHAR(40) PRIMARY KEY,
  name      VARCHAR(40) NOT NULL,
  class_code VARCHAR(20) NOT NULL,
  max_rank  SMALLINT NOT NULL DEFAULT 10,
  type      VARCHAR(12) NOT NULL, -- active | passive | ultimate
  cooldown_ms INT NOT NULL DEFAULT 0,
  cost_mp   INT NOT NULL DEFAULT 0,
  rank_formula JSONB NOT NULL      -- 랭크별 효과: {"dmg": "0.5+0.12*r", ...}
);

CREATE TABLE skill_node (
  skill_code VARCHAR(40) REFERENCES skill_def(code) ON DELETE CASCADE,
  tier        SMALLINT NOT NULL, -- 트리 단계(가로 배치)
  pos_x       SMALLINT NOT NULL, pos_y SMALLINT NOT NULL, -- UI 좌표
  requires    JSONB NOT NULL DEFAULT '[]', -- [{"code": "s001", "rank": 5}]
  PRIMARY KEY (skill_code)
);

CREATE TABLE character_skill (
  character_id BIGINT NOT NULL REFERENCES characters(id) ON DELETE CASCADE,
  skill_code   VARCHAR(40) NOT NULL REFERENCES skill_def(code),
  rank        SMALLINT NOT NULL DEFAULT 0,
  PRIMARY KEY (character_id, skill_code)
);

CREATE TABLE respec_log (
  id          BIGSERIAL PRIMARY KEY,
  character_id BIGINT NOT NULL,
  at          TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```

    refunded    INT NOT NULL,
    cost_kind    VARCHAR(20) NOT NULL    -- free_weekly | cash | gold
);

```

보조 코드 — server/skill/tree.ts — 투자/초기화 연산

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export class TreeError extends Error {
  constructor(code: "NO_POINT" | "PREREQ" | "MAX_RANK" | "LOCKED_CLASS") { super(code); }
}

export async function investSkill(
  db: Kysely<DB>, charId: number, skillCode: string, treeDefs: Map<string, { max_rank:number;
  requires:{code:string;rank:number}[]>>,
) {
  return db.transaction().execute(async tx => {
    const ch = await tx.selectFrom("characters").where("id", "=", charId)
      .select(["stat_points", "class_code"]).forUpdate().executeTakeFirstOrThrow();
    const def = treeDefs.get(skillCode);
    if (!def) throw new TreeError("LOCKED_CLASS");

    // 선행 요구 검증
    for (const req of def.requires) {
      const have = await tx.selectFrom("character_skill")
        .where("character_id", "=", charId).where("skill_code", "=", req.code)
        .select("rank").executeTakeFirst();
      if (!have || have.rank < req.rank) throw new TreeError("PREREQ");
    }

    const cur = await tx.selectFrom("character_skill")
      .where("character_id", "=", charId).where("skill_code", "=", skillCode)
      .select("rank").executeTakeFirst();
    const nextRank = (cur?.rank ?? 0) + 1;
    if (nextRank > def.max_rank) throw new TreeError("MAX_RANK");
    if (ch.stat_points < 1) throw new TreeError("NO_POINT");

    await tx.updateTable("characters").set(eb => ({ stat_points: eb("stat_points", "-", 1) }))
      .where("id", "=", charId).execute();
    await tx.insertInto("character_skill").values({ character_id: charId, skill_code: skillCode, rank:
    nextRank })
      .onConflict(oc => oc.doUpdate().set({ rank: nextRank })).execute();
    return nextRank;
  });
}

/** 리스펙: 전체 현금 + 정책 비용 */
export async function respec(
  db: Kysely<DB>, charId: number, freeAllowed: boolean, costGold: number,
) {

```

```

return db.transaction().execute(async tx => {
  const invested = await tx.selectFrom("character_skill")
    .where("character_id", "=", charId).select(["skill_code", "rank"]).execute();
  const refund = invested.reduce((s, r) => s + r.rank, 0);
  if (!freeAllowed) {
    await tx.updateTable("characters")
      .set(eb => ({ gold: sql`gold - ${costGold}` })))
      .where("id", "=", charId).where("gold", ">=", costGold).executeTakeFirst();
  }
  await tx.deleteFrom("character_skill").where("character_id", "=", charId).execute();
  await tx.updateTable("characters").set(eb => ({ stat_points: eb("stat_points", "+", refund) })))
    .where("id", "=", charId).execute();
  await tx.insertInto("respec_log").values({
    character_id: charId, refunded: refund, cost_kind: freeAllowed ? "free_weekly" : "gold",
  }).execute();
  return refund;
});
}

```

(3) 성능 최적화 팁

- 선행 요구 검증은 반드시 서버 트랜잭션 안에서 — 클라 UI 우회(패킷 위조)의 가장 흔한 표적이다.
- rank_formula 는 문자열 수식 대신 {base, perRank, cap} 구조가 안전하다 — 수식 파서는 튜닝 비용보다 버그 비용이 크다.
- 리스펙 무료 주기는 '주 1 회 접속 기준'으로 리셋해 유저가 부담 없이 실험하게 만들면 빌드 다양성이 커진다.
- 트리 UI 좌표(pos_x/pos_y)를 DB 에 두면 기획이 데이터만으로 트리 배치를 조정할 수 있다.

024. 장비 부위별 착용 슬롯 인벤토리 메커니즘

(1) 역할 설명

인벤토리는 '가방(칸 기반 목록)'과 '장착 슬롯(부위별 1 개씩)'의 이중 구조다. 부위는 weapon/helmet/armor/gloves/boots/necklace/ring1/ring2 등으로 정의하며, ring 처럼 복수 슬롯은 슬롯 코드로 구분한다. 착용은 트랜잭션으로 (1) 슬롯 점유 확인, (2) 기존 장비 탈착, (3) 새 장비 착용, (4) 스탯 캐시 더티 마킹을 한 번에 수행한다.

아이템 행(item row)은 항상 하나의 위치 소속만 갖도록 설계한다(인벤토리 슬롯 번호 또는 장비 부위). 중복 소유, 유령 아이템, 복제 버그의 상당수가 '하나의 아이템이 두 위치에 존재'하는 상태에서 발생하므로, 위치 칼럼을 단일 진실로 두고 이동은 항상 UPDATE 한 문장으로 표현한다. 파티 트레이드/우편 이동도 같은 위치 칼럼을 공유해 데이터 흐름이 단일하다.

(2) 구현 코드 — PostgreSQL DDL

```

CREATE TABLE item_def (
  code      VARCHAR(40) PRIMARY KEY,
  name      VARCHAR(40) NOT NULL,
  category  VARCHAR(16) NOT NULL,    -- weapon|armor|accessory|consumable|material
  equip_slot VARCHAR(16),           -- NULL 이면 착용 불가 소모품/재료
  req_level SMALLINT NOT NULL DEFAULT 1,
  req_class VARCHAR(20),            -- NULL = 전 클래스
  base_mods JSONB NOT NULL,         -- 기본 스탯 [{"stat":"atk","flat":30}]
  max_stack INT NOT NULL DEFAULT 1,  -- 1 = 개별 아이템(강화/옵션 존재)
  rarity     SMALLINT NOT NULL DEFAULT 1 -- 1~5 (컬러/드롭 확률 기준)
);

-- 캐릭터 소유 아이템: 위치의 단일 진실 = (bag_slot NULL ↔ equip_slot NOT NULL)
ALTER TABLE character_items
  ADD CONSTRAINT chk_pos CHECK (
    (slot IS NOT NULL AND bag_slot IS NULL) OR
    (slot IS NULL AND bag_slot IS NOT NULL)
  );
ALTER TABLE character_items
  ADD COLUMN bag_slot SMALLINT;

CREATE UNIQUE INDEX uq_equip_slot ON character_items(character_id, slot)
  WHERE slot IS NOT NULL;          -- 부위당 1 개 강제
CREATE UNIQUE INDEX uq_bag_slot ON character_items(character_id, bag_slot)
  WHERE bag_slot IS NOT NULL;      -- 가방 칸 1 개 1 아이템

```

보조 코드 — server/item/equip.ts — 착용/탈착 트랜잭션

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

const BAG_SIZE = 60;

export class EquipError extends Error {
  constructor(code: "NOT_EQUIPPABLE" | "REQ_FAIL" | "BAG_FULL" | "SLOT_EMPTY") {
    super(code);
  }
}

export async function equipItem(db: Kysely<DB>, charId: number, itemId: number, itemDef:
Map<string, any>) {
  return db.transaction().execute(async tx => {
    const it = await tx.selectFrom("character_items").where("id", "=", itemId)
      .where("character_id", "=", charId).forUpdate().executeTakeFirst();
    if (!it) throw new EquipError("SLOT_EMPTY");
    const def = itemDef.get(it.item_code);
    if (!def?.equip_slot) throw new EquipError("NOT_EQUIPPABLE");

    const ch = await tx.selectFrom("characters").where("id", "=", charId)
      .select(["level", "class_code"]).executeTakeFirstOrThrow();
    if (ch.level < def.req_level || (def.req_class && def.req_class !== ch.class_code))

```

```

    throw new EquipError("REQ_FAIL");

    // 1) 기존 장비 탈착 → 빈 가방 칸으로
    const old = await tx.selectFrom("character_items")
      .where("character_id", "=", charId).where("slot", "=", def.equip_slot)
      .forUpdate().executeTakeFirst();
    if (old) {
      const used = await tx.selectFrom("character_items")
        .where("character_id", "=", charId).where("bag_slot", "is not", null)
        .select("bag_slot").execute();
      const taken = new Set(used.map(r => r.bag_slot));
      let free = -1;
      for (let i = 0; i < BAG_SIZE; i++) if (!taken.has(i)) { free = i; break; }
      if (free < 0) throw new EquipError("BAG_FULL");
      await tx.updateTable("character_items").set({ slot: null, bag_slot: free })
        .where("id", "=", old.id).execute();
    }
    // 2) 새 장비 착용(부분 유니크 인덱스가 부위당 1 개를 물리 강제)
    await tx.updateTable("character_items").set({ slot: def.equip_slot, bag_slot: null })
      .where("id", "=", itemId).execute();

    // 3) 스탯 캐시 더티(021 번)
    await tx.updateTable("character_stats_cache").set({ dirty: true })
      .where("character_id", "=", charId).execute();
    return def.equip_slot;
  });
}

```

(3) 성능 최적화 팁

- 부분 유니크 인덱스(WHERE slot IS NOT NULL)는 '부위당 1 개'를 DB 가 물리적으로 보장하는 가장 강력한 방법이다.
- CHECK 제약으로 위치 칼럼 상호배타(가방 XOR 장착)를 강제해 복제 버그의 구조적 원인을 차단한다.
- 가방 칸 이동(swap/drag)은 클라 UI 편의일 뿐이며 서버는 항상 '아이템 id + 목표 칸'으로 재검증한다.
- 스택 가능 아이템(max_stack>1)은 분할 이동(qty 나누기)까지 트랜잭션으로 처리해야 재료 소모 버그가 사라진다.

025. 장비 강화/인챈트 확률 및 스탯 부여 연산

(1) 역할 설명

강화는 '단계별 성공률 + 실패 시 결과(유지/하락/파괴)'의 확률 표이며, 인챈트는 '속성 풀에서 가중치 랜덤 추출'의 문제다. 두 시스템 모두 서버가 결정론적 RNG(seeded)로 수행하고, 시도 로그를 남겨 유저 클레임 시 재현 검증이 가능하게 한다. 확률은 절대 클라에서 계산해 보고하지 않는다 — 서버가 표를 조회해 결과만 통보한다.

보상 설계로 천장(pity) 개념을 강화에도 도입할 수 있다. 연속 실패 횟수가 쌓이면 다음 성공률에 보정(+2%p 씩)을 주는 연속 실패 보정(soft pity)은 유저 이탈을 크게 줄이는 표준 장치다. 강화 결과 스탯은 mods JSONB 에 누적되며, 021 번 StatMath 의 equip 소스로 자연 합산된다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE enhance_table (
  level      SMALLINT PRIMARY KEY,    -- 강화 단계 0→1, 1→2 ...
  success_rate NUMERIC(5,4) NOT NULL, -- 0.9 = 90%
  on_fail     VARCHAR(10) NOT NULL,   -- keep | downgrade | destroy
  cost_gold   INT NOT NULL,
  material    JSONB NOT NULL          -- [{"code":"mat_ore","qty":3}]
);
INSERT INTO enhance_table (level, success_rate, on_fail, cost_gold, material) VALUES
(0, 1.0000, 'keep', 1000, '[{"code":"mat_ore","qty":1}]'),
(1, 0.9000, 'keep', 2000, '[{"code":"mat_ore","qty":2}]'),
(2, 0.8000, 'keep', 4000, '[{"code":"mat_ore","qty":4}]'),
(3, 0.7000, 'downgrade', 8000, '[{"code":"mat_ore","qty":8}]'),
(4, 0.5500, 'downgrade', 16000, '[{"code":"mat_ore","qty":16}]'),
(5, 0.4000, 'destroy', 32000, '[{"code":"mat_ore","qty":32}]');

CREATE TABLE enchant_pool (
  id      BIGSERIAL PRIMARY KEY,
  tier     SMALLINT NOT NULL,      -- 인챈트 등급 풀
  stat    VARCHAR(16) NOT NULL,
  min_val INT NOT NULL, max_val INT NOT NULL,
  weight  INT NOT NULL           -- 가중치 랜덤
);
INSERT INTO enchant_pool (tier, stat, min_val, max_val, weight) VALUES
(1, 'atk', 5, 12, 40), (1, 'crit', 1, 3, 25), (1, 'hp', 40, 90, 35);

ALTER TABLE character_items
ADD COLUMN pity_fail SMALLINT NOT NULL DEFAULT 0; -- 연속 실패 보정
```

보조 코드 — server/item/enhance.ts — 강화·인챈트 연산

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

/** 강화 시도 — 서버 확률표 + 연속 실패 보정 + 원자적 소모 */
export async function enhance(db: Kysely<DB>, charId: number, itemId: number, table: any[]) {
  return db.transaction().execute(async tx => {
    const it = await tx.selectFrom("character_items").where("id", "=", itemId)
      .where("character_id", "=", charId).forUpdate().executeTakeFirstOrThrow();
    const row = table[it.enhance]; // 현재 단계 → 다음 단계 확률
    if (!row) throw new Error("MAX_ENHANCE");

    // 재료·골드 소모(부족 시 예외 — CHECK 제약이 음수 차단)
    await tx.updateTable("characters").set(eb => ({ gold: eb("gold", "-", row.cost_gold) })))
      .where("id", "=", charId).where("gold", ">=", row.cost_gold).execute();
```



```

for (const m of row.material) {
  const upd = await tx.updateTable("character_items")
    .set(eb => ({ qty: eb("qty", "-", m.qty) }))
    .where("character_id", "=", charId).where("item_code", "=", m.code)
    .where("qty", ">=", m.qty).execute();
  if (Number(upd.numUpdatedRows) === 0n) throw new Error("NO_MATERIAL");
}

// 연속 실패 보정(soft pity): 실패마다 +2%p, 최대 +15%p
const pity = Math.min(15, (it.pity_fail ?? 0) * 2) / 100;
const roll = Math.random();
const success = roll < Number(row.success_rate) + pity;

if (success) {
  // 강화 보너스: 주 스탯 flat * (1 + 0.12 * 다음 단계)
  const bonus = 1 + 0.12 * (it.enhance + 1);
  await tx.updateTable("character_items")
    .set({ enhance: it.enhance + 1, pity_fail: 0 })
    .where("id", "=", itemId).execute();
  return { ok: true, level: it.enhance + 1, bonusMult: bonus };
}
if (row.on_fail === "destroy") {
  await tx.deleteFrom("character_items").where("id", "=", itemId).execute();
  return { ok: false, result: "destroy" };
}
if (row.on_fail === "downgrade" && it.enhance > 0) {
  await tx.updateTable("character_items")
    .set({ enhance: it.enhance - 1, pity_fail: it.pity_fail + 1 })
    .where("id", "=", itemId).execute();
  return { ok: false, result: "downgrade", level: it.enhance - 1 };
}
await tx.updateTable("character_items").set({ pity_fail: it.pity_fail + 1 })
  .where("id", "=", itemId).execute();
return { ok: false, result: "keep" };
});
}

/** 인첸트: 가중치 랜덤(WRS)으로 풀에서 1 개 추출 */
export function rollEnchant(pool: {stat:string; min_val:number; max_val:number;
weight:number}[]) {
  const total = pool.reduce((s, p) => s + p.weight, 0);
  let r = Math.random() * total;
  for (const p of pool) {
    r -= p.weight;
    if (r <= 0) {
      const val = p.min_val + Math.floor(Math.random() * (p.max_val - p.min_val + 1));
      return { stat: p.stat, flat: val };
    }
  }
  return { stat: pool[0].stat, flat: pool[0].min_val };
}

```

(3) 성능 최적화 팁

- 성공률 계산은 반드시 서버 Math.random(또는 CSPRNG)으로 하고, roll 값을 강화 로그에 함께 기록해 클레임 검증을 가능하게 한다.
- soft pity(+2%p)는 파괴 단계부터 적용하면 유저 불만이 눈에 띄게 줄고 경제 영향은 미미하다.
- 재료 소모는 UPDATE ... WHERE qty >= n 의 영향 행 수로 검증한다 — SELECT 후 UPDATE 방식은 동시 요청에서 음수 재고가 나온다.
- 인챈트 가중치는 stat 별 파워 밸런스가 흔들리지 않게 티어별로 관리하며, 최상위 티어는 이벤트로만 풀에 추가한다.

026. 클래스 전직 시스템 및 스킬 개방 테이블

(1) 역할 설명

전직은 '기반 클래스 → 1차 → 2차 → 각성'의 계층 구조로, 각 전직은 요구 레벨·선행 퀘스트·전용 던전 클리어 조건을 가진다. 데이터는 class_def(계층·부모 클래스)와 job_change_path(전직 경로·조건)로 정규화하고, 스킬 개방은 class_skill_unlock(클래스별 스킬 + 개방 레벨) 테이블로 관리한다. 전직 실행은 트랜잭션으로 조건 검증 → 클래스 치환 → 스킬 트리 초기화 → 외형 리소스 교체를 수행한다.

클래스 계층이 parent 참조(directed tree)로 되어 있으면, familyOf 같은 계열 판별(파티 스킬 공유, 무기 호환)이 O(1) 조회로 해결된다. 전직 시 기존 투자 스킬 포인트는 전직 전 클래스 스킬은 유지하되 신규 클래스 트리에 포인트를 추가 지급하는 정책(무료 리스펙 1 회 동봉)이 유저 경험상 표준이다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE class_def (
  code      VARCHAR(20) PRIMARY KEY, -- novice|warrior|mage|knight|pyro
  parent    VARCHAR(20) REFERENCES class_def(code), -- 계층(계열 판별용)
  tier      SMALLINT NOT NULL,        -- 0 기반, 1 1차, 2 2차
  base_mods JSONB NOT NULL,           -- 클래스 기본 스탯 성향
  sprite_set VARCHAR(40) NOT NULL     -- 기본 외형 세트
);
INSERT INTO class_def (code, parent, tier, base_mods, sprite_set) VALUES
('novice', NULL, 0, '{"str":5,"dex":5,"int":5,"vit":5}', 'cs_novice'),
('warrior', 'novice', 1, '{"str":9,"vit":7}', 'cs_warrior'),
('mage', 'novice', 1, '{"int":9,"dex":6}', 'cs_mage'),
('knight', 'warrior', 2, '{"str":11,"vit":10}', 'cs_knight'),
('pyro', 'mage', 2, '{"int":12,"dex":7}', 'cs_pyro');

CREATE TABLE job_change_path (
  from_class VARCHAR(20) REFERENCES class_def(code),
  to_class   VARCHAR(20) REFERENCES class_def(code),
  req_level  SMALLINT NOT NULL,
```

```

    req_quest VARCHAR(40),          -- 선행 퀘스트 코드
    PRIMARY KEY (from_class, to_class)
);

CREATE TABLE class_skill_unlock (
    class_code VARCHAR(20) REFERENCES class_def(code),
    skill_code VARCHAR(40) REFERENCES skill_def(code),
    unlock_lv SMALLINT NOT NULL,
    PRIMARY KEY (class_code, skill_code)
);

```

보조 코드 — server/class/jobChange.ts — 전직 실행

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface PathDef {
    from_class: string; to_class: string; req_level: number; req_quest: string | null;
}

export class JobChangeError extends Error {
    constructor(code: "WRONG_CLASS" | "LOW_LEVEL" | "QUEST_INCOMPLETE") { super(code); }
}

export async function jobChange(
    db: Kysely<DB>, charId: number, toClass: string,
    paths: PathDef[], unlocks: Map<string, {skill_code:string; unlock_lv:number}[]>,
    clearQuest: (charId: number, questCode: string) => boolean,
) {
    return db.transaction().execute(async tx => {
        const ch = await tx.selectFrom("characters").where("id", "=", charId)
            .forUpdate().select(["class_code", "level"]).executeTakeFirstOrThrow();
        const path = paths.find(p => p.to_class === toClass && p.from_class === ch.class_code);
        if (!path) throw new JobChangeError("WRONG_CLASS");
        if (ch.level < path.req_level) throw new JobChangeError("LOW_LEVEL");
        if (path.req_quest && !clearQuest(charId, path.req_quest))
            throw new JobChangeError("QUEST_INCOMPLETE");

        // 1) 클래스 치환
        await tx.updateTable("characters").set({ class_code: toClass })
            .where("id", "=", charId).execute();

        // 2) 신규 클래스 스킬 트리 시드(개방 레벨 도달분만 유효, 포인트는 유지)
        const newSkills = (unlocks.get(toClass) ?? []).
            filter(u => u.unlock_lv <= ch.level);
        for (const s of newSkills) {
            await tx.insertInto("character_skill")
                .values({ character_id: charId, skill_code: s.skill_code, rank: 0 })
                .onConflict(oc => oc.doNothing()).execute();
        }
    });
}

```

```

// 3) 외형 교체 + 스탯 캐시 더티
await tx.updateTable("character_stats_cache").set({ dirty: true })
  .where("character_id", "=", charId).execute();
return { from: ch.class_code, to: toClass };
});
}

/** 계열 판별(파티 스킬 공유 등): parent 체인 상속판별 */
export function familyOf(code: string, classes: Map<string, {parent: string|null}>): string {
  let cur: string | null = code;
  while (cur) {
    const p = classes.get(cur)?.parent ?? null;
    if (p === "novice" || p === null) return cur; // 1 차 클래스를 계열 대표로
    cur = p;
  }
  return "novice";
}

```

(3) 성능 최적화 팁

- 전직 조건(레벨+퀘스트) 검증은 트랜잭션 내 forUpdate 로 — 조건 검증과 치환 사이에 레벨 조작이 끼어들 수 없다.
- 클래스 계층은 parent 참조 하나로 계열 판별·무기 호환·스킬 상속 전부 해결되므로, 별도 '계열' 칼럼을 만들지 마라.
- 전직 후 자동 리스펙 1 회 제공은 빌드 실패 불안을 제거하는 표준 배려다 — respec_log 에 cost_kind='jobchange'로 기록한다.
- 스킬 개방 테이블은 클라 프리뷰(자물쇠 아이콘)에도 그대로 쓰이므로, 부팅 시 통째로 캐시해 조회 비용을 0 으로 만든다.

027. 외형(코스튬/스킨) 레이어 분리 및 착용 처리

(1) 역할 설명

캐릭터 외형은 '기반 신체(body) + 장비 스킨(장비가 그대로 보이는 형태) + 코스튬(장비 위에 덮는 겂옷) + 이펙트(오라)'의 레이어 스택이다. 렌더 우선순위는 코스튬 > 장비 스킨 > 기반 신체이며, 각 레이어는 파트별(머리/몸통/다리/무기) 스프라이트 키를 조합해 완성된다. 파트 분리가 핵심인 이유는 색상 변형(팔레트 스왑)과 코스튬 부분 가림(투명 헬멧 등)을 데이터만으로 지원하기 위해서다.

착용 판정은 cosmetic 슬롯을 별도 테이블로 둔다 — 장비 스탯(024 번)과 외형 표시를 분리하면 '스탯 없는 코스튬', '보이지 않는 장비(언디스가이즈)'가 자연스럽게 구현된다. 외형 조합 결과는 해시 키로 캐시해 같은 조합의 다수 유저가 한 맵에 있을 때 텍스처 아틀라스 캐시(001/009 번)가 유효하게 만든다.

(2) 구현 코드 — PostgreSQL DDL

```

CREATE TABLE cosmetic_def (
  code    VARCHAR(40) PRIMARY KEY,
  part     VARCHAR(12) NOT NULL,    -- body|head|upper|lower|weapon|aura
  layer    SMALLINT NOT NULL,      -- 렌더 순서: 낮을수록 아래(0 body, 10 skin, 20 costume, 30
aura)
  atlas_key VARCHAR(40) NOT NULL,   -- 스프라이트 아틀라스
  palette  JSONB NOT NULL DEFAULT '[]', -- 색상 변형 프리셋
  rarity    SMALLINT NOT NULL DEFAULT 1
);

CREATE TABLE character_cosmetics (
  character_id BIGINT NOT NULL REFERENCES characters(id) ON DELETE CASCADE,
  part         VARCHAR(12) NOT NULL,
  cosmetic_code VARCHAR(40) REFERENCES cosmetic_def(code),
  palette_idx  SMALLINT NOT NULL DEFAULT 0,
  PRIMARY KEY (character_id, part)    -- 파트당 1 개 착용
);

```

보조 코드 — src/game/Appearance.ts — 클라 렌더 조합

```

export interface CosPart {
  part: "body" | "head" | "upper" | "lower" | "weapon" | "aura";
  layer: number;
  atlasKey: string;
  paletteIdx: number;
}

export interface EquipVisual { part: string; atlasKey: string; } // 장비에서 온 스킨

/** 외형 스택 조합: 코스튬 > 장비 스킨 > 기반 신체 */
export function buildAppearance(
  base: { body: string; head: string },
  equips: EquipVisual[],
  cosmetics: CosPart[],
): CosPart[] {
  const layers: CosPart[] = [
    { part: "body", layer: 0, atlasKey: base.body, paletteIdx: 0 },
    { part: "head", layer: 0, atlasKey: base.head, paletteIdx: 0 },
  ];

  for (const e of equips) { // 장비 스킨 = layer 10
    layers.push({ part: e.part as CosPart["part"], layer: 10,
      atlasKey: e.atlasKey, paletteIdx: 0 });
  }

  for (const c of cosmetics) { // 코스튬 = layer 20, 오라 = 30
    layers.push(c);
  }

  return layers.sort((a, b) => a.layer - b.layer);
}

/** 조합 해시 → 아틀라스 캐시 키(009 번 MemorySweeper 보호 대상 마킹용) */

```

```
export function appearanceKey(layers: CosPart[]): string {
  return layers.map(l => l.atlasKey + "#" + l.paletteIdx).join("|");
}

// Phaser 렌더: 부위별 스프라이트를 컨테이너에 y-sort 없이 순서대로 add
// const container = scene.add.container(x, y);
// for (const layer of layers) {
//   const s = scene.add.sprite(0, 0, layer.atlasKey, frameName);
//   s.setTint(palettes[layer.paletteIdx] ?? 0xffffffff);
//   container.add(s);
// }
// 컨테이너 depth는 Y-Sorting 엔진(042 번)이 캐릭터 단위로 관리
```

(3) 성능 최적화 팁

- 스탯 슬롯(024)과 코스튬 슬롯을 테이블부터 분리해야 '같은 코스튬, 속은 강화 장비'가 설계 오염 없이 구현된다.
- 파트별 아틀라스는 전 캐릭터 공용으로 관리하면 외형 조합 수백 개에도 텍스처 수가 파트 수준으로 유지된다.
- palette JSONB 로 색상 변형을 지원하면 같은 코스튬의 컬러 변형이 레코드 추가 없이 끝난다.
- appearanceKey 캐시는 파티 화면·길드 배너 등 '다수 캐릭터 동시 표시' 시 렌더 비용을 크게 낮춘다.

028. 칭호/업적 달성 조건 및 패시브 스탯 부여 연산

(1) 역할 설명

업적(achievement)은 이벤트 기반 카운터(사냥 1000 마리, 퀘스트 100 개 등)와 조건 매칭으로 달성 되고, 칭호(title)는 업적 달성 보상의 대표 형태다. 데이터 구조는 achievement_def(조건 DSL), achievement_progress(캐릭터 카운터), title_def(패시브 스탯 부여)로 나뉜다. 조건 DSL 은 { event, target, count }의 단순 구조로 유지해 신규 업적 추가가 데이터 삽입만으로 가능하게 한다.

패시브 부여는 '장착 중인 칭호 1 개'의 mods 를 StatMath(021 번)의 passive 소스로 주입하는 방식이며, 도감·멘토링 등 전역 패시브와 같은 파이프라인을 공유한다. 진행도 업데이트는 게임 이벤트 버스에 비동기 리스너로 걸어 전투 핫패스를 절대 막지 않고, 달성 판정은 배치(틱당 큐 소화)로 수행한다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE achievement_def (
  code    VARCHAR(40) PRIMARY KEY,
  name    VARCHAR(40) NOT NULL,
  event   VARCHAR(30) NOT NULL,    -- "kill" | "quest_clear" | "gold_earned" ...
  target  VARCHAR(40),             -- 몬스터 코드/퀘스트 코드(NULL = 전체)
```

```

count    INT NOT NULL,          -- 목표 횟수
title_code VARCHAR(40),          -- 달성 시 지급 칭호
rewards   JSONB NOT NULL DEFAULT '[]'
);

CREATE TABLE achievement_progress (
  character_id BIGINT NOT NULL REFERENCES characters(id) ON DELETE CASCADE,
  achievement_code VARCHAR(40) REFERENCES achievement_def(code),
  progress     INT NOT NULL DEFAULT 0,
  completed    BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (character_id, achievement_code)
);

CREATE TABLE title_def (
  code      VARCHAR(40) PRIMARY KEY,
  name      VARCHAR(30) NOT NULL,
  mods      JSONB NOT NULL          -- 패시브: [{"stat":"atk","mul":0.03}]
);

```

보조 코드 — server/achievement/Engine.ts — 이벤트 구동 판정

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface AchDef {
  code: string; event: string; target: string | null; count: number;
  title_code: string | null; rewards: any[];
}

/** 게임 이벤트 → 업적 진행도 갱신(비동기 큐 — 전투 패스를 막지 않음) */
export class AchievementEngine {
  private queue: { charId: number; event: string; target: string; n: number }[] = [];
  private byEvent = new Map<string, AchDef[]>();

  constructor(private db: Kysely<DB>, defs: AchDef[]) {
    private onTitle: (charId: number, titleCode: string) => Promise<void> = {
      for (const d of defs) {
        const list = this.byEvent.get(d.event) ?? [];
        list.push(d);
        this.byEvent.set(d.event, list);
      }
    };
  }

  /** 월드 서버가 emit: onEnemyKilled/questClear 등에서 호출 */
  push(charId: number, event: string, target: string, n = 1) {
    this.queue.push({ charId, event, target, n });
  }

  /** RenderLoop 고정 업데이트 또는 100ms 타이머에서 소화 */
  async flush() {
    const batch = this.queue.splice(0, this.queue.length);
  }
}

```

```

if (!batch.length) return;
const db = this.db;
for (const item of batch) {
  const defs = this.byEvent.get(item.event) ?? [];
  for (const d of defs) {
    if (d.target && d.target !== item.target) continue;
    // UPSERT 진행도
    await db.insertInto("achievement_progress")
      .values({ character_id: item.charId, achievement_code: d.code,
        progress: item.n, completed: item.n >= d.count })
      .onConflict(oc => oc
        .doUpdate().set(eb => ({
          progress: eb("progress", "+", item.n),
          completed: eb("progress", "+", item.n) >= d.count ? true : false,
        }))).execute();
    // 달성 확정 처리
    const row = await db.selectFrom("achievement_progress")
      .where("character_id", "=", item.charId)
      .where("achievement_code", "=", d.code)
      .select(["progress", "completed"]).executeTakeFirst();
    if (row && row.completed && row.progress === item.n) { // 방금 달성
      if (d.title_code) await this.onTitle(item.charId, d.title_code);
      // rewards 지급은 재화 트랜잭션(018 번)으로
    }
  }
}
}

/** 장작 칭호 패시브 → StatMath 소스로 변환 */
static titleSource(mods: { stat: string; flat?: number; mul?: number }[]) {
  return { kind: "passive" as const, mods };
}
}

```

(3) 성능 최적화 팁

- 업적 진행도 갱신은 반드시 비동기 큐+배치로 — 킬당 DB 쓰기가 동기로 들어가면 대규모 사냥에서 DB가 병목이 된다.
- 조건 DSL은 {event, target, count} 3 필드로 제한한다 — 범용 조건 엔진은 만들수록 디버깅 비용이 폭증한다.
- '방금 달성' 판정(progress == n)으로 지급 중복을 막되, 재시작 후에도 completed 플래그로 이중 지급을 차단한다.
- 칭호 패시브는 StatMath passive 소스 하나로 주입하므로, 도감/멘토링/VIP 등 전역 패시브도 같은 인터페이스를 재사용한다.

029. 장비 무작위 옵션 리롤(Random Option Reroll) 로직

(1) 역할 설명

랜덤 옵션(RO)은 같은 아이템 코드라도 획득 시 가중치 기반으로 서로 다른 보조 스탯을 갖게 하는 재미 장치다. 구성은 옵션 풀(속성·최소/최대치·가중치·티어)과 '개수 보장 + 티어 분포' 규칙으로 이루어진다. 리롤은 기존 옵션을 지우고 재추출하되, '리롤 방지 옵션(잠금)'을 유료 재화로 판매하는 것이 표준 BM 구조다.

추출 알고리즘은 (1) 옵션 개수를 래리티별로 결정(일반 1~2 개, 유니크 3 개), (2) 중복 없이 가중치 랜덤(WRS)으로 속성 선택, (3) 속성별 값 범위에서 균등 추출, (4) 티어 보정(고래 아이템은 상위 티어 가중치 상향)의 4 단계다. 결과는 item.mods 에 기록되며 021 번 StatMath 의 equip 소스로 합산된다. 리롤 연산은 서버 전용이고, 클라는 결과 프리뷰만 받는다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE ro_pool (
  id      BIGSERIAL PRIMARY KEY,
  tier     SMALLINT NOT NULL,      -- 1 일반 ~ 3 유니크
  stat     VARCHAR(16) NOT NULL,
  min_val  INT NOT NULL, max_val INT NOT NULL,
  weight   INT NOT NULL
);
INSERT INTO ro_pool (tier, stat, min_val, max_val, weight) VALUES
(1, 'atk', 3, 8, 40), (1, 'hp', 30, 70, 35),
(2, 'crit', 2, 4, 20), (2, 'aspd', 3, 6, 15),
(3, 'atk', 10, 18, 8), (3, 'matk', 10, 18, 8);

ALTER TABLE character_items
ADD COLUMN ro_options JSONB NOT NULL DEFAULT '[]', -- [{stat,val,tier}]
ADD COLUMN ro_locks SMALLINT[] NOT NULL DEFAULT '{}'; -- 잠금 인덱스
```

보조 코드 — server/item/randomOption.ts — 추출·리롤

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface RoOption { stat: string; val: number; tier: number; }
export interface RoPoolRow { id: number; tier: number; stat: string; min_val: number; max_val: number; weight: number; }

/** 래리티별 옵션 개수 규칙 */
const COUNT_BY_RARITY: Record<number, number> = { 1: 1, 2: 1, 3: 2, 4: 2, 5: 3 };
/** 티어 분포 가중치(래리티별 상위 티어 확률) */
const TIER_DIST: Record<number, [number, number, number]> = {
  1: [80, 18, 2], 2: [70, 25, 5], 3: [60, 30, 10], 4: [50, 35, 15], 5: [40, 40, 20],
};

function pickTier(rarity: number): number {
  const [w1, w2, w3] = TIER_DIST[rarity] ?? TIER_DIST[1];
```

```

let r = Math.random() * (w1 + w2 + w3);
if ((r - w1) <= 0) return 1;
if ((r - w2) <= 0) return 2;
return 3;
}

function pickWeighted(pool: RoPoolRow[]): RoPoolRow {
  const total = pool.reduce((s, p) => s + p.weight, 0);
  let r = Math.random() * total;
  for (const p of pool) if ((r - p.weight) <= 0) return p;
  return pool[pool.length - 1];
}

/** 옵션 세트 추출: 티어 결정 → 중복 없는 속성 WRS → 값 균등 추출 */
export function rollOptions(rarity: number, pool: RoPoolRow[]): RoOption[] {
  const count = COUNT_BY_RARITY[rarity] ?? 1;
  const used = new Set<string>();
  const out: RoOption[] = [];
  for (let i = 0; i < count; i++) {
    const tier = pickTier(rarity);
    const tiered = pool.filter(p => p.tier === tier && !used.has(p.stat));
    if (!tiered.length) continue;
    const pick = pickWeighted(tiered);
    used.add(pick.stat);
    out.push({ stat: pick.stat,
      val: pick.min_val + Math.floor(Math.random() * (pick.max_val - pick.min_val + 1)),
      tier });
  }
  return out;
}

/** 리롤: 잠금 인덱스 유지 + 재화 소모 */
export async function reroll(db: Kysely<DB>, charId: number, itemId: number,
  pool: RoPoolRow[], costGold: number) {
  return db.transaction().execute(async tx => {
    const it = await tx.selectFrom("character_items").where("id", "=", itemId)
      .where("character_id", "=", charId).forUpdate().executeTakeFirstOrThrow();
    await tx.updateTable("characters").set(eb => ({ gold: eb("gold", "-", costGold) }));
    .where("id", "=", charId).where("gold", ">=", costGold).execute();

    const fresh = rollOptions(3, pool); // 실제로는 item_def.rarity 사용
    // 잠금된 슬롯은 기존 옵션 유지
    const locked = new Set(it.ro_locks ?? []);
    it.ro_options.forEach((old: RoOption, i: number) => {
      if (locked.has(i)) fresh[i] = old;
    });
    await tx.updateTable("character_items").set({ ro_options: fresh })
      .where("id", "=", itemId).execute();
    return fresh;
  });
}

```

(3) 성능 최적화 팁

- 중복 없는 속성 추출(used Set)은 'atk 3 개' 같은 쓰레기 옵션을 구조적으로 막는다.
- 잠금 리콜은 남은 후보 수가 줄어들수록 가치가 올라가므로, 잠금 비용을 지수적으로 (1.5^n) 설계한다.
- 추출 로그(아이템 id, 시드, 결과)를 남기면 '내 확률 거짓말' 클레임에 데이터로 대응할 수 있다.
- 옵션 밸런스는 티어 간 밸류 차이를 2~2.5 배로 유지해야 고티어 욕심이 기능한다.

030. 도감/수집품 달성률 계산 및 전역 보상 연산

(1) 역할 설명

도감(codex)은 몬스터·장비·지역 등 수집 카테고리별 등록률을 추적하고, 달성률 구간 (10%, 30%, 50%...)에 전역 패시브(모든 스탯 +1% 등)를 부여한다. 데이터는 codex_entry(수집 대상 마스터), character_codex(등록 상태), codex_reward_tier(달성률 구간 보상)로 구성된다. 달성률은 카테고리별 가중 평균으로 계산해 '몬스터 도감은 반영 낮게, 장비 도감은 높게' 같은 기획 튜닝이 가능하다.

등록 이벤트(첫 처치, 첫 획득)는 028 번 엔진과 동일한 비동기 큐로 처리하고, 달성률 구간 통과 시 보상을 재화 트랜잭션(018 번)으로 지급한다. 전역 패시브는 StatMath(021 번)의 passive 소스로 매 접속 시 재산출되며, 도감 화면의 진행도 UI는 캐시된 달성률 값을 그대로 표시한다. 재계산은 등록 이벤트 직후 1 회만 수행한다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE codex_entry (
  id          BIGSERIAL PRIMARY KEY,
  category    VARCHAR(16) NOT NULL,    -- monster | equipment | region | recipe
  ref_code    VARCHAR(40) NOT NULL,    -- 몬스터 코드 등
  weight      SMALLINT NOT NULL DEFAULT 1, -- 달성률 가중치
  UNIQUE (category, ref_code)
);

CREATE TABLE character_codex (
  character_id BIGINT NOT NULL REFERENCES characters(id) ON DELETE CASCADE,
  entry_id     BIGINT NOT NULL REFERENCES codex_entry(id),
  registered_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (character_id, entry_id)
);

CREATE TABLE codex_reward_tier (
  category    VARCHAR(16) NOT NULL,
  pct         SMALLINT NOT NULL,      -- 10 = 10%
  mods        JSONB NOT NULL,         -- [{"stat": "atk", "mul": 0.01}]
  rewards     JSONB NOT NULL DEFAULT '[]',
  PRIMARY KEY (category, pct)
```

```
);
```

보조 코드 — server/codex/Progress.ts — 달성률·보상 연산

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface TierRow { category: string; pct: number; mods: any[]; rewards: any[]; }

export class CodexService {
  constructor(private db: Kysely<DB>, private tiers: TierRow[]) {}

  /** 등록 + 즉시 달성률 재계산(등록은 드물므로 동기 처리 허용) */
  async register(charId: number, category: string, refCode: string) {
    return this.db.transaction().execute(async tx => {
      const entry = await tx.selectFrom("codex_entry")
        .where("category", "=", category).where("ref_code", "=", refCode)
        .select(["id", "weight"]).executeTakeFirst();
      if (!entry) return null;

      await tx.insertInto("character_codex")
        .values({ character_id: charId, entry_id: entry.id })
        .onConflict(oc => oc.doNothing()).execute();

      const before = await this.pctOf(tx, charId);
      await this.grantTiers(tx, charId, before);
      return before;
    });
  }

  /** 카테고리별 가중 달성률(%) */
  private async pctOf(tx: any, charId: number): Promise<Record<string, number>> {
    const rows = await tx.selectFrom("codex_entry as e")
      .leftJoin("character_codex as c", join => join
        .onRef("c.entry_id", "=", "e.id")
        .on("c.character_id", "=", charId))
      .select(["e.category", (eb: any) =>
        eb.fn.sum(eb.case().when("c.character_id", "is not", null)
          .then(eb.ref("e.weight")).else(0).end()).as("got"),
        (eb: any) => eb.fn.sum("e.weight").as("total"))]
      ).groupBy("e.category").execute();
    const out: Record<string, number> = {};
    for (const r of rows) out[r.category] = Math.floor((r.got / r.total) * 100);
    return out;
  }

  /** 새로 통과한 구간 보상 지급(중복 지급 방지: ledger ref로 검증) */
  private async grantTiers(tx: any, charId: number, pct: Record<string, number>) {
    for (const t of this.tiers) {
      if ((pct[t.category] ?? 0) < t.pct) continue;
      const ref = "codex:" + t.category + ":" + t.pct;
      const paid = await tx.selectFrom("ledger")
```

```

.where("kind", "=", "gold").where("ref", "=", ref)
.where("from_user", "is", null)
.where(sql`payload:jsonb ->> 'char'`, "=", String(charId))
.select("id").executeTakeFirst();
if (paid) continue; // 이미 지급
await tx.insertInto("ledger").values({
  at: new Date(), kind: "gold", from_user: null, to_user: charId,
  ref, payload: JSON.stringify({ char: charId, tier: t }),
}).execute();
}
}
}

// 전역 패시브 소스(021 번 StatMath 에 주입):
// sources.push({ kind: "passive", mods: flatMapOfGrantedTiers(...) })

```

(3) 성능 최적화 팁

- 달성률은 단순 등록 수 비율 대신 가중 평균으로 — 하드 코어 대상에 무게를 두면 도감이 '장기 목표'로 기능한다.
- 구간 보상 지급은 ledger ref('codex:monster:50')로 먹등성을 확보한다 — 재계산이 몇 번 돌아도 이중 지급이 없다.
- 전역 패시브는 접속 시 1 회 재산출로 충분하다 — 매 전투마다 DB 를 치지 않게 캐시 (021 번 더티 플래그)를 유지한다.
- 도감 UI 는 카테고리별 진행도를 한 번의 쿼리(GROUP BY)로 받는다 — 카테고리별 N+1 쿼리는 반드시 피한다.

제 5 장 모듈 4 — 전투 및 액션 메커니즘 (031~040)

기술 스택: TypeScript · Phaser 3/PixiJS 물리 · uWebSockets.js 본 장은 전투 판정의 정확성과 손맛을 동시에 만드는 계층이다. 3 형태 히트박스(원/부채꼴/직사각형)와 브로드페이스 2 단 판정이 성능을 확보하고, 쿨타임·자원 관리자와 CC 상태머신이 전투 규칙의 단일 진실이 된다. 어그로 AI·파티 시너지·속성 상성·피드백 연출은 전투를 게임으로 만드는 요소로, 모든 판정은 서버 권위(015~016 번)에 종속된다.

031. 2D Hitbox / Hurtbox 충돌 판정 (원, 부채꼴, 직사각형)

(1) 역할 설명

탑다운 MMORPG 의 스킬 판정은 세 형태로 충분하다. 원(AOE 폭발), 부채꼴(전방 베기), 회전 직사각형(직선 찌르기/휩쓸기)이다. 판정은 모두 '포인트(대상 중심) vs 세이프' 또는 '원 vs 세이프'의 거리 계산으로 해결하며, 서버(015 번 LagComp)와 클라(프리뷰 이펙트)가 동일 함수를 공유한다. 범위 판정은 스킬 정의에 shape+r/angle/width 를 데이터로 두어 기획 조정이 코드 수정 없이 가능해야 한다.

성능을 위해 판정 전 반드시 넓은 단계(broad phase)를 거친다. 먼저 원 반경(스킬 최대 사거리)으로 대상을 거르고, 남은 소수 대상에만 정밀 기하(부채꼴 각도, OBB)를 적용한다. 대상 200~300 개 기준으로 이 2 단계 구조는 전체 정밀 판정 대비 수십 배 빠르다.

(2) 구현 코드 — src/combat/hitbox.ts — 3 형태 판정 기하

```
export interface Vec { x: number; y: number; }

/** 원: 포인트가 반경 내 (음수 = 충돌) */
export function distPointCircle(px: number, py: number, cx: number, cy: number, r: number):
number {
  return Math.hypot(px - cx, py - cy) - r;
}

/** 부채꼴: 반경 + 중심각 내 (dir = 라디안, angle = 전체 각도) */
export function distPointSector(
  px: number, py: number, cx: number, cy: number, r: number, angle: number, dir: number,
): number {
  const dx = px - cx, dy = py - cy;
  const dist = Math.hypot(dx, dy);
  if (dist > r) return dist - r;
  // 각도 차이를 [-PI, PI]로 정규화
  let diff = Math.atan2(dy, dx) - dir;
  while (diff > Math.PI) diff -= Math.PI * 2;
  while (diff < -Math.PI) diff += Math.PI * 2;
  if (Math.abs(diff) > angle / 2) return 0.001; // 각도 밖 = 미충돌(양수)
  return -0.001;
}

/** 회전 직사각형(OBB): 로컬 좌표 변환 후 AABB 판정 */
export function distPointRect(
  px: number, py: number, cx: number, cy: number, w: number, h: number, dir: number,
): number {
  const cos = Math.cos(-dir), sin = Math.sin(-dir);
  const dx = px - cx, dy = py - cy;
  const lx = dx * cos - dy * sin; // 로컬 X (길이 방향)
  const ly = dx * sin + dy * cos; // 로컬 Y (폭 방향)
  const hw = w / 2, hh = h / 2;
  const ox = Math.max(Math.abs(lx) - hw, 0);
  const oy = Math.max(Math.abs(ly) - hh, 0);
  return Math.hypot(ox, oy) === 0 ? -0.001 : Math.hypot(ox, oy);
}

/** 원 vs 부채꼴(대상 hurtbox 반경 감안) */
export function circleInSector(
  px: number, py: number, pr: number,
  cx: number, cy: number, r: number, angle: number, dir: number,
): boolean {
  const dx = px - cx, dy = py - cy;
  const dist = Math.hypot(dx, dy);
```

```

if (dist - pr > r) return false; // 반경(대상 반경 감안)
let diff = Math.atan2(dy, dx) - dir;
while (diff > Math.PI) diff -= Math.PI * 2;
while (diff < -Math.PI) diff += Math.PI * 2;
// 대상이 클수록 각도 여유도 곱해주는 보정
const allow = angle / 2 + Math.atan2(pr, Math.max(1, dist));
return Math.abs(diff) <= allow;
}

/** broad phase: 사거리 원으로 1 차 거르기 */
export function inBroadRange(
  self: Vec, target: Vec, maxReach: number, targetRadius = 16,
): boolean {
  return Math.hypot(target.x - self.x, target.y - self.y) <= maxReach + targetRadius;
}

```

보조 코드 — src/combat/HitResolver.ts — 스킬 셰이프 기반 다중 판정

```

import { Vec, distPointCircle, circleInSector, distPointRect, inBroadRange } from "./hitbox";

export type SkillShape =
  | { kind: "circle"; r: number }
  | { kind: "sector"; r: number; angle: number }
  | { kind: "rect"; w: number; h: number };

export interface Combatant { id: number; x: number; y: number; radius: number; dead: boolean;
team: number; }

export function resolveHits(
  self: Vec & { id: number; team: number },
  dir: number,
  shape: SkillShape,
  reach: number,
  all: Combatant[],
): number[] {
  const maxReach = shape.kind === "circle" ? shape.r
    : shape.kind === "sector" ? shape.r
    : Math.max(shape.w, shape.h);
  const hits: number[] = [];

  for (const t of all) {
    if (t.dead || t.team === self.team) continue;
    if (!inBroadRange(self, t, maxReach, t.radius)) continue; // 1 차 거름
    let hit = false;
    switch (shape.kind) {
      case "circle":
        hit = distPointCircle(t.x, t.y, self.x, self.y, shape.r + t.radius) <= 0;
        break;
      case "sector":
        hit = circleInSector(t.x, t.y, t.radius, self.x, self.y, shape.r, shape.angle, dir);
        break;
      case "rect":

```

```

    hit = distPointRect(t.x, t.y, self.x, self.y, shape.w, shape.h, dir) < t.radius;
    break;
  }
  if (hit) hits.push(t.id);
}
return hits;
}

```

(3) 성능 최적화 팁

- 부채꼴 각도 차이는 반드시 atan2 정규화($[-\pi, \pi]$) 후 비교한다 — 359도와 1도의 차이를 358도로 계산하는 버그가 가장 흔하다.
- 판정 대상 반경(hurtbox radius)을 판정에 곱하면 '살짝 겹친 베기'가 자연스럽게 히트된다 — 픽셀 단위 엄격 판정은 조작감을 나쁘게 만든다.
- 서버는 클라 프리뷰와 같은 함수를 공유하되, 최종 판정은 서버(015번)만 유효하다 — 클라 판정은 이펙트 표시 용도다.
- rect 판정의 dir은 스킬 발동 순간의 조준 방향으로 고정한다 — 판정 중 방향을 추적하면 유저가 피할 수 없다.

032. 스킬 쿨타임 및 자원(MP, 기력) 소모/회복 관리자

(1) 역할 설명

쿨타임·자원 관리는 '스킬 사용 가능 여부의 단일 진실'이며, 클라 UI(버튼 활성화)와 서버(사용 승인)가 같은 규칙을 공유한다. 구조는 스킬별 상태(마지막 사용 틱, 쿨타임 ms, 소모 MP)를 Map으로 관리하고, CDR(쿨타임 감소 버프)은 '고정 감소(ms)'와 '비율 감소(%)'로 분리 계산한다. 자원은 MP(스킬)와 기력(격투 계열)처럼 복수 자원을 지원하며, 재생량은 초당 기본 재생 + 버프/스킬 효과로 합산된다.

핵심 방어는 서버 승인이다. 클라가 스킬 사용을 요청하면 서버는 (1) 쿨타임 잔여 확인, (2) MP 잔여 확인, (3) 상태(CC 여부) 확인 후 소모를 적용한다. 클라 UI는 예측으로 버튼을 즉시 돌리되, 서버 거절(reject) 수신 시 버튼 상태를 롤백하고 자원 바를 서버값으로 재동기화한다. 이 구조는 핵의 '무한 MP'와 '쿨타임 무시'를 구조적으로 막는다.

(2) 구현 코드 — src/combat/ResourceMgr.ts — 클라·서버 공용

```

export interface SkillCost { mp?: number; stamina?: number; }
export interface SkillCdState { lastUsed: number; cdMs: number; }

export class ResourceMgr {
  private cds = new Map<string, SkillCdState>();
  private mp: number; private mpMax: number;
  private stamina: number; private staminaMax: number;
  private mpRegenPerSec = 12; // 기본 재생
  private stRegenPerSec = 8;
  private cdReductionMs = 0; // 고정 CDR

```



```

private cdReductionPct = 0;      // 비율 CDR(0.2 = -20%)

constructor(mpMax: number, stMax: number, now: number) {
    this.mp = mpMax; this.stamina = stMax;
    this.mpMax = mpMax; this.staminaMax = stMax;
}

applyBuff(buff: { cdFlat?: number; cdPct?: number; mpRegen?: number }) {
    if (buff.cdFlat) this.cdReductionMs += buff.cdFlat;
    if (buff.cdPct) this.cdReductionPct = Math.min(0.6, this.cdReductionPct + buff.cdPct);
    if (buff.mpRegen) this.mpRegenPerSec += buff.mpRegen;
}

/** 실제 쿨타임: 비율 감소 후 고정 감소, 하한 100ms */
effectiveCd(skillId: string, baseMs: number): number {
    let cd = baseMs * (1 - this.cdReductionPct);
    cd -= this.cdReductionMs;
    return Math.max(100, cd);
}

/** 사용 가능 여부(클라 UI 프리뷰 & 서버 승인 동일 로직) */
canUse(skillId: string, cost: SkillCost, baseCdMs: number, now: number,
    ccState: "free" | "stunned" | "silenced"): { ok: boolean; reason?: string } {
    if (ccState === "stunned" || ccState === "silenced") return { ok: false, reason: "cc" };
    const st = this.cds.get(skillId);
    if (st && now - st.lastUsed < this.effectiveCd(skillId, baseCdMs))
        return { ok: false, reason: "cooldown" };
    if ((cost.mp ?? 0) > this.mp) return { ok: false, reason: "mp" };
    if ((cost.stamina ?? 0) > this.stamina) return { ok: false, reason: "stamina" };
    return { ok: true };
}

/** 소모 적용(서버 승인 또는 클라 예측) */
consume(skillId: string, cost: SkillCost, baseCdMs: number, now: number) {
    this.cds.set(skillId, { lastUsed: now, cdMs: this.effectiveCd(skillId, baseCdMs) });
    this.mp -= cost.mp ?? 0;
    this.stamina -= cost.stamina ?? 0;
}

/** 고정 업데이트: 자원 재생 */
fixedUpdate(dtMs: number, moving: boolean) {
    const dt = dtMs / 1000;
    this.mp = Math.min(this.mpMax, this.mp + this.mpRegenPerSec * dt * (moving ? 0.6 : 1));
    this.stamina = Math.min(this.staminaMax, this.stamina + this.stRegenPerSec * dt);
}

/** 쿨타임 잔여(버튼 게이지 표시용) */
cdRemain(skillId: string, now: number): number {
    const st = this.cds.get(skillId);
    if (!st) return 0;
    return Math.max(0, st.cdMs - (now - st.lastUsed));
}

```

```

}
get Mp() { return this.mp; } get MpMax() { return this.mpMax; }
get Stamina() { return this.stamina; }
}

```

보조 코드 — 서버 승인 핸들러 발췌

```

// server/combat/handleSkill.ts (발췌)
import type uWS from "uWebSockets.js";

export function handleSkillUse(ws: uWS.WebSocket, skillId: string) {
  const p = players.get(ws!);
  const def = skillDefs.get(skillId!);

  // CC 상태 확인(035 번) → 쿨타임/자원 검증(위 ResourceMgr 재사용)
  const gate = p.resource.canUse(skillId, def.cost, def.cdMs, p.now, p.ccState);
  if (!gate.ok) {
    // 클라에 거절 사유 통보 → 클라는 버튼 롤백
    send(ws, { t: "skill_reject", skillId, reason: gate.reason });
    return;
  }
  p.resource.consume(skillId, def.cost, def.cdMs, p.now);

  // 데미지 판정(031/015 번) → 결과 브로드캐스트(AOI 토픽, 013 번)
  const hits = lagComp.judge(buildAttackPacket(p, def), currentTick, tickStates);
  applyDamage(hits, def.dmg, p);
  publishSkillEffect(p, skillId, hits); // 이펙트는 저우선순위(002 번)로 전송
}

```

(3) 성능 최적화 팁

- canUse 는 클라 프리뷰와 서버 승인이 완전히 같은 함수여야 한다 — 조건이 같아지면 UI 와 실체가 어긋난다.
- CDR 하한(쿨타임 100ms 이상)은 반드시 둔다 — 무한 CDR 스택은 서버 틱을 포화시키는 DoS 가 된다.
- 이동 중 MP 재생 감소(60%) 같은 컨텍스트 요소는 전투 밸런스의 숨은 축이다 — 상수로 두고 튜닝한다.
- 거절 사유를 클라에 명시적으로 보내야 UI 가 정확히 반응한다(쿨타임 빨간 깜빡임 vs MP 부족 흔들림).

033. 모바일용 오토 배틀(Auto-Battle) 타겟팅 & 이동 AI

(1) 역할 설명

오토 배틀은 '타겟 선택 → 접근 이동 → 스킬 사이클 → 위험 회피'의 상태머신이다. 타겟 선택은 (1) 시야 내 유효 대상 수집, (2) 위협도 점수(거리 가깝고 HP 낮을수록 고득점), (3)

죽이기 쉬운 대상 우선(잡몹 사냥)의 가중치 합으로 결정한다. 이동은 경로(BFS/A*, 043 번)와 직선 이동을 하이브리드로 — 시야 직선이면 즉시 이동, 벽이 있으면 경로 계산한다.

생존성 장치가 핵심이다. HP 가 회복 물 소모선 이하이면 물 마시고, 주변 적이 임계 수 이상이면 열린 방향으로 후퇴하며, 쿨타임 대기 중에는 사거리 유지(카이팅)를 한다. 모바일 특성상 '전투와 무관한 패킷 최소화'도 목표로, 오토 모드 중에는 불필요한 카메라 이동·이펙트 재생을 줄여 배터리를 아낀다.

(2) 구현 코드 — src/combat/AutoBattle.ts — 상태머신

```
import type { Vec } from "./hitbox";

export interface AutoTarget {
  id: number; x: number; y: number; hp: number; maxHp: number;
  isBoss: boolean; radius: number;
}

export interface AutoSkill { id: string; range: number; cd: number; mp: number; ready: boolean; }
export type AutoState = "idle" | "approach" | "attack" | "kite" | "retreat" | "potion";

export class AutoBattle {
  state: AutoState = "idle";
  private targetId: number | null = null;
  private targetLostAt = 0;

  constructor(
    private cfg: {
      aggroRange: number;      // 시야(타겟 탐색 반경)
      retargetMs: number;      // 타겟 재평가 주기
      hpPotionPct: number;     // 물 마시는 HP 임계
      surroundedCount: number; // 이 수 이상 포위 시 후퇴
      skillPriority: string[];  // 우선 스킬 순서
    },
    private io: {
      enemies(): AutoTarget[]; skills(): AutoSkill[];
      move(dir: Vec): void; stop(): void;
      cast(skillId: string, targetId: number): boolean;
      hp(): number; maxHp(): number; mp(): number;
      usePotion(): boolean; threat(): AutoTarget[]; // 자기 주변 적 목록
      openDir(): Vec | null;           // 열린 후퇴 방향
    },
  ) {}

  /** 고정 업데이트(011 번 예측 엔진에 이동 의도 주입) */
  tick(now: number, dtMs: number) {
    const hpPct = this.io.hp() / this.io.maxHp();

    // 1) 최우선: 생존
    if (hpPct < this.cfg.hpPotionPct && this.io.usePotion()) { this.state = "potion"; return; }
```

```

if (this.io.threat().length >= this.cfg.surroundedCount) {
  const dir = this.io.openDir();
  if (dir) { this.io.move(dir); this.state = "retreat"; return; }
}

// 2) 타겟 유지/재평가
this.refreshTarget(now);
const t = this.currentTarget();
if (!t) { this.io.stop(); this.state = "idle"; return; }

// 3) 스킬 사이클(우선순위 순서로 사용 가능 첫 스킬)
for (const id of this.cfg.skillPriority) {
  const s = this.io.skills().find(x => x.id === id);
  if (!s || !s.ready || this.io.mp() < s.mp) continue;
  const dist = Math.hypot(t.x - this.pos().x, t.y - this.pos().y);
  if (dist <= s.range && this.io.cast(s.id, t.id)) { this.state = "attack"; return; }
}

// 4) 접근/카이팅: 사거리 80% 유지
const bestSkill = this.io.skills()
  .filter(s => s.ready && s.range > 0)
  .sort((a, b) => b.range - a.range)[0];
const want = (bestSkill?.range ?? 32) * 0.8;
const dist = Math.hypot(t.x - this.pos().x, t.y - this.pos().y);
const dir = { x: (t.x - this.pos().x) / (dist || 1), y: (t.y - this.pos().y) / (dist || 1) };
if (dist > want) { this.io.move(dir); this.state = "approach"; }
else if (dist < want * 0.6) { this.io.move({ x: -dir.x, y: -dir.y }); this.state = "kite"; }
else { this.io.stop(); this.state = "attack"; }
}

private refreshTarget(now: number) {
  if (this.targetId && now - this.targetLostAt < this.cfg.retargtMs) return;
  const es = this.io.enemies().filter(e =>
    Math.hypot(e.x - this.pos().x, e.y - this.pos().y) <= this.cfg.aggroRange);
  // 위협도 점수: 가까울수록, 약할수록, 보스면 가산
  let best: AutoTarget | null = null, bestScore = -Infinity;
  for (const e of es) {
    const d = Math.hypot(e.x - this.pos().x, e.y - this.pos().y);
    const score = (e.isBoss ? 500 : 0) + (e.hp / e.maxHp) * -100 - d * 0.5;
    if (score > bestScore) { bestScore = score; best = e; }
  }
  this.targetId = best?.id ?? null;
  this.targetLostAt = now;
}

private currentTarget(): AutoTarget | null {
  return this.targetId == null ? null : this.io.enemies().find(e => e.id === this.targetId) ?? null;
}

private pos(): Vec { return (this as any).pos ?? { x: 0, y: 0 }; }
}

```

(3) 성능 최적화 팁

- 생존 판단(물/후퇴)은 항상 공격 사이클보다 먼저 평가한다 — 순서가 뒤바뀌면 오토는 무작정 사망한다.
- 타겟 재평가 주기(retarget 800ms~1s)를 뒤야 몸이 떠도 계속 타겟을 바꾸는 떨림(flicker)이 사라진다.
- 사거리 80% 유지 카이팅은 근접 몸의 공격 전동작(윈드업)을 벗어나게 해 체력 소모를 크게 줄인다.
- 포위 감지는 단순 개수(주변 적 3+)로 충분하며, 후퇴 방향은 벽 아닌 방향(그리드 스캔)에서 고른다.

034. 다중 타겟팅(Tab 키, 거리순, 클릭) 선택 관리자

(1) 역할 설명

타겟 선택 관리자는 화면 중앙 표시(선택 링), 스킬 자동 조준, 타겟 정보 UI의 입력을 제공한다. 선택 소스는 세 가지다. Tab 키(사거리 내 대상 순환), 클릭/터치(직접 지정), 오토 배틀(033 번 위임). 관리자는 이 세 소스의 우선순위를 조정하고, 타겟이 사망/시야 이탈하면 자동 폴백(가장 가까운 대상 또는 해제)한다.

핵심 UX 규칙은 'Tab 순환은 시야 내 사거리 대상만, 시선 방향 근처 우선'이다. 순환 순서는 화면 중심 각도 기준 정렬해 Tab 연타가 시계 방향으로 일관되게 돌게 하며, 마지막 선택 타겟은 재선택 시 우선(스티키 타겟)으로 유지한다. 서버와의 관계에서 타겟은 클라 개념이며, 실제 스킬 판정은 히트박스(031)와 서버 판정(015)이 담당한다.

(2) 구현 코드 — src/combat/Targeting.ts

```
import type { Vec } from "../hitbox";

export interface Targetable {
  id: number; x: number; y: number; dead: boolean;
  name?: string; hpPct?: number; isBoss?: boolean;
}

export type TargetSource = "tab" | "click" | "auto" | "fallback";

export class TargetingManager {
  private currentId: number | null = null;
  private lastSource: TargetSource = "fallback";
  onSelect: ((t: Targetable | null, src: TargetSource) => void) | null = null;

  /** Tab 순환: 사거리 내 대상을 '카메라 중심 기준 각도' 순서로 다음 대상 */
  cycleTab(candidates: Targetable[], self: Vec, cameraCenter: Vec, reach: number) {
    const inReach = candidates
      .filter(c => !c.dead && Math.hypot(c.x - self.x, c.y - self.y) <= reach);
    if (!inReach.length) return this.clear();
  }
}
```

```

// 각도 정렬(시계 방향 일관 순환) — 화면 중심 기준
const sorted = inReach.sort((a, b) => angleOf(a, cameraCenter) - angleOf(b, cameraCenter));
const cur = this.currentId == null
  ? null : sorted.find(c => c.id === this.currentId);
let next: Targetable;
if (cur) {
  next = sorted[(sorted.indexOf(cur) + 1) % sorted.length];
} else {
  // 신규 선택: 가장 가까운 대상
  next = inReach.sort((a, b) =>
    Math.hypot(a.x - self.x, a.y - self.y) - Math.hypot(b.x - self.x, b.y - self.y))[0];
}
this.set(next.id, "tab");
}

/** 클릭/터치: 히트 테스트(스크린 → 월드 좌표) */
click(worldPos: Vec, candidates: Targetable[], tolerance = 24) {
  const hit = candidates
    .filter(c => !c.dead)
    .sort((a, b) => Math.hypot(a.x - worldPos.x, a.y - worldPos.y)
      - Math.hypot(b.x - worldPos.x, b.y - worldPos.y))[0];
  if (hit && Math.hypot(hit.x - worldPos.x, hit.y - worldPos.y) <= tolerance + 16) {
    this.set(hit.id, "click");
  } else {
    this.clear(); // 빈 곳 클릭 = 선택 해제
  }
}

/** 오토 배틀(033)이 결정한 타겟 위임 */
fromAuto(id: number | null) {
  if (id == null) return;
  this.set(id, "auto");
}

/** 매 프레임 유효성 검사 — 사망/이탈 시 폴백 */
validate(candidates: Targetable[], self: Vec, reach: number) {
  const cur = candidates.find(c => c.id === this.currentId);
  if (!cur || cur.dead) {
    const near = candidates
      .filter(c => !c.dead && Math.hypot(c.x - self.x, c.y - self.y) <= reach)
      .sort((a, b) => Math.hypot(a.x - self.x, a.y - self.y)
        - Math.hypot(b.x - self.x, b.y - self.y))[0];
    if (near) this.set(near.id, "fallback"); else this.clear();
  }
}

get targetId() { return this.currentId; }
private set(id: number, src: TargetSource) {
  if (this.currentId === id) return;
  this.currentId = id; this.lastSource = src;
  this.onSelect?.(this.find?.(id) ?? null, src);
}

```

```

private clear() {
  if (this.currentId == null) return;
  this.currentId = null;
  this.onSelect?.(null, "fallback");
}
find?: (id: number) => Targetable | null;
}

function angleOf(t: Targetable, center: Vec): number {
  return Math.atan2(t.y - center.y, t.x - center.x);
}

```

(3) 성능 최적화 팁

- Tab 순환에 각도 정렬을 쓰면 연타 시 타겟이 왔다 갔다 튀는 현상이 사라진다.
- 스틱키 타겟(마지막 선택 유지)은 보스전 필수 UX 다 — 다른 몹을 때려도 실수로 타겟이 바뀌면 안 된다.
- validate 는 매 프레임이 아닌 100ms 주기로 돌려도 충분하며, 사망 이벤트 혹은 있으면 이벤트 기반으로 처리한다.
- 선택 링 UI 는 타겟 종류(잡몹/엘리트/보스)별 색상·크기를 다르게 해 위험도를 즉시 전달한다.

035. CC(기절, 둔화, 속박, 밀쳐내기) 상태 이상 타이머 및 연산

(1) 역할 설명

CC(군중 제어)는 상태별로 이동/행동에 미치는 효과가 다르다. 기절(stun)은 모든 행동 차단, 둔화(slow)는 이동속도 배율 감소, 속박(root)은 이동만 차단, 밀쳐내기(knockback)는 즉시 변위 + 잠깐 행동 차단이다. 상태는 '종류 + 남은 시간 + 강도' 튜플로 관리하며, 같은 종류는 갱신(강도는 max), 다른 종류는 공존한다. CC 면역(i-frame)은 상태 추가 시점에 검사해 무한 CC 사슬을 막는다.

서버가 유일한 권위이며(016 번 철학), 클라는 CC 상태를 받아 애니메이션(기절 별, 속박 쇠사슬)과 이동 입력 차단만 처리한다. 밀쳐내기는 서버가 목표 지점을 계산해(벽 충돌 클램프) 클라에 전송하고, 클라는 예측 이동 대신 서버 지정 궤적을 따라간다. CC 적용 로그는 PVP 항의 대응 데이터가 된다.

(2) 구현 코드 — src/combat/CrowdControl.ts — 서버 권위 상태 관리

```

import type { Vec } from "../hitbox";

export type CcKind = "stun" | "slow" | "root" | "knockback";
export interface CcInstance {
  kind: CcKind;
  until: number; // 종료 시각(서버 기준 ms)
  strength?: number; // slow: 속도 배율 감소(0.4 = 40% 감소)
}

```

```

from?: Vec; dir?: number; // knockback 궤적
dist?: number;
}

export interface CcBlocked { move: boolean; act: boolean; }
export interface MovementMod { speedMult: number; }

export class CcManager {
  private list: CcInstance[] = [];
  private immuneUntil = 0;

  constructor(private now: () => number, private blocked: (x: number, y: number) => boolean) {}

  /** CC 적용 시도 — 면역/중복 규칙 처리 */
  apply(cc: CcInstance): boolean {
    const now = this.now();
    if (now < this.immuneUntil) return false;
    if (cc.kind === "knockback" && this.has("knockback")) return false; // 중복 낙백 금지

    const existing = this.list.find(c => c.kind === cc.kind);
    if (existing) {
      existing.until = Math.max(existing.until, cc.until); // 갱신 = max
      existing.strength = Math.max(existing.strength ?? 0, cc.strength ?? 0);
    } else {
      this.list.push(cc);
    }
    // CC 사슬 방지: CC 종료 후 짧은 면역(무한 스톤락 차단)
    this.immuneUntil = cc.until + 300;
    return true;
  }

  /** 현재 상태 요약(019 ResourceMgr canUse 의 ccState 입력) */
  summary(): { blocked: CcBlocked; move: MovementMod; ccState: "free" | "stunned" | "silenced" } {
    const now = this.now();
    this.list = this.list.filter(c => c.until > now);
    const has = (k: CcKind) => this.list.some(c => c.kind === k);
    const slows = this.list.filter(c => c.kind === "slow");
    const speedMult = slows.length
      ? Math.min(...slows.map(c => 1 - (c.strength ?? 0))) : 1;
    return {
      blocked: { move: has("root") || has("stun"), act: has("stun") },
      move: { speedMult },
      ccState: has("stun") ? "stunned" : "free",
    };
  }

  /** knockback 궤적 계산: 서버가 목표 지점 결정(벽 클램프) */
  computeKnockback(self: Vec, dir: number, dist: number, steps = 8): Vec {

```



```

let x = self.x, y = self.y;
const dx = Math.cos(dir) * (dist / steps);
const dy = Math.sin(dir) * (dist / steps);
for (let i = 0; i < steps; i++) {
  const nx = x + dx, ny = y + dy;
  if (this.blocked(nx, ny)) break;    // 벽에 닿으면 거기서 정지
  x = nx; y = ny;
}
return { x, y };
}

private has(k: CcKind) { return this.list.some(c => c.kind === k); }
}

```

(3) 성능 최적화 팁

- 같은 종류 CC는 시간 max + 강도 max로 갱신한다 — 합산하면 CC 지속이 폭증해 스텐락이 발생한다.
- CC 종료 후 300ms 면역(soft i-frame)은 유저가 반응할 시간을 주는 표준 장치다.
- 밀쳐내기는 서버가 궤적 종점을 벽 충돌로 클램프해 전송한다 — 클라 자유 계산은 맵 밖 추락 버그를 낳는다.
- PVP에서 CC는 딜 사이클의 핵심이므로, CC 효과 감소 속성(CC 저항)을 스탯 시스템(021번)에 조기에 녹여야 한다.

036. 투사체(직선, 유도, 곡선) 이동 궤적 연산 모듈

(1) 역할 설명

투사체는 궤적 유형별로 계산 방식이 다르다. 직선(linear)은 초기 속도 벡터로 등속 이동, 유도(homing)는 목표 방향으로 각속도 제한 회전, 곡선(arc)은 포물선 높이 렌더 + 평면 직선 이동, 관통/바운스는 히트 후 상태 전이를 추가로 가진다. 서버는 발사 시점의 궤적 매개변수(발사 위치, 방향, 속도, 목표 id)를 결정하고, 클라는 동일 매개변수로 시각적으로 재생한다 — 히트 판정은 서버가 유일 권위(015번)다.

유도탄의 핵심은 각속도 제한(턴 레이트)이다. 매 스텝 목표 방향으로 즉시 회전하면 빙글빙글 도는 우스꽝스러운 궤적이 되므로, 최대 각속도(예: 초당 120도) 이내로만 목표를 추적한다. 수명(lifetime)과 사거리 초과 시 자동 소멸, 목표 사망 시 궤적 유지 정책(직진 유지 or 소멸)도 데이터로 정의한다.

(2) 구현 코드 — src/combat/Projectile.ts

```

import type { Vec } from "../hitbox";

export type ProjKind = "linear" | "homing" | "arc";
export interface ProjDef {
  kind: ProjKind;
  speed: number;    // px/s
}

```

```

turnRate?: number;    // homing: rad/s
arcHeight?: number;   // arc: 포물선 최고 높이(px)
maxDist: number;      // 사거리 초과 소멸
pierce?: number;      // 관통 횟수
radius: number;       // 히트 반경
}

export interface ProjState {
  id: number; ownerId: number; targetId: number | null;
  x: number; y: number; dir: number; // dir = 라디안
  traveled: number; born: number; life: number;
  dead: boolean;
}

export class ProjectileSim {
  constructor(private def: ProjDef) {}

  /** 한 스텝 진행(서버 고정 틱에서 호출 — 클라는 동일 코드로 시각 재생) */
  step(p: ProjState, dtMs: number, targetPos: Vec | null): { x: number; y: number; z: number } {
    const dt = dtMs / 1000;
    if (p.traveled >= this.def.maxDist || p.dead || p.born + p.life < p.born) p.dead = true;

    switch (this.def.kind) {
      case "linear":
        break; // 방향 고정
      case "homing": {
        if (targetPos) {
          // 목표 방향으로 turnRate 이내 회전
          const want = Math.atan2(targetPos.y - p.y, targetPos.x - p.x);
          p.dir = rotateToward(p.dir, want, (this.def.turnRate ?? 2) * dt);
        }
        break; // 목표 없으면 직선 유지
      }
      case "arc":
        break; // 평면은 직선, 높이만 렌더 계산
    }
    const move = this.def.speed * dt;
    p.x += Math.cos(p.dir) * move;
    p.y += Math.sin(p.dir) * move;
    p.traveled += move;

    // 곡선(포물선) 렌더 높이: 사거리 대비 진행률로 정현파 절반
    const t = Math.min(1, p.traveled / this.def.maxDist);
    const z = this.def.kind === "arc"
      ? Math.sin(t * Math.PI) * (this.def.arcHeight ?? 0) : 0;
    return { x: p.x, y: p.y, z };
  }

  /** 히트 판정(서버) — 대상 원 반경 포함 */
  hits(p: ProjState, targets: { id: number; x: number; y: number; radius: number; dead: boolean }
[]) {

```

```

const out: number[] = [];
for (const t of targets) {
  if (t.dead || t.id === p.ownerId) continue;
  if (Math.hypot(t.x - p.x, t.y - p.y) <= this.def.radius + t.radius) out.push(t.id);
}
return out;
}

/** 각도 회전(각속도 제한) — homing 핵심 수식 */
export function rotateToward(cur: number, want: number, maxDelta: number): number {
  let diff = want - cur;
  while (diff > Math.PI) diff -= Math.PI * 2;
  while (diff < -Math.PI) diff += Math.PI * 2;
  const clamped = Math.max(-maxDelta, Math.min(maxDelta, diff));
  return cur + clamped;
}

```

(3) 성능 최적화 팁

- 투사체 동기화는 '발사 이벤트 1 회' + 양쪽 동일 시뮬레이션이 정석이다 — 위치를 매 프레임 전송하면 트래픽만 늘고 결정론만 깨진다.
- 유도탄 turnRate 는 90~180 도/초 밴드가 자연스럽다 — 초과하면 강력해 보이지만 회피가 불가능해져 조작감이 파괴된다.
- arc 투사체는 평면 판정(원 반경)과 렌더 높이(z)를 분리해야 충돌 판정 코드가 단순해진다.
- 관통(pierce)은 히트한 대상 id Set 을 유지해 같은 대상 중복 히트를 막는다.

037. 몬스터 어그로(Aggro) 수치 계산 및 타겟 전환 AI

(1) 역할 설명

어그로는 '몬스터가 누구를 공격할지'의 수치 테이블이다. 각 플레이어는 몬스터별 어그로 수치를 가지며, 데미지(1.0), 힐(0.5), 버프(0.3), 어그로 스킬(가중치 높음)로 적립되고, 시간에 따라 감쇠한다. 몬스터는 최대 어그로 대상을 공격하되, 자연 감쇠(초당 -x%)로 장기 전투에서 탱커가 어그로를 유지하기 어려워지는 것을 방지하기 위해 탱커 보정(탱커 어그로는 감쇠 50%)을 둔다.

타겟 전환 트리거는 세 가지다. 어그로 역전(상위 대상 교체), 힐탱 우선 규칙(일부 몸은 힐러 우선), 배신(hate reset, 도망/은신 시 어그로 초기화). 서버에서만 연산하며, 클라는 몬스터의 currentTargetId 만 받아 '누구를 때리는지' 시각적으로 표시(타겟 라인)한다. 파티 시스템과 결합해 '파티원이 맞는 중이면 대형 몹 어그로를 파티 탱커가 우선' 같은 규칙도 데이터로 둔다.

(2) 구현 코드 — server/combat/Aggro.ts

```

export interface AggroEntry { userId: number; value: number; lastHitAt: number; }

```

```

const DECAY_PER_SEC = 0.98;      // 초당 2% 자연 감쇠
const TANK_DECAY = 0.995;       // 탱커 보정 감쇠
const HATE_RESET_MS = 8000;     // 8 초간 무공격 시 절반 감소 가속

export class AggroTable {
  private entries = new Map<number, AggroEntry>(); // userId → entry
  private tankIds = new Set<number>();

  markTank(userId: number) { this.tankIds.add(userId); }

  add(userId: number, amount: number, now: number) {
    const e = this.entries.get(userId) ?? { userId, value: 0, lastHitAt: now };
    e.value += amount;
    e.lastHitAt = now;
    this.entries.set(userId, e);
  }

  /** 데미지/힐/버프별 어그로 가중치 */
  static fromDamage(dmg: number) { return dmg * 1.0; }
  static fromHeal(hp: number) { return hp * 0.5; }
  static fromBuff() { return 30; }
  static fromTaunt() { return 1500; }

  /** 몬스터 틱: 감쇠 + 현재 타겟 반환 */
  tickAndPick(now: number): number | null {
    for (const e of this.entries.values()) {
      const idleMs = now - e.lastHitAt;
      const isTank = this.tankIds.has(e.userId);
      let decay = Math.pow(isTank ? TANK_DECAY : DECAY_PER_SEC, 1);
      if (idleMs > HATE_RESET_MS) decay *= Math.pow(0.9, idleMs / 1000); // 무공격 가속 감쇠
      e.value *= decay;
    }
    for (const [k, e] of this.entries) if (e.value <= 1) this.entries.delete(k);

    let best: AggroEntry | null = null;
    for (const e of this.entries.values())
      if (!best || e.value > best.value) best = e;
    return best?.userId ?? null;
  }

  /** 은신/도망: 어그로 완전 제거(hate drop) */
  drop(userId: number) { this.entries.delete(userId); }
  /** 도망각 체크: 최근 피격 없이 이동 중이면 감쇠 가속 */
  onFlee(userId: number) {
    const e = this.entries.get(userId);
    if (e) e.value *= 0.6;
  }

  top(n: number): AggroEntry[] {
    return [...this.entries.values()].sort((a, b) => b.value - a.value).slice(0, n);
  }
}

```

```

    }
}

/** 몬스터 AI 통합 예시 */
export class MonsterBrain {
  private aggro = new AggroTable();
  targetUserId: number | null = null;

  onPlayerAction(userId: number, kind: "dmg" | "heal" | "buff" | "taunt", value = 0, now = Date.now()) {
    const amount = kind === "dmg" ? AggroTable.fromDamage(value)
      : kind === "heal" ? AggroTable.fromHeal(value)
      : kind === "taunt" ? AggroTable.fromTaunt() : AggroTable.fromBuff();
    this.aggro.add(userId, amount, now);
  }
  tick(now: number) {
    this.targetUserId = this.aggro.tickAndPick(now);
    // 타겟 사망/퇴장 시 다음 순위 자동 승계(tickAndPick 이 처리)
  }
}

```

(3) 성능 최적화 팁

- 감쇠(decay)는 지수 형태로 구현해야 감쇠가 프레임울 독립이다 — 고정 틱에서 $\text{pow}(0.98, \text{dt}/1000)$ 형태로 쓴다.
- 탱커 어그로 보정(감쇠 50%) 없이는 장기 보스전에서 탱커가 어그로를 잃는 '어그로 스위칭 지옥'이 벌어진다.
- 힐 어그로는 딜의 절반 — 힐러가 과도하게 어그로를 끌면 '힐 지르기 밴' 문화가 생긴다.
- 타겟 전환은 최소 지연(200~400ms 반응 시간)을 두어 유저가 회피 행동을 취할 기회를 준다.

038. 범위 내 파티 버프 및 시너지 연산

(1) 역할 설명

파티 버프는 '버퍼가 특정 범위(반경) 내 파티원에게 상태 효과를 부여'하는 연산이며, 효과는 StatMath(021 번)의 buff 소스로 주입된다. 범위 판정은 원(오라), 직사각형(지정 지역), 링크(특정 대상 추종) 세 형태다. 시너지는 파티 조합 보너스(예: 전사 2명 + 힐러 1명 = 방어력 +5%)로, 파티 구성 스냅샷에서 정의 테이블 매칭으로 계산한다.

버프 스택 규칙이 핵심이다. 같은 id 버프는 갱신(max)하고, 다른 id 의 동일 효과는 스택 허용(최대 N 개)한다. 시너지 계산은 파티원 이동/입장/퇴장 시 1 회 재평가하며, 범위 버프는 유저가 범위를 벗어날 때 효과 제거 이벤트를 보내야 '범위 밖에서 버프 유지' 버그가 사라진다. 모든 버프는 만료 시각을 가지며, 만료 일괄 처리(sweep)는 1 초 주기로 수행한다.

(2) 구현 코드 — server/combat/PartyBuff.ts

```

import type { StatMod } from "../../src/game/StatMath";

export interface BuffDef {
  id: string;          // "warrior_shout_v2" — 버전 포함해 규칙 격리
  kind: "aura" | "area" | "link";
  range?: number;      // aura: 버퍼 중심 반경
  durationMs: number;
  mods: StatMod[];
  stackLimit: number;  // 동일 id 최대 스택(1 = 갱신만)
}

export interface ActiveBuff {
  defId: string; sourceId: number; targetId: number;
  until: number; stack: number;
}

export class BuffSystem {
  private active = new Map<string, ActiveBuff>(); // key = targetId:defId:sourceId

  /** aura 버퍼 틱: 범위 내 파티원에게 부여, 벗어나면 제거 */
  tickAura(def: BuffDef, buffer: { id: number; x: number; y: number },
    party: { id: number; x: number; y: number }[], now: number) {
    for (const m of party) {
      const inRange = m.id !== buffer.id
        && Math.hypot(m.x - buffer.x, m.y - buffer.y) <= (def.range ?? 200);
      const key = m.id + ":" + def.id + ":" + buffer.id;
      if (inRange) {
        const cur = this.active.get(key);
        if (!cur) {
          this.active.set(key, { defId: def.id, sourceId: buffer.id,
            targetId: m.id, until: now + def.durationMs, stack: 1 });
        } else {
          cur.until = now + def.durationMs; // 지속 갱신(스택은 유지)
        }
      } else {
        this.active.delete(key); // 범위 이탈 즉시 제거
      }
    }
  }

  /** 스택 제한 준수 적용 */
  tryStack(def: BuffDef, targetId: number, sourceId: number, now: number): boolean {
    const sameId = [...this.active.values()].filter(b =>
      b.defId === def.id && b.targetId === targetId && b.sourceId === sourceId);
    const total = sameId.reduce((s, b) => s + b.stack, 0);
    if (total >= def.stackLimit) return false;
    const key = targetId + ":" + def.id + ":" + sourceId;
    const cur = this.active.get(key);
    if (cur) { cur.stack++; cur.until = now + def.durationMs; }
    else this.active.set(key, { defId: def.id, sourceId, targetId,
      until: now + def.durationMs, stack: 1 });
  }
}

```

```

    return true;
  }

  /** 만료 일괄 정리(1 초 주기) */
  sweep(now: number): ActiveBuff[] {
    const expired: ActiveBuff[] = [];
    for (const [k, b] of this.active)
      if (b.until <= now) { expired.push(b); this.active.delete(k); }
    return expired;
  }

  /** 대상의 현재 버프 → StatMath buff 소스 */
  modsFor(targetId: number, defs: Map<string, BuffDef>): StatMod[] {
    const out: StatMod[] = [];
    for (const b of this.active.values()) {
      if (b.targetId !== targetId) continue;
      const d = defs.get(b.defId);
      if (!d) continue;
      for (const m of d.mods) out.push({ ...m });
    }
    return out;
  }
}

/** 시너지: 파티 구성 스냅샷 → 보너스 매칭 */
export interface SynergyDef { id: string; need: Record<string, number>; mods: StatMod[]; }
export function evalSynergies(
  members: { classCode: string }[], synergyDefs: SynergyDef[],
): SynergyDef[] {
  const counts: Record<string, number> = {};
  for (const m of members) counts[m.classCode] = (counts[m.classCode] ?? 0) + 1;
  return synergyDefs.filter(s =>
    Object.entries(s.need).every(([, n]) => (counts[cls] ?? 0) >= n));
}

```

(3) 성능 최적화 팁

- 버프 키(targetId:defId:sourceId)는 중복 부여 방지의 핵심이다 — sourceId 까지 포함해야 서로 다른 버퍼의 같은 버프가 정확히 구분된다.
- aura 이탈 즉시 제거는 UX 상 1~2 초 유예(grace)를 주는 것이 부드럽다 — 톱 끊기는 버프보다 2 초 잔여가 자연스럽다.
- 시너지는 파티 입/퇴장 이벤트에서만 재평가한다 — 이동마다 재계산하면 불필요한 CPU 와 재계산 지연이 생긴다.
- 버프 버전 관리(id 에 v2 포함)로 밸런스 패치 시 구버전 버프가 남는 문제를 구조적으로 차단한다.

039. 속성(불, 물, 바람, 땅 등) 상성 피해 비율 공식

(1) 역할 설명

속성 상성은 5 원소(불/물/바람/땅/빛+어둠 확장) 상성표의 승산 배율로 구현한다. 표준 밴드는 이득 1.5 배, 상쇄 0.5 배, 무관 1.0 배이며, 대구조는 '2 단계 우위 = 1.25, 3 단계 = 1.5, 역우위 = 0.75/0.5'처럼 단계 차이로 확장한다. 상성표는 양방향 대칭 행렬(DB 시드)로 두고, 계산은 attack 속성 × defender 저항/약점의 합산으로 수행한다.

피해 공식에 상성을 끼워 넣는 순서가 중요하다. 표준 순서는 (기본 데미지 → 방어력 감산/비율 → 속성 배율 → 치명타 → 버프/디버프 → 랜덤 변동 ±5%)이며, 속성 배율을 방어력 앞에 두면 방어력이 상성의 의미를 흡수해버린다. 저항(resist)은 상성과 별개 스탯으로 합산되어 최대 75%로 클램프된다.

(2) 구현 코드 — PostgreSQL DDL + Seed

```
CREATE TABLE element_chart (
  atk_el VARCHAR(8) NOT NULL, -- fire | water | wind | earth | light | dark
  def_el VARCHAR(8) NOT NULL,
  mult NUMERIC(4,2) NOT NULL, -- 1.50 이득, 0.50 상쇄, 1.00 무관
  PRIMARY KEY (atk_el, def_el)
);

INSERT INTO element_chart (atk_el, def_el, mult) VALUES
('fire', 'water', 0.50), ('fire', 'wind', 1.50),
('fire', 'earth', 1.00), ('fire', 'fire', 0.50),
('water', 'fire', 1.50), ('water', 'earth', 1.50),
('water', 'wind', 1.00), ('water', 'water', 0.50),
('wind', 'earth', 1.50), ('wind', 'fire', 1.00),
('wind', 'water', 0.50), ('wind', 'wind', 0.50),
('earth', 'fire', 1.00), ('earth', 'water', 0.50),
('earth', 'wind', 1.00), ('earth', 'earth', 0.50);

ALTER TABLE characters
  ADD COLUMN element VARCHAR(8) DEFAULT 'fire',
  ADD COLUMN resist_fire SMALLINT DEFAULT 0,
  ADD COLUMN resist_water SMALLINT DEFAULT 0,
  ADD COLUMN resist_wind SMALLINT DEFAULT 0,
  ADD COLUMN resist_earth SMALLINT DEFAULT 0; -- 단위 %, 상한 75
```

보조 코드 — src/combat/Elemental.ts — 피해 합성 파이프라인

```
import type { StatMod } from "../game/StatMath";

export type Element = "fire" | "water" | "wind" | "earth" | "light" | "dark";

/** 상성표 → O(1) 조회용 Map (부팅 시 시드 로드) */
export function buildChart(rows: { atk_el: Element; def_el: Element; mult: number }[]) {
  const m = new Map<string, number>();
  for (const r of rows) m.set(r.atk_el + ">" + r.def_el, Number(r.mult));
  return (atk: Element, def: Element) => m.get(atk + ">" + def) ?? 1.0;
}
```



```

}

export interface DamageCtx {
  baseAtk: number;           // 공격자 2 차 스탯 atk/matk
  defVal: number;           // 방어자 def
  defLevel: number;
  element: Element;         // 공격 속성
  defenderElement: Element;
  resistPct: number;        // 방어자 해당 속성 저항(%)
  critChance: number; critMult: number;
  buffs: StatMod[];         // 공격자 증폭 버프(mul 합산)
  rng?: () => number;       // 서버 결정론 RNG 주입
}

/** 표준 파이프라인: 기본 → 방어 → 속성 → 치명 → 버프 → 랜덤 */
export function computeDamage(c: DamageCtx, chart: (a: Element, d: Element) => number) {
  const rng = c.rng ?? Math.random;

  // 1) 방어력 감산(레벨 보정): dmg = atk * (1 - def/(def + 100 + 20*lvl))
  const mitig = 1 - c.defVal / (c.defVal + 100 + 20 * c.defLevel);
  let dmg = c.baseAtk * Math.max(0.15, mitig); // 최소 관통 15%

  // 2) 속성 배율(방어 이후 적용이 표준)
  const elMult = chart(c.element, c.defenderElement);
  dmg *= elMult;

  // 3) 저항(별개 스탯, 상한 75%)
  dmg *= 1 - Math.min(75, c.resistPct) / 100;

  // 4) 치명타
  const isCrit = rng() * 100 < c.critChance;
  if (isCrit) dmg *= c.critMult; // 보통 1.5~2.0

  // 5) 버프 합산(mul 합 → 곱)
  let mul = 0;
  for (const b of c.buffs) if (b.mul) mul += b.mul;
  dmg *= 1 + mul;

  // 6) 랜덤 변동 ±5% (전투 반복 단조로움 제거)
  dmg *= 0.95 + rng() * 0.10;

  return { dmg: Math.max(1, Math.round(dmg)), crit: isCrit, elMult };
}

// 클라 UI 용 예측(전투 시뮬레이터):
// avg = computeDamage({...ctx, rng: () => 0.5, critChance: 0}, chart).dmg
// DPS 추정치를 스킬 패널에 표시할 때 사용

```

(3) 성능 최적화 팁

- 상성 배율은 방어력 이후에 적용해야 '강한 속성 = 방어 무시' 같은 해석 혼란이 없다.
- 저항 상한 75%는 필수다 — 100% 저항은 '해당 속성 콘텐츠 무력화'로 이어져 기획을 무너뜨린다.
- 랜덤 변동 $\pm 5\%$ 는 결정론 RNG(서버 시드)로 수행해 리플레이/감사에서 동일 결과를 재현 가능하게 한다.
- 상성표는 시드 데이터로만 관리하고 코드에 하드코딩하지 않는다 — 신규 원소 추가가 INSERT 한 줄로 끝나야 한다.

040. 전투 피드백 연출 (화면 흔들림, 데미지 텍스트 팝업, 피격 이펙트)

(1) 역할 설명

전투의 손맛은 판정이 아니라 피드백에서 나온다. 이 모듈은 세 계층으로 구성된다. 화면 흔들림(카메라 셰이크, 강도/지속/감쇠), 데미지 텍스트(등장-부상-소멸 3 단계 트윈, 치명타/회피/속성별 색상), 피격 이펙트(히트 스파클, 피격 플래시, 넉백 모션)다. 모든 연출은 이벤트 구동으로 — 서버 히트 이벤트를 수신하면 연출 큐에 넣고, 프레임당 연출 수 상한을 두어 대규모 전투에서 프레임이 버티게 한다.

성능 장치는 필수다. 데미지 텍스트는 비트맵 폰트(001 번)로 렌더하고 오브젝트 풀로 재사용하며, 화면 밖 히트는 텍스트를 생략한다. 셰이크는 카메라 오프셋만 조작해 물리 오브젝트에 영향을 주지 않고, 임계 이상의 동시 연출은 '중요도'로 걸러 보스 기술은 무조건 남긴다. 피격 플래시는 스프라이트 틸트(하얀색 0.1 초)로 저비용 구현한다.

(2) 구현 코드 — `src/combat/CombatFeedback.ts` — Phaser 연출 관리자

```
import Phaser from "phaser";

export interface HitEvent {
  targetX: number; targetY: number;
  dmg: number; crit: boolean; miss: boolean;
  element?: "fire" | "water" | "wind" | "earth";
  importance?: 0 | 1 | 2; // 2 = 보스기 (항상 연출)
}

const ELEMENT_COLOR: Record<string, number> = {
  fire: 0xff7a45, water: 0x4aa3ff, wind: 0x7ee8a2, earth: 0xd4a030,
};

export class CombatFeedback {
  private textPool: Phaser.GameObjects.BitmapText[] = [];
  private queue: HitEvent[] = [];
  private shakeTime = 0; private shakeIntensity = 0;
  private static MAX_TEXTS_PER_FRAME = 12;
```

```

constructor(private scene: Phaser.Scene) {}

/** 서버 히트 이벤트 수신 → 큐잉(중요도 낮은 과다 연출은 스킵) */
push(e: HitEvent) {
  if ((e.importance ?? 0) < 2 && this.queue.length > 20) return;
  this.queue.push(e);
}

/** 렌더 단계에서 소화 */
drain() {
  let budget = CombatFeedback.MAX_TEXTS_PER_FRAME;
  while (this.queue.length && budget-- > 0) {
    const e = this.queue.shift();
    this.spawnDamageText(e);
    this.spawnHitFx(e);
  }
}

private spawnDamageText(e: HitEvent) {
  const color = e.miss ? "#9aa0a6"
    : e.crit ? "#ff75e"
    : e.element ? "#" + ELEMENT_COLOR[e.element].toString(16).padStart(6, "0")
    : "#ffffff";
  const label = e.miss ? "MISS" : (e.crit ? e.dmg + "!" : String(e.dmg));

  let txt = this.textPool.pop();
  if (!txt) {
    txt = this.scene.add.bitmapText(0, 0, "dmgfont", label, 22)
      .setDepth(5000).setOrigin(0.5);
  }
  txt.setText(label).setColor(color)
    .setPosition(e.targetX + (Math.random() * 24 - 12), e.targetY - 24)
    .setAlpha(1).setScale(e.crit ? 1.5 : 1).setVisible(true);

  // 3 단계 트윈: 등장 팝 → 부상 → 페이드
  this.scene.tweens.add({
    targets: txt, y: e.targetY - 52, alpha: 0, duration: 700, ease: "Cubic.out",
    onComplete: () => { txt!.setVisible(false); this.textPool.push(txt!); },
  });
  if (e.crit) {
    this.scene.tweens.add({
      targets: txt, scale: { from: 1.6, to: 1.1 }, duration: 180, yoyo: false,
    });
  }
}

private spawnHitFx(e: HitEvent) {
  if (e.miss) return;
  // 히트 스파클: 단일 프레임 이펙트 프리팹 재사용(009 번 메모리 규율)
  const spark = this.scene.add.sprite(e.targetX, e.targetY, "fx_hit").setDepth(4900);
  spark.play("fx_hit_anim");
}

```

```

spark.once("animationcomplete", () => spark.destroy());

// 피격 플래시는 대상 스프라이트 틴트로 처리(호출부에서 setTintFill → clearTint)
if ((e.importance ?? 0) >= 1) this.shake(120, 0.004);
}

/** 화면 흔들림: 카메라 오프셋만 조작 */
shake(durationMs: number, intensity: number) {
  this.shakeTime = Math.max(this.shakeTime, durationMs);
  this.shakeIntensity = Math.max(this.shakeIntensity, intensity);
}

/** 고정 업데이트: 세이크 감쇠 */
fixedUpdate(dtMs: number) {
  const cam = this.scene.cameras.main;
  if (this.shakeTime > 0) {
    this.shakeTime -= dtMs;
    const decay = Math.max(0, this.shakeTime / 120);
    cam.setFollowOffset(
      (Math.random() - 0.5) * this.shakeIntensity * decay * 100,
      (Math.random() - 0.5) * this.shakeIntensity * decay * 100,
    );
    if (this.shakeTime <= 0) { cam.setFollowOffset(0, 0); this.shakeIntensity = 0; }
  }
}
}

```

(3) 성능 최적화 팁

- 데미지 텍스트는 비트맵 폰트 + 오브젝트 풀이 표준이다 — 일반 텍스트는 100 개만 넘어도 프레임이 무너진다.
- 프레임당 연출 예산(12 개)은 대규모 전투(파티 8 명 x 몹 50)에서 필수며, 보스기 (importance 2)는 예산 무시로 우선시한다.
- 세이크는 카메라 followOffset 만 흔들다 — 월드 좌표를 흔들면 판정 좌표와 화면이 어긋난다.
- 피격 플래시(setTintFill 흰색 0.1 초)는 가장 저렴하면서 효과가 확실한 타격감 장치다.

제 6 장 모듈 5 — 월드 & 환경 시스템 (041~050)

기술 스택: Tiled Map Editor(JSON) · Phaser 3/PixiJS · TypeScript 본 장은 월드의 공간감과 상호작용을 담당한다. Tiled 파싱에서 blocked 그리드가 서버·클라 공유의 단일 진실이 되고, Y-Sorting 엔진과 A* 길찾기가 탑다운 뷰의 2대 엔진이다. 동적 조명·날씨 파티클이 환경 몰입을 만들고, 포탈·채집·미니맵·돌발 이벤트·핫스팟 상호작용이 필드를 살아있는 공간으로 완성한다.

041. Tiled 타일맵 레이어 (바닥, 벽, 상단 가림막) 파싱 및 렌더링

(1) 역할 설명

Tiled 에디터의 JSON 맵은 계층(layer) 단위로 의미를 부여한다. 표준 구성은 floor(바닥), wall(충돌+통과 불가), overlay(상단 가림막 — 나뭇가지/다리 아래로 캐릭터가 지나가면 위에 겹쳐 보이는 계층), objects(포탈/NPC/채집물)다. 파서는 Tiled JSON 을 로드해 Phaser 타일맵으로 등록하고, 충돌 레이어는 0/1 그리드로 변환해 서버·클라가 공유하는 blocked 그리드(043 번)를 만든다.

상단 가림막(overlay)의 핵심은 렌더 순서와 투명도다. 캐릭터가 overlay 타일 아래에 위치하면 해당 타일을 반투명(30~40% 알파)으로 전환해 캐릭터가 가려지지 않게 한다. 맵이 클수록(3000x3000) 타일 레이어는 청크 단위로 분할 렌더(카메라 시야 밖 타일 그리기 생략)해 드로우콜과 메모리를 관리한다.

(2) 구현 코드 — src/world/TiledLoader.ts

```
import Phaser from "phaser";

export interface TiledMapJson {
  width: number; height: number; tilewidth: number; tileheight: number;
  layers: TiledLayerJson[]; tilesets: { firstgid: number; name: string; image: string }[];
}

export interface TiledLayerJson {
  name: string; type: "tilelayer" | "objectgroup";
  data?: number[]; objects?: TiledObjectJson[]; visible?: boolean; opacity?: number;
}

export interface TiledObjectJson {
  id: number; name: string; type: string; x: number; y: number;
  width?: number; height?: number; properties?: { name: string; value: unknown }[];
}

export const FLOOR = "floor", WALL = "wall", OVERLAY = "overlay", OBJECTS = "objects";

export class TiledLoader {
  blocked: Uint8Array; // 서버/클라 공유 충돌 그리드
  tileW = 32; tileH = 32;

  constructor(private scene: Phaser.Scene, private json: TiledMapJson,
    private tilesets: string[]) {
    this.tileW = json.tilewidth; this.tileH = json.tileheight;
    this.blocked = new Uint8Array(json.width * json.height);
  }

  build(): { map: Phaser.Tilemaps.Tilemap } {
    // 1) 타일셋 등록
    for (const ts of this.json.tilesets) {
      if (!this.scene.textures.exists(ts.name)) {
```

```

    this.scene.load.image(ts.name, "/a/maps/" + ts.image);
  }
}
const map = this.scene.make.tilemap({ data: [], tileWidth: this.tileW, tileHeight: this.tileH });

// 2) Tiled JSON 에서 레이어 순서대로 추가 (floor → wall → overlay)
const addToMap = (name: string) => {
  const layer = this.json.layers.find(l => l.name === name && l.type === "tilelayer");
  if (!layer?.data) return null;
  const data2d: number[][] = [];
  for (let y = 0; y < this.json.height; y++) {
    data2d.push(layer.data.slice(y * this.json.width, (y + 1) * this.json.width));
  }
  const t = map.addTilesetImage(name, name, this.tileW, this.tileH, 0, 0!);
  const phLayer = map.createLayer(0, [t], 0, 0!);
  phLayer.putTilesAt(data2d, 0, 0);
  phLayer.setDepth(name === OVERLAY ? 4000 : 0); // 가림막은 캐릭터 위
  return phLayer;
};
addToMap(FLOOR);
const wallLayer = addToMap(WALL);
const overlayLayer = addToMap(OVERLAY);

// 3) 충돌 그리드 구축 — 서버가 그대로 쓰는 blocked 배열
if (wallLayer) {
  wallLayer.forEachTile((tile) => {
    if (tile.index > 0) this.blocked[tile.y * this.json.width + tile.x] = 1;
  });
}

// 4) 오브젝트 레이어는 포탈/NPC/채집물 스폰 데이터로 변환(046/047 번)
const objs = this.json.layers.find(l => l.name === OBJECTS)?.objects ?? [];
this.scene.events.emit("tiled-objects", objs);

return { map };
}

isBlocked(tx: number, ty: number): boolean {
  return this.blocked[ty * this.json.width + tx] === 1;
}
}

/** overlay 반투명 전환: 캐릭터 타일 기준 주변 1 칸 */
export function updateOverlayAlpha(
  overlayLayer: Phaser.Tilemaps.TilemapLayer,
  charTileX: number, charTileY: number,
) {
  for (let dy = -1; dy <= 1; dy++) {
    for (let dx = -1; dx <= 1; dx++) {
      const t = overlayLayer.getTileAt(charTileX + dx, charTileY + dy);
      if (t) t.alpha = 0.35; // 복구는 다음 스캔에서 alpha=1
    }
  }
}

```

```
}
}
```

(3) 성능 최적화 팁

- 레이어 이름(floor/wall/overlay/objects)은 팀 컨벤션으로 고정한다 — 파서가 이름 기반이면 에디터 작업 순서가 자유로워진다.
- blocked 그리드는 맵 로드 시 1 회 구축해 서버로 전송(또는 동일 JSON 양쪽 로드)해야 클라/서버 충돌 판정이 일치한다.
- 3000x3000 맵은 청크(512px) 단위 createLayer 분할이 필수다 — 단일 레이어는 WebGL 텍스처 업로드 한계에 걸린다.
- overlay 투명화는 타일 단위 alpha 변경으로 충분하며, 매 프레임 전체 스캔 대신 이동한 캐릭터 주변만 갱신한다.

042. Y-Sorting (Y 좌표 기준 캐릭터/오브젝트 깊이 정렬) 엔진

(1) 역할 설명

탑다운 뷰에서 캐릭터가 나무 뒤를 지나가면 나무가 앞에 보여야 하고, 아래를 지나가면 캐릭터가 앞에 보여야 한다. Y-Sorting 은 렌더 대상(캐릭터, 오브젝트, 이펙트)을 '기준선(발밑 y) + 레이어 우선순위'로 정렬해 depth 를 부여하는 엔진이다. depth 계산식은 $depth = layer * 100000 + floor(y * 10)$ 로 두어, 레이어 간 충돌 없이 y 단위 0.1px 정밀 정렬을 얻는다.

성능 핵심은 매 프레임 전체 정렬을 피하는 것이다. 등록된 오브젝트를 관리하고, 이동한 오브젝트만 더티로 마킹해 다음 프레임에 부분 재정렬하며, 대규모 필드(수백 개 오브젝트)에서는 시야 컬링과 결합해 정렬 대상 자체를 줄인다. Phaser 는 setDepth 가 비교적 저비용이지만, 수천 개 대상에 매 프레임 sort 를 돌리면 GC 압박이 생기므로 사전 할당 배열을 재사용한다.

(2) 구현 코드 — [src/world/YSortEngine.ts](#)

```
import Phaser from "phaser";

const LAYER = { ground: 0, object: 1, character: 1, fx: 2, uiFx: 3 } as const;
export type Sortable = Phaser.GameObjects.Container | Phaser.GameObjects.Sprite;

export class YSortEngine {
  private items: Sortable[] = [];
  private dirty = new Set<Sortable>();
  private scratch: Sortable[] = []; // GC 방지 재사용 배열

  register(go: Sortable, layer: keyof typeof LAYER = "character") {
    (go as any).__ysortLayer = LAYER[layer];
    this.items.push(go);
    this.markDirty(go);
  }
}
```

```

unregister(go: Sortable) {
  const i = this.items.indexOf(go);
  if (i >= 0) this.items.splice(i, 1);
}
markDirty(go: Sortable) { this.dirty.add(go); }

/** 이동한 오브젝트만 depth 재계산(매 프레임 호출) */
update() {
  if (!this.dirty.size) return;
  // 1) 더티 항목 depth 갱신
  for (const go of this.dirty) {
    const layer = (go as any).__ysortLayer ?? 1;
    const y = (go as Phaser.GameObjects.Components.Origin).y ?? 0;
    go.setDepth(layer * 100000 + Math.floor(y * 10));
  }
  this.dirty.clear();

  // 2) 같은 레이어 내에서 뒤집힌 쌍만 스왑(선택적 무결성 검사)
  //   — 컨테이너 자식 순서가 depth 우선이 아닌 렌더러에서만 필요
}

/** 시야 컬링 결함: 카메라 뷰 밖은 정렬 스킵 */
updateWithCulling(cam: Phaser.Cameras.Scene2D.Camera, margin = 128) {
  const view = cam.worldView;
  for (const go of this.items) {
    const b = go.getBounds();
    if (b.right < view.x - margin || b.left > view.right + margin ||
        b.bottom < view.y - margin || b.top > view.bottom + margin) continue;
    this.dirty.add(go);
  }
  this.update();
}

/** 정렬 무결성 검사(디버그): depth 오름차순 위반 카운트 */
debugInversions(): number {
  this.scratch.length = 0;
  this.scratch.push(...this.items);
  this.scratch.sort((a, b) => a.depth - b.depth);
  let inv = 0;
  for (let i = 1; i < this.scratch.length; i++) {
    const prev = this.scratch[i - 1] as any, cur = this.scratch[i] as any;
    if (prev.__ysortLayer === cur.__ysortLayer && prev.y > cur.y) inv++;
  }
  return inv;
}

// 사용: 캐릭터 컨테이너(027 번 외형 레이어)를 그대로 register
// ysort.register(charContainer, "character");
// 이동 후 매 틱: ysort.markDirty(charContainer);
// 렌더 단계: ysort.updateWithCulling(scene.cameras.main);

```


(3) 성능 최적화 팁

- depth 식(layer*100000 + y*10)은 레이어 간 충돌과 y 정밀도를 동시에 보장한다 — y 정밀도는 맵 타일 크기에 맞춰 조정한다.
- 전체 정렬 대신 더티 마킹 + 시야 컬링만으로 수백 오브젝트 필드가 프레임 1ms 이하로 유지된다.
- 캐릭터는 컨테이너 단위로 정렬하고, 컨테이너 내부(외형 파트, 027 번)는 항상 고정 순서를 유지한다.
- 그림자는 정렬 대상에서 제외(항상 바닥 레이어)해야 캐릭터와 그림자가 뒤바뀌지 않는다.

043. 2D A* (A-Star) 길찾기 알고리즘 및 이동 가능 영역 Grid 연산

(1) 역할 설명

A*는 휴리스틱(대개 유클리드/맨해튼)으로 탐색을 목표 방향으로 유도하는 그리드 최단 경로 알고리즘이다. MMORPG에서는 (1) 클라 클릭 이동 경로, (2) 오토 배틀 접근, (3) 몬스터 추격에 쓰이며, 탐색 빈도가 높아 성능이 곧 프레임이다. 구현은 이진 힙 우선순위 큐 + Uint8Array 그리드 + 부모 노드 Int32Array 로 메모리를 고정해 GC 를 0 으로 만든다.

실무 요구는 세 가지다. 대용량 맵(2000x2000)에서 탐색 시간 상한(예: 8ms)을 두고 초과 시 부분 경로 반환, 재사용을 위한 경로 스무딩(문자형 계단 제거), 그리고 동일 경로 요청 캐시(출발/도착 타일 키). 서버 몬스터 AI 는 클라와 같은 구현(shared 모듈)을 쓰되, 서버는 8 방향 대각선 이동 가능 여부를 blocked 그리드(041 번)에서 재검증해 클라 조작을 신뢰하지 않는다.

(2) 구현 코드 — shared/astar.ts — GC-free A*

```
export class AStar {
  private w: number; private h: number;
  private blocked: Uint8Array;
  private openHeap: number[] = []; // 노드 인덱스 힙(f 값 기준)
  private fScore: Float64Array;
  private gScore: Float64Array;
  private parent: Int32Array;
  private state: Uint8Array; // 0=미방문 1=open 2=closed
  private stamp: Int32Array; // 재사용 검증(탐색마다 +1)
  private curStamp = 0;
  private deadline = 0;

  constructor(w: number, h: number, blocked: Uint8Array) {
    this.w = w; this.h = h; this.blocked = blocked;
    const n = w * h;
    this.fScore = new Float64Array(n);
    this.gScore = new Float64Array(n);
    this.parent = new Int32Array(n);
  }
}
```

```

this.state = new Uint8Array(n);
this.stamp = new Int32Array(n);
}

/** 경로: [x0,y0,x1,y1,...] 타일 좌표. 실패 시 null, 시간 초과 시 부분 경로 */
find(sx: number, sy: number, tx: number, ty: number, timeBudgetMs = 8): number[] | null {
  if (this.isBlocked(tx, ty)) return null;
  const start = sy * this.w + sx, goal = ty * this.w + tx;
  if (start === goal) return [sx, sy, tx, ty];
  if (++this.curStamp === 0x7fffffff) { // 스탬프 오버플로 방지
    this.curStamp = 1; this.state.fill(0); this.stamp.fill(0);
  }
  this.deadline = performance.now() + timeBudgetMs;
  this.openHeap.length = 0;

  const s = this.curStamp;
  this.stamp[start] = s;
  this.gScore[start] = 0;
  this.fScore[start] = this.heur(sx, sy, tx, ty);
  this.parent[start] = -1;
  this.state[start] = 1;
  this.heapPush(start);

  let best = start; // 도달 실패 시 최근접 노드
  let bestH = this.fScore[start];

  while (this.openHeap.length) {
    if (performance.now() > this.deadline) break; // 시간 예산 소진
    const cur = this.heapPop();
    if (this.state[cur] === 2) continue;
    this.state[cur] = 2;

    const cx = cur % this.w, cy = (cur / this.w) | 0;
    if (cur === goal) { best = goal; break; }
    const hNow = this.heur(cx, cy, tx, ty);
    if (hNow < bestH) { bestH = hNow; best = cur; }

    // 8 방향 이웃
    for (let dy = -1; dy <= 1; dy++) {
      for (let dx = -1; dx <= 1; dx++) {
        if (!dx && !dy) continue;
        const nx = cx + dx, ny = cy + dy;
        if (nx < 0 || ny < 0 || nx >= this.w || ny >= this.h) continue;
        if (this.isBlocked(nx, ny)) continue;
        if (dx && dy) { // 대각선: 양쪽 직교가 모두 뚫려야
          if (this.isBlocked(cx + dx, cy) || this.isBlocked(cx, cy + dy)) continue;
        }
        const ni = ny * this.w + nx;
        if (this.state[ni] === 2) continue;
        const stepCost = dx && dy ? 1.414 : 1;
        const g = this.gScore[cur] + stepCost;

```

```

    if (this.stamp[ni] !== s || g < this.gScore[ni]) {
      this.stamp[ni] = s;
      this.gScore[ni] = g;
      this.parent[ni] = cur;
      this.fScore[ni] = g + this.heur(nx, ny, tx, ty);
      this.state[ni] = 1;
      this.heapPush(ni);
    }
  }
}

// 경로 복원(역추적 → 반전)
const path: number[] = [];
let node = best;
if (this.stamp[node] !== s) return null;
while (node !== -1) {
  path.push(node % this.w, (node / this.w) | 0);
  node = this.parent[node];
}
path.reverse();
return path;
}

private isBlocked(x: number, y: number) { return this.blocked[y * this.w + x] === 1; }
private heur(x: number, y: number, tx: number, ty: number) {
  const dx = Math.abs(x - tx), dy = Math.abs(y - ty);
  return (dx + dy) + (1.414 - 2) * Math.min(dx, dy); // 옥타일 거리
}

// — 이진 힙(fScore 기준 최소 힙) —
private heapPush(i: number) {
  const h = this.openHeap; h.push(i);
  let c = h.length - 1;
  while (c > 0) {
    const p = (c - 1) >> 1;
    if (this.fScore[h[p]] <= this.fScore[h[c]]) break;
    [h[p], h[c]] = [h[c], h[p]]; c = p;
  }
}

private heapPop(): number {
  const h = this.openHeap;
  const top = h[0], last = h.pop();
  if (h.length) {
    h[0] = last;
    let p = 0;
    for (;;) {
      const l = p * 2 + 1, r = l + 1;
      let m = p;
      if (l < h.length && this.fScore[h[l]] < this.fScore[h[m]]) m = l;
      if (r < h.length && this.fScore[h[r]] < this.fScore[h[m]]) m = r;
      if (m === p) break;
      [h[p], h[m]] = [h[m], h[p]]; p = m;
    }
  }
}

```

```

    }
  }
  return top;
}
}

```

(3) 성능 최적화 팁

- TypedArray + 스택프 재사용으로 탐색마다 배열을 새로 만들지 않는다 — 몬스터 100 마리 동시 탐색에서 GC 정지가 사라진다.
- 시간 예산(8ms) 초과 시 '최근접 노드까지의 부분 경로'를 반환해 몬스터가 멈추지 않게 한다 — 다음 틱에 재탐색한다.
- 대각선 이동은 양쪽 직교 타일이 모두 뚫려 있을 때만 허용해야 코너 뚫기(벽 끼고 비스듬히 통과) 버그가 없다.
- 경로 캐시는 (start,goal) 키로 최대 200 개 LRU — 오토 배틀 반복 요청에서 CPU 를 크게 아낀다.

044. 실시간 동적 2D 조명 & 낮밤 시간 변화 셰이더/파티클

(1) 역할 설명

2D 조명은 보통 '전체 어둠 레이어(ambient) + 광원(원형 그라디언트) 마스크' 구조로 만든다. 렌더는 (1) 월드를 렌더, (2) 조명 텍스처(라이트맵)를 별도 렌더 타겟에 그림, (3) 월드 위에 multiply/normal 블렌드로 합성 — Phaser에서는 RenderTexture + BlendMode.MULTIPLY 조합으로 구현한다. 광원은 캐릭터 횃불, 가로등, 화염, 마법 오라 등이며 각 광원은 반경·색상·깜빡임 파라미터를 가진다.

낮밤 사이클은 게임 내 시간(예: 실제 30 분 = 게임 하루)에 따라 ambient 색과 강도를 보간한다. 낮(밝은 ambient) → 황혼(주황 틸트) → 밤(짙은 남색 + 광원 강조) → 새벽 순서로 색상 그라디언트를 두고, 몬스터 스폰 테이블과 연동해 밤에만 등장하는 위험 구역을 만들면 콘텐츠가 깊어진다. 파티클(불티, 반딧불)은 광원 강도와 연동해 밤에만 활성화한다.

(2) 구현 코드 — src/world/DayNightLighting.ts

```

import Phaser from "phaser";

interface LightSource {
  obj: Phaser.GameObjects.Components.Transform; // 따라갈 오브젝트
  radius: number; color: number; intensity: number;
  flicker?: number; // 0~1 깜빡임 강도
}

/** 게임 하루 = 실제 30 분. 시간대별 ambient 색상 */
const PHASES = [
  { at: 0.00, color: 0x2a3550, name: "night" }, // 0 시
  { at: 0.23, color: 0x5a5070, name: "dawn" }, // 새벽

```

```

{ at: 0.30, color: 0xfffff, name: "day" },    // 아침
{ at: 0.62, color: 0xff3dd, name: "afternoon" },
{ at: 0.78, color: 0xe8a060, name: "dusk" },    // 황혼
{ at: 0.86, color: 0x2a3550, name: "night" },
];

export class DayNightLighting {
  private rt: Phaser.GameObjects.RenderTexture;
  private lights: LightSource[] = [];
  private dayT = 0.4;          // 0~1 (0.4 = 오전)

  constructor(private scene: Phaser.Scene, w: number, h: number) {
    this.rt = scene.make.renderTexture({ width: w, height: h }, false);
    this.rt.setBlendMode(Phaser.BlendModes.MULTIPLY).setDepth(8000);
  }

  addLight(src: LightSource) { this.lights.push(src); }

  /** 게임 시간 진행(서버 동기 — 클라 로컬 시간은 신뢰하지 않음) */
  setDayTime(t: number) { this.dayT = Phaser.Math.Wrap(t, 0, 1); }
  get isNight() { return this.dayT < 0.23 || this.dayT > 0.86; }

  /** 렌더 프레임: 라이트맵 생성 */
  render(cam: Phaser.Cameras.Scene2D.Camera, time: number) {
    this.rt.clear();
    // 1) ambient 색 채움(시간대 보간)
    const amb = this.ambientColor();
    this.rt.fill(amb, 1);
    // 2) 광원: ERASE 블렌드로 어둠을 뚫는 구멍
    for (const l of this.lights) {
      const flicker = l.flicker
        ? 1 - Math.sin(time / 90) * l.flicker * 0.5 - Math.random() * l.flicker * 0.1
        : 1;
      const r = l.radius * flicker;
      const g = this.scene.make.graphics({ x: 0, y: 0 }, false);
      const cx = l.obj.x - cam.scrollX, cy = l.obj.y - cam.scrollY;
      const grad = g.createRadialGradient(cx, cy, r * 0.15, cx, cy, r);
      // Phaser 3.60+ Graphics gradient API — 내부에서 canvas gradient 구성
      grad.addColorStop(0, "rgba(0,0,0,1)");    // 중심 = 완전 밝음
      grad.addColorStop(1, "rgba(0,0,0,0)");
      g.fillCircle(cx, cy, r);
      this.rt.erase(g, cx - r, cy - r);
      g.destroy();
    }
  }

  private ambientColor(): number {
    const t = this.dayT;
    for (let i = 0; i < PHASES.length; i++) {
      const a = PHASES[i], b = PHASES[(i + 1) % PHASES.length];

```

```

const next = b.at <= a.at ? b.at + 1 : b.at;
if (t >= a.at && t < next) {
  const r = (t - a.at) / (next - a.at);
  return Phaser.Display.Color.Interpolate.ColorWithColor(
    Phaser.Display.Color.IntegerToColor(a.color),
    Phaser.Display.Color.IntegerToColor(b.color),
    100, r * 100,
  ).color;
}
}
return 0xffffffff;
}
}

```

(3) 성능 최적화 팁

- 라이트맵은 반드시 별도 RenderTexture 에 그려 한 번의 multiply 합성으로 마무리한다
— 광원마다 캔버스에 직접 그리면 드로우콜이 폭증한다.
- 깜빡임(flicker)은 sin + 약간의 랜덤 조합이 가장 자연스럽고, 순수 랜덤은 렌더 시간에 따라 성능이 불안정하다.
- 낮밤 시간은 서버가 단일 권위로 방송(맵 진입 시 1 회 + 분당 1 회 보정)해야 유저 간 시간이 어긋나지 않는다.
- 밤 전용 스폰/채집 콘텐츠는 isNight 플래그 하나로 게이팅하면 기획 데이터만으로 켜고 끌 수 있다.

045. 날씨(비, 눈, 안개) 환경 파티클 시스템

(1) 역할 설명

날씨 파티클은 카메라 시야를 채우는 스크린 공간 효과다. 비는 얇은 라인 스프라이트를 고속 낙하 + 바람 경사, 눈은 작은 원 스프라이트를 사인과 흔들림 + 저속 낙하, 안개는 큰 반투명 텍스처를 천천히 표류시킨다. 핵심은 파티클 수를 시야 크기와 날씨 강도(0~1)에 비례해 조절하고, 화면 밖으로 나간 파티클을 상단으로 재활용(순환)해 스폰 비용을 0으로 만드는 것이다.

Phaser 3.60+의 Particle Emitter 를 쓰면 웹 워커 없이도 GPU 프레임 내에 수백 파티클을 처리하지만, 대규모 눈보라는 프레임 예산(001 번)과 연동해 파티클 수를 동적으로 줄인다. 날씨 전환은 즉시가 아니라 3~5 초 보간(강도 램프)으로 진행해 화면이 뚝 끊기는 느낌을 없앤다. 서버가 날씨 상태(맵별 스케줄)를 권위로 관리하고 클라는 표현만 담당한다.

(2) 구현 코드 — src/world/WeatherFX.ts

```

import Phaser from "phaser";

export type WeatherKind = "clear" | "rain" | "snow" | "fog";
export interface WeatherState { kind: WeatherKind; intensity: number; wind: number; }

```

```

export class WeatherFX {
  private emitter: Phaser.GameObjects.Particles.ParticleEmitter | null = null;
  private fogLayers: Phaser.GameObjects.TileSprite[] = [];
  private current: WeatherState = { kind: "clear", intensity: 0, wind: 0 };
  private target: WeatherState = { kind: "clear", intensity: 0, wind: 0 };

  constructor(private scene: Phaser.Scene) {}

  /** 서버 날씨 방송 수신 */
  setState(s: WeatherState) { this.target = s; }

  /** 고정 업데이트: 4 초 램프 보간 */
  fixedUpdate(dtMs: number, cam: Phaser.Cameras.Scene2D.Camera) {
    const k = dtMs / 4000; // 보간 속도
    this.current.intensity += (this.target.intensity - this.current.intensity) * k;
    this.current.wind += (this.target.wind - this.current.wind) * k;
    if (this.current.kind !== this.target.kind && this.current.intensity < 0.05) {
      this.switchKind(this.target.kind);
    }
    this.apply(cam, dtMs);
  }

  private switchKind(kind: WeatherKind) {
    this.destroyEmitters();
    this.current.kind = kind;
    if (kind === "rain" || kind === "snow") {
      const tex = kind === "rain" ? "wx_raindrop" : "wx_snowflake";
      this.emitter = this.scene.add.particles(0, 0, tex, {
        x: { min: 0, max: this.scene.scale.width + 200 },
        y: -20,
        lifespan: kind === "rain" ? 900 : 6000,
        speedY: kind === "rain" ? { min: 700, max: 950 } : { min: 40, max: 90 },
        speedX: { min: -60, max: 60 }, // 바람은 apply 에서 곱연산
        scale: kind === "rain" ? { min: 0.7, max: 1.2 } : { min: 0.4, max: 1.0 },
        alpha: kind === "rain" ? { min: 0.4, max: 0.8 } : { min: 0.7, max: 1 },
        quantity: 4,
        frequency: 40,
        blendMode: "SCREEN",
      });
      this.emitter.setDepth(8500);
      this.emitter.setScrollFactor(0); // 스크린 공간 고정
    }
    if (kind === "fog") {
      for (let i = 0; i < 2; i++) {
        const fog = this.scene.add.tileSprite(0, i * 120,
          this.scene.scale.width, this.scene.scale.height, "wx_fog");
        fog.setOrigin(0).setAlpha(0).setDepth(8400 + i).setScrollFactor(0.3 + i * 0.2);
        this.fogLayers.push(fog);
      }
    }
  }
}

```

```
private apply(cam: Phaser.Cameras.Scene2D.Camera, dtMs: number) {
  const i = this.current.intensity;
  if (this.emitter) {
    this.emitter.frequency = 40 / Math.max(0.05, i); // 강도 → 스폰 빈도
    this.emitter.ops.speedX.onChange(this.current.wind * (this.current.kind === "rain" ? 3 : 1));
  }
  this.fogLayers.forEach((fog, idx) => {
    fog.setAlpha(Math.min(0.55, i * 0.6));
    fog.tilePositionX += (0.4 + idx * 0.25) * dtMs / 16.67 + this.current.wind * 0.02;
  });
}

private destroyEmitters() {
  this.emitter?.destroy(); this.emitter = null;
  this.fogLayers.forEach(f => f.destroy()); this.fogLayers = [];
}
}
```

(3) 성능 최적화 팁

- 파티클은 setScrollFactor(0)로 스크린 공간에 두면 카메라 이동과 무관하게 시야를 채운다.
- 강도 보간(4 초 램프)은 종류 전환 시 0.05 이하로 내린 뒤에만 emitter 를 갈아끼워 화면 끊김을 막는다.
- 눈은 speedY 를 40~90 으로 낮추고 사인 흔들림(scaleY 진동)을 주면 가격 없이 자연스러워진다.
- 날씨는 맵별 스케줄(서버 권위) + 이벤트 날씨(보스전 뇌우 등)를 구분해 데이터로 운영한다.

046. 포탈 진입 및 인스턴스 맵 이공간 텔레포트 매핑

(1) 역할 설명

포탈은 '트리거 영역 + 전송 규칙'의 데이터다. 필드 포탈은 맵 A 좌표 → 맵 B 좌표의 단순 전송이고, 던전/인스턴스 포탈은 채널/룸(017 번) 할당 후 새 인스턴스 공간 좌표로 전송된다. 포탈 정의는 Tiled objects(041 번)에서 파싱해 (x, y, 범위, 목적지, 조건) 구조로 등록하며, 조건 (레벨, 퀘스트, 파티) 불충분 시 진입 차단 + 사유 안내를 한다.

전송 절차는 서버 권위다. 클라가 포탈 영역에 진입하면 서버는 (1) 조건 검증, (2) 목적지 맵/좌표 결정, (3) AOI 구독 재계산(013 번), (4) 세이브 위치 갱신, (5) 클라에 맵 전환 명령을 순차 수행한다. 클라는 맵 로딩(004 번 번들) 후 스폰되며, 전송 중 입력은 큐잉하지 않고 폐기해 '전송 완료 후 잔여 입력으로 다시 포탈 진입' 루프 버그를 막는다.

(2) 구현 코드 — server/world/Portal.ts

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;
```



```

export interface PortalDef {
  id: string;
  fromMap: string; x: number; y: number; radius: number;
  toMap: string; toX: number; toY: number;
  kind: "field" | "instance";
  reqLevel?: number;
  reqQuest?: string;
  cooldownMs?: number;      // 재진입 쿨타임(진입 루프 방지)
}

export interface TeleportResult {
  ok: boolean; reason?: string;
  mapId?: string; x?: number; y?: number; roomId?: string;
}

export class PortalSystem {
  private lastUse = new Map<number, number>(); // charId → 마지막 사용

  constructor(private portals: PortalDef[],
    private rooms: import("../room/RoomRegistry").RoomRegistry,
    private db: Kysely<DB>) {}

  /** 이동 패킷에서 트리거 검사(서버 틱) */
  check(charId: number, mapId: string, x: number, y: number, level: number, now: number):
  TeleportResult {
    for (const p of this.portals) {
      if (p.fromMap !== mapId) continue;
      if (Math.hypot(p.x - x, p.y - y) > p.radius) continue;

      const last = this.lastUse.get(charId) ?? 0;
      if (now - last < (p.cooldownMs ?? 2000)) return { ok: false, reason: "cooldown" };
      if (p.reqLevel && level < p.reqLevel) return { ok: false, reason: "low_level" };

      this.lastUse.set(charId, now);
      return this.teleport(charId, p, now);
    }
    return { ok: false };
  }

  private async teleport(charId: number, p: PortalDef, now: number): Promise<TeleportResult>
  {
    if (p.kind === "field") {
      // 필드: 채널 자동 배정(017 번) + 세이브 갱신
      const room = await this.rooms.pickFieldChannel(p.toMap, 1);
      await this.rooms.join(room.roomId);
      await this.db.updateTable("characters").set({
        map_id: p.toMap, x: p.toX, y: p.toY,
      }).where("id", "=", charId).execute();
      return { ok: true, mapId: p.toMap, x: p.toX, y: p.toY, roomId: room.roomId };
    }
  }
}

```

```
// 인스턴스: 전용 룸 생성/참여
const room = await this.rooms.create({ kind: "dungeon", mapId: p.toMap, capacity: 5 });
await this.rooms.join(room.roomId);
return { ok: true, mapId: p.toMap, x: p.toX, y: p.toY, roomId: room.roomId };
}
}

// 클라 측 흐름(발체):
// 1) 서버 S2C_TELEPORT { mapId, x, y, roomId } 수신
// 2) AssetFlow.request(bundlesFor(mapId)) → 로딩 오버레이
// 3) 현재 씬 shutdown → 새 WorldScene.start({ mapId, x, y })
// 4) 전송 중 이동/스킬 입력은 폐기(큐잉 금지 — 포탈 루프 방지)
```

(3) 성능 최적화 팁

- 포탈 쿨타임(2 초)은 '스폰 지점이 포탈 위'인 맵 설계 실수를 안전망으로 커버하는 표준 장치다.
- 전송 중 입력 폐기는 원칙이다 — 큐잉하면 스폰 직후 캐릭터가 저절로 걸어 나가는 신기한 버그가 된다.
- 인스턴스 포탈은 반드시 룸 상태(DRAINING 등)를 확인해 회수 직전 룸에 들어가는 레이스를 막는다.
- 포탈 시각화(반짝임, 경계선)는 클라 전용 표현이며 판정 반경(radius)과 시각 크기는 별개로 튜닝한다.

047. 필드 채집/채광 리젠 오브젝트 상호작용

(1) 역할 설명

채집 오브젝트(허브, 광맥, 나무)는 '스폰 지점 정의 + 상태(사용 가능/진행중/재생대기) + 리젠 타이머'의 구조다. 유저가 상호작용(E 키/터치)하면 서버는 (1) 거리 검증, (2) 오브젝트 상태 점유(동시 채집 방지), (3) 채집 시간(틱 게이지) 진행, (4) 결과 드롭 지급, (5) 리젠 예약의 순서로 처리한다. 리젠 시간은 오브젝트 종류별 밸런스 테이블로 관리하며, 인기 필드는 개체수(스폰 밀도)로 조절한다.

멀티플레이 핵심은 '점유 상태 동기화'다. A가 채집 중이면 B에게는 진행 바가 표시되고 B는 시도할 수 없다. 채집 중 이동/피격 시 취소되며, 오브젝트는 다시 사용 가능 상태로 복귀한다. 드롭은 재화 트랜잭션(018 번)이 아닌 인벤토리 지급 트랜잭션으로 처리하고, 가방이 가득 차면 지급 실패 + 안내를 한다. 채집 기록은 도감(030 번)·업적(028 번) 이벤트로 흘려보낸다.

(2) 구현 코드 — `server/world/GatherNode.ts`

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;
```

```

export interface NodeDef {
  code: string;           // "herb_sunflower"
  type: "herb" | "ore" | "tree";
  gatherMs: number;       // 채집 소요
  respawnMs: number;      // 리젠 시간
  drops: { itemCode: string; qty: number; chance: number }[];
  reqLevel?: number;
}

export interface NodeState {
  defCode: string; mapId: string; x: number; y: number;
  status: "ready" | "gathering" | "respawning";
  gathererId: number | null;
  readyAt: number;        // respawning → ready 전환 시각
}

export class GatherSystem {
  private nodes = new Map<string, NodeState>();

  /** 상호작용 시작(서버: 점유 확정) */
  startGather(charId: number, nodeId: string, x: number, y: number, level: number, now: number) {
    const n = this.nodes.get(nodeId);
    if (!n || n.status !== "ready") return { ok: false, reason: "not_ready" };
    if (Math.hypot(n.x - x, n.y - y) > 48) return { ok: false, reason: "far" };

    const def = this.defs(n.defCode);
    if (def.reqLevel && level < def.reqLevel) return { ok: false, reason: "low_level" };

    n.status = "gathering";
    n.gathererId = charId;
    return { ok: true, gatherMs: def.gatherMs }; // 클라에 진행 바 시간 통보
  }

  /** 채집 완료(클라 게이지 완료 통보 수신 — 서버도 시간 재검증) */
  async completeGather(db: Kysely<DB>, charId: number, nodeId: string, now: number) {
    const n = this.nodes.get(nodeId)!;
    const def = this.defs(n.defCode);
    if (n.gathererId !== charId) return { ok: false, reason: "not_owner" };

    // 드롭 추출(확률 합산) — 인벤토리 지급 트랜잭션
    const drops: { itemCode: string; qty: number }[] = [];
    for (const d of def.drops) {
      if (Math.random() < d.chance) drops.push({ itemCode: d.itemCode, qty: d.qty });
    }
    await db.transaction().execute(async tx => {
      for (const d of drops) {
        await tx.updateTable("character_items")
          .set(eb => ({ qty: eb("qty", "+", d.qty) }))
          .where("character_id", "=", charId)
          .where("item_code", "=", d.itemCode).execute();
      }
    })
  }
}

```

```

    // 도감/업적 이벤트 혹은(028/030 번)은 큐로
    });

    // 리젠 예약
    n.status = "respawning";
    n.gathererId = null;
    n.readyAt = now + def.respawnMs;
    return { ok: true, drops };
}

/** 서버 틱: 리젠 전환 */
tick(now: number) {
    for (const n of this.nodes.values()) {
        if (n.status === "respawning" && now >= n.readyAt) n.status = "ready";
    }
}

cancelGather(nodeId: string, charId: number) {
    const n = this.nodes.get(nodeId);
    if (n?.gathererId === charId) { n.status = "ready"; n.gathererId = null; }
}

private defs(code: string): NodeDef { return this.defTable[code]; }
private defTable: Record<string, NodeDef> = {
    herb_sunflower: { code: "herb_sunflower", type: "herb", gatherMs: 1800,
        respawnMs: 60000, drops: [{ itemCode: "herb_sun", qty: 2, chance: 0.8 }] },
    ore_iron: { code: "ore_iron", type: "ore", gatherMs: 3200,
        respawnMs: 180000, drops: [{ itemCode: "ore_iron", qty: 1, chance: 1 }] },
};
}

```

(3) 성능 최적화 팁

- 채집 점유는 서버가 단일 권위로 확정해야 동시 채집 복제 버그가 없다 — 클라는 게이지 표시만 한다.
- 리젠은 고정 시간 + 지터($\pm 20\%$)를 섞어 리젤 패턴 자동화(봇)를 어렵게 만든다.
- 인기 필드는 리젤 시간이 아니라 스폰 개체수로 조절해야 '채집 경쟁'이 게임성으로 유지된다.
- 채집 중 피격/이동 취소는 클라와 서버 양쪽에서 처리하되, 최종 상태는 서버가 복구한다.

048. 미니맵 동기화 및 안개(Fog of War) 탐색 가림 처리

(1) 역할 설명

미니맵은 (1) 맵 기본 이미지(타일 레이어 축소본), (2) 유저/NPC/파티원 마커, (3) 안개(fog of war)의 3 계층 합성이다. 안개는 맵을 4x4 타일 셀 단위로 나누고, 방문/시야 데이터를 비트맵으로 관리한다. 시야(현재 보이는 셀)는 밝게, 과거 방문(기억)은 흐리게, 미방문은

완전히 가린다. 데이터는 서버가 권위로 유지해(대형 맵에서 치트 방지) 셀 단위로 클라에 델타 전송한다.

미니맵 렌더는 `RenderTexture` 에 기본 맵 + 안구 비트맵을 합성하고, 마커는 별도 스프라이트로 위에 올린다. 성능을 위해 안개 갱신은 캐릭터가 셀을 넘어설 때만 수행하고, 파티원 위치는 서버가 1초 주기로 방송(051 번 채팅 토픽 재사용)한다. 대형 맵(2000x2000)은 미니맵을 256x256 으로 스케일 다운해 메모리를 수십 KB 로 유지한다.

(2) 구현 코드 — `src/world/Minimap.ts`

```
import Phaser from "phaser";

const CELL = 4;          // 안개 셀 = 4x4 타일

export class Minimap {
  private fogTex: Phaser.GameObjects.RenderTexture;
  private visited: Uint8Array;    // 방문(기억)
  private visible: Uint8Array;    // 현재 시야
  private w: number; private h: number; // 셀 단위 크기

  constructor(private scene: Phaser.Scene,
    mapW: number, mapH: number,
    private baseImg: Phaser.GameObjects.Image,
    private playerMark: Phaser.GameObjects.Ellipse,
    private partyMarks: Map<number, Phaser.GameObjects.Ellipse> = new Map()) {
    this.w = Math.ceil(mapW / CELL); this.h = Math.ceil(mapH / CELL);
    this.visited = new Uint8Array(this.w * this.h);
    this.visible = new Uint8Array(this.w * this.h);

    // 안개 레이어: 검정 채움 → 방문 셀은 반투명 구멍
    this.fogTex = scene.make.renderTexture({
      width: this.w, height: this.h,
    }, false).setOrigin(0);
    this.fogTex.fill(0x000000, 1);
  }

  /** 캐릭터 이동 → 셀 단위 시야 갱신(셀 경계 통과 시만) */
  reveal(px: number, py: number, viewCells = 3) {
    const cx = Math.floor(px / 32 / CELL), cy = Math.floor(py / 32 / CELL);
    for (let dy = -viewCells; dy <= viewCells; dy++) {
      for (let dx = -viewCells; dx <= viewCells; dx++) {
        const x = cx + dx, y = cy + dy;
        if (x < 0 || y < 0 || x >= this.w || y >= this.h) continue;
        const i = y * this.w + x;
        this.visible[i] = 1;
        if (!this.visited[i]) {
          this.visited[i] = 1;
          // 기억 상태: 안개에서 반투명 구멍 (서버에도 보고 — 권위 동기화)
          this.scene.events.emit("fog-revealed", x, y);
        }
      }
    }
  }
}
```

```

    }
  }
}
}

/** 서버 델타 수신(타기기 동기화): 재접속 시 방문 기록 복원 */
applyServerFog(cells: { x: number; y: number }[]) {
  for (const c of cells) this.visited[c.y * this.w + c.x] = 1;
}

/** 미니맵 리렌더(셀 갱신 시만) */
redraw(px: number, py: number, party: Map<number, { x: number; y: number }>) {
  this.fogTex.clear();
  this.fogTex.fill(0x0a0f18, 0.92); // 미방문 가림
  for (let i = 0; i < this.visited.length; i++) {
    if (this.visited[i]) {
      const x = i % this.w, y = (i / this.w) | 0;
      this.fogTex.fill(0x0a0f18, 0.35, x, y, 1, 1); // 기억 = 흐리게
    }
  }
}
// 마커 갱신
this.playerMark.setPosition(px / 32 / CELL * this.minimapScale(),
  py / 32 / CELL * this.minimapScale());
for (const [id, p] of party) {
  const mark = this.partyMarks.get(id);
  if (mark) mark.setPosition(p.x / 32 / CELL * this.minimapScale(),
    p.y / 32 / CELL * this.minimapScale());
}
}
private minimapScale() { return 128 / Math.max(this.w, this.h); }
}

```

(3) 성능 최적화 팁

- 안개는 셀 단위(4x4 타일)가 성능/시각의 균형점이다 — 타일 단위는 데이터가 16배 커지고 8x8은 거칠다.
- 방문 기록은 서버 권위로 유지해야 재접속 복원과 치트(맵 전체 해킹)가 동시에 해결된다.
- 미니맵 리렌더는 셀 갱신 시만 — 매 프레임 RenderTexture 리드로우는 저사양 모바일에서 프레임을 잡아먹는다.
- 파티원 마커는 1초 방송(채팅 토픽 재사용)이면 충분하며, 스무딩(보간)으로 이동이 매끄럽게 보인다.

049. 필드 돌발 이벤트 영역 감지 및 트리거

(1) 역할 설명

돌발 이벤트는 '조건(시간/밀도/확률) → 영역 지정 → 이벤트 실행(스폰/연출) → 종료 처리'의 라이프사이클을 가진다. 예시는 도망치는 상자 몬스터, 몬스터 무리 습격, 떨어지는 유성, 상인 등장 등이다. 이벤트 정의는 { id, trigger, area, duration, onSpawn, onEnd }의 데이터 구조로 서버가 관리하고, 영역 진입 감지는 AOI 그리드(013 번)를 재활용해 몬스터 틱과 함께 평가한다.

트리거 종류는 (1) 시간 스케줄(하루 N 회), (2) 플레이어 밀도(구역 유저 3명 이상), (3) 확률(처치 시 1%), (4) 퀘스트 연동(특정 퀘스트 진행 중)이 표준이다. 이벤트는 구역별 동시 실행 상한(예: 2개)을 두어 스팸을 막고, 종료 시 보상은 기여도(딜량/참여 시간) 기반으로 분배한다(064 번 필드 보스와 동일 파이프라인). 클라는 이벤트 시작/종료 방송을 받아 배너 연출을 표시한다.

(2) 구현 코드 — server/world/FieldEvent.ts

```
export interface EventDef {
  id: string; // "ambush_wolves"
  name: string;
  trigger: {
    kind: "density" | "schedule" | "probability" | "quest";
    minPlayers?: number; // density
    cronExpr?: string; // schedule (서버 스케줄러 파싱)
    chance?: number; // probability (per kill)
    questCode?: string; // quest
  };
  area: { x: number; y: number; r: number };
  durationMs: number;
  spawn: { monsterCode: string; count: number; level: number }[];
  maxActivePerZone: number;
}

export interface ActiveEvent {
  defId: string; zone: string; startedAt: number; endsAt: number;
  spawnedIds: number[];
  participants: Map<number, { dmg: number; joinedAt: number }>;
}

export class FieldEventManager {
  private active = new Map<string, ActiveEvent>(); // key = zone:defId

  /** 몬스터 틱과 함께 평가(서버) */
  tryTrigger(zone: string, playersInZone: number, questState: string | null, now: number) {
    for (const def of this.definitions) {
      const key = zone + ":" + def.id;
      if (this.active.has(key)) continue; // 진행 중 스킵
      const zoneCount = [...this.active.keys()].filter(k => k.startsWith(zone + ":")).length;
```

```

if (zoneCount >= def.maxActivePerZone) continue;

const t = def.trigger;
let go = false;
switch (t.kind) {
  case "density": go = playersInZone >= (t.minPlayers ?? 3); break;
  case "probability": go = Math.random() < (t.chance ?? 0.01); break;
  case "quest": go = questState === t.questCode; break;
  case "schedule": go = false; // 스케줄러 콜백이 별도 호출
}
if (!go) continue;

const ev: ActiveEvent = {
  defId: def.id, zone, startedAt: now, endsAt: now + def.durationMs,
  spawnedIds: this.spawnMonsters(def, now),
  participants: new Map(),
};
this.active.set(key, ev);
this.broadcast(zone, { t: "event_start", id: def.id, x: def.area.x, y: def.area.y });
}
}

/** 이벤트 종료: 기여도 기반 보상(064 번과 동일 로직) */
sweep(now: number) {
  for (const [key, ev] of this.active) {
    if (now < ev.endsAt) continue;
    const def = this.defs.find(d => d.id === ev.defId!);
    const totalDmg = [...ev.participants.values()].reduce((s, p) => s + p.dmg, 0);
    for (const [userId, p] of ev.participants) {
      const share = totalDmg > 0 ? p.dmg / totalDmg : 1 / ev.participants.size;
      const bonus = share > 0.3 ? "rare" : "common"; // 상위 기여자 추가 보상
      this.grantReward(userId, def, bonus, share);
    }
    this.broadcast(ev.zone, { t: "event_end", id: ev.defId });
    this.active.delete(key);
  }
}

onDamage(evKey: string, userId: number, dmg: number) {
  const ev = this.active.get(evKey);
  if (!ev) return;
  const p = ev.participants.get(userId) ?? { dmg: 0, joinedAt: Date.now() };
  p.dmg += dmg;
  ev.participants.set(userId, p);
}

private spawnMonsters(def: EventDef, now: number): number[] { return []; }
private broadcast(zone: string, msg: unknown) {}
private grantReward(userId: number, def: EventDef, tier: string, share: number) {}
private defs: EventDef[] = [];
}

```


(3) 성능 최적화 팁

- 밀도 트리거(minPlayers)는 '구역 접속 유저 수'가 아니라 '이벤트 영역 반경 내 유저 수'로 계산해야 도심 옥외 이벤트가 자연스럽게.
- 이벤트 동시 실행 상한(zone 당 2개) 없이는 확률 트리거가 스팸 폭탄이 된다.
- 기여도 분배는 딜량 비율 + 최소 참여 시간(10 초) 게이팅으로 낚시꾼(그냥 서 있는 유저)을 걸러낸다.
- 클라 배너는 이벤트 시작 3 초 전 예고 방송이 있으면 참여율이 눈에 띄게 올라간다.

050. 핫스팟 휴식/앉기 등 오브젝트 상호작용 연출

(1) 역할 설명

핫스팟(모닥불, 벤치, 우물)은 '상호작용 가능 오브젝트 + 애니메이션 상태'다. 유저가 E 키/터치로 상호작용하면 캐릭터가 지정 자리로 이동 후 앉기 애니메이션으로 전환되고 휴식 상태에서는 HP/MP 자연 회복 배율(예: x3)이 적용된다. 상호작용 종료는 이동 입력, 피격, 일정 시간 경과로 트리거된다.

구현은 세 계층이다. 데이터(핫스팟 정의 — 위치, 타입, 회복 배율, 최대 동시 인원), 서버 (상태 검증, 회복 적용, 종료 판정), 클라(이동 → 앉기 애니메이션 → 파티클/연출). 앉기 자리(seat)는 오브젝트별 좌표 목록으로 관리해 벤치에 3명이 각각 다른 좌표로 앉게 하며, 이미 점유된 자리는 다른 유저가 볼 때 '사용 중'으로 표시된다.

(2) 구현 코드 — src/world/Hotspot.ts — 클라 상호작용

```
import Phaser from "phaser";

export interface HotspotDef {
  id: string;
  kind: "campfire" | "bench" | "well";
  x: number; y: number;
  seats: { x: number; y: number; facing: number }[]; // 앉는 좌표/방향
  regenMult: number; // 3 = x3 회복
  interactRange: number;
}

export interface SeatState { occupiedBy: number | null; }

export class HotspotManager {
  private spots: HotspotDef[] = [];
  private seats = new Map<string, SeatState>(); // key = spotId:seatIdx
  private sitting: { spotId: string; seatIdx: number } | null = null;
  onRegenMult: ((mult: number) => void) | null = null;

  register(def: HotspotDef) {
    this.spots.push(def);
    def.seats.forEach((_, i) => this.seats.set(def.id + ":" + i, { occupiedBy: null }));
  }
}
```

```

/** E 키/터치 → 가장 가까운 핫스팟에 빈 자리 요청(서버 승인 필요) */
requestInteract(charX: number, charY: number): { spot: HotspotDef; seat: number } | null {
  let best: { spot: HotspotDef; seat: number; d: number } | null = null;
  for (const s of this.spots) {
    if (Math.hypot(s.x - charX, s.y - charY) > s.interactRange) continue;
    s.seats.forEach((seatPos, i) => {
      const st = this.seats.get(s.id + ":" + i!);
      if (st.occupiedBy !== null) return; // 점유 중
      const d = Math.hypot(seatPos.x - charX, seatPos.y - charY);
      if (!best || d < best.d) best = { spot: s, seat: i, d };
    });
  }
  if (!best) return null;
  const b = best as { spot: HotspotDef; seat: number; d: number };
  return { spot: b.spot, seat: b.seat };
}

/** 서버 승인 후: 이동 → 앉기 애니메이션 */
async perform(scene: Phaser.Scene, char: Phaser.GameObjects.Container,
  spot: HotspotDef, seatIdx: number, approve: (spotId: string, seat: number) =>
  Promise<boolean>) {
  const seat = spot.seats[seatIdx];
  const ok = await approve(spot.id, seatIdx);
  if (!ok) return false;

  const dist = Phaser.Math.Distance.Between(char.x, char.y, seat.x, seat.y);
  // 1) 자리로 걸어가기(속도 140px/s — 022 번 이동 속도 재사용)
  await new Promise<void>(res => {
    scene.tweens.add({
      targets: char, x: seat.x, y: seat.y,
      duration: dist / 140 * 1000,
      onComplete: () => res(),
    });
  });
  // 2) 방향 회전 + 앉기 프레임
  char.setFlipX(seat.facing < 0);
  (char as any).playAnim?.("sit_down"); // 외형 애니메이션 시스템 호출
  this.onRegenMult?.(spot.regenMult);
  this.sitting = { spotId: spot.id, seatIdx };
  return true;
}

/** 종료: 이동/피격/쿨타임 */
standUp(reason: "move" | "hit" | "manual") {
  if (!this.sitting) return;
  this.seats.get(this.sitting.spotId + ":" + this.sitting.seatIdx)!.occupiedBy = null;
  this.onRegenMult?.(1);
  this.sitting = null;
}
get isSitting() { return this.sitting !== null; }
}

```

(3) 성능 최적화 팁

- 앓기는 '이동 완료 후 애니메이션 전환' 2 단계다 — 즉시 앓으면 캐릭터가 좌표를 순간이동한 것처럼 보인다.
- 회복 배율은 서버 권위로 적용하고 클라는 표시만 — 접속 끊김 회복 어뷰징이 서버 검증으로 차단된다.
- 좌석 점유 상태는 주변 유저에게 방송(토픽 013 번 재사용)해 같은 자리에 동시에 앓는 버그를 막는다.
- 모닥불 파티클(045 번)과 앓기 회복 배율을 조합하면 '휴식 거점'이라는 콘텐츠가 자연스럽게 완성된다.

제 7 장 모듈 6 — 소셜 및 커뮤니티 시스템 (051~060)

기술 스택: uWebSockets.js(Pub/Sub Topics) · Kysely · Redis · TypeScript 본 장은 MMORPG 의 사회 계층이다. 채팅 채널은 uWS 토픽과 1:1 매핑되고, 길드·파티·친구는 DDL 과 권한 비트로 구조화된다. 1:1 거래와 경매장은 트랜잭션 안전성이 곧 신뢰이며, 공성전 권한·Redis ZSET 랭킹·멘토링 보상은 대규모 상호작용과 장기 동기를 설계한다. 모든 재화 이동 경로는 ledger 감사 로그를 통과한다.

051. uWebSockets.js 토픽 기반 다중 채널(전체, 필드, 파티, 길드, 귓속말) 채팅 시스템

(1) 역할 설명

채팅 채널은 uWebSockets.js 의 pub/sub 토픽과 1:1 로 매핑된다. 전체 채널은 맵 단위 토픽(chat:map:world), 필드 채널은 AOI 토픽(013 번 재사용), 파티는 chat:party:<id>, 길드는 chat:guild:<id>, 귓속말은 쌍방 유저 정렬 키(chat:dm:<lowId>:<highId>)로 구성한다. 이 매핑 덕분에 채널 가입/탈퇴가 subscribe/unsubscribe 로 끝나고, 발송은 publish 로 O(수신자) 처리가 네이티브 코어에서 일어난다.

채팅은 스팸·욕설·어뷰징의 1 차 진입점이므로 서버가 반드시 중간 검증을 수행한다. 스로틀(초당 3 개), 길이 상한(200 자), 차단 목록(054 번) 확인, 금칙어 필터, 이후 Mute(099 번) 검사를 순차 적용하고, 검증 통과 시에만 토픽에 발행한다. 채팅 로그는 감사(관리자 094 번)를 위해 만료 TTL 캐시에 보관한다.

(2) 구현 코드 — server/chat/ChatChannels.ts

```
import uWS from "uWebSockets.js";

export type ChannelKind = "map" | "field" | "party" | "guild" | "dm" | "system";
```

```

export function channelTopic(kind: ChannelKind, a: string | number, b?: string | number): string
{
  switch (kind) {
    case "map": return "chat:map:" + a;
    case "field": return "chat:field:" + a;
    case "party": return "chat:party:" + a;
    case "guild": return "chat:guild:" + a;
    case "dm": {
      const [lo, hi] = [Number(a), Number(b)].sort((x, y) => x - y);
      return "chat:dm:" + lo + ":" + hi;
    }
    default: return "chat:system";
  }
}

export interface ChatMessage {
  ch: ChannelKind; target: string;      // 토픽 파라미터
  fromId: number; fromName: string;
  text: string;
}

const MAX_LEN = 200;
const RATE_MS = 350;                  // 최소 전송 간격

export class ChatService {
  private lastSent = new Map<uWS.WebSocket, number>();
  constructor(private blockList: (a: number, b: number) => Promise<boolean>,
    private muted: (id: number) => boolean,
    private badWords: (s: string) => string) {}

  /** 클라 C2S_CHAT 수신 처리 */
  async handle(ws: uWS.WebSocket, userId: number, name: string, msg: ChatMessage) {
    // 1) 스로틀
    const now = Date.now();
    if (now - (this.lastSent.get(ws) ?? 0) < RATE_MS) return;
    this.lastSent.set(ws, now);

    // 2) 제재/차단/길이/금칙어
    if (this.muted(userId)) { this.whisper(ws, "채팅 금지 상태입니다."); return; }
    if (msg.ch === "dm" && await this.blockList(userId, Number(msg.target))) return;
    if (msg.text.length > MAX_LEN) msg.text = msg.text.slice(0, MAX_LEN);
    msg.text = this.badWords(msg.text);
    if (!msg.text.trim()) return;

    // 3) 발행(자기 자신 포함 — 내 채팅도 내 화면에 떠야 함)
    const topic = channelTopic(msg.ch, msg.target, msg.ch === "dm" ? userId : undefined);
    const payload = JSON.stringify({
      ch: msg.ch, from: name, id: userId, text: msg.text, at: now,
    });
    ws.publish(topic, payload, true);
    if (msg.ch === "dm") {                  // 귓속말은 발신자 토픽에도 에코

```

```

    ws.publish(channelTopic("dm", userId, Number(msg.target)), payload, false);
  }
}

/** 채널 가입/탈퇴(파티 가입, 길드 가입, 맵 이동 시 호출) */
join(ws: uWS.WebSocket, kind: ChannelKind, a: string | number, b?: string | number) {
  ws.subscribe(channelTopic(kind, a, b));
}
leave(ws: uWS.WebSocket, kind: ChannelKind, a: string | number, b?: string | number) {
  ws.unsubscribe(channelTopic(kind, a, b));
}

private whisper(ws: uWS.WebSocket, text: string) {
  ws.send(JSON.stringify({ ch: "system", text }), false, true);
}
}

```

(3) 성능 최적화 팁

- 토픽 문자열 규약(chat:kind:id)은 클라/서버/운영 툴이 공유하므로 한 곳(이 파일)에서만 생성한다.
- 귓속말 토픽은 유저 id 오름차순 정렬 키로 단일화해야 발신/수신이 같은 토픽을 보게 된다.
- 스로틀은 유저별이 아니라 커넥션별로 두는 게 정확하다 — 계정 다중 접속 어뷰징은 별도 카운터로.
- 금치어 필터는 서버에서 최종 적용하되, 클라 프리뷰(입력 중 실시간 마스킹)도 같은 사전을 공유하면 UX가 일치한다.

052. 길드 창고, 출석, 기술 연구, 권한 관리 데이터 구조 및 DDL

(1) 역할 설명

길드 데이터는 4 축으로 나뉜다. 구성원(권한 레벨), 창고(공용 아이템 저장), 출석/기여도(일일 출석 포인트), 기술 연구(길드 단계적 효과 — 공격력 % 등)다. 권한은 master/officer/member/newbie 4 단계 + 기능별 세부 권한 비트(창고 입출, 창고 보기, 멤버 초대, 해체 등)로 이중 관리한다. 권한 비트는 기능별로 검증 미들웨어를 통과시켜 UI와 API가 같은 규칙을 쓰게 한다.

창고는 아이템 소유 주체가 길드라는 점만 다른 024 번 인벤토리와 동일 구조다. 출석은 하루 1 회 체크인 API 로 길드 포인트를 적립하고, 기술 연구는 포인트 소모 + 연구 시간(실시간 타이머) 후 효과 개방이다. 모든 길드 자산 변경은 트랜잭션 + ledger 기록(018 번)으로 감사 가능하게 만든다.

(2) 구현 코드 — PostgreSQL DDL

```
CREATE TABLE guilds (
```

```

id      BIGSERIAL PRIMARY KEY,
name    VARCHAR(20) NOT NULL UNIQUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
guild_point INT NOT NULL DEFAULT 0,
level   SMALLINT NOT NULL DEFAULT 1
);

CREATE TABLE guild_members (
  guild_id  BIGINT REFERENCES guilds(id) ON DELETE CASCADE,
  character_id BIGINT REFERENCES characters(id) ON DELETE CASCADE,
  role      VARCHAR(12) NOT NULL DEFAULT 'member', -- master|officer|member|newbie
  perms     INT NOT NULL DEFAULT 0,               -- 세부 권한 비트
  joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_attend DATE,
  attend_streak INT NOT NULL DEFAULT 0,
  contribution INT NOT NULL DEFAULT 0,
  PRIMARY KEY (guild_id, character_id)
);

-- 세부 권한 비트 상수
-- 1 창고 보기, 2 창고 입고, 4 창고 출고, 8 초대, 16 강퇴, 32 공지,
-- 64 연구 시작, 128 권한 관리
CREATE TABLE guild_warehouse (
  id      BIGSERIAL PRIMARY KEY,
  guild_id BIGINT NOT NULL REFERENCES guilds(id) ON DELETE CASCADE,
  item_code VARCHAR(40) NOT NULL,
  qty      INT NOT NULL CHECK (qty >= 0),
  deposited_by BIGINT,           -- 감사 추적
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE guild_tech (
  id      BIGSERIAL PRIMARY KEY,
  guild_id BIGINT NOT NULL REFERENCES guilds(id) ON DELETE CASCADE,
  tech_code VARCHAR(30) NOT NULL, -- "atk_up" | "hp_up" | "warehouse_plus"
  level   SMALLINT NOT NULL DEFAULT 0,
  researching BOOLEAN NOT NULL DEFAULT FALSE,
  complete_at TIMESTAMPTZ,
  UNIQUE (guild_id, tech_code)
);

```

보조 코드 — server/guild/GuildOps.ts — 권한 검증 + 창고 입출고

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export const PERM = {
  WAREHOUSE_VIEW: 1, WAREHOUSE_DEPOSIT: 2, WAREHOUSE_WITHDRAW: 4,
  INVITE: 8, KICK: 16, NOTICE: 32, TECH: 64, MANAGE_PERMS: 128,
} as const;
const ROLE_MIN_PERMS: Record<string, number> = {
  master: 255, officer: 2 | 4 | 8 | 16 | 32 | 64 | 1,

```

```

    member: 2 | 1, newbie: 1,
  };

export class GuildError extends Error {
  constructor(code: "NO_PERM" | "NOT_MEMBER" | "NO_ITEM" | "LOCK") { super(code); }
}

export async function requirePerm(db: Kysely<DB>, guildId: number, charId: number, bit:
number) {
  const m = await db.selectFrom("guild_members")
    .where("guild_id", "=", guildId).where("character_id", "=", charId)
    .select(["role", "perms"]).executeTakeFirst();
  if (!m) throw new GuildError("NOT_MEMBER");
  const effective = m.perms | (ROLE_MIN_PERMS[m.role] ?? 0);
  if ((effective & bit) === 0) throw new GuildError("NO_PERM");
  return m;
}

/** 참고 출고: 권한 검증 + 원자적 차감 + 인벤토리 지급 */
export async function withdrawItem(
  db: Kysely<DB>, guildId: number, charId: number, itemId: number, qty: number,
) {
  await requirePerm(db, guildId, charId, PERM.WAREHOUSE_WITHDRAW);
  return db.transaction().execute(async tx => {
    const row = await tx.selectFrom("guild_warehouse").where("id", "=", itemId)
      .where("guild_id", "=", guildId).forUpdate().executeTakeFirst();
    if (!row || row.qty < qty) throw new GuildError("NO_ITEM");
    const left = row.qty - qty;
    if (left === 0) await tx.deleteFrom("guild_warehouse").where("id", "=", itemId).execute();
    else await tx.updateTable("guild_warehouse").set({ qty: left }).where("id", "=",
itemId).execute();
    // 인벤토리 지급(024 번 시스템 호출로 위임 — 여기서는 원리만 표현)
    await tx.insertInto("ledger").values({
      at: new Date(), kind: "item", from_user: null, to_user: charId,
      ref: "guild_wh:" + guildId, payload: JSON.stringify({ itemId, qty }),
    }).execute();
    return { itemCode: row.item_code, qty };
  });
}

/** 일일 출석: 연속 출석 보너스 + 길드 포인트 적립 */
export async function attend(db: Kysely<DB>, guildId: number, charId: number) {
  return db.transaction().execute(async tx => {
    const today = new Date().toISOString().slice(0, 10);
    const m = await tx.selectFrom("guild_members")
      .where("guild_id", "=", guildId).where("character_id", "=", charId)
      .forUpdate().select(["last_attend", "attend_streak",
"contribution"]).executeTakeFirstOrThrow();
    if (m.last_attend && m.last_attend.toISOString().slice(0, 10) === today) return false;
    const streak = m.attend_streak + 1;
    const gain = 10 + Math.min(20, streak);    // 연속 출석 보너스
    await tx.updateTable("guild_members").set({

```

```

    last_attend: new Date(), attend_streak: streak, contribution: m.contribution + gain,
  }).where("guild_id", "=", guildId).where("character_id", "=", charId).execute();
  await tx.updateTable("guilds").set(eb => ({ guild_point: eb("guild_point", "+", gain) })))
    .where("id", "=", guildId).execute();
  return true;
});
}

```

(3) 성능 최적화 팁

- 권한은 role(간단 배정) + perms 비트(세부 오버라이드)의 OR 로 계산한다 — role 만으로는 '참고 보기만 가능한 officer' 같은 세밀함이 안 나온다.
- 창고 입출고는 반드시 ledger 기록 — 길드 창고 도난 사고(권한 남용)의 유일한 감사 근거다.
- 출석 streak 은 타임존(일 단위 경계)을 명시해야 — 서버 UTC 기준 또는 게임 시간 기준 중 하나로 통일한다.
- 기술 연구 효과는 길드 접속 시 파티 버프(038 번) 소스로 주입해 스탯 파이프라인을 재사용한다.

053. 파티 결성, 실시간 파티원 위치 공유 및 아이템 분배

(순번/균등/루팅) 로직

(1) 역할 설명

파티는 '리더 + 멤버(최대 8 명) + 정책(드롭 분배 방식)'의 상태다. 실시간 요소는 두 가지다. 파티원 위치 공유(1 초 주기 배치 방송 — 048 번 미니맵 마커와 공용)와 드롭 분배(아이템 획득 시 정책에 따라 배정)다. 분배 정책은 순번(round-robin — 순서대로 배정), 균등(비귀속 재화는 골드 환산 분할), 루팅(free-for-all — 획득자 귀속)의 세 가지를 파티 생성 시 선택한다.

분배 판정은 서버가 드롭 발생 시 즉시 수행한다. 순번 정책은 '가방 공간 있는 다음 순번 멤버'에게 귀속시키고, 공간 부족 시 다음 순번으로 스킵한다. 균등 정책은 골드만 적용해 소수점은 리더에게 절상한다. 루팅은 획득 행위(몬스터 근접 + 획득 입력)를 서버가 검증해 먼저 획득한 유저에게 귀속시킨다. 모든 분배는 파티 채팅(051 번)으로 공지되어 투명성을 확보한다.

(2) 구현 코드 — server/party/Party.ts

```

import uWS from "uWebSockets.js";

export type LootPolicy = "round_robin" | "equal" | "fffa";
export interface PartyMember {
  charId: number; name: string; online: boolean;
  x: number; y: number; mapId: string; lastPosAt: number;
}

```



```

export interface Party {
  id: string; leaderId: number;
  members: Map<number, PartyMember>;
  lootPolicy: LootPolicy;
  rrIndex: number;           // 순번 커서
  topic: string;             // chat:party:<id>
}

export class PartyManager {
  private parties = new Map<string, Party>();

  create(leaderId: number, policy: LootPolicy): Party {
    const p: Party = {
      id: "pt" + Date.now().toString(36), leaderId,
      members: new Map(), lootPolicy: policy, rrIndex: 0,
      topic: "chat:party:" + Date.now().toString(36),
    };
    this.parties.set(p.id, p);
    return p;
  }

  join(p: Party, charId: number, name: string, ws: uWS.WebSocket) {
    p.members.set(charId, { charId, name, online: true,
      x: 0, y: 0, mapId: "", lastPosAt: 0 });
    ws.subscribe(p.topic);
  }

  /** 위치 공유: 1 초 주기 배치(각 유저 이동 패킷에서 좌표 수집) */
  broadcastPositions(p: Party, ws: uWS.WebSocket) {
    const now = Date.now();
    const list = [...p.members.values()]
      .filter(m => m.online && now - m.lastPosAt < 5000)
      .map(m => ({ id: m.charId, x: m.x, y: m.y, map: m.mapId }));
    ws.publish(p.topic, JSON.stringify({ t: "party_pos", list }), false);
  }

  /** 드롭 분배 판정(서버 — 드롭 발생 시 호출) */
  distributeLoot(p: Party, drop: { itemCode: string; qty: number; isGold: boolean; amount?:
    number }) {
    const online = [...p.members.values()].filter(m => m.online);
    if (!online.length) return null;

    if (drop.isGold && p.lootPolicy === "equal") {
      // 균등: 소수점은 리더 절상
      const each = Math.floor((drop.amount ?? 0) / online.length);
      const rest = (drop.amount ?? 0) - each * online.length;
      return { kind: "equal", amounts: online.map(m => ({
        charId: m.charId, gold: each + (m.charId === p.leaderId ? rest : 0) }))) };
    }
    if (p.lootPolicy === "round_robin") {
      // 순번: 가방 공간 확인은 클라 응답이 필요하므로 후보 순서만 결정

```

```

    for (let i = 0; i < online.length; i++) {
      const m = online[(p.rrIndex + i) % online.length];
      p.rrIndex = (p.rrIndex + i + 1) % online.length;
      return { kind: "rr", assigned: m.charId, itemCode: drop.itemCode, qty: drop.qty };
    }
  }
  // fffa: 획득자 귀속(획득 입력을 서버가 검증한 charId)
  return { kind: "fffa", assigned: null }; // acquire 요청에서 결정
}

/** 획득 검증(fffa): 드롭 지점 근접 + 가장 먼저 요청 */
tryAcquire(p: Party, charId: number, dropX: number, dropY: number, charX: number, charY:
number) {
  if (Math.hypot(dropX - charX, dropY - charY) > 48) return false;
  // 동시성: 같은 드롭 id 에 대한 첫 승인만 유효(원자적 Set)
  return true;
}
}

```

(3) 성능 최적화 팁

- 위치 공유는 1초 배치 방송이면 충분하다 — 이동 패킷마다 파티 방송을 추가하면 트래픽이 2배가 된다.
- 순번 분배의 '가방 공간 부족 스킵'은 클라 응답을 기다리지 말고 서버 인벤토리 캐시로 즉시 판정해야 전투 흐름이 끊기지 않는다.
- 분배 공지를 파티 채팅에 자동으로 남기면 분배 불만이 크게 줄어든다 — 투명성은 분배 정책보다 강력하다.
- fffa 는 획득 요청을 반드시 서버가 근접 + 선착순으로 검증한다 — 클라 선언만 믿으면 중복 획득 복제가 발생한다.

054. 친구 목록, 실시간 접속 상태 모니터링 및 차단 시스템

(1) 역할 설명

친구 시스템은 (1) 목록 저장(DB), (2) 실시간 접속 상태(온라인/게임 중/오프라인), (3) 상호작용(귓속말, 파티 초대, 차단)의 3계층이다. 접속 상태는 Redis presence 해시(로그인 시 SET, 로그아웃 시 DEL + TTL 안전망)로 관리하고, 친구가 접속/종료하면 양방향 친구 관계를 따라가며 접속 알림(presence:notify:<userId> 토픽)을 발행한다. 온라인 상태는 캐시에서만 조회하므로 친구 500 명을 가져도 O(1) 해시 조회로 끝난다.

차단은 '내 화면에서 상대 제거 + 상대가 나를 보지 못하게' 양방향 필터다. 차단된 쌍은 귓속말/파티 초대/경매 연락이 서버에서 거부되고, 필드에서도 이름 표시가 익명화된다. 친구 요청은 수락 대기 큐(만료 7 일)로 운영하며, 스팸 요청은 일일 상한(20 회)으로 제한한다

(2) 구현 코드 — PostgreSQL DDL + server/friends/Presence.ts

```
-- DDL
```

```

CREATE TABLE friendships (
  user_a    BIGINT NOT NULL REFERENCES characters(id),
  user_b    BIGINT NOT NULL REFERENCES characters(id),
  status    VARCHAR(12) NOT NULL DEFAULT 'pending', -- pending|accepted|blocked
  requested_by BIGINT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_a, user_b)
);
CREATE INDEX idx_friend_of ON friendships(user_a, status);
-- 관계는 항상 user_a < user_b 로 정규화 저장(양방향 조회 단순화)

CREATE TABLE friend_blocks (
  blocker   BIGINT NOT NULL REFERENCES characters(id),
  blocked   BIGINT NOT NULL REFERENCES characters(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (blocker, blocked)
);

```

보조 코드 — server/friends/Presence.ts — Redis 접속 상태

```

import Redis from "ioredis";

export class PresenceService {
  private readonly KEY = "presence"; // 해시: charId → 상태
  private readonly NOTIFY = "presence:notify";

  constructor(private redis: Redis, private pub: Redis) {}

  /** 로그인: 상태 등록 + 친구에게 알림 */
  async online(charId: number, friends: number[]) {
    await this.redis.hset(this.KEY, charId, "online:" + Date.now());
    await this.redis.expire(this.KEY, 60 * 60 * 26); // 안전망 TTL
    for (const f of friends) {
      await this.pub.publish(this.NOTIFY + ":" + f,
        JSON.stringify({ id: charId, state: "online" }));
    }
  }

  async offline(charId: number, friends: number[]) {
    await this.redis.hdel(this.KEY, charId);
    for (const f of friends) {
      await this.pub.publish(this.NOTIFY + ":" + f,
        JSON.stringify({ id: charId, state: "offline" }));
    }
  }

  /** 목록 조회: 한 번의 HMGET 으로 N명 상태 */
  async statuses(ids: number[]): Promise<Map<number, boolean>> {
    if (!ids.length) return new Map();
    const vals = await this.redis.hmget(this.KEY, ...ids.map(String));
    const out = new Map<number, boolean>();
    ids.forEach((id, i) => out.set(id, vals[i] != null));
    return out;
  }
}

```

```

}
}

/** 친구 목록 + 상태 합성 응답 */
import { Kysely } from "kysely";
type DB = import("./schema").Database;
export async function friendList(db: Kysely<DB>, presence: PresenceService, charId: number) {
  const rows = await db.selectFrom("friendships")
    .where(eb => eb.or([
      eb("user_a", "=", charId), eb("user_b", "=", charId),
    ]))
    .where("status", "=", "accepted")
    .select(["user_a", "user_b"]).execute();
  const ids = rows.map(r => (r.user_a === charId ? r.user_b : r.user_a));
  const states = await presence.statuses(ids);
  return ids.map(id => ({ id, online: states.get(id) ?? false }));
}

/** 차단: 양방향 거부 검증(귓속말/초대 전 확인) */
export async function isBlocked(db: Kysely<DB>, a: number, b: number) {
  const row = await db.selectFrom("friend_blocks")
    .where(eb => eb.or([
      eb.and([eb("blocker", "=", a), eb("blocked", "=", b)]),
      eb.and([eb("blocker", "=", b), eb("blocked", "=", a)]),
    ])).select("blocker").executeTakeFirst();
  return !!row;
}

```

(3) 성능 최적화 팁

- presence 해시에 TTL(26 시간)을 두면 크래시로 로그아웃 처리가 누락돼도 상태가 자동 정리된다.
- 접속 알림은 pub/sub 토픽(presence:notify:<id>)으로 — 폴링으로 상태를 확인하면 친구 500 명에서 500 회 쿼리가 된다.
- 차단은 양방향으로 검증해야 '내가 차단해도 상대가 귓속말을 보낸다'는 사각지대가 없다.
- 친구 요청 일일 상한(20 회)은 신규 서버 오픈 시 스팸 요청 폭탄을 막는 필수 장치다.

055. 1:1 개인 거래 시스템 (아이템/재화 상호 수락 및 Kysely DB 트랜잭션)

(1) 역할 설명

1:1 거래는 '양쪽이 제안물을 등록하고, 양쪽이 수락하면 동시 교환'하는 상태머신이다. 상태 흐름은 open → both_locked(양쪽 잠금) → confirmed(양쪽 최종 확인) → executed/failed 다 핵심 안전장치는 세 가지다. (1) 잠금 후에는 제안물 변경 불가 — 변경 시 잠금 해제, (2) 최종

확인 후 실행은 단일 DB 트랜잭션, (3) 실행 전 소유권 재검증(거래 중 아이템이 인벤토리에서 사라진 경우 실패 처리).

구현 포인트는 거래 참여 중 다른 경로(우편, 경매, 창고)로 아이템을 옮기지 못하게 하는 '거래 잠금'이다. character_items 에 trade_lock 칼럼 또는 Redis 락 키로 구현하며, 모든 자산 이동 API 는 잠금 검사를 통과해야 한다. 실행 트랜잭션은 018 번 transferItem/transferGold 를 재사용해 소유권 이전의 안전성을 공유한다.

(2) 구현 코드 — server/trade/Trade.ts — 거래 상태머신 + 실행 트랜잭션

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface TradeOffer {
  gold: number;
  items: { itemId: number; itemCode: string }[];
}

export type TradeState = "open" | "locked_a" | "locked_both" | "executed" | "failed";

export class TradeSession {
  state: TradeState = "open";
  offers: Record<"a" | "b", TradeOffer> = { a: { gold: 0, items: [] }, b: { gold: 0, items: [] } };
  lockA = false; lockB = false;
  confirmA = false; confirmB = false;

  constructor(readonly id: string, readonly userA: number, readonly userB: number) {}

  /** 제안 수정 — 상대가 잠금했으면 잠금 해제(사기 방지 표준 규칙) */
  update(userId: number, offer: TradeOffer): boolean {
    const side = this.sideOf(userId);
    if (!side) return false;
    this.offers[side] = offer;
    if (side === "a" && this.lockB) this.lockB = false;
    if (side === "b" && this.lockA) this.lockA = false;
    this.confirmA = this.confirmB = false; // 수락 초기화
    this.lockA = side === "a" ? this.lockA : this.lockA;
    this.recomputeState();
    return true;
  }

  lock(userId: number) {
    const side = this.sideOf(userId);
    if (side === "a") this.lockA = true;
    if (side === "b") this.lockB = true;
    this.recomputeState();
  }

  confirm(userId: number) {
    if (this.state !== "locked_both") return false;
    if (this.sideOf(userId) === "a") this.confirmA = true; else this.confirmB = true;
    return this.confirmA && this.confirmB; // 둘 다 true → execute
  }
}
```

```

private recomputeState() {
  this.state = this.lockA && this.lockB ? "locked_both"
    : this.lockA || this.lockB ? "locked_a" : "open";
}
private sideOf(userId: number): "a" | "b" | null {
  return userId === this.userA ? "a" : userId === this.userB ? "b" : null;
}
}

/** 최종 실행: 단일 트랜잭션 + 소유권 재검증 */
export async function executeTrade(db: Kysely<DB>, t: TradeSession) {
  return db.transaction().execute(async tx => {
    // 1) 소유권 재검증(거래 중 변경 감지)
    for (const side of ["a", "b"] as const) {
      for (const it of t.offers[side].items) {
        const owner = await tx.selectFrom("character_items")
          .where("id", "=", it.itemId).select("character_id").executeTakeFirst();
        const expected = side === "a" ? t.userA : t.userB;
        if (!owner || owner.character_id !== expected) throw new
Error("OWNERSHIP_CHANGED");
      }
    }
    // 2) 재화 이전(018 번 트랜잭션 헬퍼 재사용)
    if (t.offers.a.gold > 0) {
      await transferGoldInTx(tx, t.userA, t.userB, t.offers.a.gold, "trade:" + t.id);
    }
    if (t.offers.b.gold > 0) {
      await transferGoldInTx(tx, t.userB, t.userA, t.offers.b.gold, "trade:" + t.id);
    }
    // 3) 아이템 소유권 이전
    for (const it of t.offers.a.items) {
      await transferItemInTx(tx, it.itemId, t.userA, t.userB, "trade:" + t.id);
    }
    for (const it of t.offers.b.items) {
      await transferItemInTx(tx, it.itemId, t.userB, t.userA, "trade:" + t.id);
    }
    t.state = "executed";
    return true;
  }).catch(e => { t.state = "failed"; throw e; });
}

async function transferGoldInTx(tx: any, from: number, to: number, amount: number, ref: string)
{
  const u = await tx.selectFrom("users").where("id", "=",
from).select("gold").forUpdate().executeTakeFirstOrThrow();
  if (u.gold < amount) throw new Error("NO_FUNDS");
  await tx.updateTable("users").set(eb => ({ gold: eb("gold", "-", amount) })).where("id", "=",
from).execute();
  await tx.updateTable("users").set(eb => ({ gold: eb("gold", "+", amount) })).where("id", "=",
to).execute();
  await tx.insertInto("ledger").values({ at: new Date(), kind: "gold",
from_user: from, to_user: to, ref, payload: JSON.stringify({ amount }) }).execute();
}

```

```

async function transferItemInTx(tx: any, itemId: number, from: number, to: number, ref: string)
{
  const it = await tx.selectFrom("character_items").where("id", "=",
itemId).forUpdate().executeTakeFirstOrThrow();
  if (it.character_id !== from) throw new Error("OWNERSHIP_CHANGED");
  if ((it as any).binding === "bound") throw new Error("BOUND_ITEM");
  await tx.updateTable("character_items").set({ character_id: to })
    .where("id", "=", itemId).execute();
  await tx.insertInto("ledger").values({ at: new Date(), kind: "item",
    from_user: from, to_user: to, ref, payload: JSON.stringify({ itemId }) }).execute();
}

```

(3) 성능 최적화 팁

- 잠금 후 제안 변경 시 상대 잠금 해제는 1:1 거래 사기(스위치 스캠)를 막는 표준 규칙이다.
- 실행 트랜잭션에서 소유권 재검증은 필수다 — 잠금 사이에 아이템이 우편/창고로 이동하는 레이스가 실제로 발생한다.
- 거래 잠금(trade_lock)은 모든 자산 이동 API(우편, 경매 등록 등)가 공통으로 검사해야 완전하다.
- 귀속(bound) 아이템은 거래 단계부터 차단해 실행 트랜잭션에서 실패하는 낭비를 없앤다.

056. 통합 경매장/위탁 판매소 (검색, 시세, 등록, 즉시 구매)

(1) 역할 설명

경매장은 '등록(판매자가 아이템+가격 업로드) → 검색/시세 조회 → 구매(즉시구매 또는 낙찰) → 정산'의 파이프라인이다. 등록 시 아이템은 소유자에서 경매장 계좌(escrow)로 이전되고, 판매 완료 시 판매자에게 금액(수수료 차감)이 지급된다. 즉시구매는 buyout 가격 필수이며, 입찰 경매는 마감 시간에 최고 입찰자가 낙찰받는다. 검색은 카테고리/이름/레벨/강화 단계 필터 + 정렬 + 페이지네이션의 표준 API다.

시세(마켓 프라이스)는 최근 24시간 거래 완료가의 이동 평균으로 집계해 상하한가 제어(087 번)와 유저 가격 참고 UI에 쓴다. 성능 핵심은 목록 조회 캐싱(30 초 Redis)과 인덱스 설계(category, item_code, buyout_price)다. 등록 취소는 상태를 canceled 로 바꾸고 escrow 에서 원상복구하는 트랜잭션으로 처리한다.

(2) 구현 코드 — PostgreSQL DDL

```

CREATE TABLE auction_listings (
  id          BIGSERIAL PRIMARY KEY,
  seller_id   BIGINT NOT NULL,
  item_id     BIGINT NOT NULL,          -- escrow 된 character_items.id
  item_code   VARCHAR(40) NOT NULL,
  category    VARCHAR(16) NOT NULL,    -- 검색 필터
  enhance     SMALLINT NOT NULL DEFAULT 0,

```

```

    qty      INT NOT NULL DEFAULT 1,
    price_now BIGINT NOT NULL,      -- 즉시구매가
    price_start BIGINT,             -- 입찰 경매 시작가(NULL = 즉시구매 전용)
    top_bid   BIGINT,               -- 현재 최고 입찰
    top_bidder BIGINT,
    status    VARCHAR(12) NOT NULL DEFAULT 'active',
            -- active|sold|canceled|expired
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX idx_auction_search ON auction_listings
(status, category, item_code, price_now) WHERE status = 'active';
CREATE INDEX idx_auction_expire ON auction_listings (expires_at)
WHERE status = 'active';

CREATE TABLE auction_trades (
    id          BIGSERIAL PRIMARY KEY,
    listing_id  BIGINT NOT NULL,
    buyer_id    BIGINT NOT NULL,
    final_price BIGINT NOT NULL,
    fee         BIGINT NOT NULL,      -- 수수료(5%)
    at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

보조 코드 — server/auction/Auction.ts — 등록/검색/구매 Kysely 쿼리

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

const FEE_RATE = 0.05;

export class AuctionService {
    constructor(private db: Kysely<DB>) {}

    /** 즉시 구매: escrow 아이템 인도 + 정산(단일 트랜잭션) */
    async buyNow(listingId: number, buyerId: number) {
        return this.db.transaction().execute(async tx => {
            const l = await tx.selectFrom("auction_listings").where("id", "=", listingId)
                .where("status", "=", "active").forUpdate().executeTakeFirst();
            if (!l) throw new Error("NOT_AVAILABLE");
            if (l.seller_id === buyerId) throw new Error("SELF_PURCHASE");

            const u = await tx.selectFrom("users").where("id", "=", buyerId)
                .forUpdate().select("gold").executeTakeFirstOrThrow();
            if (u.gold < l.price_now) throw new Error("NO_FUNDS");

            // 1) 구매자 재화 차감 → 판매자 지급(수수료 차감)
            await tx.updateTable("users").set(eb => ({ gold: eb("gold", "-", l.price_now) })))
                .where("id", "=", buyerId).execute();
            const net = Math.floor(l.price_now * (1 - FEE_RATE));
            await tx.updateTable("users").set(eb => ({ gold: eb("gold", "+", net) })))
                .where("id", "=", l.seller_id).execute();
        });
    }
}

```



```

// 2) 아이템 인도(escrow → 구매자)
await tx.updateTable("character_items").set({ character_id: buyerId })
  .where("id", "=", l.item_id).execute();
await tx.updateTable("auction_listings").set({ status: "sold" })
  .where("id", "=", listingId).execute();
await tx.insertInto("auction_trades").values({
  listing_id: listingId, buyer_id: buyerId, final_price: l.price_now,
  fee: l.price_now - net, at: new Date(),
}).execute();
return { net };
});
}

/** 검색: 필터 + 정렬 + 페이지네이션(30 초 Redis 캐시 전제) */
async search(q: {
  category?: string; itemCode?: string; minEnhance?: number;
  maxPrice?: number; sort?: "price_asc" | "price_desc" | "recent";
  page?: number; size?: number;
}) {
  let query = this.db.selectFrom("auction_listings")
    .where("status", "=", "active");
  if (q.category) query = query.where("category", "=", q.category);
  if (q.itemCode) query = query.where("item_code", "=", q.itemCode);
  if (q.minEnhance) query = query.where("enhance", ">=", q.minEnhance);
  if (q.maxPrice) query = query.where("price_now", "<=", q.maxPrice);
  const size = Math.min(50, q.size ?? 20), page = Math.max(0, q.page ?? 0);
  const sorted = q.sort === "price_asc" ? query.orderBy("price_now asc")
    : q.sort === "price_desc" ? query.orderBy("price_now desc")
    : query.orderBy("created_at desc");
  return sorted.limit(size).offset(page * size).execute();
}

/** 시세 집계: 최근 24h 완료가 평균(시세 UI + 087 번 상하한가 기준) */
async marketPrice(itemCode: string) {
  const row = await this.db.selectFrom("auction_trades")
    .innerJoin("auction_listings", "auction_trades.listing_id", "auction_listings.id")
    .where("auction_listings.item_code", "=", itemCode)
    .where("auction_trades.at", ">=", new Date(Date.now() - 86400000))
    .select(eb => eb.fn.avg("final_price").as("avg")).executeTakeFirst();
  return row ? Math.round(Number(row.avg)) : null;
}
}

```

(3) 성능 최적화 팁

- escrow 패턴(등록 시 소유 이전)은 판매자가 판매 중 아이템을 우편으로 빼가는 사기를 구조적으로 차단한다.
- 검색 인덱스는 부분 인덱스(WHERE status='active')로 — 활성 목록만 조회하므로 인덱스가 절반 이하로 작아진다.

- 구매 트랜잭션은 '재화 차감 → 아이템 인도 → 상태 변경' 순서를 지켜야 부분 실패 시 돌아갈 지점이 명확하다.
- 시세는 24 시간 거래가 이동 평균이며, 거래가 없으면 null 로 표시해 가짜 시세(조작)를 만들지 않는다.

057. 감정표현(Emote) 및 픽셀 애니메이션 동기화

(1) 역할 설명

감정표현은 '애니메이션 재생 + 주변 유저 방송'의 저비용 이벤트다. 정의는 `emote_def`(코드, 애니메이션 키, 지속 시간, 파티클/사운드 옵션)로 관리하고, 유저가 감정표현을 실행하면 서버는 스로틀 검증 후 AOI 토픽(013 번)에 이벤트 1 회를 발행한다. 수신 클라는 대상 캐릭터 컨테이너(027 번 외형)에 애니메이션 오버레이를 재생하고, 지속 시간 후 원복한다.

픽셀 애니메이션 동기화의 원칙은 '이벤트 방송, 상태 전송 금지'다. 애니메이션 프레임 인덱스를 매 프레임 보내는 것은 낭비이며, 'emote 실행' 이벤트 1 개로 시작 시각(서버 시각)과 코드를 전달하면 각 클라가 자동 재생을 맞춘다. 스폰 직후 입장한 유저는 '현재 진행 중인 emote'를 스냅샷에 포함해 받는다(남은 시간 계산 후 부분 재생).

(2) 구현 코드 — `src/world/Emote.ts` — 정의 + 클라 재생

```
import Phaser from "phaser";

export interface EmoteDef {
  code: string;           // "wave" | "dance" | "laugh" | "bow"
  animKey: string;        // 스프라이트 애니메이션 키
  durationMs: number;
  particles?: string;     // 옵션: 파티클 텍스처
  sound?: string;
  loop: boolean;
}

export const EMOTES: Record<string, EmoteDef> = {
  wave: { code: "wave", animKey: "emote_wave", durationMs: 1500, loop: false },
  dance: { code: "dance", animKey: "emote_dance", durationMs: 6000, particles: "fx_note",
    sound: "sfx_dance", loop: true },
  laugh: { code: "laugh", animKey: "emote_laugh", durationMs: 1200, loop: false },
  bow: { code: "bow", animKey: "emote_bow", durationMs: 1000, loop: false },
};

const RATE_MS = 1200;           // 연속 감정표현 스로틀

export class EmotePlayer {
  private lastAt = 0;
  private activeKey: string | null = null;
}
```

```

constructor(private scene: Phaser.Scene, private container: Phaser.GameObjects.Container) {}

/** 내가 실행(서버 승인 요청 전 로컬 예측 재생 허용) */
play(code: string, now: number): boolean {
  if (now - this.lastAt < RATE_MS) return false;
  const def = EMOTES[code];
  if (!def) return false;
  this.lastAt = now;
  this.playLocal(def, def.durationMs);
  return true;          // 호출자가 C2S_EMOTE 전송
}

/** 원격 유저 감정표현 수신(서버 시각 + 코드) */
playRemote(code: string, serverStartAt: number, nowServer: number) {
  const def = EMOTES[code];
  if (!def) return;
  const elapsed = Math.max(0, nowServer - serverStartAt);
  if (def.loop) {
    this.playLocal(def, def.durationMs);    // 루프는 남은 시간 무관 재생
  } else if (elapsed < def.durationMs) {
    this.playLocal(def, def.durationMs - elapsed); // 부분 재생
  }
}

private playLocal(def: EmoteDef, durationMs: number) {
  const sprite = this.container.list[0] as Phaser.GameObjects.Sprite;
  if (this.activeKey) this.stop();
  this.activeKey = def.animKey;
  sprite.play(def.animKey);

  if (def.particles) {
    const fx = this.scene.add.particles(this.container.x, this.container.y - 40,
      def.particles, { speedY: { min: -30, max: -10 }, lifespan: 900, quantity: 1,
        frequency: 300, alpha: { start: 0.8, end: 0 } });
    fx.setDepth(4600);
    this.scene.time.delayedCall(durationMs, () => fx.destroy());
  }
  this.scene.time.delayedCall(durationMs, () => {
    sprite.play("idle_down");    // 원복
    this.activeKey = null;
  });
}

private stop() {
  const sprite = this.container.list[0] as Phaser.GameObjects.Sprite;
  sprite.play("idle_down");
  this.activeKey = null;
}
}

```

(3) 성능 최적화 팁

- 감정표현은 이벤트 1 회 방송 + 서버 시작 시각만 전달하면 충분하다 — 프레임 동기화는 필요 없다.
- 스로틀(1.2 초)은 스팸이 아니라 '재생 충돌 방지' 목적이며, 서버가 최종 검증한다.
- 스폰 스냅샷에 '진행 중 emote + 남은 시간'을 포함해야 맵 진입 직후에도 진행 중 표현이 보인다.
- 감정표현은 파티 모집(안전장치: 파티 리더만 dance 전용 버전) 같은 비언어적 소통 도구로 확장 가치가 크다.

058. 영지전 / 공성전 대규모 PvP 인스턴스 권한 시스템

(1) 역할 설명

공성전은 '공격 길드 연합 vs 수성 길드'의 대규모(100~300 명) PvP 인스턴스다. 권한 시스템은 (1) 참가 자격(길드 가입, 레벨, 신청 완료), (2) 진영(faction) 배정, (3) 전장 내 특수 권한(공성 무기 사용, 문 개폐, 플래그 소유), (4) 관전자(사망 관전/외부 관전)의 4 계층으로 구성된다. 인스턴스는 전용 룸(017 번)으로 생성되며, 진입 시 진영 배정 정보를 세이브가 아닌 세션에 저장한다(전장 종료 후 원래 상태 복원).

핵심 안전장치는 권한의 지속 재검증이다. 전장 도중 길드 탈퇴/추방되면 즉시 참가 권한 상실(강제 관전 전환)되고, 전장 상태(문 개폐, 무기 사용)는 서버가 매 액션마다 세션 권한을 재확인한다. 점수(킬/오브젝트 파괴/기여)는 이벤트 큐로 적립해 실시간 점수판을 만들고, 종료 시 기여도 기반 보상을 지급한다(064 번 파이프라인 재사용).

(2) 구현 코드 — `server/siege/Siege.ts`

```
export type Faction = "attack" | "defense" | "spectator";
export interface SiegePermission {
  charId: number; guildId: number;
  faction: Faction;
  canUseSiegeWeapon: boolean;    // 공성 무기
  canOpenGate: boolean;         // 성문 개폐
  canCarryFlag: boolean;        // 플래그 운반
  inBattle: boolean;            // false = 관전
}
export interface SiegeInstance {
  roomId: string; phase: "register" | "prep" | "battle" | "ended";
  startsAt: number; endsAt: number;
  perms: Map<number, SiegePermission>;
  scores: Map<number, { kills: number; dmg: number; objectives: number }>;
  gateHp: number;               // 수성 성문
}

export class SiegeAuthority {
  private sieges = new Map<string, SiegeInstance>();
```

```

/** 참가 신청(등록 단계): 길드 검증 + 진영 배정 */
register(siegeId: string, charId: number, guildId: number, isDefGuild: boolean): boolean {
  const s = this.sieges.get(siegeId);
  if (!s || s.phase !== "register") return false;
  s.perms.set(charId, {
    charId, guildId,
    faction: isDefGuild ? "defense" : "attack",
    canUseSiegeWeapon: false,    // prep 단계에서 직급 기반 부여
    canOpenGate: false,
    canCarryFlag: false,
    inBattle: true,
  });
  return true;
}

/** 전장 진입 직전: 직급 기반 특수 권한 부여 */
grantRolePerms(siegeId: string, charId: number, guildRole: "master" | "officer" | "member") {
  const p = this.sieges.get(siegeId)?.perms.get(charId);
  if (!p) return;
  p.canUseSiegeWeapon = guildRole !== "member";
  p.canOpenGate = p.faction === "defense" && guildRole !== "member";
  p.canCarryFlag = true;        // 모든 참가자 가능 — 플래그 소유는 획득 판정
}

/** 액션 권위 검증(전장 내 모든 특수 액션 전 호출) */
authorize(siegeId: string, charId: number, action: "weapon" | "gate" | "flag"): boolean {
  const s = this.sieges.get(siegeId);
  if (!s || s.phase !== "battle") return false;
  const p = s.perms.get(charId);
  if (!p || !p.inBattle || p.faction === "spectator") return false;
  switch (action) {
    case "weapon": return p.canUseSiegeWeapon;
    case "gate":   return p.canOpenGate && s.gateHp > 0;
    case "flag":   return p.canCarryFlag;
  }
}

/** 길드 탈퇴 등 이벤트 → 즉시 권한 상실(강제 관전) */
revokeGuildMembership(charId: number, guildId: number) {
  for (const s of this.sieges.values()) {
    const p = s.perms.get(charId);
    if (p && p.guildId === guildId && s.phase !== "ended") {
      p.inBattle = false;
      p.faction = "spectator";
      p.canUseSiegeWeapon = p.canOpenGate = p.canCarryFlag = false;
    }
  }
}

/** 점수 적립(킬/피해/오브젝트) — 이벤트 큐로 비동기 */

```

```

score(siegeId: string, charId: number, kind: "kill" | "dmg" | "objective", value = 1) {
  const s = this.sieges.get(siegeId);
  if (!s || s.phase !== "battle") return;
  const sc = s.scores.get(charId) ?? { kills: 0, dmg: 0, objectives: 0 };
  if (kind === "kill") sc.kills += value;
  if (kind === "dmg") sc.dmg += value;
  if (kind === "objective") sc.objectives += value;
  s.scores.set(charId, sc);
}
}

```

(3) 성능 최적화 팁

- 진영 배정은 세이브가 아닌 세션 데이터로 — 전장 종료 후 원래 소속(길드/PVP 상태)을 그대로 복원해야 한다.
- 특수 권한(공성 무기 등)은 직급 기반 초기 부여 + 매 액션 재검증의 이중 구조다 — 초기 부여만 믿으면 직급 변경 후에도 권한이 남는다.
- 성문 HP 같은 공성 오브젝트는 서버 단일 권위로 두고, 클라 연출은 상태 방송(파괴 진행도)에 맞춘다.
- 관전자 수용(죽으면 관전 모드)은 전장 참여 유지보수(사망 페널티 없음) 정책과 함께 설계해야 이탈률이 관리된다.

059. Redis ZSET 기반 실시간 레벨/PvP/길드 랭킹 산출

(1) 역할 설명

랭킹은 Redis Sorted Set(ZSET)의 member=유저 id, score=랭킹 지표 구조로 실시간 산출한다. 업데이트는 ZADD/ZINCRBY(이벤트 발생 시 증분), 조회는 ZREVRANGE(순위) + ZSCORE(내 순위) + ZCOUNT(분포)다. 레벨 랭킹은 절대값(score=exp), PvP는 시즌 점수(ZINCRBY 증분), 길드 랭킹은 멤버 합산(주기 배치로 길드 ZSET 갱신)이다.

실무 포인트는 두 가지다. 첫째, 시즌 랭킹은 시즌 키(ranking:pvp:s3)로 분리해 시즌 종료 시 스냅샷(DB) 후 새 키로 재시작한다. 둘째, 랭킹 조회는 Redis에서 상위 N명의 id를 얻고 DB에서 상세(이름/클래스)를 배치 조회한다 — Redis에 이름까지 넣으면 갱신 비용이 2배가 된다. 타이 랭크(동점)는 score에 보조 정밀도(1e-6)를 더해 안정적 순서를 만든다.

(2) 구현 코드 — server/ranking/Ranking.ts

```

import Redis from "ioredis";
import { Kysely } from "kysely";
type DB = import("./schema").Database;

export type RankingKind = "level" | "pvp" | "guild";
const KEY = (kind: RankingKind, season: string) => "rank:" + kind + ":" + season;

export class RankingService {
  constructor(private redis: Redis, private db: Kysely<DB>, private season = "s3") {}

```

```

/** 절대값 랭킹(레벨/exp) — 세이브 시점 갱신 */
async setLevelRank(charId: number, totalExp: number) {
  await this.redis.zadd(KEY("level", this.season), totalExp, String(charId));
}

/** 증분 랭킹(PvP 점수, 길드 기여) */
async incr(kind: RankingKind, id: number, delta: number) {
  await this.redis.zincrby(KEY(kind, this.season), delta, String(id));
}

/** 상위 N: Redis에서 id → DB 상세 배치 조회 */
async top(kind: RankingKind, n = 50) {
  const ids = await this.redis.zrevrange(KEY(kind, this.season), 0, n - 1, "WITHSCORES");
  const out: { id: number; score: number; name?: string; classCode?: string }[] = [];
  for (let i = 0; i < ids.length; i += 2) {
    out.push({ id: Number(ids[i]), score: Number(ids[i + 1]) });
  }
  if (!out.length) return [];
  const details = await this.db.selectFrom("characters")
    .where("id", "in", out.map(o => o.id))
    .select(["id", "name", "class_code"]).execute();
  const dmap = new Map(details.map(d => [d.id, d]));
  return out.map(o => ({ ...o, name: dmap.get(o.id)?.name, classCode:
dmap.get(o.id)?.class_code }));
}

/** 내 순위 + 전후 랭커(마이 랭킹 UI) */
async around(kind: RankingKind, id: number, span = 3) {
  const rank = await this.redis.zrevrank(KEY(kind, this.season), String(id));
  if (rank == null) return { rank: null, neighbors: [] };
  const start = Math.max(0, rank - span);
  const ids = await this.redis.zrevrange(KEY(kind, this.season), start, rank + span,
"WITHSCORES");
  const neighbors: { rank: number; id: number; score: number }[] = [];
  for (let i = 0; i < ids.length; i += 2) {
    neighbors.push({ rank: start + i / 2, id: Number(ids[i]), score: Number(ids[i + 1]) });
  }
  return { rank: rank + 1, neighbors };
}

/** 시즌 종료: 스냅샷(DB 저장) 후 새 키로 롤오버 */
async rolloverSeason(kind: RankingKind, newSeason: string) {
  const oldKey = KEY(kind, this.season);
  const all = await this.redis.zrevrange(oldKey, 0, 999, "WITHSCORES");
  await this.db.transaction().execute(async tx => {
    for (let i = 0; i < all.length; i += 2) {
      await tx.insertInto("ranking_snapshot").values({
        kind, season: this.season, target_id: Number(all[i]),
        rank: i / 2 + 1, score: Number(all[i + 1]), at: new Date(),
      }).execute();
    }
  });
}

```

```

    }
  });
  this.season = newSeason;    // 이후 갱신은 새 시즌 키로
}
}

```

(3) 성능 최적화 팁

- ZSET 갱신은 이벤트 발생 즉시 ZINCRBY — 주기 배치로 모으면 '순위 반영 지연' 클레임이 생긴다.
- 타이(동점) 안정화는 score 에 소수 정밀도 보조(예: $\text{exp} * 1\text{e-}6$ + 달성시각 역수)로 해결한다.
- 상위 1000 명 외 조회는 '내 순위 + 전후' 패턴(ZREVRANK + 범위 조회)으로 절약한다 — 전체 순위 페이지네이션은 비용이 크다.
- 시즌 롤오버는 스냅샷(DB) + 새 키 생성이 원자적이어야 한다 — 롤오버 순간의 갱신 유실을 방지한다.

060. 멘토링 및 추천인 시스템 연계 보상 로직

(1) 역할 설명

멘토링은 '숙련 유저(멘토) ↔ 신규 유저(멘티) 매칭 + 함께 플레이 보상' 구조다. 자격은 레벨/플레이 시간 기준으로 판정하고, 매칭은 멘티 신청 → 멘토 수락의 큐 방식으로 진행한다. 보상은 세 단계로 지급된다. 매칭 성립 보상(즉시), 함께 플레이 보상(공동 사냥 시간 누적), 졸업 보상(멘티가 목표 레벨 도달)이다. 추천인 시스템은 가입 시점에 추천 코드를 입력한 관계를 고정하고, 멘티의 성장 지표에 따라 추천인에게 누적 보상을 준다.

어뷰징 방지가 설계의 절반이다. 자기 자신/동일 IP 다계정 매칭 차단(디바이스/IP 유사도), 함께 플레이 인정 조건(양쪽 모두 전투 참여, 최소 10 분), 졸업 보상 지급 시 재검증(멘티 실제 플레이 지표)을 둔다. 보상은 ledger ref 로 먹등 지급('mentor:grad:<pairId>')하며, 관계 종료(멘티 졸업/중도 해지) 후 재매칭은 쿨다운(24 시간)을 둔다.

(2) 구현 코드 — PostgreSQL DDL + server/mentor/Mentor.ts

```

CREATE TABLE mentor_pairs (
  id          BIGSERIAL PRIMARY KEY,
  mentor_id   BIGINT NOT NULL REFERENCES characters(id),
  mentee_id   BIGINT NOT NULL REFERENCES characters(id),
  status      VARCHAR(12) NOT NULL DEFAULT 'active',
              -- active|graduated|ended
  play_min    INT NOT NULL DEFAULT 0,      // 함께 플레이 누적(분)
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  ended_at    TIMESTAMPTZ
);

CREATE TABLE referral (
  mentee_id   BIGINT PRIMARY KEY REFERENCES characters(id),

```



```
referrer_id BIGINT NOT NULL REFERENCES characters(id),
code_used VARCHAR(12) NOT NULL,
rewarded_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

보조 코드 — server/mentor/Mentor.ts — 매칭·보상·어뷰징 검증

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

const GRADUATE_LEVEL = 30;
const CO_REWARD_MINUTES = 30; // 누적 30 분마다 보상
const REWARD_GOLD = 5000;

export class MentorService {
  constructor(private db: Kysely<DB>) {}

  /** 매칭 성립: 자격 + 어뷰징 검증 */
  async pair(mentorId: number, menteeId: number, abuseCheck: {
    similarDevice: (a: number, b: number) => boolean;
  }): Promise<boolean> {
    const [mentor, mentee] = await Promise.all([
      this.db.selectFrom("characters").where("id", "=", mentorId)
        .select(["level", "play_min"]).executeTakeFirst(),
      this.db.selectFrom("characters").where("id", "=", menteeId)
        .select(["level", "play_min"]).executeTakeFirst(),
    ]);
    if (!mentor || !mentee) return false;
    if (mentor.level < 40) return false; // 멘토 자격
    if (mentee.level > 25) return false; // 멘티 자격(신규)
    if (abuseCheck.similarDevice(mentorId, menteeId)) return false; // 다계정 차단

    const active = await this.db.selectFrom("mentor_pairs")
      .where("mentee_id", "=", menteeId).where("status", "=", "active")
      .executeTakeFirst();
    if (active) return false; // 중복 매칭 방지
    await this.db.insertInto("mentor_pairs").values({
      mentor_id: mentorId, mentee_id: menteeId, status: "active",
    }).execute();
    return true;
  }

  /** 함께 플레이 시간 누적(전투 종료 시 양쪽 세션 합산) */
  async addCoPlay(pairId: number, minutes: number) {
    await this.db.updateTable("mentor_pairs")
      .set(eb => ({ play_min: eb("play_min", "+", minutes) }))
      .where("id", "=", pairId).where("status", "=", "active").execute();
  }

  /** 보상 정산(멘토/멘티 구분 지급 — ledger 먹등) */
```

```

async settleRewards(pairId: number) {
  const p = await this.db.selectFrom("mentor_pairs").where("id", "=", pairId)
    .select(["play_min", "mentor_id", "mentee_id", "status"]).executeTakeFirst();
  if (!p || p.status !== "active") return;
  const milestones = Math.floor(p.play_min / CO_REWARD_MINUTES);
  const ref = "mentor:co:" + pairId + ":" + milestones;
  await this.grantOnce(ref, p.mentor_id, REWARD_GOLD);
  await this.grantOnce("mentor:co:mate:" + ref, p.mentee_id, REWARD_GOLD * 2);
}

/** 졸업 판정(멘티 레벨업 후에서 호출) */
async checkGraduation(menteeId: number, newLevel: number) {
  if (newLevel < GRADUATE_LEVEL) return;
  const p = await this.db.selectFrom("mentor_pairs")
    .where("mentee_id", "=", menteeId).where("status", "=", "active")
    .select(["id", "mentor_id"]).executeTakeFirst();
  if (!p) return;
  await this.db.updateTable("mentor_pairs").set({ status: "graduated" })
    .where("id", "=", p.id).execute();
  await this.grantOnce("mentor:grad:" + p.id, p.mentor_id, 50000);
  await this.grantOnce("mentor:grad:me:" + p.id, menteeId, 30000);
}

private async grantOnce(ref: string, charId: number, gold: number) {
  const paid = await this.db.selectFrom("ledger")
    .where("ref", "=", ref).where("to_user", "=", charId)
    .select("id").executeTakeFirst();
  if (paid) return; // 먹등
  await this.db.updateTable("characters").set(eb => ({ gold: eb("gold", "+", gold) }))
    .where("id", "=", charId).execute();
  await this.db.insertInto("ledger").values({
    at: new Date(), kind: "gold", from_user: null, to_user: charId,
    ref, payload: JSON.stringify({ gold }),
  }).execute();
}
}

```

(3) 성능 최적화 팁

- 보상은 ledger ref 먹등 지급이 원칙이다 — settle 이 여러 번 돌아도 이중 지급이 없다.
- '함께 플레이' 인정 조건(양쪽 전투 참여 + 최소 지속)이 없으면 두 계정이 맵에 서 있는 것만으로 시간이 쌓인다.
- 멘티 졸업 보상은 레벨 도달 시점 재검증(실제 플레이 지표) 후 지급해 수작업 성장 다계정을 걸러낸다.
- 추천인 코드는 가입 완료 7일 이내에만 입력 가능하게 해 '부계정으로 코드 수확'을 제한한다.

제 8 장 모듈 7 — 주요 콘텐츠 시스템 (061~070)

기술 스택: TypeScript · Kysely · uWebSockets.js 본 장은 유저가 매일 돌아오는 이유를 만드는 콘텐츠 계층이다. 퀘스트 트래킹과 미션 시스템은 하나의 이벤트 체커를 공유하고, 던전·필드 보스·PvP는 전용 룸 세션 관리 위에서 운영된다. 제작·펫·하우징·낚시·배틀패스는 각각의 미니 시스템이지만, 보상 지급·기여도 정산·명등 지급의 공통 패턴을 재사용한다.

061. 퀘스트 트래킹(메인, 서브, 일일) 및 대화/조건 달성 체커

(1) 역할 설명

퀘스트 시스템은 세 유형을 하나의 체커로 처리한다. 메인(진행형 스토리 체인), 서브(독립 조건), 일일(주기 리셋 반복)이다. 퀘스트 정의는 트리(선행 의존) + 목표(type: talk/hunt/reach/collect) + 보상 구조이며, 진행 상태는 (잠금 → 진행중 → 완료대기 → 보상수령)의 상태머신이다. 대화 조건(talk)은 NPC 접촉, 사냥(hunt)은 처치 이벤트, 이동(reach)은 포탈/구역 진입 이벤트가 트리거다.

조건 체커는 이벤트 버스 구독자로 — onEnemyKilled, onNpcTalk, enterPortal 등의 게임 이벤트를 받아 진행 중 퀘스트의 목표와 매칭한다. 핵심 방어는 '진행도는 서버 이벤트에서만 증가'라는 원칙이며, 클라 UI는 표시 전용이다. 일일 퀘스트는 리셋 크론(서버 시각 기준 00:00)에서 상태 초기화를 배치 수행한다.

(2) 구현 코드 — src/game/QuestEngine.ts — 트리 + 조건 체커

```
export type QuestType = "main" | "sub" | "daily";
export type ObjectiveKind = "talk" | "hunt" | "reach" | "collect";

export interface QuestObjective {
  kind: ObjectiveKind;
  targetCode: string; // NPC 코드 / 몬스터 코드 / 지역 코드 / 아이템 코드
  count: number; // 필요 횟수(1 = 1 회성)
}

export interface QuestDef {
  id: string;
  type: QuestType;
  name: string;
  summary: string;
  requires: string[]; // 선행 퀘스트 id (빈 배열 = 즉시 수락 가능)
  minLevel: number;
  objectives: QuestObjective[];
  rewards: { gold?: number; exp?: number; items?: { code: string; qty: number }[] };
  nextId?: string; // 체인 다음 퀘스트
}

export type QuestState = "locked" | "accepted" | "in_progress"
  | "ready_to_complete" | "completed";
```

```

export interface QuestProgress {
  defId: string; state: QuestState;
  counts: number[];      // objective 별 진행도
  resetAt?: number;      // 일일 퀘스트 리셋 기준
}

export class QuestEngine {
  private progress = new Map<string, QuestProgress>();

  constructor(private defs: Map<string, QuestDef>,
    private db: { saveProgress(p: QuestProgress[]): Promise<void> } ) {}

  /** 수락 가능 판정: 선행 완료 + 레벨 */
  canAccept(defId: string, level: number): boolean {
    const d = this.defs.get(defId);
    if (!d || level < d.minLevel) return false;
    return d.requires.every(r => this.progress.get(r)?.state === "completed");
  }

  accept(defId: string) {
    const d = this.defs.get(defId)!;
    this.progress.set(defId, {
      defId, state: "in_progress", counts: d.objectives.map(() => 0),
    });
  }

  /** 게임 이벤트 → 조건 매칭(서버 이벤트에서만 호출) */
  onEvent(kind: ObjectiveKind, targetCode: string, amount = 1) {
    for (const p of this.progress.values()) {
      if (p.state !== "in_progress") continue;
      const d = this.defs.get(p.defId)!;
      let changed = false;
      d.objectives.forEach((o, i) => {
        if (o.kind !== kind || o.targetCode !== targetCode) return;
        if (p.counts[i] >= o.count) return;
        p.counts[i] = Math.min(o.count, p.counts[i] + amount);
        changed = true;
      });
      if (changed && d.objectives.every((o, i) => p.counts[i] >= o.count)) {
        p.state = "ready_to_complete";
      }
    }
  }

  /** 보상 수령(서버 트랜잭션 — 재화/아이템 시스템 위임) */
  async complete(defId: string, grant: (def: QuestDef) => Promise<void>) {
    const p = this.progress.get(defId);
    if (!p || p.state !== "ready_to_complete") return false;
    const d = this.defs.get(defId)!;
    p.state = "completed";
    await grant(d);
  }
}

```

```

// 체인 연결: nextId 는 즉시 수락 가능 상태가 됨(canAccept 가 처리)
return true;
}

/** 일일 리셋(크론): state/completed 를 진행 가능으로 되돌림 */
resetDaily(todayStart: number) {
  for (const p of this.progress.values()) {
    const d = this.defs.get(p.defId);
    if (d?.type !== "daily") continue;
    if ((p.resetAt ?? 0) < todayStart) {
      p.state = "in_progress";
      p.counts = d.objectives.map(() => 0);
      p.resetAt = todayStart;
    }
  }
}
}
}
}

```

보조 코드 — 이벤트 혹은 연결 예시(WorldScene 발췌)

```

// 클라/서버 게임 이벤트 → 퀘스트 체커 위임(서버에서만 진행도 증가)
// onEnemyKilled(monsterCode) → quest.onEvent("hunt", monsterCode)
// onNPCTalk(npcCode) → quest.onEvent("talk", npcCode)
// enterPortal(mapId) → quest.onEvent("reach", mapId)
// onItemGained(itemCode) → quest.onEvent("collect", itemCode)

// 세이브 구조: questIdx(메인 체인 커서) + subState(서브/일일 진행도)
// 재접속 복구 시: 퀘스트 정의의 requires 체인으로 "다음 메인 퀘스트"를 자동 재배치
// — v3.0.28 reach 버그 교훈: enterPortal에서 reach 목표 advance 를 반드시 수행

```

(3) 성능 최적화 팁

- 메인 체인은 nextId 링크 + requires 이중 결합으로 — 세이브 커서가 유실돼도 체인 순회로 복구 가능하다.
- reach 목표는 포탈/구역 진입 이벤트에서 반드시 처리한다 — 씬 전환만으로 완료 처리를 빼먹는 것이 이동 퀘스트 불완료의 최대 원인이다.
- 일일 리셋은 유저별 리셋 시각(접속 시 보정)이 아니라 서버 단일 시각으로 — 다계정 리셋 어뷰징을 막는다.
- 대화(talk) 조건은 NPC 접촉 반경 + 대화 UI 오픈 이벤트에서만 인정해 '지나가기만 해도 완료' 버그를 막는다.

062. 일일/주간 미션 및 활동도 게이지 보상 시스템

(1) 역할 설명

미션 시스템은 '개별 미션 목록 + 활동도(전체 진행도 게이지)'의 이중 보상 구조다. 미션은 일일(매일 리셋)과 주간(월요일 리셋)으로 나뉘고, 각 미션은 고정 목표(로그인, 몬스터 50 마리, 던전 1 회 등)를 가진다. 활동도는 미션 완료 시 점수를 누적해 게이지 구간

(30/60/100/140)에 도달하면 구간 보상을 지급한다 — 개별 미션과 별개로 '전체 활동성'을 보상하는 장치다.

구현 핵심은 미션 정의의 통합 이벤트 스키마(061 번 QuestObjective 재사용)다. 퀘스트 체커와 같은 이벤트 버스를 구독하되, 리셋 주기만 다르게 관리한다. 활동도 게이지 구간 보상은 ledger ref 먹등 지급(060 번 패턴)으로 이중 지급을 방지하고, 리셋 크론은 퀘스트 리셋과 같은 배치에서 수행한다.

(2) 구현 코드 — server/mission/DailyMission.ts

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface MissionDef {
  id: string; period: "daily" | "weekly";
  name: string; event: string; target: string | null; count: number;
  activityScore: number; // 완료 시 활동도 점수
  rewards: { gold?: number; items?: { code: string; qty: number }[] };
}

export interface GaugeTier { score: number; rewards: { gold?: number; items?: any[] }; }

export class MissionService {
  constructor(private db: Kysely<DB>, private defs: MissionDef[],
    private tiers: GaugeTier[]) {}

  /** 이벤트 진행도 증가(퀘스트 체커와 동일 이벤트 버스 사용) */
  async onEvent(charId: number, event: string, target: string, amount = 1) {
    const applicable = this.defs.filter(d => d.event === event
      && (!d.target || d.target === target));
    if (!applicable.length) return;
    const today = new Date().toISOString().slice(0, 10);
    await this.db.transaction().execute(async tx => {
      for (const d of applicable) {
        const ref = "mission:" + d.id + ":" + (d.period === "daily" ? today : weekKey());
        const done = await tx.selectFrom("ledger")
          .where("ref", "=", ref).where("to_user", "=", charId).select("id").executeTakeFirst();
        if (done) continue;
        const row = await tx.selectFrom("mission_progress")
          .where("character_id", "=", charId).where("mission_id", "=", d.id)
          .select(["count", "at"]).executeTakeFirst();
        // 주간/일일 경계 판정: 마지막 갱신이 현재 주기 밖이면 리셋
        const periodKey = d.period === "daily" ? today : weekKey();
        const count = row && row.at.toISOString().slice(0, 10) === periodKey
          ? row.count + amount : amount;
        if (count >= d.count) {
          await tx.updateTable("mission_progress")
            .set({ count, completed: true, at: new Date() })
            .where("character_id", "=", charId).where("mission_id", "=", d.id).execute();
          await tx.insertInto("ledger").values({
            at: new Date(), kind: "gold", from_user: null, to_user: charId,
            ref, payload: JSON.stringify({ mission: d.id, score: d.activityScore }),

```

```

    }).execute();
    // 활동도 점수 누적
    await tx.updateTable("mission_activity")
      .set(eb => ({ score: eb("score", "+", d.activityScore) }))
      .where("character_id", "=", charId).execute();
  } else {
    await tx.updateTable("mission_progress")
      .set({ count, at: new Date() })
      .where("character_id", "=", charId).where("mission_id", "=", d.id).execute();
  }
}
});
}

/** 활동도 게이지 구간 보상 지급(먹등) */
async settleGauge(charId: number) {
  const act = await this.db.selectFrom("mission_activity")
    .where("character_id", "=", charId).select("score").executeTakeFirst();
  if (!act) return;
  for (const t of this.tiers) {
    if (act.score < t.score) continue;
    await this.grantOnce("gauge:" + weekKey() + ":" + t.score, charId, t.rewards);
  }
}

private async grantOnce(ref: string, charId: number, rewards: any) {
  const paid = await this.db.selectFrom("ledger").where("ref", "=", ref)
    .where("to_user", "=", charId).select("id").executeTakeFirst();
  if (paid) return;
  await this.db.insertInto("ledger").values({
    at: new Date(), kind: "gold", from_user: null, to_user: charId,
    ref, payload: JSON.stringify(rewards),
  }).execute();
}

function weekKey(): string {
  const d = new Date();
  const day = (d.getDay() + 6) % 7; // 월요일 = 0
  const monday = new Date(d.getFullYear(), d.getMonth(), d.getDate() - day);
  return monday.toISOString().slice(0, 10);
}

```

(3) 성능 최적화 팁

- 미션과 퀘스트는 같은 이벤트 버스(061 번)를 구독하되 리셋 주기만 다르게 — 이벤트 정의가 이중으로 흩어지면 불일치 버그가 난다.
- 활동도 점수는 '미션 완료 시점'에만 누적한다 — 진행도에 비례해 점수를 주면 계산이 복잡해지고 조작 여지가 생긴다.
- 구간 보상 ref 에 주기 키(weekKey)를 포함해야 주간 리셋 후 재지급이 자연스럽게 동작한다.

- 일일 미션은 '로그인+사냥+던전'의 3축 골격을 유지해 유저 루틴을 만들고, 변주는 보상에서 준다.

063. 인스턴스 던전 솔로/파티 매칭 및 전용 룸 세션 관리

(1) 역할 설명

던전 시스템은 '매칭 큐 → 전용 룸 생성 → 세션 진행(진도/전멸/보상) → 정산·해체'의 사이클이다. 솔로는 즉시 룸 생성, 파티는 파티 단위 진입, 매칭 던전은 레벨 밴드별 큐에서 4명이 모이면 룸을 생성한다. 룸 세션은 017번 RoomRegistry로 관리되며, 세션 데이터(진도, 몬스터 상태, 시간 제한)는 룸 소유 프로세스가 단일 권위로 운영한다.

세션 관리의 핵심은 비정상 종료 대응이다. 유저 접속 끊김 시 60초 재접속 유예(세션 유지), 유예 경과 시 자동 탈주 처리, 전원 이탈 시 룸 DRAINING 전환을 상태머신으로 처리한다. 탈주 페널티(재매칭 대기 5분)는 매칭 품질의 핵심이며, 던전 완료 보상은 기여도(딜량+지속 시간) 기반으로 개별 지급한다.

(2) 구현 코드 — server/dungeon/DungeonSession.ts

```
import uWS from "uWebSockets.js";

export type DungeonPhase = "matching" | "prep" | "running" | "cleared" | "failed" | "draining";

export interface DungeonRoom {
  roomId: string; dungeonCode: string;
  phase: DungeonPhase;
  members: Map<number, {
    online: boolean; disconnectedAt: number | null;
    dmg: number; deaths: number; joinedAt: number;
  }>;
  progress: number;           // 진도(0~100)
  startedAt: number; timeLimitMs: number;
  readySet: Set<number>;      // prep 단계 준비 완료
}

export class DungeonSession {
  constructor(private room: DungeonRoom, private ws: uWS.WebSocket,
    private broadcast: (msg: unknown) => void) {}

  /** prep: 전원 준비 → running */
  toggleReady(charId: number): boolean {
    if (this.room.phase !== "prep") return false;
    if (this.room.readySet.has(charId)) this.room.readySet.delete(charId);
    else this.room.readySet.add(charId);
    if (this.room.readySet.size >= this.room.members.size) {
      this.room.phase = "running";
      this.room.startedAt = Date.now();
      this.broadcast({ t: "dungeon_start", timeLimit: this.room.timeLimitMs });
    }
  }
}
```



```

    }
    return true;
}

/** 틱: 시간 제한/전멸/이탈 감시 */
tick(now: number) {
    if (this.room.phase !== "running") return this.sweepDisconnect(now);
    if (now - this.room.startedAt > this.room.timeLimitMs) {
        this.room.phase = "failed";           // 시간 초과
        this.broadcast({ t: "dungeon_fail", reason: "timeout" });
        return;
    }
    this.sweepDisconnect(now);
}

/** 접속 끊김 유예(60 초) → 탈주 처리 */
private sweepDisconnect(now: number) {
    for (const [id, m] of this.room.members) {
        if (m.online || m.disconnectedAt == null) continue;
        if (now - m.disconnectedAt > 60_000) {
            m.online = false;
            m.disconnectedAt = null;
            this.broadcast({ t: "dungeon_member_left", charId: id });
            // 파티 던전에서 나머지가 계속 진행 가능(솔로 유저 룸만 실패 처리)
            const alive = [...this.room.members.values()].filter(x => x.online).length;
            if (alive === 0) {
                this.room.phase = "draining";
            }
        }
    }
}

onReconnect(charId: number) {
    const m = this.room.members.get(charId);
    if (m) { m.online = true; m.disconnectedAt = null; }
}

/** 클리어: 기여도 기반 개별 정산 */
clear(now: number) {
    this.room.phase = "cleared";
    const results = [...this.room.members.entries()]
        .filter(([ , m]) => m.online)
        .map(([charId, m]) => ({
            charId,
            share: m.dmg / Math.max(1, [...this.room.members.values()]
                .reduce((s, x) => s + x.dmg, 0)),
            deaths: m.deaths,
        }));
    this.broadcast({ t: "dungeon_cleared", results });
    return results;
}
}

```

(3) 성능 최적화 팁

- 매칭 밴드는 레벨 $\pm 10\%$ 가 표준이며, 큐 대기 60 초 초과 시 밴드를 단계적으로 확장 ($\pm 15\% \rightarrow \pm 20\%$)하는 '대기 확장'이 품질과 속도의 균형이다.
- prep 단계 전원 준비는 필수 UX 다 — 준비 없이 시작하면 텔레포트 스폰 즉시 전투에 휘말리는 불만이 쏟아진다.
- 탈주 유예 60 초는 네트워크 재접속(005 번)과 같은 값으로 맞춰야 '끊김 = 탈주' 오해가 사라진다.
- 던전 진도는 체크포인트(구역 진입) 단위로 저장해 전멸 재시작이 구역 단위로 되게 하면 난이도 체감이 급격히 완화된다는 점.

064. 필드 보스 스폰 타이머, 알림 및 기여도(딜량)별 아이템 분배 로직

(1) 역할 설명

필드 보스는 '스폰 스케줄(죽은 시각 + 리젠 주기) → 사전 알림 → 등장 → 전투 → 기여도 정산'의 라이프사이클을 갖는다. 스폰 타이머는 서버 단일 권위로 관리하되, 예상 리젠 시각을 클라 UI에 노출(몇 분 후)해 유저가 계획하게 한다. 알림은 3 단계(등장 5 분 전, 직전 30 초, 등장 즉시)로 채널 방송(051 번 시스템 채널 + 049 번 이벤트 배너)한다.

정산은 '기여도 테이블(딜량 누적) → 등급 분배' 방식이 표준이다. 전체 딜량에 대한 각자 비율로 등급(상위 10% = 3 개, 30% = 2 개, 그 외 = 1 개, 최소 참여 미달 = 0 개)을 나눠 지급하고, 라스트 히트 보너스는 별도 소액으로 준다. 어그로를 끌지 않고 딜만 넣는 유저도 기여도로 인정되며, 첫 10 초 미참여(딜 0) 유저는 배제해 관전 참여를 막는다.

(2) 구현 코드 — `server/boss/FieldBoss.ts`

```
import uWS from "uWebSockets.js";

export interface BossSchedule {
  bossCode: string; mapId: string;
  spawnIntervalMs: number;      // 리젠 주기
  announceBeforeMs: number;      // 등장 전 알림(5 분)
  dropTier: { ratio: number; items: { code: string; qty: number }[] }[];
  minParticipationMs: number;    // 최소 참여 시간
}

export interface BossInstance {
  bossCode: string; spawnedAt: number;
  aggroTable: Map<number, { dmg: number; firstHitAt: number; lastHitAt: number }>;
  dead: boolean;
}

export class FieldBossManager {
  private nextSpawnAt = new Map<string, number>();
  private current: BossInstance | null = null;
```

```

constructor(private defs: BossSchedule[], private notify: (msg: unknown) => void) {}

/** 서버 틱: 스폰/알림 감시 */
tick(now: number) {
  for (const def of this.defs) {
    const next = this.nextSpawnAt.get(def.bossCode) ?? 0;
    if (this.current || next === 0) {
      if (!next) this.nextSpawnAt.set(def.bossCode, now + def.spawnIntervalMs);
      continue;
    }
    const remain = next - now;
    if (remain <= 0) {
      this.current = { bossCode: def.bossCode, spawnedAt: now,
        aggroTable: new Map(), dead: false };
      this.notify({ t: "boss_spawn", boss: def.bossCode, map: def.mapId });
    } else if (remain <= def.announceBeforeMs && !this.warned.get(def.bossCode)) {
      this.warned.set(def.bossCode, true);
      this.notify({ t: "boss_preannounce", boss: def.bossCode,
        inMs: remain });
    }
  }
}
private warned = new Map<string, boolean>();

/** 데미지 기록(서버 전투 판정에서 호출) */
onDamage(charId: number, dmg: number, now: number) {
  if (!this.current || this.current.dead) return;
  const e = this.current.aggroTable.get(charId)
    ?? { dmg: 0, firstHitAt: now, lastHitAt: now };
  e.dmg += dmg; e.lastHitAt = now;
  this.current.aggroTable.set(charId, e);
}

/** 사망 정산: 기여도 등급별 지급 */
async onDeath(now: number, def: BossSchedule, grant: (charId: number, items: any[]) =>
Promise<void>) {
  const inst = this.current!;
  inst.dead = true;
  this.current = null;
  this.warned.delete(def.bossCode);
  this.nextSpawnAt.set(def.bossCode, now + def.spawnIntervalMs);

  // 최소 참여(시간) 필터
  const entries = [...inst.aggroTable.entries()]
    .filter(([_, e]) => e.lastHitAt - e.firstHitAt >= def.minParticipationMs
      || e.dmg > 0);
  const totalDmg = entries.reduce((s, [_, e]) => s + e.dmg, 0);
  if (!totalDmg) return;

  // 비율 누적 → 등급 배정
  const sorted = entries.sort((a, b) => b[1].dmg - a[1].dmg);

```

```

let cumulative = 0;
for (let i = 0; i < sorted.length; i++) {
  const [charId, e] = sorted[i];
  cumulative += e.dmg / totalDmg;
  const tier = i === 0 && cumulative <= 0.1 ? def.dropTier[0]
    : cumulative <= 0.3 ? def.dropTier[1]
    : def.dropTier[def.dropTier.length - 1];
  await grant(charId, tier.items);
}
// 라스트 히트 소액 보너스
const last = sorted[0];
if (last) await grant(last[0], [{ code: "bonus_last_hit", qty: 1 }]);
this.notify({ t: "boss_defeated", boss: def.bossCode, participants: sorted.length });
}
}

```

(3) 성능 최적화 팁

- 리젠 시각 노출(몇 분 후)은 서버가 권위로 알려줘야 합니다 — 클라 추측 시각은 어긋나면 불신을 산다.
- 기여도 등급 분배는 '비율 누적(cumulative)' 방식으로 상위 점유율을 안정적으로 반영한다 — 절대 딜량 컷은 파티 구성에 따라 공정성이 깨진다.
- 최소 참여 시간(10 초) 게이팅은 관전자 무료 승차를 막는 최소한의 장치다.
- 라스트 히트 보너스는 소액으로 — 커지면 '아군 기술 뺏기' 전쟁이 벌어진다.

065. PvP 전장 및 PK 카오스(성향치) 시스템 메커니즘

(1) 역할 설명

PvP 시스템은 두 축으로 나뉜다. 전장(인스턴스 — 진영전, 8v8 등, 규칙성 보상)과 야외 PK(필드 자유 공격 + 성향치 패널티)다. 야외 PK는 성향치(karma)로 통제한다. 타인을 죽이면 성향이 음수로 떨어지고, 음수 구간(카오스)에서는 사망 페널티(장비 드롭 확률), 상점 이용 제한, 가드 NPC 공격 대상이 된다. 성향은 시간 경과 또는 정화 아이템으로 회복된다.

구현 포인트는 PK 판정의 무죄 조건이다. 동의 전투(결투 신청 수락), 전장 내부, 자기방어(먼저 공격한 상대 처치)는 성향 패널티가 없다. 서버는 전투 관계 테이블(누가 누구를 먼저 때렸는지, 결투 동의 여부)을 유지해 사망 시점에 판정한다. 카오스 유저는 시각 표시(빨간 이름)로 피해자가 위험을 인지하게 한다.

(2) 구현 코드 — server/pvp/Karma.ts

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface PvpContext {
  inBattlefield: boolean; // 전장/결투장 내부
  duelAccepted: boolean; // 상호 결투 동의
  victimAttackedFirst: boolean; // 피해자가 먼저 공격(자기방어)

```

```

}

const KARMA_KILL = -30;
const KARMA_RECOVER_PER_HOUR = 5;
export const KARMA_CHAOS = -100; // 이 이하 = 카오스
export const KARMA_NEUTRAL = 0;

export class KarmaService {
  constructor(private db: Kysely<DB>) {}

  /** 킬 판정: 무죄 조건 확인 후 성향 감소 */
  async onPlayerKill(killerId: number, ctx: PvpContext) {
    const innocent = ctx.inBattlefield || ctx.duelAccepted || ctx.victimAttackedFirst;
    if (innocent) return { penalty: 0 };
    const delta = KARMA_KILL;
    await this.db.updateTable("characters")
      .set(eb => ({ karma: eb("karma", "+", delta) }))
      .where("id", "=", killerId).execute();
    return { penalty: delta };
  }

  /** 시간 회복(로그인 시 1 회 + 주기 배치) */
  async recoverOnLogin(charId: number, offlineHours: number) {
    const ch = await this.db.selectFrom("characters").where("id", "=", charId)
      .select("karma").executeTakeFirstOrThrow();
    if (ch.karma >= KARMA_NEUTRAL) return ch.karma;
    const recovered = Math.min(KARMA_NEUTRAL,
      ch.karma + KARMA_RECOVER_PER_HOUR * Math.floor(offlineHours));
    await this.db.updateTable("characters").set({ karma: recovered })
      .where("id", "=", charId).execute();
    return recovered;
  }

  /** 카오스 상태 판정(클라 이름색/상점/가드 로직 입력) */
  static isChaos(karma: number) { return karma <= KARMA_CHAOS; }
  static nameColor(karma: number): number {
    if (KarmaService.isChaos(karma)) return 0xff4444; // 빨강
    if (karma < 0) return 0xffaa44; // 주황
    return 0x9ad8ff; // 기본
  }

  /** 카오스 사망 페널티: 장비 드롭 확률 */
  static deathDropChance(karma: number): number {
    if (!KarmaService.isChaos(karma)) return 0;
    const depth = Math.min(3, Math.floor((KARMA_CHAOS - karma) / 100)); // 단계
    return 0.15 + depth * 0.15; // 15/30/45%
  }
}

/** 전장(인스턴스) 스코어: 킬/오브젝트 → 팀 점수, 종료 보상 */

```

```
export class Battlefield {
  scores = { red: 0, blue: 0 };
  constructor(private limit: number, private onEnd: (winner: "red" | "blue") => void) {}
  addScore(team: "red" | "blue", v = 1) {
    this.scores[team] += v;
    if (this.scores[team] >= this.limit) this.onEnd(team);
  }
}
```

(3) 성능 최적화 팁

- 성향 판정은 사망 시점의 관계 테이블(먼저 공격한 자, 결투 동의)로 — '먼저 때린 쪽이 피해자 사망 시 무죄' 규칙이 자기방어의 핵심이다.
- 카오스 드롭 확률은 단계별(15/30/45%)로 두되 절대 100%가 아니게 — 지옥이 아니라 경제적 부담으로 설계해야 복귀가 가능하다.
- 전장은 인스턴스 룸(017 번)으로 완전 격리해 필드 유저에게 영향을 주지 않게 한다.
- 가드 NPC(카오스 유저 공격)는 필드 도시 안전성의 핵심이며, 가드 공격도 성향 판정 테이블에 포함해야 한다.

066. 제작/연금술 재료 합성 레시피 연산 모듈

(1) 역할 설명

제작 시스템은 '레시피(재료 → 결과물 + 확률/대체재) → 재고 검증 → 소모 → 산출'의 연산 파이프라인이다. 레시피 정의는 (1) 재료 목록(코드, 수량, 대체재 허용 여부), (2) 결과물 (아이템/수량/랜덤 옵션 여부), (3) 스킬·도구 요구(연금술 레벨, 제작대 접근)로 구성된다. 재고 검증과 소모는 단일 트랜잭션으로 — 검증과 소모 사이의 동시 제작 요청이 재료를 이중 소모하는 레이스를 차단한다.

성공률/품질 계산도 서버 연산이다. 기본 성공률 + 제작 스킬 보정 + 행운 버프로 성공을 판정하고, 실패 시 재료의 일부(예: 50%)만 회수하는 정책으로 실패 부담을 조절한다. 연금술 (포션)은 개수 배치 제작(1 회 시도로 N 개)을 지원해 재고 연산을 수율 (multiplier)로 일반화한다. 제작 이력은 로그로 남겨 경제 지표(재료 소비량) 분석에 쓴다.

(2) 구현 코드 — server/craft/Craft.ts

```
import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface Ingredient { itemCode: string; qty: number; substitutes?: string[]; }
export interface RecipeDef {
  id: string; name: string;
  craftMs: number; // 제작 소요(0 = 즉시)
  requiredSkillLevel?: number; // 연금술/대장장이 레벨
  ingredients: Ingredient[];
  output: { itemCode: string; qty: number };
  successRate: number; // 0~1
```

```

failRefundRate: number;      // 실패 시 회수율(0.5 = 절반)
allowBatch: boolean;        // 개수 배치(연금술)
}

export class CraftService {
  constructor(private db: Kysely<DB>, private recipes: Map<string, RecipeDef>) {}

  /** 제작 실행: 재고 검증 → 원자적 소모 → 산출 */
  async craft(charId: number, recipeId: string, batch = 1) {
    const def = this.recipes.get(recipeId);
    if (!def) throw new Error("NO_RECIPE");
    if (batch < 1 || (!def.allowBatch && batch > 1)) throw new Error("BATCH_FORBIDDEN");

    return this.db.transaction().execute(async tx => {
      // 1) 스킬 요구
      if (def.requiredSkillLevel) {
        const sk = await tx.selectFrom("character_skill")
          .where("character_id", "=", charId)
          .where("skill_code", "=", "craft_" + recipeId.split("_")[0])
          .select("rank").executeTakeFirst();
        if (!sk || sk.rank < def.requiredSkillLevel) throw new Error("LOW_SKILL");
      }
      // 2) 재료 소유 검증 + 소모(수량 n 배)
      for (const ing of def.ingredients) {
        const need = ing.qty * batch;
        // 대체재 포함 보유량 합산 후 차감
        const codes = [ing.itemCode, ...(ing.substitutes ?? [])];
        const owned = await tx.selectFrom("character_items")
          .where("character_id", "=", charId)
          .where(eb => eb("item_code", "in", codes))
          .select(["id", "item_code", "qty"]).forUpdate().execute();
        const have = owned.reduce((s, r) => s + r.qty, 0);
        if (have < need) throw new Error("NO_MATERIAL");
        let left = need;
        for (const row of owned.sort((a, b) => a.qty - b.qty)) {
          if (left <= 0) break;
          const take = Math.min(row.qty, left);
          left -= take;
          await tx.updateTable("character_items")
            .set(eb => ({ qty: eb("qty", "-", take) }))
            .where("id", "=", row.id).execute();
        }
      }
      // 3) 성공 판정 + 산출
      const ok = Math.random() < def.successRate;
      if (!ok) {
        // 실패: 재료 일부 회수(재고로 환원)
        for (const ing of def.ingredients) {
          const refund = Math.floor(ing.qty * batch * def.failRefundRate);
          if (refund > 0) {
            await tx.updateTable("character_items")
              .set(eb => ({ qty: eb("qty", "+", refund) }))

```

```

        .where("character_id", "=", charId)
        .where("item_code", "=", ing.itemCode).execute();
    }
}
return { ok: false, output: null };
}
const outQty = def.output.qty * batch;
await tx.updateTable("character_items")
    .set(eb => ({ qty: eb("qty", "+", outQty) }))
    .where("character_id", "=", charId)
    .where("item_code", "=", def.output.itemCode).execute();
// 제작 이력(경제 지표)
await tx.insertInto("ledger").values({
    at: new Date(), kind: "item", from_user: null, to_user: charId,
    ref: "craft:" + recipeId, payload: JSON.stringify({ batch, outQty }),
}).execute();
return { ok: true, output: { code: def.output.itemCode, qty: outQty } };
});
}
}

```

(3) 성능 최적화 팁

- 재료 소모는 '보유량 합산 → 차감' 패턴으로 대체재를 지원한다 — 개별 재고 검증 방식은 대체재 로직이 중복된다.
- 실패 회수율(50%)은 제작 실패의 스트레스와 재료 경제 사이의 균형점이며, 고급 레시피는 60~70%로 완화한다.
- 배치 제작(연금술)은 반복 클릭을 없애는 UX 장치이자, 재고 연산을 수율로 일반화하는 설계 이점이 있다.
- 제작 이력 로그는 인플레이션 모니터링(085 번)의 원천 데이터로 필수다.

067. 펫/탈것 이동속도 보정 및 자동 루팅 AI

(1) 역할 설명

펫/탈것은 (1) 이동 속도 보정(스탯 파이프라인 021 번의 speedMult), (2) 팔로우 AI(주인 뒤따르기), (3) 자동 루팅(드롭 근접 시 획득)의 세 기능으로 구성된다. 속도 보정은 탈것 등급(일반 1.3배, 희귀 1.5배, 레전더리 1.8배)과 펫 보너스를 합산하되, 전투 중 페널티(전투 시 0.6배)와 지형 제한(실내 탑승 금지) 규칙이 함께 적용된다.

자동 루팅은 '드롭 스폰 → 펫이 목표로 이동 → 반경 도달 시 획득 요청'의 AI 사이클이다. 획득은 서버가 검증(053 번 fffa 규칙과 동일 — 근접 + 선착)하고, 펫은 주인과 드롭 사이 우선순위(드롭이 3타일 이내면 폐치, 아니면 팔로우)로 결정한다. 붓 탐지(093 번)와의 경계 때문에, 자동 루팅은 '주인이 유효한 전투 상태일 때만' 활성화하는 게이팅을 둔다.

(2) 구현 코드 — src/game/PetAI.ts

```
import type { Vec } from "../combat/hitbox";
```



```

export interface MountDef { code: string; speedMult: number; canCombat: boolean; }
export interface PetState {
  x: number; y: number;
  mode: "follow" | "fetch";
  fetchTarget: { dropId: number; x: number; y: number } | null;
}

export class PetBrain {
  private static FOLLOW_DIST = 64;    // 팔로우 유지 거리
  private static FETCH_RANGE = 96;    // 루팅 판정 반경
  private static PET_SPEED = 170;     // 주인보다 약간 빠르게

  constructor(private rng: () => number = Math.random) {}

  /** 고정 업데이트: 우선순위(페치 > 팔로우)로 이동 벡터 산출 */
  tick(pet: PetState, owner: Vec & { inCombat: boolean; mounted: boolean },
    drops: { id: number; x: number; y: number; claimed: boolean }[],
    dtMs: number): { vx: number; vy: number; wantAcquire: number | null } {
    const dt = dtMs / 1000;

    // 1) 루팅: 주인이 비전투 + 미탑승 + 미점유 드롭 존재
    if (!owner.inCombat && !owner.mounted) {
      const near = drops
        .filter(d => !d.claimed
          && Math.hypot(d.x - owner.x, d.y - owner.y) < PetBrain.FETCH_RANGE)
        .sort((a, b) => Math.hypot(a.x - pet.x, a.y - pet.y)
          - Math.hypot(b.x - pet.x, b.y - pet.y))[0];
      if (near) {
        pet.mode = "fetch";
        pet.fetchTarget = { dropId: near.id, x: near.x, y: near.y };
      }
    }
    if (pet.mode === "fetch" && pet.fetchTarget) {
      const t = pet.fetchTarget;
      const d = Math.hypot(t.x - pet.x, t.y - pet.y);
      if (d <= 16) { // 도달 → 획득 요청
        pet.mode = "follow"; pet.fetchTarget = null;
        return { vx: 0, vy: 0, wantAcquire: t.dropId };
      }
      const ux = (t.x - pet.x) / d, uy = (t.y - pet.y) / d;
      pet.x += ux * PetBrain.PET_SPEED * dt;
      pet.y += uy * PetBrain.PET_SPEED * dt;
      return { vx: ux, vy: uy, wantAcquire: null };
    }

    // 2) 팔로우: 유지 거리 밖이면 접근(약간의 지연/브레이크로 자연스럽게)
    const dist = Math.hypot(owner.x - pet.x, owner.y - pet.y);
    if (dist > PetBrain.FOLLOW_DIST) {
      const ux = (owner.x - pet.x) / dist, uy = (owner.y - pet.y) / dist;
      const speed = PetBrain.PET_SPEED * (dist > 200 ? 1.4 : 1); // 멀면 가속
    }
  }
}

```

```

    pet.x += ux * speed * dt;
    pet.y += uy * speed * dt;
    return { vx: ux, vy: uy, wantAcquire: null };
  }
  return { vx: 0, vy: 0, wantAcquire: null };
}
}

/** 속도 보정 함수(021 번 StatMath 와 연결) */
export function movementSpeedMult(mount: MountDef | null,
    petBufs: { stat: string; mul?: number }[],
    inCombat: boolean): number {
  let mult = mount?.speedMult ?? 1;
  for (const b of petBufs) if (b.stat === "aspd" && b.mul) mult += b.mul;
  if (inCombat) mult *= 0.6;      // 전투 중 탈것 페널티
  return Math.min(2.2, mult);    // 상한 클램프
}

```

(3) 성능 최적화 팁

- 펫 속도는 주인보다 15~20% 빠르게 — 느리면 항상 뒤쳐져 '버려진 펫'처럼 보인다.
- 페치(루팅) 조건에 '주인 비전투 + 미탐승' 게이팅은 봇 탐지(093 번) 오탐을 크게 줄인다.
- 팔로우는 거리 멀수록 가속(멀면 1.4 배)해 유저가 워프하는 느낌 없이 따라붙게 만든다.
- 드롭 획득은 반드시 서버 근접+선착 검증(053 번)으로 — 펫이 자동 획득해도 규칙은 사람과 동일하다.

068. 하우스/개인 영지 가구 배치 및 인스턴스 공간 생성

(1) 역할 설명

하우스는 '개인 전용 인스턴스 공간 + 가구 배치(그리드 스냅) + 방문자 초대'의 구조다. 집은 017 번 룸 시스템에서 kind=housing 룸으로 생성되고, 주인 접속 시 1 회 생성(유효 10 분 후 회수), 방문자는 초대 토큰으로 입장한다. 가구 배치 데이터는 (가구 코드, 타일 좌표, 회전, 스킨)의 배열이며, 세이브는 JSONB 로 통째로 저장한다(부분 업데이트가 아니라 배치 편집 완료 시 덤프).

배치 편집 UX 의 핵심은 그리드 스냅 + 충돌(가구 겹침) 검증 + 배치 유효 좌표(벽/문 위 배치 금지)다. 클라는 드래그 편집 중 미리보기를 제공하고, 저장 시 서버가 전체 배치를 재검증(코드 유효성, 좌표 범위, 겹침)한다. 가구는 스탯 효과(편의도 — 회복 배율 등)를 가질 수 있어 050 번 핫스팟 회복과 파이프라인을 공유한다.

(2) 구현 코드 — server/housing/Housing.ts

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface FurnitureDef {

```

```

code: string;          // "bed_wood", "table_oak"
w: number; h: number;  // 그리드 점유 크기(타일)
category: "bed" | "table" | "decor" | "storage";
facility?: { kind: "rest"; regenMult: number } | { kind: "storage"; slots: number };
priceGold: number;
}
export interface PlacedFurniture {
  defCode: string; x: number; y: number; rot: 0 | 90 | 180 | 270; skin?: string;
}

const HOUSE_W = 24, HOUSE_H = 18;    // 집 내부 그리드(타일)

export class HousingService {
  constructor(private db: Kysely<DB>, private defs: Map<string, FurnitureDef>) {}

  /** 배치 검증: 범위/코드/겹침/벽 */
  validate(placed: PlacedFurniture[], blocked: Uint8Array): string | null {
    const occupied = new Set<string>();
    for (const p of placed) {
      const def = this.defs.get(p.defCode);
      if (!def) return "unknown:" + p.defCode;
      // 회전 시점에 w/h 스왑
      const [w, h] = p.rot % 180 === 90 ? [def.h, def.w] : [def.w, def.h];
      if (p.x < 0 || p.y < 0 || p.x + w > HOUSE_W || p.y + h > HOUSE_H) return "out_of_bounds";
      for (let dy = 0; dy < h; dy++) {
        for (let dx = 0; dx < w; dx++) {
          const gx = p.x + dx, gy = p.y + dy;
          if (blocked[gy * HOUSE_W + gx]) return "on_wall:" + gx + "," + gy;
          const key = gx + "," + gy;
          if (occupied.has(key)) return "overlap:" + key;
          occupied.add(key);
        }
      }
    }
    return null;
  }

  /** 배치 저장(주인 세이브 — JSONB 덤프) */
  async saveLayout(charId: number, placed: PlacedFurniture[], blocked: Uint8Array) {
    const err = this.validate(placed, blocked);
    if (err) throw new Error("INVALID_LAYOUT:" + err);
    const convenience = placed.reduce((s, p) => {
      const def = this.defs.get(p.defCode);
      return s + (def?.facility?.kind === "rest" ? 1 : 0);
    }, 0);
    await this.db.updateTable("housing")
      .set({ layout: JSON.stringify(placed), convenience })
      .where("owner_id", "=", charId).execute();
    return { convenience }; // 편의도 → 회복 배율(050 번 연계)
  }
}

```

```

/** 입장(주인/방문자): 룸 세션 연결 */
async enter(db: DB, charId: number, ownerId: number) {
  // 룸 존재 확인/생성(017 번 Registry 위임) + 방문자 권한(초대 목록 검증)
  return { roomId: "housing:" + ownerId, isOwner: charId === ownerId };
}
}

```

(3) 성능 최적화 팁

- 배치 데이터는 '편집 완료 시 덤프 저장'이 정답이다 — 가구 하나당 UPDATE 는 N+1 쓰기 폭탄이 된다.
- 그리드 스냅(타일 단위) + 서버 재검증(겹침/벽)은 필수다 — 픽셀 자유 배치는 충돌 검증이 수학적으로 지옥이 된다.
- 편의도(convenience) 같은 가구 누적 효과는 회복 배율(050 번)로 자연스럽게 콘텐츠화된다.
- 방문자 초대는 토큰(만료 24 시간) 방식이 간단하고, 친구 목록 연동(054 번)은 2 단계로 추가한다.

069. 캐주얼 미니게임(낚시 타이밍 미니게임 등) 상호작용

(1) 역할 설명

낚시 미니게임은 '기다림(랜덤 대기) → 입질 신호 → 타이밍 판정(성공 구간 밴드) → 결과 (어종/등급)'의 사이클이다. 타이밍 판정은 막대(게이지)가 이동하는 동안 유저가 클릭한 순간의 위치가 성공 밴드에 들어가는지로 판정하며, 성공 밴드 폭과 막대 속도가 난이도 파라미터다. 서버는 시도 기록(시작/판정 시각)으로 검증해 자동화(매크로)를 통계로 잡아낸다(093 번).

구현 원칙은 '연출은 클라, 판정은 서버'다. 클라는 입질 확률/타이밍 창을 받아 게이지를 그리고, 클릭 시 로컬 판정 결과를 서버에 보고하면 서버는 시간 검증(입질 후 최소 150ms 이내 클릭은 인위적) + 밴드 재계산으로 최종 승인한다. 결과 보상은 어종 테이블(등급별 무게 분포)로 추출하고, 도감(030 번)/미션(062 번) 이벤트로 흘러보낸다.

(2) 구현 코드 — src/game/Fishing.ts — 클라 미니게임 + 서버 검증

```

import Phaser from "phaser";

export interface FishingWindow {
  startAt: number;      // 입질 시각(서버 기준)
  bandStart: number;    // 성공 밴드(0~1 정규화)
  bandWidth: number;
  speed: number;        // 마커 속도(초당 양복 횟수)
  timeoutMs: number;    // 판정 제한
}

```

```

/** 클라: 게이지 렌더 + 클릭 판정 보고 */
export class FishingMinigame {
  private win: FishingWindow | null = null;
  private marker = 0;           // 0~1
  private direction = 1;

  start(w: FishingWindow) { this.win = w; }

  update(dtMs: number) {
    if (!this.win) return;
    const dt = dtMs / 1000;
    this.marker += this.direction * this.win.speed * dt;
    if (this.marker > 1) { this.marker = 1; this.direction = -1; }
    if (this.marker < 0) { this.marker = 0; this.direction = 1; }
  }

  /** 클릭: 로컬 판정 + 서버 보고 */
  onClick(nowLocal: number) {
    if (!this.win) return null;
    const elapsed = nowLocal - this.win.startAt;
    const inBand = this.marker >= this.win.bandStart
      && this.marker <= this.win.bandStart + this.win.bandWidth;
    this.win = null;
    return { hit: inBand, marker: this.marker, elapsedMs: elapsed };
  }
}

/** 서버: 시간 검증 + 밴드 재계산으로 최종 승인 */
import { Kysely } from "kysely";
type DB = import("./schema").Database;
export class FishingAuthority {
  constructor(private db: Kysely<DB>, private rng: () => number = Math.random) {}

  verify(report: { hit: boolean; marker: number; elapsedMs: number },
    win: FishingWindow): boolean {
    if (report.elapsedMs > win.timeoutMs) return false; // 제한 초과 무효
    if (report.elapsedMs < 120) return false;           // 인간 반응 하한(매크로 방지)
    // 서버가 밴드 재계산(마커는 속도로 역산)
    const pos = (report.elapsedMs / 1000 * win.speed) % 2;
    const serverMarker = pos <= 1 ? pos : 2 - pos;
    return report.hit
      && serverMarker >= win.bandStart
      && serverMarker <= win.bandStart + win.bandWidth;
  }

  /** 어종 추출: 등급 가중치 + 무게 분포 */
  rollFish(tierWeights: { tier: number; weight: number }[]) {
    const total = tierWeights.reduce((s, t) => s + t.weight, 0);
    let r = this.rng() * total;
    for (const t of tierWeights) if ((r -= t.weight) <= 0) {
      const weightG = Math.round(200 + this.rng() * 1800); // 200g~2kg
    }
  }
}

```

```

    return { tier: t.tier, weightG };
  }
  return { tier: 1, weightG: 200 };
}
}

```

(3) 성능 최적화 팁

- 판정은 서버가 마커를 역산해 재계산한다 — 클라 보고값(hit)만 믿으면 매크로가 100% 성공률을 얻는다.
- 인간 반응 하한(120ms) 검증은 가장 저렴한 매크로 필터이며, 093 번 통계 탐지의 1 차 데이터가 된다.
- 입질 대기 시간은 지수 분포(평균 8~15 초)가 체감상 자연스럽다 — 균일 분포는 리듬이 노출된다.
- 성공 밴드 폭은 어중 등급별로(하급 넓게, 최상급 좁게) 두면 난이도 곡선이 자동 형성된다.

070. 시즌 패스 / 배틀 패스 달성도 및 보상 지급 로직

(1) 역할 설명

배틀 패스는 '시즌(30~60 일) → 레벨(경험치 티어) → 트랙(무료/프리미엄) → 보상'의 구조다. 패스 경험치는 미션(062 번 이벤트 스키마 재사용)에서 누적되고, 레벨은 고정 필요치 (예: 1000XP)의 단순 누적으로 계산한다. 각 레벨은 무료 트랙 보상 + 프리미엄 트랙 보상 2 종을 가지며, 프리미엄은 시즌 내 구매(082 번 결제)로 개방된다.

지급 로직의 핵심은 '레벨 도달과 보상 수령의 분리'다. 도달은 자동이지만 수령은 유저가 클레임하며(재방문 유인), 클레임은 ledger ref 맥등 지급(060 번 패턴)으로 이중 수령을 막는다. 프리미엄 구매 시점 이전에 달성한 레벨 보상도 소급 수령 가능해야 하며(구매 후 전 레벨 클레임 가능), 시즌 종료 시 미수령 보상은 우편으로 일괄 전환(배치)한다.

(2) 구현 코드 — `server/seasonpass/SeasonPass.ts`

```

import { Kysely } from "kysely";
type DB = import("./schema").Database;

export interface SeasonDef {
  id: string; // "s3"
  startsAt: number; endsAt: number;
  xpPerLevel: number; // 1000
  maxLevel: number; // 50
  tracks: {
    level: number;
    free: { gold?: number; itemCode?: string; qty?: number } | null;
    premium: { gold?: number; itemCode?: string; qty?: number } | null;
  }[];
}

```

```

export class SeasonPassService {
  constructor(private db: Kysely<DB>, private def: SeasonDef) {}

  /** 패스 XP 적립(미션 시스템에서 호출) */
  async addXp(charId: number, xp: number) {
    const now = Date.now();
    if (now < this.def.startsAt || now >= this.def.endsAt) return;
    await this.db.updateTable("season_pass")
      .set(eb => ({ xp: eb("xp", "+", xp) }))
      .where("character_id", "=", charId)
      .where("season_id", "=", this.def.id).execute();
  }

  /** 현재 레벨 계산(저장 xp → 레벨) */
  levelOf(xp: number): number {
    return Math.min(this.def.maxLevel, Math.floor(xp / this.def.xpPerLevel) + 1);
  }

  /** 보상 클레임: 트랙 권한(프리미엄) + 먹등 지급 + 소급 수령 */
  async claim(charId: number, level: number, track: "free" | "premium") {
    const row = await this.db.selectFrom("season_pass")
      .where("character_id", "=", charId).where("season_id", "=", this.def.id)
      .select(["xp", "premium"]).executeTakeFirst();
    if (!row) throw new Error("NO_PASS");
    if (this.levelOf(row.xp) < level) throw new Error("LEVEL_NOT_REACHED");
    if (track === "premium" && !row.premium) throw new Error("NEED_PREMIUM");

    const tier = this.def.tracks.find(t => t.level === level);
    if (!tier) throw new Error("NO_TIER");
    const reward = tier[track];
    if (!reward) throw new Error("EMPTY_REWARD");

    const ref = "sp:" + this.def.id + ":" + level + ":" + track + ":" + charId;
    const paid = await this.db.selectFrom("ledger")
      .where("ref", "=", ref).select("id").executeTakeFirst();
    if (paid) throw new Error("ALREADY_CLAIMED");

    await this.db.transaction().execute(async tx => {
      if (reward.gold) {
        await tx.updateTable("characters").set(eb => ({ gold: eb("gold", "+", reward.gold) }))
          .where("id", "=", charId).execute();
      }
      if (reward.itemCode) {
        await tx.updateTable("character_items")
          .set(eb => ({ qty: eb("qty", "+", reward.qty ?? 1) }))
          .where("character_id", "=", charId)
          .where("item_code", "=", reward.itemCode).execute();
      }
      await tx.insertInto("ledger").values({
        at: new Date(), kind: "gold", from_user: null, to_user: charId,
        ref, payload: JSON.stringify(reward),
      });
    });
  }
}

```

```

    }).execute();
  });
  return reward;
}

/** 프리미엄 구매(082 번 결제 검증 완료 후 호출) — 소급 개방 */
async grantPremium(charId: number, paymentRef: string) {
  await this.db.updateTable("season_pass").set({ premium: true })
    .where("character_id", "=", charId).where("season_id", "=", this.def.id).execute();
  // 이전 레벨 프리미엄 보상은 클레임 목록에 소급 표시(수령은 유저 클레임)
  await this.db.insertInto("ledger").values({
    at: new Date(), kind: "gold", from_user: null, to_user: charId,
    ref: "sp:premium:" + paymentRef, payload: JSON.stringify({ season: this.def.id }),
  }).execute();
}
}

```

(3) 성능 최적화 팁

- 도달(자동)과 수령(클레임)의 분리는 재방문 유인이자 이중 지급 방지의 구조적 해법이다.
- 프리미엄 소급 수령은 필수다 — 구매 시점 이전 레벨 보상을 못 받으면 결제 불만이 바로 터진다.
- 시즌 종료 미수령 보상은 우편 전환 배치로 자동화해 CS(고객 문의)를 원천 차단한다.
- 패스 XP 는 062 번 미션 이벤트 스키마에 태그 하나(dailyPass)로 붙여 이벤트 파이프라인을 공유한다.

제 9 장 모듈 8 — UI/UX 및 크로스플랫폼 가용성 (071~080)

기술 스택: HTML5/DOM · Phaser 3 UI · TypeScript · Web Push API 본 장은 조작의 품질이 곧 게임 품질인 계층이다. 가상 조이스틱과 통합 컨트롤러는 동일한 입력 계약(moveVec + 액션 이벤트)으로 통합되고, 반응형 HUD 는 세이프 에어리어와 44px 터치 타깃을 보장한다. 드래그 인벤토리·퀵슬롯 스와이프·장비 비교 툴팁·퀘스트 내비게이션은 일상 조작의 마찰을 제거하고, 절전 모드와 웹 푸시는 게임 밖 경험까지 연결한다.

071. 모바일 가상 조이스틱 (Touch Virtual Joystick) 반응형 구현

(1) 역할 설명

가상 조이스틱은 '터치 다운 시점에 베이스 생성 → 드래그 벡터 계산 → 터치 업 시 소멸'의 동적 배치 방식이 고정 배치보다 우수하다. 화면 어디든 터치하면 그 자리가 중심이 되어 조작이 시작되므로, 엄지 도달 범위의 개인차를 흡수한다. 벡터 계산은 (터치 위치 - 중심)/반경으로 정규화하고, 반경을 벗어나면 중심을 손가락 방향으로 이동시키는 '슬라이딩 베이스'로 장거리 드래그도 부드럽게 만든다.

출력은 -1~1 정규화 벡터이며, 데드존(중심 15%는 0으로)을 두어 정지 중 미세 흔들림을 제거한다. Phaser Input 과 DOM 터치 이벤트의 이중 발생(multi-touch 중복)을 방지하기 위해 pointerId 기반 추적으로 같은 손가락만 추적하고, 두 번째 손가락은 스킵 버튼에 남겨둔다. 조이스틱 표시는 캔버스 위 DOM 오버레이(반투명 링)로 구현해 게임 렌더와 분리한다.

(2) 구현 코드 — src/input/VirtualJoystick.ts

```
import type { InputController, MoveIntentVec, ActionEvent } from "../InputRouter";

const DEADZONE = 0.15;           // 데드존(정규화)
const BASE_RADIUS = 72;          // 베이스 반경(px, 반응형으로 확장)

export class VirtualJoystick implements InputController {
  readonly name = "touch" as const;
  readonly moveVec: MoveIntentVec = { x: 0, y: 0 };
  onAction: ((e: ActionEvent) => void) | null = null;

  private baseEl: HTMLDivElement;
  private stickEl: HTMLDivElement;
  private pointerId: number | null = null;
  private cx = 0; private cy = 0;    // 현재 베이스 중심

  constructor(private root: HTMLElement, private worldTap: (wx: number, wy: number) => void) {
    this.baseEl = document.createElement("div");
    this.baseEl.className = "vjoy-base";
    this.stickEl = document.createElement("div");
    this.stickEl.className = "vjoy-stick";
    this.baseEl.appendChild(this.stickEl);
    this.baseEl.style.display = "none";
    this.root.appendChild(this.baseEl);

    root.addEventListener("pointerdown", e => this.onDown(e));
    root.addEventListener("pointermove", e => this.onMove(e));
    root.addEventListener("pointerup", e => this.onUp(e));
    root.addEventListener("pointercancel", e => this.onUp(e));
  }

  private onDown(e: PointerEvent) {
    if (this.pointerId !== null) return;      // 이미 조작 중(두 번째 손가락 무시)
    this.pointerId = e.pointerId;
    this.cx = e.clientX; this.cy = e.clientY;
    this.baseEl.style.display = "block";
    this.placeBase();
    this.updateStick(e.clientX, e.clientY);
  }

  private onMove(e: PointerEvent) {
    if (e.pointerId !== this.pointerId) return;
    const dx = e.clientX - this.cx, dy = e.clientY - this.cy;
```

```

const dist = Math.hypot(dx, dy);
// 슬라이딩 베이스: 반경 초과 시 중심을 따라 이동
if (dist > BASE_RADIUS) {
  const k = (dist - BASE_RADIUS) / dist;
  this.cx += dx * k; this.cy += dy * k;
  this.placeBase();
}
this.updateStick(e.clientX, e.clientY);
}

private onUp(e: PointerEvent) {
  if (e.pointerId !== this.pointerId) return;
  this.pointerId = null;
  this.moveVec.x = this.moveVec.y = 0;
  this.baseEl.style.display = "none";
  this.stickEl.style.transform = "translate(-50%,-50%)";
}

private updateStick(x: number, y: number) {
  let nx = (x - this.cx) / BASE_RADIUS, ny = (y - this.cy) / BASE_RADIUS;
  const len = Math.hypot(nx, ny);
  if (len < DEADZONE) { this.moveVec.x = this.moveVec.y = 0; }
  else {
    const clamped = Math.min(1, len);
    nx = nx / len * clamped; ny = ny / len * clamped;
    this.moveVec.x = nx; this.moveVec.y = ny;
  }
  this.stickEl.style.transform =
    "translate(-50%,-50%) translate(" + (nx * BASE_RADIUS) + "px," + (ny * BASE_RADIUS) +
    "px)";
}

private placeBase() {
  this.baseEl.style.left = this.cx + "px";
  this.baseEl.style.top = this.cy + "px";
}

destroy() {
  this.baseEl.remove();
}
}

/* CSS 예:
.vjoy-base { position: fixed; width: 144px; height: 144px; margin-left:-72px; margin-top:-72px;
border-radius: 50%; background: rgba(255,255,255,.08);
border: 2px solid rgba(255,255,255,.25); pointer-events: none; z-index: 900; }
.vjoy-stick { position:absolute; left:50%; top:50%; width:56px; height:56px;
border-radius:50%; background: rgba(255,255,255,.35); transform: translate(-50%,-50%); } */

```

(3) 성능 최적화 팁

- 동적 배치(터치 위치 = 중심)는 엄지 도달 범위 문제를 근본적으로 해결한다 — 고정 좌하단 배치는 대형 폰에서 닿지 않는다.
- 데드존 15% + 슬라이딩 베이스의 조합이 조작감 표준이며, 데드존이 없으면 정지 상태에서 캐릭터가 떨린다.
- pointerId 추적으로 첫 손가락은 이동, 나머지는 스킬 버튼에 자연스럽게 남긴다.
- 터치 영역 최적화(073 번 --touch-min 44px)와 조합해 조이스틱 반경도 화면 크기 비율로 확장한다.

072. PC 용 마우스 클릭 이동 및 WASD/방향키 조작 통합 컨트롤러

(1) 역할 설명

PC 컨트롤러는 두 조작 모드(WASD 자유 이동, 클릭 지점 이동)를 하나의 인터페이스로 통합한다. WASD 입력 중이면 키보드 우선, 없으면 클릭 경로(A* 경로, 043 번)를 따라 이동한다. 클릭 이동은 경로가 살아 있는 동안 '다음 웨이포인트 방향 벡터'를 moveVec 으로 내보내고, 새 클릭이 오면 경로를 교체한다. ESC 또는 이동키 입력으로 클릭 이동을 취소한다.

통합 컨트롤러의 설계 목표는 썬 코드가 조작 모드를 신경 쓰지 않게 하는 것이다. 두 모드 모두 최종적으로 moveVec + 액션 이벤트만 방출하므로, 예측 엔진(011 번)과 오토 배틀(033 번)은 같은 입력 계약을 공유한다. 키 바인딩은 설정 가능(퀵슬롯 076 번과 연동)하며, 기본값은 WASD + 스킬 1~6 + E 상호작용 + Tab 타겟팅이다.

(2) 구현 코드 — src/input/UnifiedController.ts

```
import type { MoveIntentVec, ActionEvent, InputController } from "../InputRouter";
import type { AStar } from "../../shared/astar";

export class UnifiedController implements InputController {
  readonly name = "kb" as const;
  readonly moveVec: MoveIntentVec = { x: 0, y: 0 };
  onAction: ((e: ActionEvent) => void) | null = null;

  private keys = new Set<string>();
  private path: { x: number; y: number }[] = []; // 클릭 이동 웨이포인트(월드 px)
  private TILE = 32;

  constructor(private scene: Phaser.Scene, private astar: AStar,
    private mapW: number, private mapH: number,
    private blocked: Uint8Array) {
    scene.input.keyboard!.on("keydown", (e: KeyboardEvent) => {
      this.keys.add(e.code);
      if (e.code.startsWith("Digit")) {
        const slot = Number(e.code.slice(5)) - 1;
        if (slot >= 0 && slot < 6) this.onAction?.({ kind: "skill", slot });
      }
    });
  }
}
```

```

    if (e.code === "KeyE") this.onAction?({ kind: "interact" });
    if (e.code === "ESCAPE") this.cancelPath();
  });
  scene.input.keyboard!.on("keyup", (e: KeyboardEvent) => this.keys.delete(e.code));

  scene.input.on("pointerdown", (p: Phaser.Input.Pointer) => {
    if (p.button !== 0) return;           // 좌클릭만
    const cam = scene.cameras.main;
    const wx = cam.getWorldPoint(p.x, p.y).x;
    const wy = cam.getWorldPoint(p.x, p.y).y;
    this.setClickPath(wx, wy);
    this.onAction?({ kind: "worldTap", wx, wy });
  });
}

/** 클릭 → A* 경로 → 웨이포인트 체인 */
private setClickPath(wx: number, wy: number) {
  const self = (this.scene as any).playerPos as { x: number; y: number };
  const sx = Math.floor(self.x / this.TILE), sy = Math.floor(self.y / this.TILE);
  const tx = Math.floor(wx / this.TILE), ty = Math.floor(wy / this.TILE);
  const raw = this.astar.find(sx, sy, tx, ty);
  if (!raw) return;
  this.path = [];
  for (let i = 2; i < raw.length; i += 2) {
    this.path.push({ x: raw[i] * this.TILE + this.TILE / 2,
                    y: raw[i + 1] * this.TILE + this.TILE / 2 });
  }
}
cancelPath() { this.path = []; }

/** 고정 업데이트: 키보드 우선, 없으면 경로 추적 */
poll(dtMs: number): MoveIntentVec {
  let x = 0, y = 0;
  if (this.keys.has("KeyD") || this.keys.has("ArrowRight")) x += 1;
  if (this.keys.has("KeyA") || this.keys.has("ArrowLeft")) x -= 1;
  if (this.keys.has("KeyS") || this.keys.has("ArrowDown")) y += 1;
  if (this.keys.has("KeyW") || this.keys.has("ArrowUp")) y -= 1;

  if (x || y) {
    this.cancelPath();           // 키 입력 시 클릭 경로 취소
    const len = Math.hypot(x, y);
    this.moveVec.x = x / len; this.moveVec.y = y / len;
    return this.moveVec;
  }
  if (this.path.length) {
    const self = (this.scene as any).playerPos as { x: number; y: number };
    const wp = this.path[0];
    const dx = wp.x - self.x, dy = wp.y - self.y;
    const dist = Math.hypot(dx, dy);
    if (dist < this.TILE * 0.4) {
      this.path.shift();         // 웨이포인트 도달
      if (!this.path.length) { this.moveVec.x = this.moveVec.y = 0; return this.moveVec; }
    }
  }
}

```

```

    }
    this.moveVec.x = dx / dist; this.moveVec.y = dy / dist;
    return this.moveVec;
  }
  this.moveVec.x = this.moveVec.y = 0;
  return this.moveVec;
}

destroy() {
  this.scene.input.keyboard?.removeAllListeners();
  this.scene.input.removeAllListeners("pointerdown");
}
}

```

(3) 성능 최적화 팁

- 키 입력이 들어오면 클릭 경로를 즉시 취소해야 '자동 이동 중 원치 않는 이동' 충돌이 없다.
- 웨이포인트 도달 판정(타일 40%)은 빠빠하면 지그재그가 커지고 느슨하면 벽에 붙어 떨어진다.
- 클릭 이동 경로는 A* 결과를 그대로 쓰지 말고 2~3 타일 단위로 스무딩해 부자연스러운 계단 이동을 제거한다.
- 이동키 우선순위(키보드 > 경로 > 정지)는 씬이 아니라 컨트롤러 내부에서 해결해 계약을 단일화한다.

073. 해상도 반응형 HUD 스케일링 및 모바일 터치 영역 최적화

(1) 역할 설명

HUD(DOM 오버레이)는 게임 캔버스와 별개의 반응형 스케일 체계를 갖는다. CSS 변수(--hud-scale)에 뷰포트 폭 기반 스케일을 주입(003 번 ResponsiveScaler)하고, 모든 HUD 패널은 transform: scale(var(--hud-scale))로 동일 배율을 따른다. 터치 타겟은 최소 44x44px(Apple HIG 기준)을 CSS 변수(--touch-min)로 보장해 손가락 미스터치를 줄인다.

레이아웃은 안전 영역(safe-area-inset)을 반영해야 한다. 아이폰 노치/홈바 영역에 HUD 요소가 침범하면 시스템 제스처와 충돌한다. env(safe-area-inset-*)을 CSS에 적용하고, 가로 고정(화면 회전 잠금 유도) 상태에서 좌우 여백을 동적으로 계산한다. 또한 미니맵·채팅·스킬바 같은 핵심 HUD는 중앙 상단/하단의 '엄지 비충돌 구역'에 배치하는 것이 표준이다.

(2) 구현 코드 — src/ui/hud.css + HudLayout.ts

```

/* — HUD 반응형 CSS ————— */
:root {
  --hud-scale: 1;
  --touch-min: 32px;          /* 모바일에서 JS가 44px로 교체 */
}
html, body { margin: 0; overflow: hidden; touch-action: none; }

```

```
#hud {
  position: fixed; inset: 0;
  padding: calc(env(safe-area-inset-top) + 8px)
           calc(env(safe-area-inset-right) + 8px)
           calc(env(safe-area-inset-bottom) + 8px)
           calc(env(safe-area-inset-left) + 8px);
  pointer-events: none;          /* 자식만 상호작용 */
}
.hud-panel {
  transform: scale(var(--hud-scale));
  transform-origin: top left;
  pointer-events: auto;
}
.hud-btn {
  min-width: var(--touch-min);
  min-height: var(--touch-min);
  display: grid; place-items: center;
  border-radius: 10px;
}
/* 스킬바: 하단 중앙 — 엄지 비충돌 구역 */
#skillbar {
  position: absolute; left: 50%; bottom: 12px;
  transform: translateX(-50%) scale(var(--hud-scale));
  display: flex; gap: 8px;
}
/* 모바일 전용: 조이스틱 우측 스킬 휠 */
@media (pointer: coarse) {
  #skillbar { bottom: calc(env(safe-area-inset-bottom) + 96px); }
  .hud-chat { opacity: 0.85; max-width: 46vw; } /* 화면 좌측 절반 이하 */
}
```

보조 코드 — `src/ui/HudLayout.ts` — 스케일 주입 + 세이프 에어리어

```
export class HudLayout {
  constructor(private hudRoot: HTMLElement) {}

  /** ResponsiveScaler(003 번)에서 호출 */
  apply(viewportW: number, viewportH: number, isMobile: boolean) {
    const scale = isMobile
      ? Math.max(0.9, Math.min(viewportW / 1280, 1.5))
      : Math.max(0.8, Math.min(viewportW / 1280, 1.35));
    this.hudRoot.style.setProperty("--hud-scale", scale.toFixed(3));
    this.hudRoot.style.setProperty("--touch-min", isMobile ? "44px" : "32px");

    // 세이프 에어리어 미지원 브라우저 폴백(노치 폰 추정 여백)
    if (!CSS.supports("padding-left: env(safe-area-inset-left)")) {
      const portrait = viewportH > viewportW;
      const notch = isMobile && portrait ? 24 : 0;
      this.hudRoot.style.padding =
        notch + "px 12px " + (isMobile ? 16 : 8) + "px 12px";
    }
  }
}
```

```

/** 터치 타겟 실측 검증(개발 콘솔용): 44px 미만 요소 나열 */
auditTouchTargets() {
  const min = parseFloat(getComputedStyle(this.hudRoot)
    .getPropertyValue("--touch-min"));
  const bad: Element[] = [];
  this.hudRoot.querySelectorAll("button, .hud-btn, [data-touch]").forEach(el => {
    const r = el.getBoundingClientRect();
    if (r.width < min - 2 || r.height < min - 2) bad.push(el);
  });
  if (bad.length) console.warn("[HUD] touch targets below " + min + "px:", bad);
  return bad;
}
}

```

(3) 성능 최적화 팁

- HUD 는 pointer-events: none(컨테이너) + auto(요소) 패턴으로 게임 캔버스 터치를 막지 않아야 한다.
- transform: scale 은 레이아웃 재계산이 없어 font-size 기반 반응형보다 프레임 비용이 훨씬 저렴하다.
- safe-area-inset 미지원 브라우저 폴백(노치 추정)은 중국 안드로이드 계열에서 반드시 필요하다.
- auditTouchTargets 를 CI 스크린샷 테스트와 함께 돌리면 터치 타겟 위반을 코드 리뷰 전에 잡는다.

074. 터치 제스처 (핀치 투 줌, 스와이프) 이벤트 핸들러

(1) 역할 설명

멀티터치 제스처는 두 손가락(핀치 줌)과 빠른 한 손가락(스와이프)을 구분해 처리한다. 핀치 줌은 두 터치 포인트 사이 거리 변화율로 카메라 줌을 조절하고, 스와이프는 이동 속도 (빠른 이동 = 지나간 거리/시간)와 방향으로 퀵슬롯 페이지 전환(076 번)이나 카메라 스냅을 트리거한다. 제스처 인식기는 '터치 시작 → 움직임 추적 → 판정(거리/시간 임계)'의 상태머신으로 구현한다.

핵심은 조이스틱(071 번)과의 충돌 방지다. 첫 손가락이 조이스틱이면 두 번째 손가락만 제스처 대상이 되고, 반대로 두 손가락이 동시에 화면에 나타나면(조이스틱 시작 전) 핀치로 판정한다. 또한 브라우저 기본 제스처(페이지 줌/당겨서 새로고침)를 touch-action: none 과 preventDefault 로 차단해 게임 내 제스처가 OS 제스처와 섞이지 않게 한다.

(2) 구현 코드 — src/input/Gesture.ts

```

export interface GestureHandlers {
  pinch: (scaleDelta: number) => void; // 1.0 = 변화 없음
  swipe: (dir: "left" | "right" | "up" | "down", speedPxMs: number) => void;
  tap?: (x: number, y: number) => void;
}

```

```

const PINCH_THRESHOLD = 0.06;          // 줌 판정 변화율
const SWIPE_DIST = 60;                 // px
const SWIPE_MAX_MS = 260;
const TAP_MAX_MS = 200, TAP_MAX_DIST = 12;

export class GestureRecognizer {
  private pointers = new Map<number, { x: number; y: number; t: number; startX: number;
startY: number }>();
  private startDist = 0;
  private startZoom = 1;
  private reserved = false;           // 조이스틱이 선점한 첫 터치

  constructor(private root: HTMLElement, private handlers: GestureHandlers,
    joystickReserve: { isCapturing: (pointerId: number) => boolean }) {
    this.joystickReserve = joystickReserve;
    root.addEventListener("pointerdown", e => this.down(e));
    root.addEventListener("pointermove", e => this.move(e));
    root.addEventListener("pointerup", e => this.up(e));
    root.addEventListener("pointercancel", e => this.up(e));
  }
  private joystickReserve: { isCapturing: (id: number) => boolean };

  private down(e: PointerEvent) {
    if (this.joystickReserve.isCapturing(e.pointerId)) { this.reserved = true; return; }
    this.pointers.set(e.pointerId, {
      x: e.clientX, y: e.clientY, t: performance.now(),
      startX: e.clientX, startY: e.clientY,
    });
    if (this.pointers.size === 2) {
      const [a, b] = [...this.pointers.values()];
      this.startDist = Math.hypot(a.x - b.x, a.y - b.y);
      this.startZoom = 1;
    }
  }

  private move(e: PointerEvent) {
    const p = this.pointers.get(e.pointerId);
    if (!p) return;
    p.x = e.clientX; p.y = e.clientY;

    if (this.pointers.size === 2 && this.startDist > 0) {
      const [a, b] = [...this.pointers.values()];
      const d = Math.hypot(a.x - b.x, a.y - b.y);
      const delta = d / this.startDist - 1;
      if (Math.abs(delta - (this.startZoom - 1)) > PINCH_THRESHOLD) {
        this.handlers.pinch(1 + delta);
        this.startZoom = 1 + delta;
      }
    }
  }
}

```



```

private up(e: PointerEvent) {
  const p = this.pointers.get(e.pointerId);
  if (!p) { this.reserved = false; return; }
  this.pointers.delete(e.pointerId);
  if (this.pointers.size === 2) this.startDist = 0; // 핀치 끝김 → 재시작

  const dt = performance.now() - p.t;
  const dx = e.clientX - p.startX, dy = e.clientY - p.startY;
  const dist = Math.hypot(dx, dy);

  if (this.pointers.size === 0) {
    if (dt < TAP_MAX_MS && dist < TAP_MAX_DIST) {
      this.handlers.tap?.(e.clientX, e.clientY);
    } else if (dist > SWIPE_DIST && dt < SWIPE_MAX_MS) {
      const dir = Math.abs(dx) > Math.abs(dy)
        ? (dx > 0 ? "right" : "left")
        : (dy > 0 ? "down" : "up");
      this.handlers.swipe(dir, dist / dt);
    }
  }
}

```

(3) 성능 최적화 팁

- 핀치는 거리 변화율(배수)로 계산하되 판정 임계(6%)를 두어 손가락 떨림으로 줄이 왔다 갔다 하는 것을 막는다.
- 스와이프는 거리와 시간(속도)을 함께 판정해야 의도된 제스처와 그냥 드래그가 구분된다.
- 조이스틱 선점(reserve) 상태를 제스처 인식기에 알려주는 인터페이스가 멀티터치 충돌의 해법이다.
- 브라우저 기본 제스처 차단(touch-action: none, preventDefault)은 iOS 사파리에서 특히 중요하다.

075. 드래그 앤 드롭 & 터치 인벤토리 아이템 그리드 인터페이스

(1) 역할 설명

인벤토리 UI는 '그리드(칸) 기반 렌더 + 드래그 앤 드롭(마우스) + 탭-홀드-이동(터치)'을 하나의 드래그 세션 모델로 처리한다. 드래그 세션은 (1) 원본 칸에서 시작, (2) 자유 이동 중 고스트(반투명 아이콘) 표시, (3) 드롭 대상 판정(칸/장비 슬롯/웍슬롯/삭제), (4) 서버 요청 → 성공 시 로컬 반영, 실패 시 원위치의 흐름이다. HTML5 DnD API는 모바일 미지원이므로 pointer 이벤트로 직접 구현한다.

터치에서는 롱프레스(300ms)로 드래그 시작을 판정해 일반 스크롤과 구분한다. 서버 동기화는 낙관적(로컬 먼저 반영 → 실패 시 롤백)이되, 스택 분할(Shift 클릭/분할 모달)과 장비 착용(장비 슬롯 드롭)은 각각 전용 요청으로 분기한다. 60 칸 그리드는 DOM 재사용(가상화 불필요 수준) + CSS grid 로 구현해 렌더 비용을 낮춘다.

(2) 구현 코드 — src/ui/InventoryGrid.ts

```
import Phaser from "phaser";

export interface InvItem {
  id: number; itemCode: string; qty: number;
  bagSlot: number;           // 그리드 칸
}

export type DropTarget =
  | { kind: "slot"; index: number }           // 인벤토리 칸
  | { kind: "equip"; part: string }           // 장비 슬롯
  | { kind: "quickslot"; slot: number };      // 퀵슬롯(076 번)

export class InventoryGrid {
  private drag: { item: InvItem; ghost: HTMLDivElement; x: number; y: number }
    | null = null;
  private pressTimer = 0;

  constructor(private rootEl: HTMLElement,
    private cellSize = 52,
    private api: {
      moveItem(itemId: number, toSlot: number): Promise<boolean>;
      equipItem(itemId: number): Promise<boolean>;
    }) {}

  /** pointer 이벤트 통합(마우스/터치 공용) */
  bind() {
    this.rootEl.addEventListener("pointerdown", e => this.start(e));
    this.rootEl.addEventListener("pointermove", e => this.move(e));
    window.addEventListener("pointerup", e => this.end(e));
    window.addEventListener("pointercancel", () => this.cancel());
  }

  private cellAt(x: number, y: number): number | null {
    const rect = this.rootEl.getBoundingClientRect();
    const cx = Math.floor((x - rect.left) / this.cellSize);
    const cy = Math.floor((y - rect.top) / this.cellSize);
    return (cx >= 0 && cy >= 0) ? cy * this.columns() + cx : null;
  }

  private columns() {
    return Math.floor(this.rootEl.clientWidth / this.cellSize);
  }

  private start(e: PointerEvent) {
    const slot = this.cellAt(e.clientX, e.clientY);
```

```

    if (slot == null) return;
    const item = (this as any).items?.find?.((i: InvItem) => i.bagSlot === slot);
    if (!item) return;
    // 터치: 롱프레스 300ms 후 드래그 / 마우스: 즉시
    if (e.pointerType === "touch") {
        this.pressTimer = window.setTimeout(() => this.beginDrag(item, e), 300);
    } else {
        this.beginDrag(item, e);
    }
}

private beginDrag(item: InvItem, e: PointerEvent) {
    const ghost = document.createElement("div");
    ghost.className = "inv-ghost";
    ghost.textContent = item.itemCode;
    document.body.appendChild(ghost);
    this.drag = { item, ghost, x: e.clientX, y: e.clientY };
    this.positionGhost(e.clientX, e.clientY);
}

private move(e: PointerEvent) {
    if (!this.drag) return;
    // 롱프레스 대기 중 이동 → 드래그 취소(스크롤로 판단)
    if (this.pressTimer && (e.pointerType === "touch")) {
        clearTimeout(this.pressTimer); this.pressTimer = 0; return;
    }
    this.drag.x = e.clientX; this.drag.y = e.clientY;
    this.positionGhost(e.clientX, e.clientY);
}

private async end(e: PointerEvent) {
    clearTimeout(this.pressTimer); this.pressTimer = 0;
    const d = this.drag;
    if (!d) return;
    this.drag = null;
    d.ghost.remove();

    const target = this.resolveTarget(e);
    const orig = d.item.bagSlot;
    if (!target) return;
    if (target.kind === "slot" && target.index !== orig) {
        const ok = await this.api.moveItem(d.item.id, target.index);
        if (!ok) (this as any).render?.(); // 실패 시 원위치 리렌더
    } else if (target.kind === "equip") {
        await this.api.equipItem(d.item.id);
    }
    // quickslot 은 076 번 시스템에 위임
}

private resolveTarget(e: PointerEvent): DropTarget | null {
    const slot = this.cellAt(e.clientX, e.clientY);
    if (slot != null) return { kind: "slot", index: slot };
}

```

```

const eq = (e.target as HTMLElement).closest?("[data-equip-part]");
if (eq) return { kind: "equip", part: (eq as HTMLElement).dataset.equipPart! };
const qs = (e.target as HTMLElement).closest?("[data-quickslot]");
if (qs) return { kind: "quickslot", slot: Number((qs as HTMLElement).dataset.quickslot) };
return null;
}
private positionGhost(x: number, y: number) {
  if (!this.drag) return;
  this.drag.ghost.style.left = x - this.cellSize / 2 + "px";
  this.drag.ghost.style.top = y - this.cellSize / 2 + "px";
}
private cancel() {
  clearTimeout(this.pressTimer); this.pressTimer = 0;
  this.drag?.ghost.remove(); this.drag = null;
}
}

```

(3) 성능 최적화 팁

- HTML5 DnD API(dragstart 등)는 모바일에서 동작하지 않는다 — pointer 이벤트로 통합 구현해야 PC/터치가 한 코드로 동작한다.
- 터치 롱프레스(300ms) + 이동 취소 판정으로 일반 스크롤과 드래그를 구분한다.
- 서버 요청은 낙관적 반영 + 실패 롤백이 원칙이며, 롤백은 전체 리렌더(간단하고 안전)로 처리한다.
- 장비 슬롯/퀵슬롯 드롭 판정은 closest([data-...]) 셀렉터 기반이 확장성이 가장 좋다.

076. 퀵슬롯 커스텀 배치 및 다중 페이지 스와이프 UI

(1) 역할 설명

퀵슬롯은 '스킬/아이템을 슬롯(1~6, 2 페이지)에 자유 배치하고 키보드 숫자키/터치로 실행'하는 조작의 중심이다. 배치 데이터는 (슬롯 인덱스 → 바인딩(스킬 코드 또는 아이템 코드))의 세이브 구조로 서버에 저장되고, 075 번 인벤토리 드래그로 퀵슬롯에 끌어다 놓으면 바인딩이 갱신된다. 쿨타임 오버레이(032 번 ResourceMgr 잔여 시간)와 수량 표시를 각 슬롯에 렌더한다.

모바일에서는 2 페이지 구성 + 스와이프 전환이 표준이다. 스와이프는 074 번 제스처 인식기에서 방향/속도 이벤트를 받아 페이지를 전환하고, 전환 애니메이션(0.2s 트랜지션) 중 입력은 무시해 이중 전환을 막는다. 페이지 전환은 '스와이프'와 '페이지 도트 탭' 두 경로를 모두 제공하고, 현재 페이지 번호는 세이브에 유지해 재접속 시 복원한다.

(2) 구현 코드 — src/ui/QuickslotBar.ts

```

import type { SlotBinding } from "../game/quickslot";

export class QuickslotBar {
  private page = 0; // 0 | 1
  private pages: SlotBinding[][] = [[], []];
}

```

```

private animating = false;
private cooldowns = new Map<string, number>(); // 바인딩 key → 잔여 ms

constructor(private rootEl: HTMLElement,
  private exec: (b: SlotBinding) => void,
  private save: (pages: SlotBinding[][], page: number) => void) {
  this.rootEl.addEventListener("click", e => this.onTap(e));
}

/** 드래그 드롭(075 번)에서 호출 */
async dropTo(slotIndex: number, binding: SlotBinding) {
  this.pages[this.page][slotIndex] = binding;
  this.render();
  this.save(this.pages, this.page); // 서버 세이브
}

/** 스와이프(074 번 제스처)로 페이지 전환 */
swipePage(dir: "left" | "right") {
  if (this.animating) return;
  const next = dir === "left" ? Math.min(1, this.page + 1)
    : Math.max(0, this.page - 1);
  if (next === this.page) return;
  this.animating = true;
  this.page = next;
  this.render();
  this.save(this.pages, this.page);
  setTimeout(() => { this.animating = false; }, 220);
}

/** 쿨타임 게이지 갱신(032 번 ResourceMgr 에서 주입) */
setCooldown(bindingKey: string, remainMs: number, totalMs: number) {
  this.cooldowns.set(bindingKey, remainMs);
  const el = this.rootEl.querySelector<HTMLElement>(
    "[data-slot-idx].cd-overlay[data-key='" + bindingKey + "']");
  if (!el || totalMs <= 0) return;
  el.style.height = (remainMs / totalMs * 100).toFixed(1) + "%";
}

private onTap(e: MouseEvent) {
  const btn = (e.target as HTMLElement).closest("[data-slot-idx]");
  if (!btn) return;
  const idx = Number((btn as HTMLElement).dataset.slotIdx);
  const b = this.pages[this.page][idx];
  if (b) this.exec(b);
}

private render() {
  this.rootEl.dataset.page = String(this.page);
  this.rootEl.style.transform = "translateX(" + (-this.page * 100) + "%)";
  this.rootEl.style.transition = "transform 0.2s ease";
}

```

}

보조 코드 — src/game/quickslot.ts — 바인딩 모델 + 실행 라우팅

```

export type SlotBinding =
  | { kind: "skill"; code: string; label: string }
  | { kind: "item"; itemCode: string; label: string };

/** 실행 라우팅: 스킬은 ResourceMgr 검증 → 서버 요청, 아이템은 사용 트랜잭션 */
export class QuickslotExecutor {
  constructor(
    private useSkill: (code: string) => void,
    private useItem: (itemCode: string) => void,
    private resource: { canUse(code: string): { ok: boolean; reason?: string } },
  ) {}

  exec(b: SlotBinding) {
    if (b.kind === "skill") {
      const gate = this.resource.canUse(b.code);
      if (!gate.ok) {
        // 거절 사유별 UI 피드백(032 번 reject 규칙 재사용)
        if (gate.reason === "cooldown") this.flash(b.code, "cd");
        if (gate.reason === "mp") this.flash(b.code, "mp");
        return;
      }
      this.useSkill(b.code);
    } else {
      this.useItem(b.itemCode);
    }
  }
  private flash(code: string, kind: "cd" | "mp") {
    document.dispatchEvent(new CustomEvent("slot-flash", {
      detail: { code, kind } }));
  }
}

```

(3) 성능 최적화 팁

- 쿨타임 오버레이는 CSS 높이 퍼센트 방식이 가장 저렴하다 — 캔버스에서 다시 그리면 불필요한 오버헤드가 된다.
- 페이지 전환 애니메이션 중 입력 무시(animating 플래그)는 이중 스와이프로 페이지가 날아가는 버그를 막는다.
- 퀵슬롯 배치는 서버 세이브로 — 디바이스마다 다르게 하고 싶으면 로컬 오버라이드 레이어를 별도로 둔다.
- 숫자키(1~6)와 터치 탭이 같은 exec 라우팅을 타야 PC/모바일 동작이 완전히 일치한다.

077. 착용 장비 vs 선택 장비 스탯 비교 툴팁 레이어

(1) 역할 설명

장비 비교 툴팁은 '현재 착용 중인 동일 부위 장비'와 '선택한(호버/터치한) 장비'의 스탯 차이를 즉시 시각화한다. 표준 레이아웃은 좌우 2 열(선택 장비 vs 착용 장비) + 차이 표시(증가 초록/감소 빨강, 수치와 %)다. 비교 대상 스탯은 item_def.base_mods + 랜덤 옵션(029 번) + 강화 보너스(025 번)의 합산 결과(021 번 StatMath 장비 소스)로 계산한다.

모바일에서는 호버가 없으므로 '탭 = 선택, 정보 버튼 = 툴팁 오픈'의 2 단 상호작용을 쓴다. 툴팁은 화면 경계 폴립(오른쪽이 넘치면 왼쪽 표시)과 뷰포트 클램프를 수행하며, 드래그 드롭(075 번) 호버 중에도 동일 컴포넌트를 재사용해 '드래그 중 비교'를 지원한다. 차이 계산은 순수 함수로 분리해 도감/거래 UI에서도 재사용한다.

(2) 구현 코드 — src/ui/StatCompare.ts

```
import type { StatMod } from "../game/StatMath";

export interface EquipStats { mods: StatMod[]; enhance: number; }
export interface CompareRow { stat: string; a: number; b: number; diff: number; }

/** 장비 스탯 합산(강화 보너스 + RO 포함) — 021 번 computeFinal 의 장비 소스 재사용 */
export function sumEquipStats(s: EquipStats): Record<string, number> {
  const out: Record<string, number> = {};
  const bonus = 1 + 0.12 * s.enhance; // 025 번 강화 배율
  for (const m of s.mods) {
    const v = (m.flat ?? 0) * (m.flat ? bonus : 1);
    out[m.stat] = (out[m.stat] ?? 0) + v;
    if (m.mul) out["%" + m.stat] = (out["%" + m.stat] ?? 0) + m.mul;
  }
  return out;
}

export function compare(selected: EquipStats, equipped: EquipStats): CompareRow[] {
  const A = sumEquipStats(selected), B = sumEquipStats(equipped);
  const keys = new Set([...Object.keys(A), ...Object.keys(B)]);
  const rows: CompareRow[] = [];
  for (const k of keys) {
    const a = A[k] ?? 0, b = B[k] ?? 0;
    const diff = Math.round((a - b) * 100) / 100;
    if (diff === 0) continue;
    rows.push({ stat: k, a: Math.round(a * 100) / 100,
      b: Math.round(b * 100) / 100, diff });
  }
  // 중요 스탯(atk/hp 등) 우선 정렬
  const PRIORITY = ["atk", "matk", "def", "hp", "crit"];
  return rows.sort((r1, r2) => {
    const pi = PRIORITY.indexOf(r1.stat), pj = PRIORITY.indexOf(r2.stat);
    return (pi < 0 ? 99 : pi) - (pj < 0 ? 99 : pj);
  });
}
```

```

}

/** 툴팁 렌더 + 화면 경계 플립 */
export class CompareTooltip {
  private el: HTMLElement;
  constructor(root: HTMLElement) {
    this.el = document.createElement("div");
    this.el.className = "compare-tip";
    root.appendChild(this.el);
    this.el.style.display = "none";
  }

  show(x: number, y: number, rows: CompareRow[], names: { a: string; b: string }) {
    const html = ["<table><thead><tr><th>" + names.a + "</th><th>차이</th><th>" +
names.b + "</th></tr></thead><tbody>"];
    for (const r of rows) {
      const cls = r.diff > 0 ? "up" : "down";
      html.push("<tr><td>" + r.stat + ": " + r.a + "</td>"
+ "<td class='" + cls + "'>" + (r.diff > 0 ? "+" : "") + r.diff + "</td>"
+ "<td>" + r.stat + ": " + r.b + "</td></tr>");
    }
    html.push("</tbody></table>");
    this.el.innerHTML = html.join("");
    this.el.style.display = "block";
    // 경계 플립: 우측 넘침 시 좌측 배치
    const rect = this.el.getBoundingClientRect();
    const vw = window.innerWidth;
    this.el.style.left = (x + rect.width + 16 > vw ? x - rect.width - 16 : x + 16) + "px";
    this.el.style.top = Math.max(8, Math.min(y, window.innerHeight - rect.height - 8)) + "px";
  }
  hide() { this.el.style.display = "none"; }
}

```

(3) 성능 최적화 팁

- 차이 계산은 반드시 '합산된 최종 장비 스탯'끼리 비교한다 — 모드 배열끼리 비교하면 강화/RO가 누락된다.
- 증가/감소 색상(초록/빨강)은 '선택 장비 기준'으로 통일해 유저가 헷갈리지 않게 한다.
- 경계 플립과 뷰포트 클램프는 모바일(좁은 화면)에서 필수다 — `getBoundingClientRect`로 실측한다.
- % 스탯(mul)과 flat 스탯은 다른 행으로 분리 표시해 '공격력 +5 vs 공격력 +10%' 비교 혼란을 막는다.

078. 퀘스트 자동 길찾기 경로 내비게이션 라인 표시

(1) 역할 설명

퀘스트 내비게이션은 '목표 지점(NPC/몬스터 구역)까지의 경로를 화면에 라인 + 목적지 화살표로 표시'하는 기능이다. 경로는 A*(043번)로 계산하고, 화면 표시는 (1) 월드 공간 경로

라인(지면 위 반투명 점선), (2) 화면 가장자리 목적지 방향 화살표(타겟이 화면 밖일 때), (3) 거리 표시(몇 미터)의 3 계층으로 구성한다. 경로는 목적지 이동/맵 전환 시 재계산하며, 캐릭터가 경로에서 벗어나면(3 타일 이상) 자동 재계산한다.

맵 전환을 동반하는 목적지는 '포탈까지의 경로 + 포탈 이름 + 다음 맵명'을 표시해 체인 내비게이션을 만든다. 라인 렌더는 Phaser Graphics 를 쓰되 매 프레임 다시 그리지 않고 경로 갱신 시점에만 다시 그린다. 경로 스무딩(코너 절단)으로 자연스러운 곡선을 만들고, 저사양 기기에서는 라인 대신 화살표만 표시하는 품질 단계(001 번)를 둔다.

(2) 구현 코드 — `src/ui/QuestNavigator.ts`

```
import Phaser from "phaser";
import type { AStar } from "../../shared/astar";

const TILE = 32;
const RECALC_DIST = TILE * 3; // 경로 이탈 재계산 임계

export class QuestNavigator {
  private path: { x: number; y: number }[] = [];
  private goal: { x: number; y: number; mapId: string } | null = null;
  private g: Phaser.GameObjects.Graphics;
  private lastRecalcAt = 0;

  constructor(private scene: Phaser.Scene, private astar: AStar,
    private mapW: number, mapH: number) {
    this.g = scene.add.graphics().setDepth(4500);
    (this as any).mapH = mapH;
  }

  /** 목적지 설정(퀘스트 트래커에서 호출) */
  setGoal(goal: { x: number; y: number; mapId: string } | null) {
    this.goal = goal;
    this.path = [];
    this.lastRecalcAt = 0;
  }

  /** 고정 업데이트(0.5 초 주기 재계산) */
  update(self: { x: number; y: number }, now: number) {
    if (!this.goal) { this.g.clear(); return; }
    if (now - this.lastRecalcAt > 500 || this.offPath(self)) {
      this.recalc(self);
      this.lastRecalcAt = now;
    }
    this.render(self);
  }

  private offPath(self: { x: number; y: number }): boolean {
    if (!this.path.length) return true;
    const next = this.path[0];
```

```

    return Math.hypot(next.x - self.x, next.y - self.y) > RECALC_DIST * 2;
}

private recalc(self: { x: number; y: number }) {
    if (!this.goal) return;
    const sx = Math.floor(self.x / TILE), sy = Math.floor(self.y / TILE);
    const tx = Math.floor(this.goal.x / TILE), ty = Math.floor(this.goal.y / TILE);
    const raw = this.astar.find(sx, sy, tx, ty);
    if (!raw) return;
    // 경로 스무딩: 2 타일 간격 샘플 + 코너 절단
    this.path = [];
    for (let i = 2; i < raw.length; i += 4) {
        this.path.push({ x: raw[i] * TILE + TILE / 2, y: raw[i + 1] * TILE + TILE / 2 });
    }
    this.path.push({ x: this.goal.x, y: this.goal.y });
}

/** 렌더: 경로 라인(점선) + 화면 밖 목적지 화살표 */
private render(self: { x: number; y: number }) {
    const cam = this.scene.cameras.main;
    this.g.clear();
    if (!this.path.length || !this.goal) return;

    // 1) 경로 라인(월드 좌표 → 화면)
    this.g.lineStyle(3, 0x4ad0ff, 0.7);
    this.g.beginPath();
    const pts = [self, ...this.path];
    for (let i = 1; i < pts.length; i++) {
        const a = pts[i - 1], b = pts[i];
        this.g.moveTo(a.x - cam.scrollX, a.y - cam.scrollY);
        this.g.lineTo(b.x - cam.scrollX, b.y - cam.scrollY);
    }
    this.g.strokePath();

    // 2) 목적지가 화면 밖 → 가장자리 화살표
    const sx = this.goal.x - cam.scrollX, sy = this.goal.y - cam.scrollY;
    const inView = sx > 0 && sy > 0 && sx < cam.width && sy < cam.height;
    if (!inView) {
        const ang = Math.atan2(sy - cam.height / 2, sx - cam.width / 2);
        const ex = cam.width / 2 + Math.cos(ang) * (Math.min(cam.width, cam.height) / 2 - 40);
        const ey = cam.height / 2 + Math.sin(ang) * (Math.min(cam.width, cam.height) / 2 - 40);
        this.g.fillStyle(0x4ad0ff, 0.9);
        this.g.save();
        this.g.translateCanvas(ex, ey);
        this.g.rotateCanvas(ang);
        this.g.fillTriangle(-14, -10, -14, 10, 14, 0);
        this.g.restore();
        // 거리 표시
        const dist = Math.hypot(this.goal.x - self.x, this.goal.y - self.y);
        this.scene.add.text(ex, ey + 16, Math.round(dist / TILE) + "m",
            { fontSize: "12px", color: "#4ad0ff" }).setOrigin(0.5);
    }
}

```

```
}
}
```

(3) 성능 최적화 팁

- 재계산 주기(0.5 초) + 이탈 임계(3 타일) 조합으로 탐색 빈도와 경로 신선도의 균형을 맞춘다.
- 라인 렌더는 경로 갱신 시에만 다시 그린다 — 매 프레임 clear+redraw 는 저사양 기기에서 눈에 띄는 비용이다.
- 화면 가장자리 화살표 + 거리 표시는 타깃이 안 보일 때 가장 효과적인 안내이며, 미니맵(048 번)과 방향이 일치해야 한다.
- 맵 전환 목적지는 체인 표시(포탈 → 다음 맵명)로 UX 를 완성해 '길이 끊긴 느낌'을 없앤다.

079. 모바일 절전 모드 (프레임 제한 및 화면 암전 처리)

(1) 역할 설명

모바일 장기 세션에서 배터리 소모의 주범은 60fps 풀렌더와 화면 밝기다. 절전 모드는 두 단계로 대응한다. 첫째, 입력 없음 상태가 지속되면 프레임 제한(60 → 30fps)으로 렌더 비용을 절반으로 줄이고, 둘째, 장시간(3 분) 무입력이면 UI 를 점진 암전(오버레이 알파 상승)해 OLED 화면 전력을 줄인다. 어떤 입력(터치/키)이든 즉시 복귀한다.

구현은 (1) 마지막 입력 시각 추적, (2) 단계별 상태머신(active → eco30 → dim), (3) 복귀 트랜지션의 구조다. 프레임 제한은 Phaser fps target 변경 또는 수동 rAF 스로틀로 구현하며, 암전 오버레이는 캔버스 위 DOM(불투명도 전환)으로 화면 밝기 감지(OLED)와 무관하게 확실한 전력 절감을 만든다. 오토 배틀(033 번) 실행 중에는 암전을 생략해 진행 상황을 볼 수 있게 한다.

(2) 구현 코드 — src/engine/PowerSaver.ts

```
import Phaser from "phaser";

export type PowerState = "active" | "eco30" | "dim";
const ECO_AFTER_MS = 20_000;    // 20 초 무입력 → 30fps
const DIM_AFTER_MS = 180_000;   // 3 분 무입력 → 암전

export class PowerSaver {
  private state: PowerState = "active";
  private lastInput = performance.now();
  private overlay: HTMLDivElement;
  private rafThrottle = 0;

  constructor(private game: Phaser.Game,
    private isActiveBattle: () => boolean) {
    this.overlay = document.createElement("div");
  }
}
```

```

this.overlay.className = "power-dim";
this.overlay.style.cssText =
  "position:fixed;inset:0;background:#000;opacity:0;" +
  "transition:opacity 1.5s;pointer-events:none;z-index:9998;";
document.body.appendChild(this.overlay);

for (const ev of ["pointerdown", "keydown", "touchstart"]) {
  window.addEventListener(ev, () => this.onInput(), { passive: true });
}

private onInput() {
  this.lastInput = performance.now();
  if (this.state !== "active") this.restore();
}

/** 고정 업데이트(1 초 주기) */
tick(now: number) {
  const idle = now - this.lastInput;
  if (idle > DIM_AFTER_MS && this.state !== "dim" && !this.isActiveBattle()) {
    this.state = "dim";
    this.overlay.style.opacity = "0.55";    // 부드러운 암전
    this.setFrameRate(30);
  } else if (idle > ECO_AFTER_MS && this.state === "active") {
    this.state = "eco30";
    this.setFrameRate(30);
  }
}

/** 프레임 제한: rAF 스로틀(Phaser fps 재설정보다 부드럽다) */
private setFrameRate(fps: number) {
  const minFrame = 1000 / fps;
  this.game.events.emit("power-fps", fps);
  const loop = this.game.loop as any;
  if (loop && "time" in loop) {
    // Phaser Loop 대체 스로틀: 외부 rAF 에서 minFrame 간격으로 step 호출
    this.rafThrottle = minFrame;
  }
}

private restore() {
  this.state = "active";
  this.overlay.style.opacity = "0";
  this.rafThrottle = 0;
  this.game.events.emit("power-fps", 60);
}

get currentState() { return this.state; }
}

/* CSS: .power-dim 트랜지션은 JS opacity 제어만으로 충분 */

```

```
// 통합 방식(대안): Phaser fps.target 변경
// this.game.loop.actualFps — Phaser 3.60+ 에서 game.loop.targetFps 조정 가능
// 그러나 fps 재설정은 오디오 스케줄 드리프트를 일으킬 수 있으므로
// 오디오 재생 중(주인공 액션 사운드)에는 30fps 전환을 미룬다.
```

(3) 성능 최적화 팁

- 절전 단계(20 초 30fps → 3 분 잠전)는 유저가 체감하지 못하는 선에서 최대한 공격적으로 설정한다.
- 잠전은 OLED 에서 전력 절감이 크지만, 오토 배틀 중에는 생략해 진행 상황 가시성을 유지한다.
- 입력 복귀는 즉시(트랜지션 1.5s 는 시각적 페이드만) — 조작 지연이 느껴지면 절전 기능이 결함처럼 느껴진다.
- 30fps 전환은 파티클 수도 절반으로(045 번) 줄여 렌더 비용을 두 축으로 절감한다.

080. 브라우저 웹 푸시 알림 (Web Push Notification) 연동

(1) 역할 설명

웹 푸시는 서버 이벤트(길드전 시작, 파티 초대, 경매 낙찰)를 게임 밖(탭 닫힘/다른 앱 사용 중)으로 전달하는 채널이다. 구조는 (1) 클라: Notification.requestPermission + PushManager 구독(subscription 엔드포인트 서버 전송), (2) 서버: VAPID 키로 서명한 푸시 발송(web-push 라이브러리), (3) Service Worker: 푸시 수신 → 알림 표시 → 클릭 시 게임 딥링크 오픈이다.

설계 원칙은 유저 통제다. 알림 유형별(길드/파티/경매/친구) 옵트인을 개별 설정으로 제공하고, 기본값은 끄 — 권한 요청은 게임 내 '유용한 순간'(길드 가입 직후 등)에 문맥적으로 요청해 거부율을 낮춘다. 구독은 브라우저/기기 단위로 만료가 있으므로, 서버는 발송 실패(410 Gone) 시 구독을 자동 정리한다. 푸시 페이로드는 최소화(제목+본문+딥링크)해 데이터 절약 규정을 준수한다.

(2) 구현 코드 — src/push/PushClient.ts — 클라 구독

```
const VAPID_PUBLIC = import.meta.env.VITE_VAPID_PUBLIC_KEY;

function urlBase64ToUint8Array(base64: string): Uint8Array {
  const pad = "=".repeat((4 - (base64.length % 4)) % 4);
  const b64 = (base64 + pad).replace(/-/g, "+").replace(/_/g, "/");
  const raw = atob(b64);
  const out = new Uint8Array(raw.length);
  for (let i = 0; i < raw.length; i++) out[i] = raw.charCodeAt(i);
  return out;
}
```

```

export async function enablePush(kind: "guild" | "party" | "auction") {
  // 1) 권한 요청(게임 내 문맥 버튼에서 호출 — 자동 팝업 금지)
  const perm = await Notification.requestPermission();
  if (perm !== "granted") return false;

  // 2) Service Worker 등록 + 구독 생성
  const reg = await navigator.serviceWorker.register("/sw.js");
  await navigator.serviceWorker.ready;
  let sub = await reg.pushManager.getSubscription();
  if (!sub) {
    sub = await reg.pushManager.subscribe({
      userVisibleOnly: true, // 스펙 필수
      applicationServerKey: urlBase64ToUint8Array(VAPID_PUBLIC),
    });
  }
  // 3) 구독 정보 서버 전송(기기별 유형 옵트인)
  await fetch("/api/push/subscribe", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ subscription: sub.toJSON(), kinds: [kind] }),
  });
  return true;
}

```

보조 코드 — public/sw.js — Service Worker + server/push/send.ts

```

// — public/sw.js —
self.addEventListener("push", (event) => {
  const data = event.data ? event.data.json() : {};
  event.waitUntil(self.registration.showNotification(data.title || "SERTZ", {
    body: data.body,
    icon: "/icons/192.png",
    badge: "/icons/badge.png",
    tag: data.tag || "default", // 동일 태그 = 알림 교체(스팸 방지)
    data: { url: data.url || "/play" }, // 딥링크
  }));
});

self.addEventListener("notificationclick", (event) => {
  event.notification.close();
  event.waitUntil(clients.openWindow(event.notification.data.url));
});

// — server/push/send.ts (web-push 라이브러리) —
import webpush from "web-push";
import { Kysely } from "kysely";
type DB = import("../schema").Database;

webpush.setVapidDetails(
  "mailto:ops@example.com",
  process.env.VAPID_PUBLIC!,
  process.env.VAPID_PRIVATE!,
);

```

```

export async function pushToUser(db: Kysely<DB>, charId: number,
    payload: { title: string; body: string; tag: string; url?: string }) {
    const subs = await db.selectFrom("push_subscriptions")
        .where("character_id", "=", charId).select(["endpoint", "p256dh", "auth"]).execute();
    const results = await Promise.allSettled(subs.map(s =>
        webpush.sendNotification({
            endpoint: s.endpoint,
            keys: { p256dh: s.p256dh, auth: s.auth },
        }, JSON.stringify(payload))));
    // 410 Gone(구독 만료) → 정리
    await Promise.all(results.map(async (r, i) => {
        if (r.status === "rejected" && (r as any).reason?.statusCode === 410) {
            await db.deleteFrom("push_subscriptions")
                .where("endpoint", "=", subs[i].endpoint).execute();
        }
    }));
}

```

(3) 성능 최적화 팁

- 권한 요청은 반드시 유저 클릭(게임 내 설정 버튼)에서 — 페이지 로드 즉시 요청하면 거부 후 영구 차단될 수 있다.
- 알림 tag 를 이벤트 유형별로 두면 동일 유형이 교체되어 스팸처럼 쌓이지 않는다.
- 410 Gone 자동 정리 없이는 만료 구독이 쌓여 발송 실패율이 지표를 오염시킨다.
- 푸시는 게임 밖 채널이므로 온라인 유저에게는 보내지 않는다(중복 알림 방지) — presence(054 번)로 게이팅한다.

제 10 장 모듈 9 — 경제 및 BM(수익화) 시스템 (081~090)

기술 스택: Node.js · Kysely(PostgreSQL) · 결제 PG/IAP API · TypeScript 본 장은 게임 경제의 원자성과 수익화의 공정성을 동시에 담보한다. 다중 재화는 단일 진입점(지급/소모 분리)과 ledger 감사로 통제되고, 결제·가차·구독은 서버 검증과 먹통 지급이 절대 규칙이다. 싱크 매커니즘·상하한가·기간제 교환소·트랙 분기·스테미너는 경제 순환과 BM 가치를 데이터로 운영한다.

081. 다중 재화(골드, 캐시, 이벤트 코인, 길드 포인트) 관리 시스템

(1) 역할 설명

재화 시스템은 '재화 정의(특성: 저장소, 상한, 소스/싱크) + 잔액 변경 API(단일 진입점) + 흐름 로그(ledger)'의 3 계층이다. 골드/캐시는 유저 계정, 길드 포인트는 길드, 이벤트 코인은 이벤트 기간 캐릭터 단위 등 저장소 주체가 다르므로, 변경 API 는 (ownerType, ownerId, currency, delta, reason)의 범용 시그니처로 단일화한다. 모든 변경은 트랜잭션 + CHECK 제약 (음수 금지) + ledger 기록으로 안전장치를 삼중으로 둔다.

핵심은 '지급(grant)과 소모(spend)의 분리된 진입점'이다. 소모는 잔액 검증 + 원자적 차감이 하나의 UPDATE 문(WHERE balance >= n)으로 수행되어 동시 요청에서 음수가 발생할 수 없다. 지급은 이벤트 소스(퀘스트, 보상, 결제)를 reason 으로 남겨 감사·복구·인플레이션 분석(085 번)의 원천 데이터가 된다.

(2) 구현 코드 — PostgreSQL DDL

```
CREATE TABLE wallets (
  owner_type VARCHAR(10) NOT NULL, -- char | guild | event
  owner_id   BIGINT NOT NULL,
  currency   VARCHAR(12) NOT NULL, -- gold | cash | event_coin | guild_pt
  balance    BIGINT NOT NULL DEFAULT 0,
  version    INT NOT NULL DEFAULT 0, -- 낙관적 잠금(필요 시)
  PRIMARY KEY (owner_type, owner_id, currency)
);
ALTER TABLE wallets ADD CONSTRAINT chk_balance CHECK (balance >= 0);

CREATE TABLE currency_ledger (
  id          BIGSERIAL PRIMARY KEY,
  at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  owner_type  VARCHAR(10) NOT NULL,
  owner_id    BIGINT NOT NULL,
  currency    VARCHAR(12) NOT NULL,
  delta       BIGINT NOT NULL,      -- 양수 지급 / 음수 소모
  reason      VARCHAR(30) NOT NULL, -- quest | shop | trade | pay | sink_...
  ref         VARCHAR(60),          -- 먹등 키
  balance_after BIGINT NOT NULL     -- 변경 후 잔액(감사)
);
CREATE INDEX idx_ledger_owner ON currency_ledger(owner_type, owner_id, at);
CREATE UNIQUE INDEX uq_ledger_ref ON currency_ledger(ref) WHERE ref IS NOT NULL;
```

보조 코드 — server/economy/Wallet.ts — Kysely 단일 진입점

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export type OwnerType = "char" | "guild" | "event";
export type Currency = "gold" | "cash" | "event_coin" | "guild_pt";

export class WalletError extends Error {
  constructor(code: "NO_FUNDS" | "NO_WALLET" | "OVER_CAP") { super(code); }
}

export class Wallet {
  constructor(private db: Kysely<DB>, private caps: Map<Currency, number> = new Map()) {}

  /** 소모: 잔액 검증 + 원자적 차감(WHERE balance >= n) */
  async spend(owner: { type: OwnerType; id: number }, c: Currency, amount: number,
    reason: string, ref?: string): Promise<number> {
    if (amount <= 0) throw new Error("amount must be positive");
    return this.db.transaction().execute(async tx => {
```



```

const upd = await tx.updateTable("wallets")
  .set(eb => ({ balance: eb("balance", "-", amount) })))
  .where(eb => eb.and([
    eb("owner_type", "=", owner.type),
    eb("owner_id", "=", owner.id),
    eb("currency", "=", c),
    eb("balance", ">=", amount),
  ]))
  .returning("balance").executeTakeFirst();
if (!upd) throw new WalletError("NO_FUNDS");
await this.log(tx, owner, c, -amount, reason, ref, upd.balance);
return upd.balance;
});
}

/** 지급: 상한 클램프 + 역등(ref 중복 무시) */
async grant(owner: { type: OwnerType; id: number }, c: Currency, amount: number,
  reason: string, ref?: string): Promise<number> {
  if (amount <= 0) throw new Error("amount must be positive");
  if (ref) {
    const paid = await this.db.selectFrom("currency_ledger")
      .where("ref", "=", ref).select("id").executeTakeFirst();
    if (paid) return -1; // 이미 지급됨(역등)
  }
  return this.db.transaction().execute(async tx => {
    // 지급 보장(없으면 생성)
    await tx.insertInto("wallets").values({
      owner_type: owner.type, owner_id: owner.id, currency: c, balance: 0,
    }).onConflict(oc => oc.doNothing()).execute();
    const cap = this.caps.get(c) ?? Number.MAX_SAFE_INTEGER;
    const cur = await tx.selectFrom("wallets")
      .where(eb => eb.and([
        eb("owner_type", "=", owner.type), eb("owner_id", "=", owner.id),
        eb("currency", "=", c)]))
      .forUpdate().select("balance").executeTakeFirstOrThrow();
    const next = Math.min(cap, cur.balance + amount);
    await tx.updateTable("wallets").set({ balance: next })
      .where(eb => eb.and([
        eb("owner_type", "=", owner.type), eb("owner_id", "=", owner.id),
        eb("currency", "=", c)]))
      .execute();
    await this.log(tx, owner, c, next - cur.balance, reason, ref, next);
    return next;
  });
}

private async log(tx: any, owner: { type: OwnerType; id: number },
  c: Currency, delta: number, reason: string,
  ref: string | undefined, after: number) {
  await tx.insertInto("currency_ledger").values({
    owner_type: owner.type, owner_id: owner.id, currency: c,
    delta, reason, ref: ref ?? null, balance_after: after, at: new Date(),
  }).execute();
}

```

```
}
}
```

(3) 성능 최적화 팁

- 소모는 UPDATE ... WHERE balance >= n 의 영향 행으로 판정한다 — SELECT-then-UPDATE 는 동시 요청에서 반드시 구멍이 난다.
- CHECK(balance >= 0)는 앱 버그의 최후 방어선이며, 지급 상한(caps)은 캐시 계정 도용 피해 한도로도 기능한다.
- ledger 의 balance_after 칼럼이 있으면 잔액 불일치 사고 시 '어느 시점에 어긋났는지'를 순회로 재구성할 수 있다.
- 이벤트 코인은 기간 만료 시 소각 배치(0×잔액)를 뒤야 기간제 재화가 다음 이벤트로 유출되지 않는다.

082. 웹 PG 결제 API 및 모바일 인앱 결제(IAP) 영수증 서버 검증

(1) 역할 설명

결제 파이프라인은 '주문 생성(서버) → 결제 수행(웹 PG or 스토어 IAP) → 영수증 검증(서버 → PG/스토어 API) → 재화 지급(먹등)'의 4 단계다. 웹 PG(토스/페이팔 등)는 결제 승인 API 를 서버에서 재호출해 금액·상태를 검증하고, 모바일 IAP(구글/애플)는 영수증/토큰을 스토어 검증 API 로 확인한다. 검증 성공 시에만 지급하며, 지급은 orderId 기준 먹등(중복 요청 무시)으로 처리한다.

보안 원칙은 세 가지다. 첫째, 금액/상품은 클라가 아니라 서버 주문 테이블이 단일 진실이다 — 클라가 보낸 금액은 절대 신뢰하지 않는다. 둘째, 검증 실패/타임아웃은 주문을 pending 으로 남겨 재조정(reconcile) 배치가 나중에 처리한다. 셋째, 환불(refund) 이벤트를 웹훅으로 받아 지급된 재화를 차감(음수 허용 로그)한다.

(2) 구현 코드 — server/pay/PaymentService.ts

```
import { Kysely } from "kysely";
import crypto from "node:crypto";
type DB = import("../schema").Database;

export interface OrderRow {
  order_id: string; char_id: number; sku: string;
  amount: number; currency: string;
  status: "pending" | "verified" | "granted" | "failed" | "refunded";
  provider: "webpg" | "google" | "apple";
  created_at: Date;
}

export class PaymentService {
  constructor(private db: Kysely<DB>,
```

```

        private pg: { confirmPayment(orderId: string, amount: number): Promise<{ ok:
boolean; raw: string }> },
        private stores: {
            google: (token: string, sku: string) => Promise<{ ok: boolean; orderId: string }>;
            apple: (receipt: string) => Promise<{ ok: boolean; orderId: string }>;
        },
        private wallet: import("../economy/Wallet").Wallet,
        private skuTable: Map<string, { price: number; cash: number; bonusCash?: number }
>) {}

/** 1) 주문 생성(서버가 금액 결정 — 클라 금액 무시) */
async createOrder(charId: number, sku: string) {
    const item = this.skuTable.get(sku);
    if (!item) throw new Error("UNKNOWN_SKU");
    const orderId = crypto.randomUUID();
    await this.db.insertInto("orders").values({
        order_id: orderId, char_id: charId, sku,
        amount: item.price, currency: "KRW", status: "pending", provider: "webpg",
        created_at: new Date(),
    }).execute();
    return { orderId, amount: item.price };    // PG SDK 에 이 금액으로 결제 위임
}

/** 2) 웹 PG 결제 콜백: 서버 승인 재검증 → 지급 */
async confirmWeb(orderId: string) {
    const order = await this.db.selectFrom("orders")
        .where("order_id", "=", orderId).select().executeTakeFirst();
    if (!order || order.status !== "pending") throw new Error("BAD_ORDER");
    const res = await this.pg.confirmPayment(orderId, order.amount);
    if (!res.ok) {
        await this.setStatus(orderId, "failed");
        throw new Error("PG_DECLINED");
    }
    return this.grantForOrder(orderId);
}

/** 3) IAP 영수증 검증(스토어 API → 지급) */
async confirmIap(provider: "google" | "apple", charId: number, token: string, sku: string) {
    const store = this.stores[provider];
    const verified = await store(token, sku);
    if (!verified.ok) throw new Error("RECEIPT_INVALID");
    // 스토어 orderId 를 먹등 키로 사용(재전송 무시)
    const orderId = verified.orderId;
    const dup = await this.db.selectFrom("orders")
        .where("order_id", "=", orderId).select("status").executeTakeFirst();
    if (dup && dup.status === "granted") return { already: true };
    await this.db.insertInto("orders").values({
        order_id: orderId, char_id: charId, sku, amount: this.skuTable.get(sku)!.price,
        currency: "USD", status: "verified", provider, created_at: new Date(),
    }).onConflict(oc => oc.doNothing()).execute();
    return this.grantForOrder(orderId);
}

```

```

/** 4) 지급(먹등) — cash = 상품 캐시 + 이벤트 보너스 */
private async grantForOrder(orderId: string) {
  const order = await this.db.selectFrom("orders")
    .where("order_id", "=", orderId).executeTakeFirstOrThrow();
  const sku = this.skuTable.get(order.sku!);
  await this.wallet.grant(
    { type: "char", id: order.char_id }, "cash",
    sku.cash + (sku.bonusCash ?? 0), "pay", "pay:" + orderId);
  await this.setStatus(orderId, "granted");
  return { granted: sku.cash + (sku.bonusCash ?? 0) };
}
private async setStatus(orderId: string, s: OrderRow["status"]) {
  await this.db.updateTable("orders").set({ status: s })
    .where("order_id", "=", orderId).execute();
}
}

```

(3) 성능 최적화 팁

- 주문 금액은 서버가 결정한다 — 클라 금액을 승인 API에 그대로 쓰면 조작 결제가 즉시 가능해진다.
- IAP 먹등 키는 스토어 orderId — 앱 재시도/서버 재시도 양쪽에서 이중 지급이 차단된다.
- pending 주문에 대한 재조정 배치(5분 주기, PG 상태 재확인)는 결제 성공했는데 콜백이 유실된 사고의 유일한 회복 수단이다.
- 환불 웹훅 처리(재화 음수 차감 + 사용 잔액 부채 기록)는 상점 도용 경제를 막는 필수 요소다.

083. 확률형 상품(가차) 뽑기 및 천장(Pity) 시스템 연산

(1) 역할 설명

가차는 '등급 가중치 추출 → 등급 내 아이템 균등/가중 선택 → 천장 계산 → 지급'의 연산이다. 등급 확률(SR 5%, SSR 0.6% 등)은 가중치 랜덤(WRS)으로 추출하고, 천장은 'N 회차 내 확정' 보장(soft pity: 확률 상승, hard pity: 100%)으로 구현한다. 표준 설계는 (1) hard pity(예: 80 회차 확정 SSR), (2) soft pity(70 회차부터 회차마다 +6%p), (3) 천장 카운터는 배너별·등급별로 분리 관리다.

공정성 요건은 두 가지다. 확률·천장 정보를 게임 내에 표시(법규 준수 — 한국 확률형 아이템 정보공개)하고, 뽑기 기록(history)을 서버에 보존해 클레임 검증이 가능하게 한다. 10 연차는 1 회차~10 회차로 각각 카운터를 진행하되, 하드 천장 도달 시 즉시 확정 등급을 강제한다. 중복 아이템은 파편(교환 재화)으로 전환하는 규칙도 연산에 포함한다.

(2) 구현 코드 — server/gacha/Gacha.ts

```

import { Kysely } from "kysely";
type DB = import("../schema").Database;

```

```

export interface GachaTier { tier: number; weight: number; items: string[]; }
export interface BannerDef {
  id: string;
  tiers: GachaTier[];
  pityHard: number;           // 80 회차 확정(최상등급)
  pitySoftFrom: number;       // 70 회차부터
  pitySoftStep: number;       // 회차당 +0.06(6%p)
  dupeShards: number;         // 중복 전환 파편 수
}

export interface PullResult {
  tier: number; itemCode: string; dupe: boolean;
  pityCount: number; pityTriggered: boolean;
}

export class GachaService {
  constructor(private db: Kysely<DB>, private rng: () => number = Math.random) {}

  /** 1 회 뽑기: 천장 우선 → 등급 WRS → 아이템 선택 */
  async pull(charId: number, banner: BannerDef, count = 1): Promise<PullResult[]> {
    const out: PullResult[] = [];
    for (let i = 0; i < count; i++) {
      // 1) 천장 카운터 로드/증가(배너별)
      const state = await this.loadCounter(charId, banner.id);
      const pityCount = state.count + 1;

      // 2) 하드 천장: 최상등급 확정
      const topTier = Math.max(...banner.tiers.map(t => t.tier));
      let tier: number, pityTriggered = false;
      if (pityCount >= banner.pityHard) {
        tier = topTier;
        pityTriggered = true;
      } else {
        // soft pity: 상승분을 최상등급 가중치에 더함
        const softBoost = pityCount >= banner.pitySoftFrom
          ? (pityCount - banner.pitySoftFrom + 1) * banner.pitySoftStep : 0;
        tier = this.rollTier(banner.tiers, topTier, softBoost);
      }

      // 3) 등급 내 아이템 균등 선택
      const pool = banner.tiers.find(t => t.tier === tier)!.items;
      const itemCode = pool[Math.floor(this.rng() * pool.length)];

      // 4) 중복 판정(보유 이력) → 파편 전환
      const owned = state.owned.has(itemCode);
      if (owned) {
        await this.walletGrant(charId, "shard", banner.dupeShards);
      } else {
        state.owned.add(itemCode);
      }
    }
  }
}

```

```

    out.push({ tier, itemCode, dupe: owned, pityCount, pityTriggered });

    // 카운터 갱신(최상등급 획득 시 리셋)
    await this.saveCounter(charId, banner.id,
        tier === topTier ? 0 : pityCount, state.owned);
    }
    return out;
}

/** 등급 WRS: soft boost 는 최상등급 가중치에 가산 */
private rollTier(tiers: GachaTier[], topTier: number, softBoost: number): number {
    const adjusted = tiers.map(t => ({
        tier: t.tier,
        weight: t.weight + (t.tier === topTier ? softBoost : 0),
    }));
    const total = adjusted.reduce((s, t) => s + t.weight, 0);
    let r = this.rng() * total;
    for (const t of adjusted) if ((r -= t.weight) <= 0) return t.tier;
    return adjusted[adjusted.length - 1].tier;
}

private async loadCounter(charId: number, bannerId: string) {
    const row = await this.db.selectFrom("gacha_counter")
        .where("character_id", "=", charId).where("banner_id", "=", bannerId)
        .select(["count", "owned"]).executeTakeFirst();
    return {
        count: row?.count ?? 0,
        owned: new Set<string>(row ? (row.owned as string[]) : []),
    };
}

private async saveCounter(charId: number, bannerId: string,
    count: number, owned: Set<string>) {
    await this.db.insertInto("gacha_counter").values({
        character_id: charId, banner_id: bannerId, count, owned: [...owned],
    }).onConflict(oc => oc.doUpdate().set({ count, owned: [...owned] })).execute();
}

private async walletGrant(charId: number, c: string, n: number) {
    // 081 번 Wallet.grant 위임(발체)
}
}

```

(3) 성능 최적화 팁

- soft pity(+6%p/회차)는 '후반 스트레스를 확률로 해소'하는 표준이며, hard pity(80 회)는 절대 보장선이다.
- 천장 카운터는 배너별·등급별로 분리해야 — 최상등급 천장과 중간등급 천장이 섞이면 보장이 깨진다.
- 10 연차도 회차별로 개별 카운터를 진행해야 10 연차가 천장을 10 회 우회하는 어뷰징이 없다.

- 중복 전환(파편)은 수확 체감을 줄이는 표준 장치이며, 전환율은 교환소 환율(088 번)과 균형을 맞춘다.

084. VIP / 월정액 매일 자동 보상 지급 스케줄러

(1) 역할 설명

월정액(VIP) 상품은 '구매 → 자격 부여(기간) → 매일 접속 보상 지급'의 구조다. 자격은 subscriptions 테이블(시작/종료일, 자동 갱신 여부)로 관리하고, 일일 보상은 '첫 접속 시 1 회' 정책으로 지급한다 — 접속하지 않으면 그날 보상은 소멸(또는 적립 옵션)되며, 이것이 앱 실행 유인의 핵심 동력이다. 지급은 (유저, 상품, 날짜) 먹등 키로 이중 지급을 막는다.

스케줄러의 역할은 두 가지다. 첫째, 접속 시점 즉시 지급(로그인 핸들러 혹은) — 대부분의 보상은 이 경로다. 둘째, 만료/자동 갱신/기간 종료 정산을 크론(일 1 회)으로 처리한다. 자동 갱신 결제 실패 시 유예 기간(3 일)을 두고, 유예 중에도 자격 유지 정책은 BM 설계로 결정한다. 모든 상태 변화는 구독 로그로 남겨 CS 대응 데이터를 확보한다.

(2) 구현 코드 — server/vip/VipService.ts

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export interface VipSku {
  skuId: string; days: number;
  dailyCash: number; dailyItem?: { code: string; qty: number };
  autoRenewable: boolean;
}

export class VipService {
  constructor(private db: Kysely<DB>, private skus: Map<string, VipSku>,
    private wallet: import("../economy/Wallet").Wallet) {}

  /** 구매 완료(082 번 결제 검증 후) → 자격 부여(기존 잔여기간 + 신규 기간) */
  async activate(charId: number, skuId: string, paymentRef: string) {
    const sku = this.skus.get(skuId);
    if (!sku) throw new Error("NO_SKU");
    const now = Date.now();
    const cur = await this.db.selectFrom("vip_subscriptions")
      .where("character_id", "=", charId).where("sku_id", "=", skuId)
      .select(["expires_at"]).executeTakeFirst();
    const base = cur && cur.expires_at.getTime() > now ? cur.expires_at.getTime() : now;
    const expiresAt = new Date(base + sku.days * 86400000);
    await this.db.insertInto("vip_subscriptions").values({
      character_id: charId, sku_id: skuId, expires_at: expiresAt,
      started_at: new Date(), auto_renew: sku.autoRenewable,
    }).onConflict(oc => oc.doUpdate().set({ expires_at: expiresAt })).execute();
    await this.db.insertInto("vip_log").values({
      character_id: charId, event: "activate", sku_id: skuId,
      ref: paymentRef, at: new Date(),
    });
  }
}
```

```

    }).execute();
    return expiresAt;
}

/** 첫 접속 일일 보상(로그인 혹은 — 먹등: (char, sku, date)) */
async grantDailyOnLogin(charId: number, dateKey: string) {
    const subs = await this.db.selectFrom("vip_subscriptions")
        .where("character_id", "=", charId)
        .where("expires_at", ">", new Date())
        .select(["sku_id"]).execute();
    for (const s of subs) {
        const sku = this.skus.get(s.sku_id!);
        const ref = "vip:daily:" + charId + ":" + s.sku_id + ":" + dateKey;
        await this.wallet.grant({ type: "char", id: charId }, "cash",
            sku.dailyCash, "vip_daily", ref);
        if (sku.dailyItem) {
            await this.wallet.grant({ type: "char", id: charId }, "event_coin",
                1, "vip_daily_item", ref + ":i");
        }
    }
}

/** 크론(일 1 회): 만료 처리 + 자동 갱신 시도 */
async cron(now: Date) {
    // 1) 만료
    await this.db.updateTable("vip_subscriptions")
        .set({ status: "expired" })
        .where("expires_at", "<=", now).where("status", "=", "active").execute();
    // 2) 자동 갱신(결제 시스템 082 번 호출) — 실패 시 유예 기록
    const due = await this.db.selectFrom("vip_subscriptions")
        .where("auto_renew", "=", true)
        .where("expires_at", "<=", now + 3 * 86400000)
        .where("status", "=", "active")
        .select(["character_id", "sku_id"]).execute();
    for (const s of due) {
        // 실제 결제는 082 번 confirmIap/confirmWeb 파이프라인 사용
        await this.db.insertInto("vip_log").values({
            character_id: s.character_id, event: "renew_attempt",
            sku_id: s.sku_id, ref: "cron", at: now,
        }).execute();
    }
}
}

```

(3) 성능 최적화 팁

- 일일 보상은 '첫 접속 시 지급 + 미접속 소멸'이 기본이며, 적립형(최대 3일 누적)은 유예 정책으로 채택 가능하다.
- 기간 연장은 '잔여 기간 + 신규 기간' 합산으로 — 조기 구매가 손해가 되면 재구매율이 떨어진다.

- 구독 상태 변화(활성/갱신/만료/유예)는 모두 vip_log 로 — CS '보상 안 나왔다' 클레임의 1차 근거다.
- 자동 갱신은 결제 실패 유예(3 일) 중에도 보상을 지급하는 정책이 이탈을 줄인다(갱신 실패 곧 해지 방지).

085. 인플레이션 방지를 위한 재화 소모(Sink) 매커니즘

(1) 역할 설명

MMORPG 경제는 '소스(지급)와 싱크(소모)의 균형'이 생명이다. 소스는 퀘스트·몬스터 드롭·이벤트에서 계속 늘어나므로, 싱크가 부족하면 물가가 폭등해 신규 유저 진입이 불가능해진다. 이 시스템은 재화 유동성 대시보드(소스/싱크 흐름 집계)와 싱크 설계(강화 비용, 제작 비용, 시세 수수료, 수리비, 텔레포트 비용 등)를 데이터로 운영한다. 싱크 강도는 소스 대비 목표 소비율(보통 60~80%)로 역산해 조정한다.

구현은 (1) ledger 기반 흐름 집계(reason 별 소스/싱크 합산, 081 번 데이터 재사용), (2) 물가 지표(경매장 거래가 이동 평균 — 056 번), (3) 동적 조정(특정 싱크 비용 배율 테이블)의 3 단계다. 중요한 것은 싱크가 '강제 세금'이 아니라 '가치 소비'여야 한다는 점이다 — 강화/제작처럼 유저가 자발적으로 소비하는 싱크가 지속 가능하다.

(2) 구현 코드 — server/economy/EconomyMonitor.ts

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export interface FlowRow {
  reason: string; inflow: number; outflow: number; net: number;
}

export class EconomyMonitor {
  constructor(private db: Kysely<DB>) {}

  /** 소스/싱크 집계(기간별) — 대시보드 + 싱크 조정 근거 */
  async flows(currency = "gold", days = 7): Promise<FlowRow[]> {
    const rows = await this.db.selectFrom("currency_ledger")
      .where("currency", "=", currency)
      .where("at", ">=", new Date(Date.now() - days * 86400000))
      .select(eb => [
        eb("reason").as("reason"),
        eb.fn.sum(eb.case().when("delta", ">", 0)
          .then(eb.ref("delta")).else(0).end()).as("inflow"),
        eb.fn.sum(eb.case().when("delta", "<", 0)
          .then(eb.ref("delta").neg()).else(0).end()).as("outflow"),
      ])
      .groupBy("reason").execute();
    return rows.map(r => ({
      reason: r.reason,
      inflow: Number(r.inflow), outflow: Number(r.outflow),
      net: Number(r.inflow) - Number(r.outflow),
    }));
  }
}
```

```

    }));
  }

  /** 싱크 강도 지표: 소스 대비 소모율 */
  static sinkRate(flows: FlowRow[]): number {
    const inflow = flows.reduce((s, f) => s + f.inflow, 0);
    const outflow = flows.reduce((s, f) => s + f.outflow, 0);
    return inflow > 0 ? outflow / inflow : 0;
  }

  /** 동적 싱크 배율: 소모율 낮을수록 싱크 비용 상향(전략적 조정) */
  static sinkMultiplier(sinkRate: number): number {
    if (sinkRate >= 0.8) return 1.0; // 건강: 기본 배율
    if (sinkRate >= 0.6) return 1.05;
    if (sinkRate >= 0.4) return 1.15;
    return 1.3; // 심각: 싱크 비용 +30%
  }
}

/** 싱크 설계 예시 — 유저가 자발적으로 소비하는 경로 */
// 1) 강화 비용(025 번): 단계별 골드 소모 — 최대 싱크
// 2) 제작/연금술(066 번): 재료 + 수수료
// 3) 경매장 수수료(056 번): 거래 5% — 유동성 싱크
// 4) 수리비: 사망 시 장비 내구도 소모
// 5) 텔레포트/이동 주문서: 편의 소모
// 6) 명함/외형 합성: 수집형 소비

/** 경고 규칙(크론 일 1 회) */
// sinkRate < 0.5 지속 3 일 → 운영팀 알림 + 싱크 배율 상향 검토
// 특정 reason inflow 급증(이벤트 드롭 오류 등) → 이상 흐름 감지

```

(3) 성능 최적화 팁

- 싱크는 '자발적 소비(강화/제작/외형)' 중심으로 설계한다 — 강제 세금형 싱크(수리비 폭탄)는 이탈리아의 주범이다.
- 소모율(sinkRate) 0.6~0.8 밴드가 건강 구간이며, 0.4 미만이면 싱크 비용 배율로 서서히 조정한다(급격한 인상 금지).
- 이벤트 기간 소스 폭증은 예측 가능하므로, 이벤트와 함께 전용 싱크(이벤트 코인 교환소 088 번)를 동시에 오픈해야 한다.
- ledger reason 표준화가 없으면 흐름 분석 자체가 불가능하다 — reason 은 서버 상수 테이블로 관리한다.

086. 귀속 / 비귀속 아이템 판정 및 거래 제한 상태 관리

(1) 역할 설명

귀속(binding)은 아이템의 이동 가능성을 정의하는 상태다. 비귀속(tradeable)은 모든 경로(거래/경매/창고/우편)로 이동 가능하고, 획득 귀속(acquire-bound)은 획득 즉시 귀속, 장착 귀속(equip-bound)은 장착한 순간 귀속된다. 판정은 단일 함수(canTrade(item))로 수행하며, 모든 이동 경로(055 번 거래, 056 번 경매, 052 번 창고, 우편)가 동일 함수를 호출해 규칙을 하나로 유지한다.

거래 제한 상태는 귀속 외에도 여러 겹이다. 거래 쿨타임(획득 후 24 시간 거래 불가 — RMT 차단), 스택 분해(스택 아이템의 부분 거래), 상한가/하한가(087 번), 기간제 아이템(만료 시 소각) 등이다. 상태는 item row 의 칼럼(bound_at, acquired_at, expires_at)으로 관리하며, 귀속 전환은 이벤트(장착/사용) 혹은 원자적으로 기록된다.

(2) 구현 코드 — server/item/Binding.ts

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export type BindingKind = "tradeable" | "acquire_bound" | "equip_bound";

export interface ItemDefBinding {
  code: string;
  binding: BindingKind;
  tradeCooldownMs?: number; // 획득 후 거래 쿨타임(RMT 방지)
  expiresAtMs?: number; // 기간제(0 = 무기한)
}

export interface ItemRuntime {
  id: number; itemCode: string;
  binding: BindingKind;
  boundAt: number | null; // 귀속 시각(null = 미귀속)
  acquiredAt: number; // 획득 시각(쿨타임 계산)
  expiresAt: number | null;
  equipLockedPart?: string | null; // 장착 중 부위(장착 귀속 판정)
}

export class BindingPolicy {
  /** 귀속 판정: 정의 + 런타임 상태 합산 */
  static isBound(rt: ItemRuntime): boolean {
    if (rt.binding === "acquire_bound") return true;
    if (rt.binding === "equip_bound" && rt.boundAt != null) return true;
    return false;
  }

  /** 거래 가능 판정(모든 이동 경로의 공통 게이트) */
  static canTrade(rt: ItemRuntime, def: ItemDefBinding, now: number):
    { ok: boolean; reason?: string } {
```

```

    if (BindingPolicy.isBound(rt)) return { ok: false, reason: "bound" };
    const cooldown = def.tradeCooldownMs ?? 0;
    if (now - rt.acquiredAt < cooldown)
        return { ok: false, reason: "trade_cooldown" };
    if (rt.expiresAt && rt.expiresAt <= now) return { ok: false, reason: "expired" };
    return { ok: true };
}

/** 장착 이벤트 혹은: 장착 귀속 전환(원자적 기록) */
static async onEquip(db: Kysely<DB>, itemId: number, now: number) {
    const it = await db.selectFrom("character_items").where("id", "=", itemId)
        .forUpdate().select(["binding", "bound_at"]).executeTakeFirstOrThrow();
    if (it.binding === "equip_bound" && it.bound_at == null) {
        await db.updateTable("character_items").set({ bound_at: new Date(now) })
            .where("id", "=", itemId).execute();
    }
}
}

```

보조 코드 — PostgreSQL DDL — 상태 칼럼

```

ALTER TABLE character_items
    ADD COLUMN binding    VARCHAR(16) NOT NULL DEFAULT 'tradeable',
    ADD COLUMN bound_at   TIMESTAMPTZ,      -- 귀속 전환 시각
    ADD COLUMN acquired_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    ADD COLUMN expires_at TIMESTAMPTZ;      -- 기간제 아이템

-- 기간제 아이템 만료 소각(크론 시간당)
-- DELETE FROM character_items WHERE expires_at IS NOT NULL AND expires_at <= now();
-- 대량 테이블이므로 배치 삭제 + ledger 기록으로 실제 구현

```

(3) 성능 최적화 팁

- 규칙 판정은 BindingPolicy.canTrade 한 곳으로 — 경로별로 따로 만들면 '거래는 되는데 경매는 안 되는' 불일치가 난다.
- 장착 귀속은 이벤트 혹은: bound_at 을 기록해야 CS 에서 '언제 귀속됐는지'를 증명할 수 있다.
- 거래 쿨타임(획득 후 24 시간)은 RMT(현금거래) 흐름을 끊는 가장 효과적인 완충 장치다.
- 기간제 아이템은 만료 소각을 크론으로 처리하되, 만료 24 시간 전 알림(080 번 푸시)을 함께 두면 불만이 사라진다.

087. 어뷰징/현금거래 방지용 거래소 시세 상하한가 제어 로직

(1) 역할 설명

시세 상하한가는 '기준 시세 대비 등록/거래 가능 가격의 밴드'로, 시세 조작(자기 매수 반복)과 RMT(극단가 거래)를 동시에 차단한다. 기준 시세는 056 번 marketPrice(최근 24h

거래가 평균)이며, 밴드는 기본 $\pm 30\%$ (등록)와 $\pm 60\%$ (거래 허용 임계) 이중으로 둔다. 밴드 밖 가격은 등록 자체가 거부되고, 이유(UI 안내: 시세 밖 가격입니다)를 명확히 전달한다.

고급 장치는 유동성 부족 보호다. 거래량이 적은 아이템은 24h 평균이 왜곡되기 쉬우므로, 최소 표본(거래 5 건) 미만이면 7일 평균으로 풀백하고 그것도 부족하면 item_def 기준가(정가)를 사용한다. 또한 '가격 급변 감지(1시간 내 평균 대비 $\pm 50\%$)'는 시세 조작 시도 알림(094 번 어드민)으로 연결된다. 시세 밴드는 카테고리별로 다르게 설정해고가 희귀품은 완화한다.

(2) 구현 코드 — server/auction/PriceGuard.ts

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export interface PriceBand {
  min: number; max: number;
  basePrice: number; source: "24h" | "7d" | "def_price";
  sampleCount: number;
}

export class PriceGuard {
  /** 등록 밴드( $\pm 30\%$ ), 거래 밴드( $\pm 60\%$ ) */
  private static REGISTER_BAND = 0.30;
  private static TRADE_BAND = 0.60;
  private static MIN_SAMPLES = 5;

  constructor(private db: Kysely<DB>,
    private defPrice: (itemCode: string) => number) {}

  /** 기준 시세 산출: 24h → 7d → 정가 풀백(유동성 보호) */
  async basePrice(itemCode: string): Promise<PriceBand> {
    const day = await this.avg(itemCode, 1);
    if (day && day.count >= PriceGuard.MIN_SAMPLES) {
      return this.band(day.avg, "24h", day.count);
    }
    const week = await this.avg(itemCode, 7);
    if (week && week.count >= PriceGuard.MIN_SAMPLES) {
      return this.band(week.avg, "7d", week.count);
    }
    return this.band(this.defPrice(itemCode), "def_price", 0);
  }
  private band(base: number, source: PriceBand["source"], n: number): PriceBand {
    return {
      basePrice: Math.round(base),
      source, sampleCount: n,
      min: Math.max(1, Math.round(base * (1 - PriceGuard.REGISTER_BAND))),
      max: Math.round(base * (1 + PriceGuard.REGISTER_BAND)),
    };
  }
  private async avg(itemCode: string, days: number) {
```

```

const row = await this.db.selectFrom("auction_trades")
  .innerJoin("auction_listings", "auction_trades.listing_id", "auction_listings.id")
  .where("auction_listings.item_code", "=", itemCode)
  .where("auction_trades.at", ">=",
    new Date(Date.now() - days * 86400000))
  .select(eb => [
    eb.fn.avg("final_price").as("avg"),
    eb.fn.count("auction_trades.id").as("cnt"),
  ]).executeTakeFirst();
return row && Number(row.cnt) > 0
? { avg: Number(row.avg), count: Number(row.cnt) } : null;
}

/** 등록 검증(056 번 등록 경로에서 호출) */
async validateListing(itemCode: string, price: number):
  Promise<{ ok: boolean; reason?: string; hint?: PriceBand }> {
  const band = await this.basePrice(itemCode);
  if (price < band.min) return { ok: false, reason: "below_floor", hint: band };
  if (price > band.max) return { ok: false, reason: "above_ceiling", hint: band };
  return { ok: true, hint: band };
}

/** 가격 급변 감지(시세 조작 알림) — 1h 평균 vs 이전 1h */
async detectAnomaly(itemCode: string): Promise<boolean> {
  const now = await this.avg(itemCode, 1 / 24);
  const prev = await this.avgRange(itemCode,
    Date.now() - 2 * 3600000, Date.now() - 1 * 3600000);
  if (!now || !prev) return false;
  const swing = Math.abs(now.avg - prev.avg) / Math.max(1, prev.avg);
  return swing > 0.5; // ±50% 급변
}

private async avgRange(itemCode: string, from: number, to: number) {
  const row = await this.db.selectFrom("auction_trades")
    .innerJoin("auction_listings", "auction_trades.listing_id", "auction_listings.id")
    .where("auction_listings.item_code", "=", itemCode)
    .where("auction_trades.at", ">=", new Date(from))
    .where("auction_trades.at", "<", new Date(to))
    .select(eb => [eb.fn.avg("final_price").as("avg")]).executeTakeFirst();
  return row ? { avg: Number(row.avg) } : null;
}
}

```

(3) 성능 최적화 팁

- 밴드 폴백 체인(24h → 7d → 정가) 없이는 신규 아이템이 '시세 없음'으로 등록 자체가 불가능해진다.
- 등록 밴드(±30%)와 거래 밴드(±60%)를 분리하면 '이미 등록된 것의 급락'과 '신규 등록 조작'을 다르게 통제할 수 있다.
- 급변 감지는 자동 제재가 아니라 어드민(094 번) 알림으로 — 오탐(이벤트로 인한 자연 급등)을 사람이 판단한다.

- 시세 표시 UI에 표본 수(최근 거래 N 건 기준)를 함께 보여주면 '가짜 시세' 불신을 예방한다.

088. 이벤트 코인 교환소 기간 제한 매커니즘

(1) 역할 설명

이벤트 코인 교환소는 '획득(코인 드롭) → 교환(코인 → 아이템) → 만료(잔여 코인 소각)'의 기간제 경제다. 교환소 상품은 (상품 코드, 코인 가격, 구매 한도(1 인/전체), 재고)로 정의되고, 이벤트 기간(start~end) 동안만 거래가 가능하다. 종료 시점 이후에는 교환 API가 기간 검증에서 거부되고, 잔여 코인은 만료 배치에서 0으로 소각해(081번 ledger 기록) 다음 이벤트로 이월되지 않게 한다.

설계 포인트는 세 가지다. 첫째, 전체 재고 한도(서버 전체 판매량 제한)로 희소품 가치를 유지한다. 둘째, 구매 한도는 1인 기준 + 서버 재고의 원자적 차감(UPDATE WHERE stock >= n)으로 동시 구매 경쟁을 안전하게 처리한다. 셋째, 교환은 이벤트 종료 직전(마지막 24시간) 트래픽 폭증이 예상되므로, 상품별 캐시(재고/한도 Redis)로 DB 부담을 분산한다.

(2) 구현 코드 — server/exchange/CoinExchange.ts

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export interface ExchangeDef {
  eventId: string;
  startsAt: number; endsAt: number;
  goods: {
    code: string;           // 상품 코드
    coinCost: number;       // 코인 가격
    perUserLimit?: number;  // 1인 구매 한도
    stock?: number;         // 전체 재고(null = 무제한)
    itemCode: string; qty: number;
  }[];
}

export class CoinExchange {
  constructor(private db: Kysely<DB>, private def: ExchangeDef,
    private wallet: import("../economy/Wallet").Wallet) {}

  /** 교환: 기간 → 한도 → 재고 → 코인 소모 → 지급(단일 트랜잭션) */
  async exchange(charId: number, goodsCode: string) {
    const now = Date.now();
    if (now < this.def.startsAt) throw new Error("NOT_OPENED");
    if (now >= this.def.endsAt) throw new Error("EXPIRED");    // 기간 만료

    const g = this.def.goods.find(x => x.code === goodsCode);
    if (!g) throw new Error("NO_GOODS");
```

```

return this.db.transaction().execute(async tx => {
  // 1) 1인 한도 검증
  if (g.perUserLimit) {
    const bought = await tx.selectFrom("exchange_purchases")
      .where("character_id", "=", charId)
      .where("event_id", "=", this.def.eventId)
      .where("goods_code", "=", goodsCode)
      .select(eb => eb.fn.sum("qty").as("n")).executeTakeFirst();
    if (Number(bought?.n ?? 0) + g.qty > g.perUserLimit)
      throw new Error("LIMIT_REACHED");
  }
  // 2) 전체 재고 원자적 차감
  if (g.stock !== null) {
    const upd = await tx.updateTable("exchange_stock")
      .set(eb => ({ remaining: eb("remaining", "-", g.qty) })))
      .where("event_id", "=", this.def.eventId)
      .where("goods_code", "=", goodsCode)
      .where("remaining", ">=", g.qty)
      .returning("remaining").executeTakeFirst();
    if (!upd) throw new Error("SOLD_OUT");
  }
  // 3) 코인 소모(081번 spend — 잔액 부족 시 예외)
  await this.wallet.spend({ type: "event", id: charId },
    "event_coin", g.coinCost, "exchange:" + this.def.eventId,
    "ex:" + this.def.eventId + ":" + goodsCode + ":" + charId + ":" + Date.now());
  // 4) 아이템 지급
  await tx.updateTable("character_items")
    .set(eb => ({ qty: eb("qty", "+", g.qty) })))
    .where("character_id", "=", charId)
    .where("item_code", "=", g.itemCode).execute();
  await tx.insertInto("exchange_purchases").values({
    character_id: charId, event_id: this.def.eventId,
    goods_code: goodsCode, qty: g.qty, at: new Date(),
  }).execute();
  return { itemCode: g.itemCode, qty: g.qty };
});
}

/** 만료 배치(크론): 이벤트 종료 후 잔여 코인 소각 */
async expireEvent() {
  const now = Date.now();
  if (now < this.def.endsAt) return;
  // 코인 잔액 전체 소각 + ledger 기록(sink: ev_expire)
  // UPDATE wallets SET balance = 0 WHERE currency='event_coin'
  // AND owner_type='event' AND owner_id IN (이벤트 참여자);
  // 실제 구현은 페이지네이션 배치(수만 건)로 분할 수행
}
}

```


(3) 성능 최적화 팁

- 기간 검증은 서버 단일 시각으로 — 클라 시계 조작으로 만료 후 교환하는 것을 차단한다.
- 전체 재고 차감은 UPDATE ... WHERE remaining >= n 으로 — SELECT 후 UPDATE 는 동시 구매에서 초판매가 발생한다.
- 잔여 코인 소각은 반드시 ledger 기록(reason: ev_expire)으로 — 이벤트 경제 회고(085 번)의 핵심 데이터다.
- 마지막 24 시간 트래픽 폭증 대비 상품별 재고 캐시(Redis) + DB 원자 차감의 이중 구조가 안전하다.

089. 무료/프리미엄 배틀패스 보상 트랙 분기 처리

(1) 역할 설명

배틀패스 트랙 분기는 '같은 레벨 구간에서 무료 트랙과 프리미엄 트랙이 다른 보상을 지급'하는 구조다(070 번 기본 파이프라인의 확장). 핵심 차별점은 세 가지다. (1) 프리미엄 전용 레벨 구간(무료는 30 레벨, 프리미엄은 50 레벨까지), (2) 프리미엄 보상의 가치(외형/탈것 등), (3) 프리미엄 XP 가속(+20% 등)의 이중 이점이다. 분기 판정은 claim 시점에 트랙 권한(구독 여부)으로 수행한다.

추가 요건은 시즌 종료 후 처리다. 무료 트랙 미수령 보상은 우편 일괄 전환, 프리미엄 미수령 보상은 '다음 시즌 구매 시 소급 클레임 가능' 기간(7 일)을 두는 정책이 표준이다. 트랙 데이터는 JSON 정의로 관리해 시즌마다 리밸런싱이 코드 수정 없이 가능하며, 레벨별 보상이 비어있으면(null) 스킵 마커로 UI 에 표시한다.

(2) 구현 코드 — server/seasonpass/TrackPolicy.ts

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export interface TrackReward { gold?: number; itemCode?: string; qty?: number; }
export interface SeasonTracks {
  seasonId: string;
  freeMaxLevel: number;           // 무료 트랙 상한(30)
  maxLevel: number;              // 프리미엄 상한(50)
  premiumXpBoost: number;        // 0.2 = +20%
  levels: { free: TrackReward | null; premium: TrackReward | null }[];
}

export class TrackPolicy {
  constructor(private db: Kysely<DB>, private tracks: SeasonTracks,
    private wallet: import("../economy/Wallet").Wallet) {}

  /** XP 적립: 프리미엄 가속 적용(062 번 미션 이벤트에서 호출) */
```

```

async addXp(charId: number, baseXp: number) {
  const row = await this.db.selectFrom("season_pass")
    .where("character_id", "=", charId).where("season_id", "=", this.tracks.seasonId)
    .select(["xp", "premium"]).executeTakeFirst();
  const boost = row?.premium ? 1 + this.tracks.premiumXpBoost : 1;
  const gained = Math.floor(baseXp * boost);
  await this.db.updateTable("season_pass")
    .set(eb => ({ xp: eb("xp", "+", gained) })))
    .where("character_id", "=", charId).where("season_id", "=", this.tracks.seasonId)
    .execute();
  return gained;
}

/** 트랙 분기 클레임(070 번 claim 확장 — 상한 레벨 구간 처리) */
async claim(charId: number, level: number, track: "free" | "premium") {
  const row = await this.db.selectFrom("season_pass")
    .where("character_id", "=", charId).where("season_id", "=", this.tracks.seasonId)
    .select(["xp", "premium"]).executeTakeFirstOrThrow();
  const userLevel = Math.min(this.levelOf(row.xp), this.tracks.maxLevel);

  // 무료 트랙은 freeMaxLevel 이후 구간 자체가 없음
  if (track === "free" && level > this.tracks.freeMaxLevel)
    throw new Error("FREE_TRACK_LIMIT");
  // 프리미엄 전용 구간(31~50): 무료 유저는 도달 불가 개념
  if (track === "premium" && !row.premium) throw new Error("NEED_PREMIUM");
  if (userLevel < level) throw new Error("LEVEL_NOT_REACHED");

  const tier = this.tracks.levels[level - 1];
  const reward = tier[track];
  if (!reward) throw new Error("EMPTY_REWARD");
  const ref = "sp:" + this.tracks.seasonId + ":" + level + ":" + track + ":" + charId;
  await this.wallet.grant({ type: "char", id: charId }, "gold",
    reward.gold ?? 0, "season_pass", ref);
  return reward;
}

/** 시즌 종료 배치: 미수령 보상 우편 전환(먹등) */
async sweepUnclaimed() {
  // 실제 구현: season_pass 전체 순회 → levelOf(xp) 이하 레벨 중
  // ledger 에 ref 없는 보상을 우편 테이블로 이관
  // 무료/프리미엄 각각 처리, 프리미엄은 premium 여부 확인
}
private levelOf(xp: number): number {
  const lv = Math.floor(xp / 1000) + 1; // 070 번 xpPerLevel 재사용
  return Math.min(this.tracks.maxLevel, lv);
}
}

```

(3) 성능 최적화 팁

- 무료 트랙 상한(30 레벨)과 프리미엄 상한(50 레벨)의 격차가 프리미엄 가치의 본질이다 — 같은 상한이면 구매 동기가 없다.
- 프리미엄 XP 가속(+20%)은 '구매 후 성장 속도' 이점으로, 트랙 보상 외에 체감 이점을 하나 더 준다.
- 빈 보상(null) 스킵 마커를 UI에 명확히 표시해 '보상 누락' 클레임을 예방한다.
- 시즌 정의는 JSON 데이터로 관리해 리밸런싱(레벨 보상 조정)이 배포 없이 이뤄지게 한다.

090. 스테미너 / 피로도 자동 회복 및 차감 연산

(1) 역할 설명

스테미너는 '행동 게이팅(던전 진입, 보스 도전 횟수) + 시간 자동 회복(분당 n 점)'의 자원이다. 핵심 연산은 '오프라인 회복'이다. 접속 종료 중에도 시간이 흐르므로, 재접속 시 (현재 시각 - 마지막 회복 시각) / 회복 주기로 회복량을 계산하고 상한(예: 120)까지 클램프한다. 저장은 '현재 잔량 + 마지막 회복 시각' 두 값만 유지하면 계산이 프레임율/접속 패턴과 무관하게 결정론적으로 성립한다.

차감은 서버 권위로 — 던전 진입 요청 시 잔량 검증 + 원자적 차감을 한 트랜잭션으로 수행한다. 회복 속도 부스트(VIP/아이템)는 '회복 주기 단축'과 '상한 증가'의 두 파라미터로 모델링하며, 스테미너 충전 아이템은 상한 초과 허용 여부를 정책으로 명시한다(보통 상한 초과 허용, 최대 2배).

(2) 구현 코드 — `server/stamina/Stamina.ts`

```
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export interface StaminaConfig {
  max: number;          // 자연 상한 120
  regenMs: number;      // 6 분당 1 점 회복
  hardCapMult: number;   // 아이템 충전 상한 배율(2.0)
}

export class StaminaService {
  constructor(private db: Kysely<DB>, private cfg: StaminaConfig) {}

  /** 현재 스테미너 계산(저장값 + 오프라인 회복 재구성) */
  async current(charId: number, regenBoost = 1): Promise<number> {
    const row = await this.db.selectFrom("stamina")
      .where("character_id", "=", charId)
      .select(["value", "last_regen_at"]).executeTakeFirst();
    if (!row) return this.cfg.max;          // 신규 = 풀 충전
  }
}
```

```

const elapsed = Date.now() - row.last_regen_at.getTime();
const regen = Math.floor(elapsed / (this.cfg.regenMs / regenBoost));
return Math.min(this.cfg.max, row.value + regen);
}

/** 접속 시 동기화: 계산 결과를 저장(이후 차감은 저장값 기준) */
async sync(charId: number, regenBoost = 1): Promise<number> {
  const value = await this.current(charId, regenBoost);
  await this.db.insertInto("stamina").values({
    character_id: charId, value, last_regen_at: new Date(),
  }).onConflict(oc => oc.doUpdate().set({
    value,
    last_regen_at: value >= this.cfg.max
      ? new Date() // 상한 도달 — 회복 시각 리셋
      : this.lastRegenShift(charId, regenBoost),
  })).execute();
  return value;
}

private async lastRegenShift(charId: number, boost: number) {
  const row = await this.db.selectFrom("stamina")
    .where("character_id", "=", charId).select("last_regen_at").executeTakeFirst();
  const prev = row?.last_regen_at.getTime() ?? Date.now();
  const consumed = Math.floor((Date.now() - prev) / (this.cfg.regenMs / boost))
    * (this.cfg.regenMs / boost);
  return new Date(prev + consumed);
}

/** 차감(던전 진입 등) — 원자적 검증 + 차감 */
async consume(charId: number, cost: number): Promise<boolean> {
  return this.db.transaction().execute(async tx => {
    const row = await tx.selectFrom("stamina")
      .where("character_id", "=", charId)
      .forUpdate().select(["value", "last_regen_at"]).executeTakeFirstOrThrow();
    // 저장값을 현재 시점으로 갱신 후 검증
    const elapsed = Date.now() - row.last_regen_at.getTime();
    const regen = Math.floor(elapsed / this.cfg.regenMs);
    const value = Math.min(this.cfg.max, row.value + regen);
    if (value < cost) return false; // 부족
    const next = value - cost;
    const consumedMs = regen * this.cfg.regenMs;
    await tx.updateTable("stamina").set({
      value: next,
      last_regen_at: new Date(row.last_regen_at.getTime() + consumedMs),
    }).where("character_id", "=", charId).execute();
    return true;
  });
}

/** 충전 아이템: 상한 배율(hardCap)까지 허용 */
async charge(charId: number, amount: number): Promise<number> {
  const row = await this.db.selectFrom("stamina")
    .where("character_id", "=", charId)

```

```

    .forUpdate().select(["value", "last_regen_at"]).executeTakeFirstOrThrow();
    const cap = this.cfg.max * this.cfg.hardCapMult;
    const next = Math.min(cap, row.value + amount);
    await this.db.updateTable("stamina").set({ value: next })
      .where("character_id", "=", charId).execute();
    return next;
  }
}

```

(3) 성능 최적화 팁

- 저장은 '잔량 + 마지막 회복 시각' 두 값으로 충분하다 — 분당 회복량을 전부 저장하면 데이터가 계속 변한다.
- 회복 소수점 처리는 '소모된 ms 를 last_regen_at 에 반영'으로 — 매 계산마다 잔여 시간이 유실되지 않는다.
- 차감은 forUpdate + 잔량 검증으로 — 동시 던전 진입 요청에서 음수 스테미너가 발생하지 않는다.
- 상한 도달 시 last_regen_at 을 리셋해야 '상한 도달 후 다시 빠지면 즉시 회복되는' 무한 회복 버그가 막힌다.

제 11 장 모듈 10 — 보안, 어드민 및 라이브 서비스 (091~100)

기술 스택: uWebSockets.js · Kysely · Fastify(어드민 API) · OAuth2 · TypeScript 본 장은 라이브 서비스의 방어선과 운영 손잡이다. 스피드해킹·벽뚫기·매크로 탐지는 서버 권위 재연산과 통계 기반 점수화로 수행되고, 어드민 대시보드는 권한 비트와 감사 로그가 본질이다. 텔레메트리·핫픽스·i18n·OAuth2·제재 미들웨어·백업/패일오버는 서비스가 '살아있는 동안' 계속 요구되는 인프라다.

091. 클라이언트 메모리 변조 및 스피드해킹 탐지 검증

(1) 역할 설명

웹 게임의 클라 메모리는 공개되어 있다 — 디버거로 HP/골드/속도 변수를 직접 고칠 수 있다. 방어 원칙은 '클라 값은 참조용이고 서버가 유일 권위(016 번)'지만, 탐지 계층을 두면 어뷰징을 조기에 잡을 수 있다. 스피드해킹 탐지는 (1) 클라 자체 시계 이중 검증(performance.now vs Date.now vs rAF 누적), (2) 서버측 이동 속도 검증(016 번), (3) 액션 빈도 통계(초당 공격 횟수)의 3 중 구조로 수행한다.

클라 셀프 체크는 소프트 계층이다. 중요 변수(전투 상태, 세이브 데이터)에 체크섬을 유지해 변조 시 서버 동기화가 불일치로 드러나게 하고, rAF 간격의 통계적 이상(고정 10ms 간격 반복)을 서버로 보고한다. 최종 판정은 항상 서버가 하며, 클라 탐지 신호는 '참고 데이터'로 취급해 오탐(느린 기기, 스로틀링 브라우저)을 배제한다.

(2) 구현 코드 — server/security/SpeedHackDetector.ts

```

import type uWS from "uWebSockets.js";

export interface MoveSample { at: number; x: number; y: number; }

export class SpeedHackDetector {
  private samples = new Map<number, MoveSample[]>(); // charId → 최근 샘플
  private strikes = new Map<number, number>();

  /** 서버 틱에서 이동 변위 샘플 수집(016 번 MoveAuthority 와 결합) */
  sample(charId: number, x: number, y: number, now: number) {
    const arr = this.samples.get(charId) ?? [];
    arr.push({ at: now, x, y });
    if (arr.length > 60) arr.shift(); // 최근 60 샘플(1 초)
    this.samples.set(charId, arr);
  }

  /** 초당 변위 기반 속도 검증 */
  detect(charId: number, maxSpeedPx: number, now: number): boolean {
    const arr = this.samples.get(charId);
    if (!arr || arr.length < 30) return false; // 표본 부족 → 판정 보류
    const span = (arr[arr.length - 1].at - arr[0].at) / 1000;
    if (span <= 0) return false;
    const dist = Math.hypot(
      arr[arr.length - 1].x - arr[0].x,
      arr[arr.length - 1].y - arr[0].y);
    const speed = dist / span;
    if (speed > maxSpeedPx * 1.6) { // 60% 초과 = 의심
      this.addStrike(charId, "speed", speed / maxSpeedPx);
      return true;
    }
    return false;
  }

  /** 액션 빈도 검증(스킬/공격) — 초당 상한 대비 비율 */
  detectActionSpam(charId: number, actions: number[], perSecLimit = 8): boolean {
    if (actions.length < 10) return false;
    // 정규 분포 이상치: 평균 + 4 시그마 초과 지속
    const perSec = new Map<number, number>();
    for (const t of actions) {
      const sec = Math.floor(t / 1000);
      perSec.set(sec, (perSec.get(sec) ?? 0) + 1);
    }
    const values = [...perSec.values()];
    const avg = values.reduce((s, v) => s + v, 0) / values.length;
    const peak = Math.max(...values);
    if (peak > avg * 3 && peak > perSecLimit * 2) {
      this.addStrike(charId, "action_spam", peak);
      return true;
    }
  }
}

```

```

    return false;
  }

  addStrike(charId: number, kind: string, severity: number) {
    const n = (this.strikes.get(charId) ?? 0) + 1;
    this.strikes.set(charId, n);
    // 3 회 누적 → 어드민 알림(094 번) + 자동 CAPTCHA(093 번) 트리거
    if (n === 3) this.onSuspect?.(charId, kind, severity);
  }
  onSuspect: ((charId: number, kind: string, severity: number) => void) | null = null;
}

/** 클라 셀프 체크(참고 신호) — 발췌 */
// const CHECK_INTERVAL = 5000;
// let drift = 0;
// setInterval(() => {
//   const pNow = performance.now();
//   const dNow = Date.now();
//   drift += Math.abs((pNow % 1000000) - (dNow % 1000000)) / 1000000;
//   if (drift > 0.5) socket.send(reportSpeedAnomaly()); // 클록 불일치 보고
// }, CHECK_INTERVAL);
// 최종 판정은 서버 — 클라 신호는 참고로만 기록(095 번 텔레메트리)

```

(3) 성능 최적화 팁

- 속도 판정은 '60% 초과 지속'처럼 여유 마진을 두어야 네트워크 지터/탭 복귀 스파이크를 오탐하지 않는다.
- 클라 클록 이중 검증(performance.now vs Date.now)은 스피드해킹이 훅(hook)한 시계를 걸러내는 저비용 신호다.
- 탐지는 즉시 밴이 아니라 strike 누적 + CAPTCHA(093 번) 순서로 — 오탐 비용(정상 유저 제재)이 탐지 이득보다 크면 안 된다.
- 모든 탐지 이벤트는 텔레메트리(095 번)로 남겨 패턴을 축적해야 2차 자동화 스크립트 (회피형)도 잡힌다.

092. 서버 측 이동 거리 및 타일 충돌 재연산 (벽뚫기 방지)

(1) 역할 설명

벽뚫기 방지의 핵심은 서버가 클라 좌표를 수신하는 것이 아니라, 클라의 이동 '입력'으로부터 서버가 직접 시뮬레이션(016 번 ServerAuthority)한다는 점이다. 이 항목은 그 위에 두 개의 독립 검증을 얹는다. (1) 거리 검증 — 이전 승인 위치에서 한 틱 허용 변위를 초과하면 롤백, (2) 타일 충돌 재연산 — blocked 그리드(041 번, 043 번 충돌 판정과 동일 그리드)로 새 위치가 벽 위인지 확인해 벽 위 위치는 이전 유효 위치로 되돌린다.

충돌 재연산의 세부는 '축 분리 슬라이딩'이다. X 이동과 Y 이동을 분리해 각각 충돌 검사하면 코너에서 벽에 붙어 끼는 현상 없이 벽면 미끄러짐이 자연스럽게, 클라의 이동 함수

(shared/movement.ts, 011 번)와 완전히 동일한 결과를 보장한다. 또한 반경(circle) 충돌(캐릭터 반경 12px)을 타일 검사에 곁해 '타일 모서리 꿰뚫기'를 방지한다.

(2) 구현 코드 — server/security/MoveValidator.ts

```
import type { Vec } from "../../shared/hitbox";

export interface MoveCheckResult {
  ok: boolean;
  corrected: Vec;          // 서버가 최종 인정하는 위치
  reason?: "wall" | "too_far" | "corner";
}

export class MoveValidator {
  private static CHAR_RADIUS = 12;
  private static TILE = 32;

  constructor(private blocked: Uint8Array,
    private mapW: number, private mapH: number,
    private maxTickDist = 16) {}    // 한 틱 최대 변위(속도 140 * 1/60 * 1.5)

  /** 축 분리 이동 + 충돌 재연산(클라 shared/movement.ts 와 동일 규칙) */
  simulate(from: Vec, dx: number, dy: number, dtMs: number): MoveCheckResult {
    const dist = Math.hypot(dx, dy);
    if (dist === 0) return { ok: true, corrected: from };

    // 1) 변위 상한 검증(거리 핵 방지)
    const move = Math.min(dist, this.maxTickDist) * Math.min(1, dtMs / 16.67);
    const ux = dx / dist, uy = dy / dist;

    // 2) 축 분리(슬라이딩) — X 먼저
    let cur: Vec = { ...from };
    const nx = cur.x + ux * move;
    if (!this.circleBlocked(nx, cur.y)) cur.x = nx;
    const ny = cur.y + uy * move;
    if (!this.circleBlocked(cur.x, ny)) cur.y = ny;

    // 3) 최종 위치가 여전히 벽(코너)이면 이전 위치 유지
    if (this.circleBlocked(cur.x, cur.y)) {
      return { ok: false, corrected: from, reason: "corner" };
    }
    const totalMove = Math.hypot(cur.x - from.x, cur.y - from.y);
    if (totalMove > this.maxTickDist * 1.05) {
      return { ok: false, corrected: from, reason: "too_far" };
    }
    return { ok: true, corrected: cur };
  }

  /** 원 반경 충돌: 캐릭터 반경을 감안한 4 코너 검사 */
  private circleBlocked(x: number, y: number): boolean {
```



```

const r = MoveValidator.CHAR_RADIUS;
const T = MoveValidator.TILE;
const corners = [
  [x - r, y - r], [x + r, y - r],
  [x - r, y + r], [x + r, y + r],
  [x, y], // 중심(빈 타일 통과 보장)
];
for (const [cx, cy] of corners) {
  const tx = Math.floor(cx / T), ty = Math.floor(cy / T);
  if (tx < 0 || ty < 0 || tx >= this.mapW || ty >= this.mapH) return true;
  if (this.blocked[ty * this.mapW + tx]) return true;
}
return false;
}

/** 텔레포트/웨이프인 감지: 틱 간 변위가 물리 상한 초과 */
static isWarp(prev: Vec, next: Vec, maxTickDist: number): boolean {
  return Math.hypot(next.x - prev.x, next.y - prev.y) > maxTickDist * 2;
}

```

(3) 성능 최적화 팁

- 충돌 그리드는 클라(TiledLoader 041 번)와 완전히 동일한 바이트열을 써야 — 클라/서버 판정 불일치로 '평평한 곳에서 끼임' 클레임이 생긴다.
- 원 반경 4 코너 검사로 '타일 모서리 꿰뚫기'를 막는다 — 중심점만 검사하면 반쪽 위치가 통과된다.
- 축 분리 슬라이딩은 벽면 미끄러짐 UX와 서버 검증을 동시에 해결하는 표준 기법이다.
- 거리 검증(maxTickDist)과 타일 검증은 독립적으로 실행해 — 어느 쪽 조작이든 정확한 reason 으로 로그가 남는다.

093. 매크로/봇 패턴 탐지 및 자동 캡차(CAPTCHA) 트리거

(1) 역할 설명

봇 패턴의 특징은 '너무 완벽한 인간 행동'이다. 반복 주기가 정확히 일정($\pm 5\text{ms}$), 이동 경로가 매번 동일, 무반응 구간이 없다(24 시간 연속 조작), 클릭 좌표가 픽셀 단위 동일 등이다. 탐지는 통계 기반으로 — 이동 방향 시퀀스 엔트로피, 액션 간격의 표준편차, 세션 지속 시간 분포를 윈도우별로 계산해 점수화한다.

트리거 단계는 3 단계다. (1) 점수 임계 초과 → 스텔스 챌린지(투명 CAPTCHA — 클라에 응답 요구, 정상 유저는 티 안 나게 배경 클릭 1 회), (2) 미응답/반복 실패 → 가시 CAPTCHA(이미지/수학), (3) 반복 회피 → 제재(099 번) 대상 기록. 캡차는 게임 중단을 최소화해야 — 대화형 이벤트(간단한 퍼즐)나 마우스 궤적 인간성 검증(자연스러운 커브 여부)을 선호한다.

(2) 구현 코드 — server/security/BotDetector.ts

```

export interface BehaviorWindow {
  moveDirs: number[];          // 이동 방향(라디안) 시퀀스
  actionGapsMs: number[];      // 액션 간격
  sessionStartAt: number;
  clicks: { x: number; y: number; t: number }[];
}

export class BotDetector {
  /** 행동 인간성 점수(0~1, 낮을수록 봇 의심) */
  score(w: BehaviorWindow): { score: number; flags: string[] } {
    const flags: string[] = [];
    let score = 1;

    // 1) 이동 방향 엔트로피(인간은 방향이 다양)
    if (w.moveDirs.length > 50) {
      const entropy = dirEntropy(w.moveDirs);
      if (entropy < 0.35) { flags.push("low_move_entropy"); score -= 0.3; }
    }

    // 2) 액션 간격 표준편차(인간은 변동이 크다)
    if (w.actionGapsMs.length > 30) {
      const cv = coefficientOfVariation(w.actionGapsMs);
      if (cv < 0.08) { flags.push("perfect_rhythm"); score -= 0.35; } // ±8% 미만 = 기계
      const avg = w.actionGapsMs.reduce((s, v) => s + v, 0) / w.actionGapsMs.length;
      if (avg < 250) { flags.push("inhuman_speed"); score -= 0.2; } // 250ms 미만 연타
    }

    // 3) 클릭 좌표 동일성(픽셀 단위 반복)
    const same = w.clicks.filter(c =>
      w.clicks.some(c2 => c2 !== c && c2.x === c.x && c2.y === c.y)).length;
    if (w.clicks.length > 20 && same / w.clicks.length > 0.8) {
      flags.push("pixel_repeat"); score -= 0.25;
    }

    // 4) 무중단 세션(인간은 휴식이 있다)
    const sessionMin = (Date.now() - w.sessionStartAt) / 60000;
    if (sessionMin > 480) { flags.push("session_8h+"); score -= 0.15; }

    return { score: Math.max(0, Math.min(1, score)), flags };
  }

  /** 점수 → 트리거 단계 결정 */
  triggerOf(score: number): "none" | "stealth_challenge" | "captcha" | "review" {
    if (score >= 0.55) return "none";
    if (score >= 0.35) return "stealth_challenge";
    if (score >= 0.2) return "captcha";
    return "review"; // 어드민(094 번) 수동 검토 큐
  }
}

function dirEntropy(dirs: number[]): number {
  const bins = new Array(8).fill(0); // 45 도씩 8 분할

```

```

for (const d of dirs) bins[Math.floor(((d + Math.PI) / (Math.PI * 2)) * 8) % 8]++;
const total = dirs.length;
let h = 0;
for (const b of bins) {
  if (!b) continue;
  const p = b / total;
  h -= p * Math.log2(p);
}
return h / 3;          // log2(8)=3 정규화
}

function coefficientOfVariation(xs: number[]): number {
  const mean = xs.reduce((s, v) => s + v, 0) / xs.length;
  const variance = xs.reduce((s, v) => s + (v - mean) ** 2, 0) / xs.length;
  return Math.sqrt(variance) / mean;
}

```

(3) 성능 최적화 팁

- 탐지 신호의 왕은 '액션 간격 표준편차'다 — 인간은 절대 $\pm 8\%$ 이하의 정확한 리듬을 유지할 수 없다.
- 스텔스 챌린지(투명 CAPTCHA)는 정상 유저에게 보이지 않는 검증으로 — 가시 CAPTCHA는 최후 단계로 남긴다.
- 8시간 이상 무중단 세션은 강한 신호지만 단독으로 제재하지 않는다 — 커피 마시며 방치한 정상 유저도 있다.
- 클릭 좌표 픽셀 동일성은 해상도별로 다르게 임계를 잡아야 — 고해상도 모니터와 저해상도 노트북의 클릭 분포가 다르다.

094. 어드민 웹 대시보드 (유저 조회, 아이템 지급, 공지, 제재 API)

(1) 역할 설명

어드민 대시보드는 Fastify 기반 REST API + 감사 로그 구조로 만든다. 4계층 권한(viewer / support / admin / root)이 모든 엔드포인트를 게이팅하고, 민감 작업(아이템 지급, 제재)은 반드시 (1) 2인 승인 또는 사유 필수, (2) 감사 로그(admin_audit) 기록, (3) 결과 확인 응답을 거친다. 유저 조회는 검색(이름/ID/IP/디바이스) + 상태(재화/아이템/로그 요약)의 읽기 전용 뷰로 시작해, 제재·지급은 별도 엔드포인트로 분리한다.

보안 요건은 운영망 분리(내부 네트워크/VPN 만 접근), JWT 세션 + IP 화이트리스트, 모든 요청의 operator 기록이다. 공지(긴급 점검/이벤트 안내)는 게임 서버로 방송 이벤트를 전달 (013 번 시스템 채널 재사용)하는 API 로 노출하고, 제재(099 번)는 즉시 적용 + 사전 안내 정책을 설정으로 선택할 수 있게 한다.

(2) 구현 코드 — server/admin/AdminApi.ts — Fastify 엔드포인트

```

import Fastify from "fastify";
import { Kysely } from "kysely";
type DB = import("../schema").Database;

```

```

const ROLE_BITS = { viewer: 1, support: 3, admin: 7, root: 15 };
const PERM = {
  READ_USER: 1, GRANT_ITEM: 2, NOTICE: 2, BAN: 4, AUDIT_VIEW: 4, ROOT_OP: 8,
};

const app = Fastify({ logger: true });
let db: Kysely<DB>;
let gameBus: { broadcastNotice(msg: string): void };

/** 인증/권한 미들웨어 */
app.addHook("preHandler", async (req, reply) => {
  const token = req.headers["x-admin-token"];
  const session = verifyAdminJwt(String(token)); // JWT + IP 화이트리스트 검사
  if (!session) return reply.code(401).send({ error: "unauthorized" });
  (req as any).admin = session;
});

function requirePerm(bit: number) {
  return async (req: any, reply: any) => {
    const role = (req as any).admin.role as keyof typeof ROLE_BITS;
    if ((ROLE_BITS[role] & bit) === 0) return reply.code(403).send({ error: "forbidden" });
  };
}

/** 유저 조회(읽기 전용) */
app.get("/admin/users/:id", { preHandler: requirePerm(PERM.READ_USER) },
  async (req, reply) => {
    const id = Number((req.params as any).id);
    const [user, items, wallet, recent] = await Promise.all([
      db.selectFrom("characters").where("id", "=", id).select().executeTakeFirst(),
      db.selectFrom("character_items").where("character_id", "=", id).limit(20).execute(),
      db.selectFrom("wallets").where(eb => eb.and([
        eb("owner_type", "=", "char"), eb("owner_id", "=", id)]).execute(),
      db.selectFrom("currency_ledger").where("owner_id", "=", id)
        .orderBy("at desc").limit(30).execute(),
    ]);
    if (!user) return reply.code(404).send({ error: "not_found" });
    return { user, items, wallet, recent };
  });

/** 아이템 지급(사유 필수 + 감사 로그) */
app.post("/admin/users/:id/grant", { preHandler: requirePerm(PERM.GRANT_ITEM) },
  async (req, reply) => {
    const id = Number((req.params as any).id);
    const { itemCode, qty, reason } = (req.body as any);
    if (!reason || reason.length < 10)
      return reply.code(400).send({ error: "reason_required_min10chars" });
    const ref = "admin:" + (req as any).admin.id + ":" + Date.now();
    await db.transaction().execute(async tx => {
      await tx.updateTable("character_items")

```

```

        .set(eb => ({ qty: eb("qty", "+", qty ?? 1) })))
        .where("character_id", "=", id).where("item_code", "=", itemCode).execute();
    await tx.insertInto("admin_audit").values({
        operator: (req as any).admin.id, action: "grant_item",
        target_id: id, payload: JSON.stringify({ itemCode, qty, reason }),
        ref, at: new Date(),
    }).execute();
    });
    return { ok: true, ref };
    });

/** 공지 방송 */
app.post("/admin/notice", { preHandler: requirePerm(PERM.NOTICE) },
    async (req) => {
        const { message, channel = "system" } = (req.body as any);
        gameBus.broadcastNotice(message);
        await db.insertInto("admin_audit").values({
            operator: (req as any).admin.id, action: "notice",
            target_id: 0, payload: JSON.stringify({ message, channel }), at: new Date(),
        }).execute();
        return { ok: true };
    });

/** 제재(099 번 미들웨어와 연결) */
app.post("/admin/users/:id/ban", { preHandler: requirePerm(PERM.BAN) },
    async (req) => {
        const { kind, durationH, reason } = (req.body as any);
        return applyBan(Number((req.params as any).id), kind, durationH, reason,
            (req as any).admin.id, db);
    });

async function applyBan(charId: number, kind: string, durationH: number,
    reason: string, operator: number, db: Kysely<DB>) {
    const until = durationH > 0 ? new Date(Date.now() + durationH * 3600000) : null;
    await db.insertInto("sanctions").values({
        character_id: charId, kind, until,
        reason, operator, at: new Date(),
    }).execute();
    return { ok: true, until };
}

app.listen({ port: 4000, host: "127.0.0.1" }); // 운영망 내부 바인딩

```

(3) 성능 최적화 팁

- 어드민은 감사 로그가 본질이다 — 모든 쓰기 작업에 operator + 사유 + ref 가 없으면 내부 사고 추적이 불가능하다.
- 권한은 비트(role)로 관리해 '지급은 되지만 제재는 안 되는 support' 같은 세분화가 한 줄로 가능하다.

- 운영망 바인딩(127.0.0.1 + VPN)과 JWT 는 필수이며, 2 인 승인(root 작업)은 대형 사고의 마지막 안전장치다.
- 유저 조회 API 는 읽기 전용 엔드포인트로 먼저 완성해야 — 제재/지급은 그 위에 얹는다.

095. 인게임 재화 유동성 및 유저 데이터 로그/텔레메트리 수집

(1) 역할 설명

텔레메트리는 (1) 이벤트 로그(행동 데이터 — 퀘스트 완료, 결제, 전투), (2) 성능 메트릭 (RTT 019 번, fps 001 번), (3) 경제 지표(085 번 흐름)의 3 계층을 수집한다. 클라는 배치 큐(5 초/50 개)로 전송해 네트워크 비용을 낮추고, 서버는 비동기 버퍼 → Kafka/파일 → 집계 파이프라인(일 단위 OLAP)으로 흘려보낸다. 개인정보 규정상 IP/기기 식별자는 해시 처리 + 보관 기간(90 일)을 명시한다.

수집 설계 원칙은 '게임 패스를 막지 않는다'다. 전송 실패는 드롭(재시도 없음), 큐 상한 초과는 샘플링(10%만 수집), 중요 이벤트(결제/제재)만 우선 전송이다. 이벤트 스키마는 (name, ts, charId, props JSON)의 단일 구조로 유지해 분석 쿼리가 스키마 진화에 강건하다.

(2) 구현 코드 — src/telemetry/Telemetry.ts — 클라 배치 전송

```
export interface TelemetryEvent {
  name: string; // "quest_complete" | "boss_kill" | "fps_sample"
  props?: Record<string, unknown>;
}

export class TelemetryClient {
  private queue: { name: string; ts: number; props: Record<string, unknown> }[] = [];
  private timer = 0;
  private static MAX_QUEUE = 200;
  private static FLUSH_MS = 5000;
  private sampled = Math.random() < 1; // 샘플링 비율(운영에서 0.1 로 조정)

  constructor(private endpoint = "/api/telemetry") {}

  start() {
    this.timer = window.setInterval(() => this.flush(), TelemetryClient.FLUSH_MS);
    window.addEventListener("pagehide", () => this.flush(true)); // 종료 시 마지막 전송
  }

  track(name: string, props?: Record<string, unknown>) {
    if (!this.sampled) return;
    this.queue.push({ name, ts: Date.now(), props: props ?? {} });
    if (this.queue.length > TelemetryClient.MAX_QUEUE) this.queue.shift(); // 오래된 것 드롭
  }

  async flush(sendBeacon = false) {
    if (!this.queue.length) return;
    const batch = this.queue.splice(0, this.queue.length);
    const body = JSON.stringify({ events: batch });
```

```

if (sendBeacon && navigator.sendBeacon) {
  navigator.sendBeacon(this.endpoint, new Blob([body], { type: "application/json" }));
  return;
}
try {
  await fetch(this.endpoint, {
    method: "POST", headers: { "Content-Type": "application/json" },
    body, keepalive: true,
  });
} catch { /* 실패 = 드롭(게임에 영향 없음) */ }
}
}

```

보조 코드 — server/telemetry/Ingest.ts — 서버 수집 버퍼

```

import uWS from "uWebSockets.js";
import fs from "node:fs";
import { createGzip } from "node:zlib";

/** JSONL 버퍼 → 분 단위 파일 플러시(간단 파이프라인; Kafka 로 치환 가능) */
export class TelemetryIngest {
  private buffer: string[] = [];
  private lastFlush = Date.now();
  private stream = fs.createWriteStream("/var/log/game/tel.jsonl", { flags: "a" });
  private gzip = createGzip();

  /** 클라 배치 수신(uWS POST 핸들러) */
  handleBatch(raw: ArrayBuffer) {
    try {
      const parsed = JSON.parse(Buffer.from(raw).toString());
      if (!Array.isArray(parsed.events)) return;
      for (const e of parsed.events) {
        if (typeof e.name !== "string") continue;
        this.buffer.push(JSON.stringify({
          n: e.name, ts: e.ts,
          // 개인정보: charId 는 서버가 재매핑한 pseudo id
          pid: pseudoId(e.charId),
          p: sanitize(e.props ?? {}),
        }));
      }
    } catch { /* malformed = 드롭 */ }
    if (this.buffer.length > 5000 || Date.now() - this.lastFlush > 60000) {
      this.flushFile();
    }
  }

  private flushFile() {
    const chunk = this.buffer.join("\n") + "\n";
    this.buffer = [];
    this.lastFlush = Date.now();
    this.stream.write(chunk);
  }
}

```

```

function pseudoId(charId: number): string {
  // salt 해시로 재식별 불가(개인정보 최소화)
  return require("node:crypto")
    .createHmac("sha256", process.env.TEL_SALT!)
    .update(String(charId)).digest("hex").slice(0, 16);
}

function sanitize(p: Record<string, unknown>): Record<string, unknown> {
  // 금지 필드 제거(ip, email, 디바이스 ID)
  const out: Record<string, unknown> = {};
  for (const [k, v] of Object.entries(p)) {
    if (/ip|email|device|token/i.test(k)) continue;
    out[k] = v;
  }
  return out;
}

/** 집계 예시(일 단위): 퀘스트 완료율, 결제 전환율, 재화 유동성(085 번 연계) */
// SELECT n, count(*) FROM tel WHERE ts BETWEEN ... GROUP BY n;

```

(3) 성능 최적화 팁

- 클라 배치(5 초/50 개) + sendBeacon(페이지 종료) 조합이 텔레메트리 표준이며, 실패는 절대 재시도하지 않는다.
- charId 는 서버에서 salt 해시(pseudo id)로 변환해 저장해야 개인정보 규정(보관 최소화)을 통과한다.
- 샘플링 비율은 유저 수에 반비례해 조정한다 — DAU 10 만이면 10% 샘플링으로도 통계가 충분하다.
- 이벤트 스키마는 name+props 단일 구조로 — 스키마 버전 관리가 아니라 props 내 확장으로 진화시킨다.

096. 웹 브라우저 무중단 핫픽스(Hot-Fix) 리소스 업데이트

(1) 역할 설명

웹 게임의 장점은 재접속 없이 코드를 갈아끼울 수 있다는 것이다. 핫픽스 파이프라인은 (1) 빌드 해시가 담긴 manifest.json, (2) 주기적(3 분) manifest 폴링, (3) 해시 변경 감지 → 새 체크 프리로드 → 안전 시점(싌 전환/메인 화면)에 교체, (4) 실패 시 롤백의 4 단계다. JS 모듈은 동적 import()로 교체하고, 게임 밸런스 데이터(JSON)는 캐시 무효화로 즉시 적용한다.

무중단 핵심은 '안전 시점에만 교체'다. 전투 중 코드 교체는 싌 참조가 깨지므로, 교체 지점을 (싌 전환, 로비 복귀, 리스폰)으로 제한한다. 또한 교체 실패(체크 로드 실패) 시 이전 버전 유지 + 재시도 백오프로 — 실패가 유저 세션을 죽이면 핫픽스가 아니라 사고가 된다. 긴급 서버 코드 수정은 이 파이프라인과 별개로 서버 재시작(100 번 롤링)으로 처리한다.

(2) 구현 코드 — src/hotfix/HotfixManager.ts

```

export interface HotfixManifest {
  version: string;           // "3.0.29-hotfix2"
  builtAt: number;
  chunks: Record<string, string>; // 모듈명 → 해시 파일 경로
  dataFiles: Record<string, string>; // JSON 밸런스 → 경로
  minClientVersion: string;    // 하위 호환 하한
}

export class HotfixManager {
  private current: HotfixManifest;
  private pollTimer = 0;
  private pending: HotfixManifest | null = null;

  constructor(private manifestUrl: string,
    private currentVersion: string,
    private hooks: { safePoint(): boolean; applyModule(path: string): Promise<void> } ) {}

  start() {
    this.pollTimer = window.setInterval(() => this.poll(), 3 * 60_000);
  }

  /** 1) manifest 폴링 → 해시 변화 감지 */
  private async poll() {
    try {
      const res = await fetch(this.manifestUrl, { cache: "no-store" });
      const m = (await res.json()) as HotfixManifest;
      if (m.version === this.currentVersion) return;
      if (compareVersion(m.minClientVersion, this.currentVersion) > 0) {
        // 하위 호환 하한 초과 → 전체 새로그침 유도(재접속 안내)
        this.promptReload(m);
        return;
      }
    }
    // 2) 새 청크 사전 프리로드(실패해도 현재 세션 유지)
    const ok = await this.preload(m);
    if (ok) this.pending = m; // 안전 시점까지 대기
  } catch { /* 네트워크 실패 = 다음 폴링 */ }

  private async preload(m: HotfixManifest): Promise<boolean> {
    try {
      for (const path of Object.values(m.chunks)) {
        await import(/* @vite-ignore */ path); // 브라우저 캐시에 미리 적재
      }
      for (const path of Object.values(m.dataFiles)) {
        await fetch(path, { cache: "reload" });
      }
      return true;
    } catch {
      return false; // 실패 → 이전 버전 유지, 다음 폴링 재시도
    }
  }
}

```

```

    }
  }

  /** 3) 안전 시점(씬 전환/로비)에서 호출 */
  tryApplyAtSafePoint(): boolean {
    if (!this.pending || !this.hooks.safePoint()) return false;
    const m = this.pending;
    this.pending = null;
    (async () => {
      try {
        for (const [name, path] of Object.entries(m.chunks)) {
          await this.hooks.applyModule(path);      // 모듈 핫스왑(010 번 패턴)
          console.info("[hotfix] applied " + name);
        }
        this.currentVersion = m.version;
        document.dispatchEvent(new CustomEvent("hotfix-applied",
          { detail: { version: m.version } }));
      } catch (e) {
        console.error("[hotfix] apply failed — keep old version", e);
        // 롤백: 상태는 이전 버전 유지(이미 로드된 모듈은 교체 실패 무시)
      }
    })();
    return true;
  }

  private promptReload(m: HotfixManifest) {
    // 메이저 업데이트: 유저 동의 후 새로고침
    document.dispatchEvent(new CustomEvent("update-available",
      { detail: { version: m.version } }));
  }
}

function compareVersion(a: string, b: string): number {
  const pa = a.split(/[.]/).map(Number), pb = b.split(/[.]/).map(Number);
  for (let i = 0; i < 3; i++) {
    if ((pa[i] ?? 0) !== (pb[i] ?? 0)) return (pa[i] ?? 0) - (pb[i] ?? 0);
  }
  return 0;
}

```

(3) 성능 최적화 팁

- 교체 시점은 씬 전환/로비로 제한한다 — 전투 중 모듈 스왑은 씬 참조 붕괴로 즉시 크래시가 난다.
- 프리로드(캐시 워밍) 후 안전 시점에 실제 교체하면 적용이 밀리초 단위로 끝난다.
- 적용 실패는 '이전 버전 유지'가 원칙이다 — 반쪽 교체 상태보다 구버전이 훨씬 안전하다.
- manifest 에 minClientVersion(하위 호환 하한)을 두면 급격한 프로토콜 변경 시 강제 재접속으로 안내할 수 있다.

097. 다국어(i18n) localization 텍스트 파싱 및 폰트 레이아웃

(1) 역할 설명

i18n 은 (1) 키 기반 텍스트 조회(플레이스홀더 치환), (2) 복수형/성별 규칙(CLDR), (3) RTL(아랍어) 레이아웃 반전, (4) CJK 폰트 폴백 체인의 4 요소다. 텍스트는 JSON 리소스(언어별 분리, 지연 로딩 — 004 번 번들)로 관리하고, 키는 코드에 하드코딩하지 않는다. 플레이스홀더({name}, {count})는 Intl.MessageFormat 스타일의 단순 치환기로 처리한다.

폰트 레이아웃 핵심은 CJK/라틴 혼합 폭 문제다. 한글+영문 혼합 텍스트는 이상적인 행 래핑이 언어마다 다르므로, UI 컴포넌트는 텍스트 길이가 아니라 실측 너비(ctx.measureText)로 배치해야 한다. 폰트 폴백 체인(맑은 고딕 → Noto Sans KR → sans-serif)을 CSS/캔버스 양쪽에 명시하고, 비트맵 폰트(데미지 텍스트)는 언어별로 별도 생성해야 글자가 네모(□)로 깨지지 않는다.

(2) 구현 코드 — src/i18n/i18n.ts

```
import en from "./locales/en.json";
import ko from "./locales/ko.json";
import ja from "./locales/ja.json";

const BUNDLES: Record<string, Record<string, string>> = { en, ko, ja };
type Lang = "en" | "ko" | "ja";

export class I18n {
  private lang: Lang;
  private listeners = new Set<() => void>();

  constructor(private fallback: Lang = "en") {
    this.lang = (localStorage.getItem("lang") as Lang)
      ?? this.detectFromBrowser();
  }

  /** 브라우저 언어 감지 → 지원 언어 매핑 */
  private detectFromBrowser(): Lang {
    const nav = navigator.language.toLowerCase(); // "ko-KR" → "ko"
    if (nav.startsWith("ko")) return "ko";
    if (nav.startsWith("ja")) return "ja";
    return this.fallback;
  }

  setLang(l: Lang) {
    this.lang = l;
    localStorage.setItem("lang", l);
    document.documentElement.lang = l;
    document.documentElement.dir = l === "ar" ? "rtl" : "ltr"; // RTL 확장 대비
    this.listeners.forEach(fn => fn());
  }
}
```

```

get current() { return this.lang; }

/** 텍스트 조회 + 플레이스홀더 치환: t("quest.complete", {count: 3}) */
t(key: string, params?: Record<string, string | number>): string {
  let text = BUNDLES[this.lang][key] ?? BUNDLES[this.fallback][key];
  if (!text) return key; // 누락 키는 키 자체 표시(디버그)
  if (params) {
    for (const [k, v] of Object.entries(params)) {
      text = text.replaceAll("{" + k + "}", String(v));
    }
  }
  return text;
}

/** 복수형: Intl.PluralRules(CLRD 규칙) */
tPlural(key: string, count: number): string {
  const rules = new Intl.PluralRules(this.lang);
  const cat = rules.select(count); // one/other/zero...
  return this.t(key + "." + cat, { count });
}
onChange(fn: () => void) { this.listeners.add(fn); }
}

// 캔버스 텍스트 폭 실측(Phaser 비트맵/일반 텍스트 레이아웃)
export function measureFit(
  ctx: CanvasRenderingContext2D, text: string, maxWidth: number,
): { text: string; lines: string[] } {
  ctx.font = '14px "Malgun Gothic", "Noto Sans KR", sans-serif';
  const words = text.split(" ");
  const lines: string[] = [];
  let cur = "";
  for (const w of words) {
    const test = cur ? cur + " " + w : w;
    if (ctx.measureText(test).width > maxWidth && cur) {
      lines.push(cur); cur = w;
    } else cur = test;
  }
  if (cur) lines.push(cur);
  return { text: lines.join("\n"), lines };
}

```

보조 코드 — locales/ko.json 예시 + 폰트 풀백

```

// — locales/ko.json —
// {
//   "quest.complete": "퀘스트 {name} 완료!",
//   "inventory.item_count": "보유 {count}개",
//   "chat.muted": "채팅 금지 상태입니다.",
//   "common.plural.item.one": "아이템 {count}개",
//   "common.plural.item.other": "아이템 {count}개"
// }

```

```
// — 폰트 폴백 체인(CSS + 캔버스 공통) —
// CSS: font-family: "Malgun Gothic", "Noto Sans KR", "Apple SD Gothic Neo", sans-serif;
// 캔버스: ctx.font = '14px "Malgun Gothic", "Noto Sans KR", sans-serif';
// 비트맵 폰트는 언어별 생성(한글 2350 자 기본 + 사용 빈도 상위 확장)

// — UI 폭 안전 규칙 —
// 1) 버튼 라벨은 길이 기준(한글 6 자 이내) + overflow 시 폰트 축소(scale)
// 2) 데미지 텍스트(비트맵)는 숫자 전용 폰트 — 언어 무관
// 3) RTL(ar) 확장 시 레이아웃 방향만 반전(logical property 사용)
```

(3) 성능 최적화 팁

- 누락 키는 키 자체를 표시하면(디버그 모드) 번역 누락을 런타임에 즉시 발견할 수 있다.
- CJK/라틴 혼합 텍스트는 실측(measureText) 기반 줄바꿈만 안전하다 — 글자 수 기반은 언어별 폭 차이로 깨진다.
- 비트맵 폰트(데미지 텍스트)는 언어별 생성이 필수다 — 라틴 전용 비트맵 폰트에 한글을 넣으면 네모가 된다.
- RTL 지원은 처음부터 logical property(margin-inline-start 등)로 작성해야 나중에 아랍어 추가가 반나절로 끝난다.

098. 소셜 로그인(구글, 카카오 등) OAuth2 & 게스트 계정 연동/이전

(1) 역할 설명

OAuth2 연동은 '인증 코드 → 토큰 교환(서버) → 유저 식별(provider+subject) → 세션 발급'의 서버 권위 흐름이다. 클라는 provider 별 SDK/리디렉션으로 인증 코드를 얻고, 서버는 code 를 provider 토큰 엔드포인트로 교환해 사용자 정보(id_token/sub)를 확인한다. state 파라미터(CSRF 방지)와 redirect_uri 정확 매칭은 필수다. 여러 프로바이더를 하나의 accounts 테이블(provider, subject)로 정규화해 계정 이중화(같은 유저가 구글+카카오 동시 연동)를 지원한다.

게스트 계정은 디바이스 토큰(localStorage + 서버 발급)으로 운영하며, 소셜 연동 시점에 '게스트 → 소셜' 계정 이전(마이그레이션)을 트랜잭션으로 수행한다. 이전 규칙은 (1) 대상 소셜 계정이 기존 유저면 '병합 요청' 확인 UI, (2) 기존 유저가 없으면 바로 연결, (3) 이미 연동된 계정은 거부의 3 분기다. 세션은 JWT(단기) + refresh 토큰(장기, 로테이션)으로 관리한다.

(2) 구현 코드 — server/auth/OAuth2.ts

```
import { Kysely } from "kysely";
import crypto from "node:crypto";
type DB = import("../schema").Database;

export type Provider = "google" | "kakao" | "guest";
export interface OAuthConfig {
```

```

clientId: string; clientSecret: string;
tokenUrl: string; userInfoUrl: string;
redirectUri: string;
}
const CONFIGS: Record<"google" | "kakao", OAuthConfig> = {
  google: {
    clientId: process.env.GOOGLE_CLIENT_ID!,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
    tokenUrl: "https://oauth2.googleapis.com/token",
    userInfoUrl: "https://openidconnect.googleapis.com/v1/userinfo",
    redirectUri: process.env.OAUTH_REDIRECT!,
  },
  kakao: {
    clientId: process.env.KAKAO_CLIENT_ID!,
    clientSecret: process.env.KAKAO_CLIENT_SECRET!,
    tokenUrl: "https://kauth.kakao.com/oauth/token",
    userInfoUrl: "https://kapi.kakao.com/v2/user/me",
    redirectUri: process.env.OAUTH_REDIRECT!,
  },
};

export class AuthService {
  constructor(private db: Kysely<DB>) {}

  /** 1) 인증 URL 생성(state 로 CSRF 방지) */
  authorizeUrl(provider: "google" | "kakao"): { url: string; state: string } {
    const c = CONFIGS[provider];
    const state = crypto.randomBytes(16).toString("hex");
    // state 는 임시 저장(TTL 10 분) 후 콜백에서 검증
    const params = new URLSearchParams({
      client_id: c.clientId, redirect_uri: c.redirectUri,
      response_type: "code", scope: provider === "google" ? "openid email" : "profile_nickname",
      state,
    });
    return { url: (provider === "google"
      ? "https://accounts.google.com/o/oauth2/v2/auth"
      : "https://kauth.kakao.com/oauth/authorize") + "?" + params, state };
  }

  /** 2) 콜백: code → 토큰 → 사용자 식별 → 세션 발급 */
  async callback(provider: "google" | "kakao", code: string, state: string, expectedState: string) {
    if (state !== expectedState) throw new Error("CSRF_STATE_MISMATCH");
    const c = CONFIGS[provider];
    // 토큰 교환(서버 전용 secret — 클라에 노출 금지)
    const tokenRes = await fetch(c.tokenUrl, {
      method: "POST",
      headers: { "Content-Type": "application/x-www-form-urlencoded" },
      body: new URLSearchParams({
        grant_type: "authorization_code", code,
        client_id: c.clientId, client_secret: c.clientSecret,
        redirect_uri: c.redirectUri,
      }),
    });
  }
}

```

```

});
const tokens = await tokenRes.json() as { access_token: string; id_token?: string };
// 사용자 식별
const infoRes = await fetch(c.userInfoUrl, {
  headers: { Authorization: "Bearer " + tokens.access_token },
});
const info = await infoRes.json() as { sub?: string; id?: number };
const subject = String(info.sub ?? info.id);
return this.loginWithIdentity(provider, subject);
}

/** 3) 정규화 계정 조회/생성 + 게스트 이전 */
async loginWithIdentity(provider: Provider, subject: string, guestCharId?: number) {
  return this.db.transaction().execute(async tx => {
    let account = await tx.selectFrom("oauth_accounts")
      .where("provider", "=", provider).where("subject", "=", subject)
      .select("character_id").executeTakeFirst();
    if (account && guestCharId && account.character_id !== guestCharId) {
      throw new Error("OAUTH_ALREADY_LINKED"); // 병합 요청 UI 로 유도
    }
    let charId = account?.character_id ?? guestCharId ?? 0;
    if (!charId) {
      // 신규 유저 생성
      const created = await tx.insertInto("characters")
        .values({ account_id: 0, name: "user_" + subject.slice(0, 6),
          class_code: "novice" }).returning("id").executeTakeFirstOrThrow();
      charId = created.id;
    }
    await tx.insertInto("oauth_accounts").values({
      provider, subject, character_id: charId,
    }).onConflict(oc => oc.doNothing()).execute();
    // 게스트 이전이면 디바이스 토큰 무효화(중복 접속 차단)
    if (guestCharId) {
      await tx.updateTable("device_tokens").set({ revoked: true })
        .where("character_id", "=", guestCharId).execute();
    }
    return { charId };
  });
}
}

```

보조 코드 — PostgreSQL DDL — 계정 정규화

```

CREATE TABLE oauth_accounts (
  provider  VARCHAR(12) NOT NULL, -- google | kakao | guest
  subject   VARCHAR(64) NOT NULL, -- provider 내부 고유 id(sub)
  character_id BIGINT NOT NULL REFERENCES characters(id),
  linked_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (provider, subject)
);

CREATE UNIQUE INDEX uq_char_provider ON oauth_accounts(character_id, provider)
WHERE provider != 'guest'; -- 동일 유저, 동일 provider 중복 연동 방지

```

```
CREATE TABLE device_tokens (
  token      VARCHAR(64) PRIMARY KEY, -- 게스트 디바이스 토큰
  character_id BIGINT NOT NULL REFERENCES characters(id),
  revoked     BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

(3) 성능 최적화 팁

- client_secret 은 절대 클라에 두지 않는다 — 토큰 교환은 반드시 서버에서 수행한다 (코드 교환 흐름, code flow).
- state(CSRF) 검증과 redirect_uri 정확 매칭은 OAuth2 의 최소 보안선이며, 생략하면 계정 탈취가 가능해진다.
- 계정 정규화(oauth_accounts)로 '게스트 → 소셜 이전'이 UPDATE 한 줄이 된다 — 유저 데이터를 통째로 옮기는 방식은 절대 금물이다.
- 게스트 토큰 폐기(revoke)를 이전 트랜잭션에 포함해야 동일 디바이스 토큰으로 두 계정 접속하는 사고가 막힌다.

099. 계정/IP/디바이스 단위 제재(Ban) 및 채팅 금지(Mute) 미들웨어

(1) 역할 설명

제재는 대상(계정/디바이스/IP), 종류(접속 차단/채팅 금지/거래 금지), 기간(임시/영구) 의 3 차원이며, sanctions 테이블의 행으로 표현된다. 적용 지점은 두 곳이다. (1) 연결 수립 시 (소켓 핸드셰이크 후 인증 단계) 접속 제재 검사, (2) 채팅/거래 등 액션 단계(094 번 어드민과 연결)별 미들웨어 검사. 디바이스/IP 제재는 '새 계정 생성 우회'를 막는 2 차 방어선이다.

미들웨어 설계 원칙은 '검사는 캐시, 원본은 DB'다. 제재 목록을 Redis(또는 프로세스 캐시)로 미러링해 소켓 패킷 핫패스에서 O(1) 검사를 수행하고, 제재 부여/해제 시 캐시를 즉시 갱신한다. Mute 는 채팅 전용(게임 플레이는 허용)이며, 임시 제재 만료는 TTL 로 자동 해제된다. 제재 이력은 유저 단위 조회 API(094 번)로 CS 에 노출한다.

(2) 구현 코드 — server/security/SanctionGate.ts — 제재 게이트 + 캐시

```
import Redis from "ioredis";
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export type SanctionKind = "ban" | "mute" | "trade_ban";
export interface SanctionRow {
  id: number; character_id: number; kind: SanctionKind;
  until: Date | null; reason: string;
  ip_prefix?: string; device_hash?: string;
}

export class SanctionGate {
  private cache = new Map<string, { kinds: Set<SanctionKind>; until: number | null }>();
```



```

constructor(private redis: Redis, private db: Kysely<DB>) {}

/** 제재 부여(094 번 어드민 API에서 호출) — DB + 캐시 동시 갱신 */
async apply(s: Omit<SanctionRow, "id">) {
  await this.db.insertInto("sanctions").values({
    character_id: s.character_id, kind: s.kind,
    until: s.until, reason: s.reason,
    ip_prefix: s.ip_prefix ?? null, device_hash: s.device_hash ?? null,
  }).execute();
  const key = "sanc:" + s.character_id;
  const ttl = s.until ? Math.max(1, Math.floor((s.until.getTime() - Date.now()) / 1000)) : 86400;
  await this.redis.sadd(key, s.kind);
  await this.redis.expire(key, ttl);
  this.cache.set(key, { kinds: new Set([s.kind]), until: s.until?.getTime() ?? null });
}

/** 연결/액션 게이트(핫패스 O(1)) */
check(charId: number, need: SanctionKind): boolean {
  const c = this.cache.get("sanc:" + charId);
  if (c) {
    if (c.until && c.until < Date.now()) { this.cache.delete("sanc:" + charId); return true; }
    return !c.kinds.has(need);
  }
  return true; // 캐시 미스 = 정상(폴백은 비동기 로드)
}

/** 채팅 미들웨어(051 번 ChatService에서 호출) */
chatAllowed(charId: number): { ok: boolean; reason?: string } {
  if (!this.check(charId, "mute")) return { ok: false, reason: "muted" };
  if (!this.check(charId, "ban")) return { ok: false, reason: "banned" };
  return { ok: true };
}

/** 접속 게이트(소켓 open 후 인증 직후) — 디바이스/IP 확장 검사 */
async connectionAllowed(charId: number, ip: string, deviceHash: string): Promise<boolean> {
  if (!this.check(charId, "ban")) return false;
  // 디바이스/IP 제재(우회 계정 방지) — DB 조회(저빈도 경로)
  const rows = await this.db.selectFrom("sanctions")
    .where(eb => eb.or([
      eb.and([eb("device_hash", "=", deviceHash), eb("kind", "=", "ban")]),
      eb.and([eb("ip_prefix", "=", ip.split(".").slice(0, 3).join(".") + "."),
        eb("kind", "=", "ban")]),
    ]))
    .where(eb => eb.or([
      eb("until", "is", null),
      eb("until", ">", new Date()),
    ])).select("id").execute();
  return rows.length === 0;
}
}

```

(3) 성능 최적화 팁

- 제재 검사는 핫패스(캐시)와 콜드패스(DB)를 분리해 — 패킷 처리 성능에 영향을 주지 않고 정확성을 유지한다.
- IP 제재는 /24 프리픽스 단위로 과하게 잡으면 공유기/학교 네트워크 정상 유저까지 막힌다 — 마지막 수단으로만.
- Mute 는 '만료 TTL + 사유'를 함께 기록해 CS 문의에 즉시 답할 수 있게 한다.
- 제재 대상 계정의 파티/길드 영향(팀 이탈)은 제재 API에서 이벤트로 방송해 자동 정리되게 한다.

100. DB 자동 스냅샷 백업 및 분산 서버 패일오버(Failover)

(1) 역할 설명

백업 전략은 '전일 스냅샷 + WAL 아카이브(포인트 인 타임 복구)'의 PostgreSQL 표준이다. 스냅샷은 매일 새벽(basebackup)을 S3 로 업로드하고, WAL 은 5 분 단위로 아카이브해 RPO(복구 시점 손실)를 5 분 이하로 유지한다. 복구 훈련(월 1 회 리스토어 리허설) 없는 백업은 없는 것이나 다름 없으므로, 복구 소요 시간(RTO) 측정을 운영 절차에 포함한다.

패 일 오 버 는 (1) PostgreSQL 스트리밍 복제(대기 노드) + 자동 승격(패트로니테/pg_auto_failover), (2) 게임 서버 노드 헬스 체크 + 세션 이관(017 번 룸 레지스트리 재할당), (3) 로드밸런서 헬스 체크로 구성된다. 게임 서버는 스테이트리스(세션은 Redis/DB 에)가 이상적이지만, 룸 상태는 프로세스 내에 있으므로 '룸 소유 노드 다운 → 룸 재생성(진행 중 던전은 실패 처리 + 보상 조정)'의 명시적 정책이 필요하다.

(2) 구현 코드 — ops/backup.sh + server/failover/Health.ts

```
#!/bin/bash
# — ops/backup.sh — 일일 스냅샷 + WAL 아카이브(S3 업로드) —
set -euo pipefail
STAMP=$(date +%Y%m%d_%H%M%S)
PG_HOST=${DB_HOST} PG_PORT=5432

# 1) basebackup(전일 스냅샷)
pg_basebackup -h $PG_HOST -U replicator -D /backup/base_$STAMP \
  -Ft -z -X stream --checkpoint=fast
# 2) 압축 + S3 업로드(3개월 보존 정책)
tar -czf /backup/base_$STAMP.tar.gz -C /backup base_$STAMP
aws s3 cp /backup/base_$STAMP.tar.gz \
  s3://sertz-backups/base/ --storage-class STANDARD_IA
# 3) 30일 경과 로컬 정리
find /backup -name "base_*.tar.gz" -mtime +30 -delete

# PITR 복구 절차(훈련 문서화 필수):
# 1) 최신 basebackup 다운로드/복원
# 2) recovery.signal + restore_command(archived WAL)
```

```
# 3) recovery_target_time = "사고 1 분 전"
# 4) 승격 → 앱 재연결
```

보조 코드 — server/failover/Health.ts — 노드 헬스 + 룸 이관

```
import Redis from "ioredis";
import { Kysely } from "kysely";
type DB = import("../schema").Database;

export class FailoverCoordinator {
  private readonly NODE_KEY = "nodes:alive"; // ZSET: nodeId → heartbeat
  constructor(private redis: Redis, private db: Kysely<DB>,
    private nodeId: string) {}

  /** 하트비트(5 초 주기) */
  async heartbeat() {
    await this.redis.zadd(this.NODE_KEY, Date.now(), this.nodeId);
  }

  /** 감시자(리더 노드가 크론 30 초): 죽은 노드 감지 → 룸 재할당 */
  async sweep(onRoomLost: (roomId: string) => void) {
    const now = Date.now();
    const dead = (await this.redis.zrangebyscore(this.NODE_KEY, "-inf", now - 15000))
      .filter(n => n !== this.nodeId);
    for (const deadNode of dead) {
      // 죽은 노드 소유 룸 조회 → 재생성 이벤트
      const all = await this.redis.hvals("rooms:");
      for (const raw of all) {
        const r = JSON.parse(raw);
        if (r.ownerId === deadNode && r.phase !== "CLOSED") {
          onRoomLost(r.roomId); // 던전은 실패 처리 + 보상 조정, 필드는 재생성
          await this.redis.hdel("rooms:", r.roomId);
        }
      }
      await this.redis.zrem(this.NODE_KEY, deadNode);
    }
  }

  /** PostgreSQL 헬스(복제 지표) — pgbouncer/패트로니테가 1 차, 앱은 2 차 검사 */
  async dbHealthy(): Promise<boolean> {
    try {
      const r = await this.db.selectFrom("characters")
        .select(eb => eb.fn.count("id").as("n")).executeTakeFirst();
      return Number(r?.n ?? 0) >= 0;
    } catch { return false; }
  }

  /** 절차 요약(운영 반복):
  * 1) DB 장애: 패트로니테 자동 승격(대기 노드 → 주) → 앱은 재시도로 재연결
  * 2) 게임 노드 장애: 감시자가 15 초 내 감지 → 룸 재생성 방송 → 유저 재접속 유도
```

- * 3) 리전 전체 장애: DNS/로드밸런서 → 백업 리전(롤백 RPO 5 분 선언)
- * 4) 모든 조치는 상태 페이지 + 인게임 공지(094 번)로 즉시 안내
- */

(3) 성능 최적화 팁

- 백업의 진짜 지표는 RTO/RPO 다 — 월 1 회 리스토어 리허설로 실측하지 않으면 문서상 수치는 허구다.
- WAL 아카이브 5 분 = RPO 5 분이며, 결제 관련 트랜잭션은 별도 즉시 복제(동기 커밋)로 보호한다.
- 게임 노드 패일오버는 '룸 소유 상태'가 스테이트리스가 아니므로, 진행 중 던전/전장은 명시적 실패 정책(보상 조정)을 미리 선언해야 유저 분쟁이 없다.
- 장애 공지(094 번 어드민 → 051 번 시스템 채널 + 080 번 푸시)가 유저 이탈을 절반으로 줄인다 — 무소식이 최악이다.